Reconocimiento de flores por imagen

Cómo se construyó un sistema de inteligencia artificial que identifica 102 especies de flores

Autor: Luis Alfredo Jaramillo Restrepo

Materia: Inteligencia Artificial

Profesor: Andrés Atehortúa Gutiérrez

Proyecto final

13/10/2026

Índice

Parte I. Para empezar

Qué es este proyecto y qué encontrarás en este documento

En una frase

La idea, contada como se la contarías a alguien que no programa

Qué hace bien y qué no hace

Cómo leer este documento

Ficha técnica del proyecto

Una advertencia útil

Conceptos básicos: de una foto a una respuesta

1. Qué es una imagen para un ordenador

2. Qué es clasificar una imagen y qué es una etiqueta

3. Qué es un modelo de inteligencia artificial

4. Qué es una red neuronal y qué es una red convolucional

5. Entrenar: épocas, lotes, función de pérdida y learning rate

6. Sobreajuste y los tres conjuntos de datos

7. Transfer learning: la clave de este proyecto

8. Aumentación de datos

9. Cómo se mide si el modelo es bueno

10. GPU, Google Colab y Gradio

11. Qué es el umbral de confianza

Resumen en una frase

Guía práctica: cómo usarlo sin saber programar

Para qué sirve esta guía

Qué necesitas antes de empezar

Lo que vas a hacer, en cinco líneas

Las dos corridas: primero la rápida, después la completa

Paso a paso numerado en Google Colab

Qué verás en pantalla, mensaje por mensaje

La corrida rápida primero y la corrida completa después

Cómo usar la demo

Cómo recoger los resultados y dónde queda el informe

Problemas frecuentes y cómo resolverlos

Advertencia importante sobre la corrida rápida

Chuleta final

Parte II. El codigo explicado

Los cimientos: configuración y utilidades

Para qué sirve este archivo (en lenguaje llano)

Qué hay dentro

Por qué se hizo asi

Los datos: cómo entran las fotos al sistema

Para qué sirve este archivo (en lenguaje llano)

Qué hay dentro

Por qué se hizo asi

El modelo: el cerebro que aprende a distinguir flores

Para qué sirve este archivo (en lenguaje llano)

Por qué se hizo así

El entrenamiento: las 25 pasadas que convierten el modelo en experto

Para qué sirve este archivo (en lenguaje llano)

Por qué se hizo así

Evaluar y predecir: cómo sabemos si acierta

Para qué sirve este archivo (en lenguaje llano)

Predecir: dar una respuesta a una foto suelta

Por qué se hizo asi

La demo: la ventana donde cualquiera puede probarlo

Para qué sirve este archivo (en lenguaje llano)

La función que hace la predicción: predecir

build_app: construir la ventana

main y cli: encender la ventana

Por qué se hizo asi

El cuaderno de Colab, las pruebas automáticas y el empaquetado

Qué es un cuaderno (notebook) y qué es una celda

Recorrido del cuaderno, paso a paso

Las pruebas automáticas

El empaquetado: empaquetar_colab.py

El archivo requirements.txt

Resumen del capítulo

Parte III. Las decisiones y los errores

Por qué se hizo así: las decisiones y los errores del camino

1. Por qué se usaron las flores de Oxford y no fotos propias
2. Por qué PyTorch y no TensorFlow
3. Por qué se usó una red ya entrenada y no una red construida desde cero
4. Por qué congelar la parte preentrenada 3 épocas y luego descongelarla con una velocidad menor
5. Por qué se aumentaron los datos y por qué se suavizaron las etiquetas
6. Por qué se usaron las particiones oficiales (y la salvedad de las fotos repetidas)
7. Por qué existe un umbral de confianza y por que acabó convertido en un aviso
8. Por qué todo se comprueba con pruebas automáticas y por que hubo una auditoría adversarial
9. Por qué el código "degrada" en lugar de fallar
10. Los errores reales que aparecieron y cómo se resolvieron
11. Por qué se documenta lo que NO se ha podido verificar

Resumen del capítulo

Parte IV. Diccionario

Diccionario: todos los términos en lenguaje llano

Anexo A. Codigo fuente completo

Anexo B. El cuaderno de Colab, celda por celda

Anexo C. Estructura del proyecto y como regenerarlo

Parte I. Para empezar

Qué es este proyecto y qué encontrarás en este documento

En una frase

Es un programa que **mira una fotografía de una flor y dice de qué especie se trata**, eligiendo entre **102 especies distintas**, y que funciona en Google Colab, es decir, en el navegador y sin instalar nada en tu ordenador.

La idea, contada como se la contarías a alguien que no programa

Imagina que quieres construir un experto en flores. No puedes escribirle una lista de reglas del tipo "si los pétalos son rosas y hay cinco, entonces es una rosa", porque las flores son demasiado variadas y las reglas se vuelven infinitas. Lo que haces es otra cosa: le enseñas **miles de fotos ya etiquetadas** con su nombre correcto (esta es una rosa, esta es un tulipán) y dejas que él solo descubra las diferencias. Eso es, en esencia, la inteligencia artificial de este proyecto.

El programa hace tres cosas:

1. **Aprende** mirando 8189 fotos de flores que ya vienen clasificadas por la Universidad de Oxford.
2. **Se evalúa** con fotos que nunca vio durante el aprendizaje, para saber si de verdad ha aprendido o solo ha memorizado.
3. **Se usa**: le subes una foto cualquiera y te devuelve las cinco especies más probables con su porcentaje de confianza.

Qué hace bien y qué no hace

Es honesto decirlo desde el principio.

Sí hace:

- Distinguir entre las 102 especies del catálogo con una precisión que, en la corrida completa, es alta.
- Decirte "no estoy seguro" cuando la foto no se parece a nada de lo que conoce, en lugar de inventarse una respuesta.
- Funcionar sin GPU (más lento) y con GPU (rápido), tanto en un ordenador normal como en la nube.

No hace:

- No reconoce flores que no estén entre esas 102 especies. Si le enseñas una flor de tu jardín que no está en el catálogo, lo más probable es que elija la que más se le parezca y te avise de que la confianza es baja.
- No detecta varias flores en una misma foto: asume que la foto es de una sola flor.
- No es infalible. Con 102 clases y flores que se parecen mucho entre sí (por ejemplo, distintas variedades de petunia), se equivoca.

Cómo leer este documento

No hace falta leerlo entero ni en orden. Está pensado para que puedas elegir:

Si eres...	**Lee...**
Alguien que solo quiere entender qué es y cómo se	Las partes I y IV, y el diccionario

usa	
Alguien que quiere entender cómo funciona por dentro	Las partes I y II completas
Alguien que va a leer o modificar el código	Todo, y usa el anexo A como referencia
Alguien que solo quiere ejecutarlo	La guía práctica (sección 3)

El documento está dividido en cuatro partes:

- **Parte I. Para empezar.** Qué es el proyecto, los conceptos básicos explicados desde cero (qué es una imagen para un ordenador, qué es un modelo, qué es entrenar) y la guía práctica paso a paso.
- **Parte II. El código explicado.** Un capítulo por archivo. En cada uno se explica qué hace, se muestra el código real y se justifica por qué se escribió así.
- **Parte III. Las decisiones y los errores.** Por qué se eligió cada cosa y qué problemas reales aparecieron por el camino, contados sin adornos, incluidos los errores que cometimos nosotros.
- **Parte IV. Diccionario.** Todos los términos técnicos en lenguaje llano, en orden alfabético.

Después vienen los **anexos**, que contienen el código completo tal cual está en el proyecto, para que nada quede fuera de este documento.

Ficha técnica del proyecto

Dato	**Valor**
Qué resuelve	Clasificar una foto entre 102 especies de flores
Datos de aprendizaje	Oxford Flowers-102: 8189 fotos (1020 entrenamiento, 1020 validación, 6149 prueba)
Técnica principal	Transfer learning sobre una red EfficientNet-B0 preentrenada con ImageNet
Lenguaje y librerías	Python con PyTorch, TorchVision, Gradio y Matplotlib
Dónde se ejecuta	Google Colab (gratuito) o cualquier ordenador con Python
Entrenamiento completo	25 épocas, lotes de 32 imágenes de 224x224, 15-25 minutos en una GPU T4
Entrenamiento rápido de prueba	2 épocas sobre 408 imágenes, unos minutos
Resultado medido con 4 épocas	37,8 % de acierto exacto y 67,4 % entre las 5 primeras propuestas, sobre las 6149 fotos oficiales de prueba
Cómo se sabe que funciona	78 pruebas automáticas, una auditoría independiente y varias ejecuciones completas
Qué se entrega	Un archivo comprimido con el cuaderno de Colab, el

	código, las pruebas y los informes

Una advertencia útil

En este documento aparecen a veces números pequeños, como el 37,8 % de acierto. **No son el resultado final del proyecto**: son el resultado de una corrida corta de 4 épocas que se hizo para comprobar que todo funcionaba. El resultado bueno se obtiene con la corrida completa de 25 épocas, que tarda entre 15 y 25 minutos en una GPU de Colab y que tienes que lanzar tú. Cuando lo hagas, el propio cuaderno te imprimirá un resumen con tus cifras, listo para copiar en el informe.

Y una última cosa, quizá la más importante de este documento: al final de la parte III contamos **los errores que cometimos** y cómo los encontramos. En un proyecto real eso no es un detalle vergonzoso, es la parte que más enseña.

Conceptos básicos: de una foto a una respuesta

Esta sección está escrita para alguien que nunca ha programado y que no ha estudiado inteligencia artificial. No hace falta saber nada de computación para leerla: cada idea se explica primero con una comparación cotidiana y solo después se conecta con el proyecto. Al final deberías poder explicarle a otra persona, con tus palabras, qué hace el sistema "IA flores" cuando le entregas la foto de una flor.

1. Qué es una imagen para un ordenador

Cuando miras una foto ves una flor. Un ordenador no ve nada: **solo maneja números**. La forma que tiene de "ver" la foto es guardarla como una enorme tabla de números, y nada más.

Imagina que divides la foto en una cuadrícula finísima, como un mosaico de azulejos diminutos. Cada cuadradito se llama **píxel** ("píxel" viene de *picture element*, elemento de imagen). Una foto de 224 por 224 píxeles, que es el tamaño que usa este proyecto, es una rejilla de 224 columnas por 224 filas: en total 50 176 cuadraditos.

Cada cuadradito tiene un color, y un ordenador representa el color con tres números: cuánto rojo, cuánto verde y cuánto azul lleva esa mezcla. Cada uno de esos tres números va de 0 a 255, donde 0 es "nada de ese color" y 255 es "todo ese color". Con esa receta:

- (255, 0, 0) es rojo puro.
- (0, 0, 0) es negro, porque no hay nada de ningún color.
- (255, 255, 255) es blanco, porque están los tres al máximo.
- (255, 160, 200) es un rosa claro, mezcla de mucho rojo y bastante azul.

Si juntamos todo, una foto de 224 por 224 en color es una tabla de números con 224 filas, 224 columnas y 3 valores por casilla. Eso son 150 528 números para una sola foto.

En el proyecto, cuando el programa dice que carga una imagen y la convierte en algo llamado *tensor*, lo único que hace es leer la foto y colocarla en esa tabla de números, ordenada y lista para operar con ella. Eso es todo el misterio de "la máquina ve la imagen".

```
Una foto pequeña de 3 x 3 píxeles, vista como tabla de números
(en cada casilla, los tres canales rojo-verde-azul):

  (255,255,255)  (255,  0,  0)  (255,255,255)
  (  0,255,  0)  (255,255,  0)  (  0,  0,255)
  (128,128,128)  (255,255,255)  (255,255,255)
```

2. Qué es clasificar una imagen y qué es una etiqueta

Clasificar es elegir una opción de una lista cerrada. Cuando ordenas tu ropa por "camisetas, pantalones, abrigos" estás clasificando: miras la prenda y decides a qué cajón va. No describes la prenda entera, solo dices a qué grupo pertenece.

Clasificar una imagen es exactamente eso: mirar la foto y decidir a cuál de las opciones de la lista pertenece. Cada una de esas opciones se llama **etiqueta** o **clase**. Las dos palabras significan lo mismo en este contexto: el nombre del grupo.

La lista de clases de este proyecto tiene **102 etiquetas**, porque el catálogo que usamos es el del dataset público *Oxford Flowers-102*, que contiene **102 especies de flores**. La respuesta del sistema siempre tiene que salir de esa lista: no puede inventarse una especie nueva ni responder "no sé".

Y aquí aparece un matiz que conviene entender desde el principio, porque no es lo mismo que clasificar la ropa:

- Ordenar una camiseta es fácil: la etiqueta es "camiseta".
- Distinguir la especie número 47 de la especie número 48 puede ser difícil **incluso para una persona**, si las dos flores son casi idénticas salvo por el número de pétalos o el tono.

Clasificar no es "entender" la foto. Es elegir una etiqueta de una lista cerrada. Un sistema puede acertar mucho y, aun así, no tener ni idea de botánica.

En el código, cada etiqueta es simplemente un número del 0 al 101, y al final se traduce al nombre de la especie para mostrarlo.

3. Qué es un modelo de inteligencia artificial

Un **modelo** es la parte del sistema que sabe hacer la tarea. No es un programa con instrucciones escritas a mano: es una gran cantidad de números ajustables que, combinados, producen una respuesta.

La analogía del aprendiz

Piensa en alguien que quiere aprender a reconocer flores y no tiene maestro de botánica, pero sí tiene un montón de fotos ya etiquetadas con el nombre correcto. El aprendiz hace lo siguiente:

4. Mira una foto y arriesga un nombre.
5. Compara su respuesta con el nombre correcto que viene en la ficha.
6. Anota en qué se equivocó.
7. Ajusta un poquito su forma de mirar, para acercarse la próxima vez.
8. Repite esto miles de veces.

Ese ciclo es todo el aprendizaje automático. Al principio el aprendiz acierta por casualidad, una vez de cada cien. Después de ver muchas fichas, empieza a fijarse en detalles útiles (color del centro de la flor, forma de los pétalos, hojas) y su porcentaje de acierto sube.

Un detalle importante: el aprendiz **no guarda las fotos** que ve. Lo que guarda son los ajustes de su forma de mirar. Eso es el modelo entrenado: un archivo con esos números ajustados. En este proyecto ese archivo se llama *checkpoint* y ocupa muy poco (unos pocos megas), muchísimo menos que las 8189 fotos del catálogo.

Reglas a mano frente a aprender de ejemplos

Hay dos maneras de conseguir que un ordenador reconozca flores, y la diferencia es la clave de todo el campo.

Escribir reglas a mano. Tú escribes las instrucciones. Por ejemplo: "si la flor tiene más de 20 pétalos y el centro es amarillo, es la especie A". Este enfoque es transparente y controlable, pero se rompe enseguida: las flores cambian de forma según el ángulo, la luz, la estación del año y la cámara. Harían falta millones de reglas y ni así funcionaría, porque nadie es capaz de escribir en palabras qué hace exactamente que dos flores se parezcan.

Aprender de ejemplos. Tú no escribes ninguna regla sobre flores. En su lugar le das al modelo miles de fotos con su nombre correcto y dejas que él mismo descubra qué detalles distinguen una especie de otra. Tú solo decides cómo se le presentan los ejemplos y cómo se mide su error.

	Reglas a mano	**Aprender de ejemplos**
Quién decide las características	La persona que programa	El propio modelo
Cuánto trabajo humano	Muchísimo, y nunca termina	Preparar datos y configurar
Si aparecen flores nuevas o fotos raras	Suele fallar	Suele aguantar mejor

Se puede explicar fácilmente	Sí, es una lista de reglas	No del todo, es un cuadro de números

En este proyecto no hay ni una sola regla escrita sobre pétalos, colores de flor ni hojas. Todo lo que el sistema sabe sobre flores lo aprendió de los ejemplos.

4. Qué es una red neuronal y qué es una red convolucional

Una red neuronal, en palabras simples

Una **red neuronal** es una forma concreta de organizar esos números ajustables. La imagen se descompone en señales pequeñas, esas señales se combinan en señales un poco más complejas, y esas a su vez en otras más complejas todavía, en capas sucesivas, hasta que al final sale una lista de 102 puntuaciones: una por especie.

Una comparación útil es una cadena de montaje de una cocina. En la primera estación se cortan las verduras. En la segunda se mezclan. En la tercera se sazona. En la cuarta se emplata. Nadie de esas estaciones "sabe" qué plato es; cada una hace su parte y pasa el resultado a la siguiente. En una red neuronal, las estaciones son las **capas** y el resultado que cada una pasa a la siguiente es un conjunto de números.

La analogía del aprendiz del apartado anterior sigue valiendo: "ajustar su forma de mirar" es literalmente retocar los números de todas esas capas, un poquito en cada ejemplo.

La red convolucional: una lupa que recorre la foto

Una **red neuronal convolucional** (en inglés *convolutional neural network*, de ahí la abreviatura CNN) es una red neuronal especializada en imágenes. Su idea central es la **convolución**, y la mejor forma de imaginarla es una lupa pequeña que se desliza por encima de la foto.

Imagina una lupa del tamaño de 3 por 3 píxeles. La apoyas en la esquina superior izquierda y miras solo ese trocito diminuto. Anotas una conclusión sencilla, por ejemplo "aquí hay un borde que va de claro a oscuro y apunta hacia abajo". Luego mueves la lupa un píxel a la derecha y repites. Y así, deslizándola por toda la imagen. Al terminar tienes un mapa que dice en qué zonas de la foto aparece ese tipo de borde.

Ahora usas muchas lupas a la vez, cada una especializada en algo distinto: una detecta bordes horizontales, otra manchas de color intenso, otra curvas suaves. Después pones una segunda ronda de lupas que ya no mira la foto original, sino esos mapas: "donde hay un borde curvo rodeado de una mancha amarilla, hay un pétalo". Y una tercera ronda que combina pétalos:

"muchos pétalos amarillos alrededor de un centro oscuro". De lo simple a lo complejo.

Esto explica por qué funciona tan bien en fotos:

- **Reconoce el rasgo sin importar dónde esté.** Si la flor está en el centro o en una esquina, la lupa la encuentra igual, porque recorre toda la imagen. Esto ahorra muchísimo trabajo: la misma lupa sirve para cualquier posición.
- **Aprende los rasgos por sí sola.** Nadie le dijo "busca bordes" ni "busca pétalos". La forma de cada lupa es uno de esos números ajustables, y el aprendiz los va afinando con los ejemplos.

El modelo concreto que usa este proyecto se llama **EfficientNet-B0**. Es una red convolucional conocida y bastante eficiente, pensada para dar buenos resultados gastando pocos recursos. No hace falta recordar el nombre: basta con saber que es una red de lupa deslizante, con muchas capas, y que ya viene con experiencia previa (lo veremos en el apartado 7).

5. Entrenar: épocas, lotes, función de pérdida y learning rate

Entrenar es el proceso de aprendizaje del apartado 3: pasarle al modelo los ejemplos etiquetados una y otra vez, medir sus errores y ajustar sus números internos. Terminado el entrenamiento, el modelo queda guardado y ya no necesita aprender más: solo responde.

Vamos con las piezas, una por una.

Época

Una **época** es una vuelta completa por todos los ejemplos de entrenamiento. Como estudiar un libro de texto de principio a fin: no basta con leerlo una vez, se repasa varias veces y en cada repaso se entiende mejor.

Este proyecto entrena **25 épocas**. Con muy pocas se queda corto de aprendizaje; con demasiadas, empieza a memorizar (el problema del apartado 6). Veinticinco es un punto razonable para este catálogo.

Lote

Si le mostraras al modelo las 1020 fotos de entrenamiento de golpe antes de corregir nada, sería lentísimo y la corrección sería torpe. En su lugar se le dan de a poquito: un **lote** (en inglés *batch*) es un grupo pequeño de imágenes que se procesan juntas y que provocan un único ajuste conjunto.

Aquí el lote es de **32 imágenes** de 224 por 224 píxeles. Piensa en un profesor que corrige 32 exámenes, saca la conclusión general de ese montón y ajusta su manera de explicar antes de pasar al siguiente montón. Cada lote produce una corrección pequeña.

Función de pérdida

La **función de pérdida** es el marcador que mide qué tan mal lo está haciendo el modelo. Le das la respuesta correcta y la respuesta que dio el modelo, y te devuelve un número: cuanto más grande, peor.

Es el marcador de un partido. Sin marcador no puedes saber si vas ganando, y tampoco sabes hacia dónde moverte. Toda la corrección de la red se guía por este número: se ajustan los números internos en la dirección que hace bajar la pérdida. Pérdida que baja de forma sostenida significa que el aprendiz está mejorando de verdad.

Learning rate

El **learning rate** (ritmo de aprendizaje) es el tamaño del paso que das al corregir. Y aquí la analogía es la de bajar una cuesta con niebla:

- Si das pasos **demasiado grandes**, pasas de largo el fondo del valle, rebotas de un lado al otro y nunca te asientas.
- Si das pasos **demasiado pequeños**, el descenso es seguro pero tardas una eternidad y puede que te quedes atascado a mitad de camino.

En este proyecto se usan dos ritmos distintos a la vez, uno más lento para la parte que ya tiene experiencia (el *backbone*, del apartado 7) y otro más rápido para la parte nueva que acaba de nacer (la cabeza que decide entre las 102 especies). La idea: lo que ya sabe bien se retoca con delicadeza, y lo que es nuevo se aprende con decisión.

Qué significa que el modelo "aprenda"

Decir que el modelo **aprende** significa, ni más ni menos, que sus números internos se van ajustando de manera que la pérdida baja y el porcentaje de aciertos sube. No hay comprensión ni conciencia de por medio: hay números que cambian en la dirección correcta, miles de millones de veces.

Hay una señal de que está aprendiendo bien y es fácil de reconocer: en el historial de entrenamiento, la pérdida de entrenamiento baja y la de validación también baja. Si la primera baja y la segunda empieza a subir, lo que está pasando es el problema del apartado siguiente.

El entrenamiento completo de este proyecto (25 épocas, lotes de 32 imágenes de 224 por 224) está pensado para tardar entre 15 y 25 minutos en una GPU T4, la tarjeta gráfica gratuita que ofrece Google Colab.

6. Sobreajuste y los tres conjuntos de datos

Sobreajuste: memorizar en vez de aprender

El **sobreajuste** (en inglés *overfitting*) es el riesgo más importante de todo el aprendizaje automático. La analogía clásica es el estudiante que se aprende las respuestas del examen de memoria, sin entender la materia.

Ese estudiante saca un 10 en el examen exacto que memorizó. Pero si le cambias los números del problema, o le pones un ejercicio con otras palabras, se queda en blanco: no aprendió el método, aprendió la lista. En un modelo pasa exactamente lo mismo. Si memoriza las fotos de entrenamiento, acierta casi todas esas fotos y falla las que nunca vio.

Es especialmente peligroso en este proyecto porque hay **muy pocos ejemplos por clase**: el conjunto oficial de entrenamiento de Flowers-102 tiene 1020 fotos para 102 especies, es decir **10 fotos por especie**. Con tan pocos ejemplos es fácil memorizar en lugar de aprender, y por eso el proyecto usa varias defensas: aumento de datos (apartado 8), parada temprana y evaluación sobre conjuntos que el modelo nunca vio.

Entrenamiento, validación y prueba

Para detectar el sobreajuste hay que examinar al modelo con fotos que no haya visto. Por eso los datos se reparten en tres grupos con papeles distintos. En este proyecto se usan los repartos **oficiales** del dataset Oxford Flowers-102, no unos inventados:

Conjunto	Para qué sirve	Fotos
Entrenamiento (train)	Son los ejemplos de los que el modelo aprende	1020
Validación (val)	Es el ensayo general: sirve para tomar decisiones durante el entrenamiento	1020
Prueba (test)	Es el examen final, se usa una sola vez y nunca se toca antes	6149

En total son **8189 fotos** repartidas en 102 especies.

La lógica de los tres conjuntos:

9. **Entrenamiento** funciona como los ejercicios que se hacen en casa: se pueden consultar las soluciones, y de hecho se consultan en cada lote.
10. **Validación** funciona como el simulacro de examen. No se usa para aprender directamente, sino para decidir cuándo parar y qué versión guardar. Es el conjunto que revela si el modelo

está empezando a memorizar: cuando el acierto en entrenamiento sube y el de validación deja de subir, esa es la señal de alarma y se detiene el entrenamiento.

11. **Prueba** funciona como el examen final que no se ha visto nunca. Si el modelo rinde bien ahí, la confianza es legítima. Si el examen final fuera el mismo que el simulacro, el resultado dejaría de ser creíble, porque se habrían tomado decisiones mirándolo.

Regla de oro: el conjunto de prueba no se toca hasta el final. Usarlo para decidir cosas durante el entrenamiento es como hacer trampa en el examen y luego presumir la nota.

7. Transfer learning: la clave de este proyecto

El **transfer learning** (aprendizaje por transferencia) es la idea central que hace que este proyecto funcione con solo 10 fotos por especie. La analogía es directa:

Imagina a alguien que ya sabe distinguir mil objetos distintos: latas, ruedas, plumas, hojas, monedas, tejidos, cielos, animales. Sabe reconocer bordes, texturas, curvas, brillos y sombras porque lleva años practicando con miles de objetos. Ahora le pides que aprenda a nombrar 102 especies de flores.

Esa persona no empieza de cero. No necesita volver a aprender qué es un borde ni qué es una textura. Solo necesita aprender **cómo se llaman** las combinaciones de rasgos que ya sabe ver: "esta combinación de pétalos y hojas, que ya reconozco, se llama especie 47". Con diez ejemplos por flor basta, porque lo difícil (ver) ya está resuelto.

Eso es exactamente lo que hace el proyecto: parte de **EfficientNet-B0 preentrenado con ImageNet**, un banco público de** 1,2 millones de fotos de 1000 categorías** ajenas a las flores. El modelo llega sabiendo ver; lo único que hay que enseñarle es la lista de nombres de flores.

Los pasos, en palabras llanas:

12. **Se toma el experto prestado.** Se usa esa red que ya sabe mirar imágenes, con todos sus números ya ajustados por 1,2 millones de fotos.
13. **Se le cambia la última pieza.** La parte final, que antes decía "esto es un perro" o "esto es una bicicleta", se sustituye por una pieza nueva que puede decir 102 nombres. En el proyecto esa pieza nueva termina en una decisión entre 102 opciones.
14. **Primero se entrena solo la pieza nueva.** Al principio el experto se queda quieto y solo la pieza nueva aprende los nombres. Esto es rápido y seguro.
15. **Después se afina todo, con cuidado.** Luego se permite que el experto también se retoque, pero con pasos muy pequeños, para afinar su manera de mirar hacia las flores sin destrozar lo

que ya sabía. Esto es lo que en el código se llama *descongelar* el backbone.

Beneficios concretos de este enfoque:

- **Menos datos.** Con 10 fotos por especie sería inviable aprender de cero, pero es suficiente para afinar un experto.
- **Menos tiempo.** El entrenamiento completo cabe en 15 a 25 minutos de GPU T4.
- **Mejor resultado.** El modelo aprovecha patrones visuales generales que le costaría muchísimo redescubrir solo con flores.

Esa es la razón de que el proyecto sea viable como trabajo de curso: no entrena un sistema desde cero, sino que aprovecha el trabajo de otros y lo especializa.

8. Aumentación de datos

La **aumentación de datos** (en inglés *data augmentation*) consiste en fabricar variantes artificiales de las fotos de entrenamiento. La analogía: si solo tienes diez fotos de una especie y quieres que el aprendiz la vea desde muchos ángulos, recortas, giras y cambias la luz de esas fotos para obtener versiones distintas sin necesidad de salir a tomar más fotos.

Aquí está la clave: son variantes que **no cambian la respuesta correcta**. Si giras la foto de una rosa, sigue siendo una rosa, y su etiqueta sigue siendo la misma. El aprendiz aprende así que la especie no depende del encuadre, del lado hacia el que mira ni del brillo, sino de la forma de la flor. Dicho de forma técnica: se le enseña a ser **invariante** a esas transformaciones.

Las tres variantes que aplica este proyecto son:

RandomResizedCrop. Recorta un trozo de la foto al azar y lo agranda hasta los 224 por 224 píxeles que el modelo espera. En este proyecto el recorte cubre entre el 60 % y el 100 % del área original, así que a veces se ve la flor casi entera y a veces solo una parte con el centro desplazado. Es como fotografiar la misma flor desde más cerca o desde otro encuadre. Sirve porque en la vida real nadie encuadra la flor exactamente igual que en el catálogo.

Volteo horizontal (*flip*). Se refleja la foto como en un espejo, de izquierda a derecha. Una flor no tiene "mano derecha": la versión reflejada sigue siendo la misma especie. Sirve porque duplica los puntos de vista sin costo alguno.

RandAugment. Es un paquete de muchas pequeñas transformaciones de aspecto, aplicadas al azar y con intensidad aleatoria: cambios de brillo, contraste, saturación, tono, giros leves, desenfoques, ruido. En lugar de elegir a mano una receta fija, se deja que la receta varíe en cada lote. Sirve porque las fotos reales tienen luces, cámaras y condiciones muy variadas, y el

modelo debe aprender a no depender de ninguna en particular.

*Detalle importante: la aumentación se aplica **solo durante el entrenamiento**. Cuando el sistema evalúa o responde a una foto nueva, la imagen se prepara de forma sencilla y estable (se redimensiona y se recorta al centro, sin trucos). Si se aumentara también al evaluar, la nota final sería un sinsentido, como dejar que el alumno cambie las preguntas del examen.*

Efecto práctico: la aumentación es una de las defensas contra el sobreajuste. Con 10 fotos por especie, ver "muchas versiones" de cada foto equivale a tener más ejemplos, y además ejemplos que le enseñan lo que no debe importar.

9. Cómo se mide si el modelo es bueno

Un modelo no es bueno porque "parece que funciona". Hay que medirlo, y para eso se usan métricas claras sobre el conjunto de prueba, que el modelo nunca vio.

Exactitud top-1

Es la métrica más directa: de cada 100 fotos, en cuántas la especie más votada por el modelo es la correcta. Si el sistema pone la respuesta correcta en primer lugar 37,8 veces de cada 100, decimos que su exactitud **top-1** es **37,8 %**.

Es como un examen de opción múltiple donde solo cuenta la primera casilla marcada.

Exactitud top-5

Aquí la regla es más generosa: se considera acierto si la especie correcta está **entre las cinco más probables** que propone el modelo, sin importar en qué puesto. Con las mismas fotos, el resultado medido de este proyecto es **67,4 %** en top-5.

Es como un examen donde puedes dar cinco respuestas y basta con que la buena esté entre ellas.

Por qué con 102 clases el top-5 tiene sentido

Con 102 especies hay que juzgar el resultado contra el azar, y el azar es durísimo:

Métrica	Azar puro con 102 clases	Este sistema (4 épocas)
Top-1	1 de 102, es decir 0,98 %	37,8 %
Top-5	5 de 102, es decir 4,9 %	67,4 %

Es decir, el sistema está muy por encima de tirar una moneda de 102 caras. Además hay una razón práctica: varias de las 102 especies son casi idénticas a simple vista, y los errores se concentran justo ahí. Para una persona que quiere saber qué flor tiene en la mano, recibir una lista corta de cinco candidatos con su probabilidad es mucho más útil que una única respuesta que puede ser la equivocada. El top-5 convierte una respuesta seca en una sugerencia útil.

Macro-F1

La exactitud tiene un problema: cuenta todas las fotos por igual, y puede esconder un desastre en algunas especies. Imagina un modelo que acierta perfecto en 90 especies y falla siempre en las otras 12; la exactitud global puede verse aceptable mientras hay 12 especies para las que el sistema es inútil.

Macro-F1 corrige eso. Se calcula una nota de calidad para cada una de las 102 especies combinando dos cosas: cuántas veces la especie correcta fue efectivamente detectada, y cuántas veces el sistema la eligió sin serlo. Después se **promedia dando el mismo peso a cada especie**, sin importar si tiene muchas o pocas fotos en el conjunto de prueba.

La analogía es el boletín de notas frente al promedio del curso. El promedio del curso puede ser bueno aunque a ti te vaya mal; el boletín te obliga a mirar asignatura por asignatura. Con macro-F1, una especie olvidada arrastra la nota hacia abajo y queda a la vista. Esto es especialmente útil en Flowers-102, donde la cantidad de fotos por especie no es uniforme.

Matriz de confusión

La **matriz de confusión** es una tabla de 102 filas por 102 columnas. Las filas son la especie verdadera y las columnas, la especie que el sistema respondió. La casilla donde la fila y la columna coinciden es un acierto; cualquier otra casilla ocupada es un error, y la tabla dice exactamente **de qué especie a qué especie** se confundió.

Es como un cuadro que anota, en cada partido, no solo quién ganó, sino a quién le ganó. Visto entero, revela patrones que un solo número esconde: por ejemplo, que las confusiones se concentran en tres parejas de especies casi gemelas, o que dos especies se confunden sistemáticamente entre ellas en las dos direcciones.

Es una herramienta de diagnóstico, no una nota: no se aprueba o se reprueba con ella, se aprende qué arreglar. Si dos especies se confunden mucho, eso orienta la siguiente mejora (conseguir más fotos de esas dos, revisar si las etiquetas del catálogo son ambiguas).

Las cifras de este proyecto

Las cifras reales que se citan aquí provienen de una corrida de prueba del sistema, y conviene leerlas con honestidad:

Métrica	Resultado	Contexto
Exactitud top-1	37,8 %	6149 fotos del conjunto oficial de prueba
Exactitud top-5	67,4 %	Las mismas fotos

Macro-F1	0,353	Promedio por especie, con clases de desempeño desigual

Esos números se obtuvieron con **solo 4 épocas** de las 25 previstas, es decir con el entrenamiento interrumpido a propósito para verificar que todo el montaje funciona. En los artículos científicos que usan este mismo catálogo y este tipo de red, el resultado reportado por **otros autores** ronda el **92 % de exactitud top-1** con el entrenamiento completo. Ese 92 % no es un resultado de este proyecto ni una promesa: es la referencia externa que sirve para juzgar si una corrida propia salió razonable. Lo que sí está medido aquí es que el sistema ya aprende algo real y muy por encima del azar incluso a medio entrenar, después de solo 4 épocas.

*Además de las métricas, el proyecto se comprueba con **78 pruebas automáticas** que verifican que cada pieza hace lo que promete (por ejemplo, que el top-5 nunca puede ser peor que el top-1, o que el umbral de confianza responde "no reconocida" cuando corresponde). Esas pruebas no miden la calidad de las predicciones: miden que la maquinaria está bien armada.*

10. GPU, Google Colab y Gradio

GPU: la potencia extra

La **GPU** (unidad de procesamiento gráfico) es un componente que nació para dibujar videojuegos y que resulta ideal para estos cálculos, porque hace muchísimas operaciones sencillas en paralelo. Un modelo convolucional es exactamente eso: millones de operaciones simples, todas parecidas, y todas independientes entre sí.

La analogía: una persona cortando verduras es lenta; veinte personas cortando verduras a la vez son rapidísimas. Multiplicar números para una red neuronal es cortar verduras. La CPU, el procesador normal, es el chef que hace todo en orden y con mucha habilidad; la GPU es el equipo grande de cocina que hace lo repetitivo en paralelo.

Efecto medible: un entrenamiento que en un portátil común puede tardar horas, en una GPU T4 tarda **15 a 25 minutos**.

Google Colab: la cocina prestada

Google Colab es un servicio gratuito de Google que presta un ordenador en la nube, con su GPU, y te deja ejecutar el proyecto desde el navegador. No hay que instalar nada en tu computadora ni comprar hardware.

Es la cocina prestada: no tienes el equipo en casa, pero puedes entrar, cocinar y llevarte el

plato. Dos consecuencias prácticas que conviene tener presentes:

- La sesión no es eterna. Si te distraes o se corta la conexión, el ordenador prestado se apaga y se pierde lo que estaba solo en memoria. Por eso los resultados se guardan en archivos (el *checkpoint* y las métricas) y por eso el notebook recomienda guardarlos también en tu Drive, que equivale a llevarte la comida a casa antes de que cierren la cocina.
- Hay que comprobar que la GPU está encendida antes de empezar; si no lo está, todo funciona igual pero muchísimo más lento.

Gradio: el mostrador

Gradio es una herramienta que crea, en unas pocas líneas, una página web sencilla con un botón para subir una foto. Es el mostrador donde el usuario final entrega su imagen y recibe la respuesta.

La analogía completa queda así: el modelo es la receta, la GPU es el equipo de cocina potente, Colab es la cocina prestada donde se cocina, y Gradio es el mostrador donde tú entregas la foto de tu flor y te devuelven una lista de cinco especies posibles con su probabilidad.

Gracias a Gradio, la persona que usa el sistema no necesita saber programar ni abrir ningún archivo de código: arrastra una foto, pulsa un botón y lee el resultado.

11. Qué es el umbral de confianza

El **umbral de confianza** es una línea de corte que el sistema aplica a su propia seguridad antes de responder. En este proyecto se llama `min_conf` y su valor es 0,25, es decir un 25 %.

Durante una predicción, el modelo reparte su opinión entre las 102 especies en forma de probabilidades, algo parecido a los porcentajes de una encuesta. Sumando todo da el 100 %. Si la especie mejor posicionada se lleva el 62 %, el sistema está bastante seguro. Si la mejor se lleva el 11 %, está prácticamente adivinando entre varias posibilidades.

La regla es: **si la probabilidad de la mejor candidata no llega a 0,25, el sistema no elige ninguna especie** y responde "no reconocida / fuera del catálogo" en lugar de forzar una respuesta.

Por qué existe

La razón es que el catálogo tiene exactamente 102 especies y el mundo tiene muchísimas más flores. Si le entregas la foto de una flor que no está en la lista, de una planta, de una persona o de una pared, el modelo **igual dará 102 probabilidades**, porque su salida siempre tiene 102 casillas y no puede decir "no sé" por sí mismo. Sin umbral, elegiría la menos mala de esas 102 opciones y te la presentaría con toda tranquilidad. Eso es peor que no responder: es

afirmar algo falso con apariencia de seguridad.

El umbral es, entonces, la única manera de que el sistema **admita que no sabe**. La analogía: un médico que no está seguro de un diagnóstico no debe elegir una enfermedad al azar; debe decir "con estos datos no puedo concluir, hace falta otra prueba". El umbral es esa prudencia incorporada.

El valor 0,25 no es mágico y no está escondido en el código: es un parámetro configurable, y subirlo o bajarlo cambia el carácter del sistema.

- **Umbral más alto** (por ejemplo 0,60): el sistema responde menos veces, pero cuando responde suele estar más seguro. Se pierden respuestas correctas junto con las dudosas.
- **Umbral más bajo** (por ejemplo 0,10): el sistema responde casi siempre, incluidos los casos en que no tiene ni idea, y por tanto aumenta el riesgo de afirmar algo falso.

Es un equilibrio entre dos errores: quedarse callado cuando sí sabía, o hablar cuando no sabía.

El valor 0,25 es una elección razonable para una demo, no una verdad universal.

Como dato útil: la propia interfaz muestra siempre las cinco candidatas con su probabilidad. Así, incluso cuando la confianza es baja, la persona que usa el sistema ve que el modelo está dudando y puede juzgar por sí misma, en lugar de recibir una respuesta seca y aparentemente segura.

Resumen en una frase

Una foto es una tabla de números; el sistema aprendió, mirando miles de ejemplos ya etiquetados y aprovechando la experiencia que otro modelo adquirió con 1,2 millones de fotos, a decir a cuál de las 102 especies del catálogo se parece más; cuando no está seguro, lo admite, y cuando sí lo está, muestra sus cinco mejores candidatas con su probabilidad para que tú decidas.

Guía práctica: cómo usarlo sin saber programar

Para qué sirve esta guía

Esta guia explica como poner en marcha el reconocedor de flores del proyecto **IA flores** usando solamente el navegador. No hace falta saber programar, ni instalar programas, ni tener un ordenador potente: todo el trabajo pesado lo hace un ordenador de Google, a traves de una herramienta gratuita que se llama **Google Colab**.

Al terminar tendras tres cosas: un modelo entrenado que reconoce 102 especies de flores, una demostracion interactiva donde subes una foto y ves el resultado, y un archivo comprimido con todos los resultados para adjuntar al informe.

Qué necesitas antes de empezar

- **Una cuenta de Google** (la misma de Gmail). Es gratis y sirve tanto para Colab como para Drive.
- **Un navegador actualizado**, por ejemplo Chrome, Edge o Firefox. Nada mas: no hay que descargar ningun programa.
- **El archivo del proyecto**, que se llama IA_flores_colab.zip. Es el paquete que te entrega el equipo que desarrollo el proyecto. Si no lo tienes, pidelo antes de empezar: es lo unico que hay que subir a mano.
- **Un rato de tiempo.** Unos 5 minutos para la prueba rapida y entre 20 y 30 minutos para la corrida completa, contando la primera descarga de imagenes.

Nota: no hace falta tarjeta de credito ni instalar nada. El acelerador gratuito de Google, la GPU, se activa con un par de clics y no se paga.

Lo que vas a hacer, en cinco líneas

16. Abrir Google Colab en el navegador.
17. Activar la GPU gratuita.
18. Subir el paquete del proyecto, que se llama IA_flores_colab.zip.
19. Ejecutar todo y esperar.
20. Mirar los resultados, probar la demostracion con una foto y descargar el archivo comprimido con todo.

Las dos corridas: primero la rápida, después la completa

El cuaderno (asi se llama el documento de trabajo de Colab) puede funcionar en dos velocidades. Conviene hacerlas en este orden y no al reves.

Corrida	Vueltas sobre los datos	Cuanto tarda	Para que sirve	Fiabilidad de las respuestas
Rapida (modo de prueba, 2 epocas)	2	Unos minutos	Comprobar que todo funciona: la descarga, el modelo, la evaluacion y la demo	Baja: las respuestas no son fiables
Completa (25 epocas)	25	15 a 25 minutos en una GPU T4	Obtener las cifras buenas y un modelo que reconoce de verdad	Alta

Por que en este orden. La corrida rapida es un ensayo barato. Si falta un archivo o algo esta mal configurado, por ejemplo la GPU sin activar, lo descubres en un par de minutos en lugar de despues de media hora. Ademas, la primera vez hay que descargar las imagenes (330 MB), y esa descarga ocurre igual durante la corrida rapida; asi, cuando lanzas la corrida completa los datos ya estan en su sitio.

Muy importante. Lo que devuelve la corrida rapida no sirve para el informe ni para sacar conclusiones. Sirve para comprobar que la maquinaria esta bien montada. Los numeros buenos salen siempre de la corrida completa.

Paso a paso numerado en Google Colab

21. **Abre Google Colab.** Busca Google Colab en tu buscador habitual y entra en la primera pagina oficial que aparezca, o escribelo directamente en la barra de direcciones si ya conoces la direccion. Inicia sesion con tu cuenta de Google. Veras una ventana con una lista de cuadernos recientes y un boton para crear uno nuevo.
22. **Crea un cuaderno nuevo.** Pulsa el boton de cuaderno nuevo. Se abre una pagina con una caja vacia arriba: esa caja es la primera celda, y a su izquierda hay un boton redondo de reproduccion. Una celda es una caja donde se escribe una orden; en este proyecto no escribiras ninguna, solo las ejecutaras.
23. **Activa la GPU.** En el menu superior entra en Entorno de ejecucion y despues en Cambiar tipo de entorno de ejecucion. En la ventana que aparece, busca Acelerador de hardware, elige T4 GPU y pulsa Guardar. Si Colab avisa de que la sesion se reiniciara, acepta. Este es el paso mas importante de toda la guia: sin GPU, la corrida completa puede tardar horas en lugar de minutos.
24. **Sube el paquete del proyecto.** En la barra lateral izquierda de Colab hay un icono con forma de carpeta. Pulsalo, pulsa despues el boton de subir y elige el archivo IA_flores_colab.zip de tu ordenador. Espera a que la barra de progreso termine: son unos pocos megabytes y sube rapido. Si el paquete no aparece, comprueba que estas en la pestana de archivos y no en la de busqueda.
25. **Abre el cuaderno que va dentro del paquete.** Dentro del paquete hay una carpeta llamada notebooks y, dentro, el cuaderno del proyecto. En el menu Archivo elige Subir cuaderno y selecciona ese archivo: se abrira en Colab con todas las celdas ya escritas. Si no encuentras esa opcion, descomprime el paquete en tu ordenador y sube el cuaderno igual que subiste el paquete.
26. **Ejecuta todo.** En el menu Entorno de ejecucion elige Ejecutar todo. Colab empezara a ejecutar las celdas una detras de otra y a mostrar los resultados debajo de cada una. Junto a la celda que esta trabajando en ese momento vera un circulo girando; cuando termina, aparece una marca de visto bueno.
27. **Espera la descarga la primera vez.** Las primeras celdas comprueban el entorno y preparan el respaldo en Drive. Despues el sistema descarga las fotografias de flores, que ocupan 330 MB. Esta descarga solo ocurre la primera vez de cada sesion; si tienes Drive conectado, las sesiones siguientes las encuentran ya preparadas.
28. **Deja que entrene.** Cuando empieza el entrenamiento van apareciendo lineas de texto, una por cada vuelta completa sobre los datos. En la corrida rapida son 2 lineas; en la completa, hasta 25. Cada linea muestra el castigo por los errores y la proporcion de aciertos. No tienes que hacer nada, solo esperar sin cerrar la pestana.
29. **Lee el resumen y prueba la demo.** Al final el cuaderno imprime un resumen con las cifras y despues abre la demostracion. En la demostracion puedes subir una foto de una flor o hacerla con la camara, y veras las cinco especies mas probables con su porcentaje.
30. **Descarga los resultados.** La ultima celda crea un archivo comprimido llamado resultados_flores.zip y lanza su descarga automaticamente. Dentro esta todo: el modelo entrenado, las cifras, las figuras y el resumen listo para el informe.

Nota: si Colab te pide permiso para acceder a tu Drive, puedes aceptarlo (es lo recomendado) o rechazarlo. Si lo rechazas, todo funciona igual, pero si la sesion se corta perderas el progreso, porque la sesion gratuita se borra al desconectarse.

Qué verás en pantalla, mensaje por mensaje

La comprobacion de la GPU

Lo primero que aparece es un resumen del entorno. Busca estas dos lineas:

```
CUDA disponible: True
GPU            : Tesla T4
```

Que ponga True, es decir verdadero, significa que la GPU esta activa y que el entrenamiento ira rapido. Si en su lugar aparece False acompanado de un aviso que empieza por AVISO: no hay GPU, has olvidado el paso 3. Vuelve a Entorno de ejecucion, Cambiar tipo de entorno de ejecucion, elige T4 GPU, guarda y ejecuta otra vez desde la primera celda.

Tambien veras un mensaje tranquilizador sobre el programa base:

```
torch y torchvision ya estan disponibles: no se tocan (regla de Colab).
```

Eso significa que Colab ya trae las piezas grandes y que no hay que instalar nada pesado. Si en su lugar aparece la linea Instalando por pip (solo lo que falta), tambien es normal: son dos o tres piezas pequenas que Colab no trae de serie y que se instalan solas en menos de un minuto.

La primera vez: se esta descargando el dataset de 330 MB

La primera vez que se ejecuta, el sistema descarga de internet las 8189 fotografias de flores del conjunto de datos. Ocupa 330 MB y tarda entre uno y tres minutos segun la velocidad de la conexion. Veras que la celda se queda trabajando un rato largo sin imprimir casi nada: es normal, esta bajando archivos. No cierres la pestana ni interrumpas la celda. Las veces siguientes reutiliza la copia ya descargada y no vuelve a bajar nada.

Si tienes Drive conectado, veras ademas un mensaje que dice que no hay copia en Drive todavia y que se descargara: significa que es la primera vez de verdad. Al final del cuaderno, esa misma copia se guardara en tu Drive para no repetir la descarga.

Las lineas del entrenamiento con la perdida y la exactitud

Durante el entrenamiento aparece una linea por cada vuelta completa sobre los datos. Tienen este aspecto:

```
[  1/  2] backbone congelado | train loss 4.1023 acc 0.1500 | val loss 3.7801 acc 0.2100 top5 0.4900 |
lr 3.000e-04 *mejor
```

Como se lee, trozo a trozo:

- El primer numero entre corchetes es la vuelta actual y el segundo el total. Si pone 1 de 2 estas en la corrida rapida; si pone 1 de 25, en la corrida completa.
- backbone congelado significa que en esa vuelta solo se entrena la parte nueva del modelo. Mas adelante la linea dira backbone entrenando, que es cuando se afina tambien la parte ya preentrenada.
- train loss es la perdida, el castigo por los errores, medido sobre las fotos de aprendizaje. Cuanto mas bajo, mejor.
- train acc es la exactitud sobre esas mismas fotos: la proporcion de aciertos entre 0 y 1. Un 0.1500 equivale a un 15 por ciento.

- val loss y val acc son lo mismo, pero medidos sobre imagenes que el modelo no ha usado para aprender. Estas son las cifras honestas: si la exactitud de aprendizaje sube mucho y la de validacion se queda atras, el modelo esta memorizando en lugar de aprender.
- top5 es la proporcion de veces que la especie correcta aparece entre las cinco propuestas.
- lr es la velocidad de aprendizaje. Veras que empieza baja, sube durante las primeras vueltas y despues baja sola poco a poco.
- La marca *mejor aparece en las vueltas que mejoran el resultado de validacion. El modelo que se guarda al final es el de la ultima vuelta marcada asi, no el de la ultima vuelta a secas.

Al terminar el entrenamiento aparece un bloque de cierre con el tiempo total y la mejor vuelta:

```
Listo en 842.3 s (14.0 min) | epocas ejecutadas: 22/25
Mejor val_acc: 0.9531 (epoca 18)
Checkpoint   : resultados/best.pt (existe: True)
```

Si el entrenamiento se para antes de la ultima vuelta, veras una linea que empieza por Early stopping. No es un error: significa que el modelo dejo de mejorar y el sistema prefirio no perder el tiempo.

El resumen final con las metricas

Despues del entrenamiento viene la evaluacion, que mide el modelo sobre las 6149 fotos de prueba que nunca ha visto. Aparece una linea como esta:

```
[evaluate] split=test imagenes=6149 top1=0.9420 top5=0.9880 macro_f1=0.9380 (61.42s)
```

Los tres numeros importantes son:

- top1: la exactitud top-1, es decir, la proporcion de fotos en las que la primera propuesta es la especie correcta.
- top5: la exactitud top-5, la proporcion de fotos en las que la especie correcta esta entre las cinco propuestas.
- macro_f1: una nota de conjunto que reparte el merito por igual entre las 102 especies, de modo que una especie dificil no quede tapada por las faciles.

Justo despues, el cuaderno imprime un bloque de texto delimitado por unas marcas de tijera, con las cifras principales, el tiempo, el numero de parametros y las confusiones mas frecuentes. Ese bloque esta pensado para copiarse tal cual en el informe: no hay que escribir ningun numero a mano. Se guarda ademas en un archivo llamado resumen_informe.md.

El enlace de la demo

Al final el cuaderno levanta la demostracion y muestra algo parecido a esto:

```
Demo lanzada.
  URL local    : 127.0.0.1:7860
```

Dentro de Colab veras ademas la propia interfaz incrustada en la pagina, lista para usarse. Si se activa la opcion de compartir, aparece ademas una linea con una direccion publica temporal de internet: ese es el enlace publico de la demo. Funciona desde el movil o desde otro ordenador, pero caduca en unas 72 horas y cualquiera que lo tenga puede usar la demo mientras siga vivo, asi que no lo dejes abierto con imagenes privadas.

El ZIP de resultados

La ultima celda crea y descarga el archivo comprimido:

```
ZIP creado: /content/resultados_flores.zip (312.4 MB)
```

El tamano exacto varia de una corrida a otra. Si la descarga automatica no arranca, abre el panel de archivos de la izquierda, busca resultados_flores.zip, pulsa los tres puntos que hay a su derecha y elige Descargar.

La corrida rápida primero y la corrida completa después

El cuaderno trae preparada una prueba rapida que se enciende y se apaga con un interruptor llamado MODO_DEBUG, que significa modo de prueba. Viene activado de fabrica y hace dos cosas: entrena solo 2 vueltas sobre un trozo pequeno del conjunto de imagenes, y guarda sus resultados en una carpeta aparte llamada resultados/debug, de modo que nunca pisa la corrida buena.

La prueba rapida tarda unos minutos y responde a una sola pregunta: funciona todo. De una vez comprueba la descarga de las imagenes, la carga del modelo, el entrenamiento, la evaluacion y la demostracion. Si algo tiene que fallar, que falle aqui, donde lo arreglas en minutos.

Para la corrida buena hay que apagar ese interruptor: busca en el cuaderno la linea que dice MODO_DEBUG = True, cambiala por MODO_DEBUG = False y vuelve a ejecutar desde esa celda hasta el final. Entonces si: 25 vueltas completas, entre 15 y 25 minutos en una GPU T4, y las cifras que si valen.

La razon de este orden es sencilla: no tiene sentido gastar media hora de GPU si falta un archivo o la GPU no esta activada. Primero se comprueba que la maquinaria gira; despues se fabrica la pieza buena.

Cómo usar la demo

Subir una foto o hacerla con la camara

La demostracion es una pagina con dos formas de darle una imagen:

- Subir una foto: arrastra el archivo de imagen desde tu ordenador hasta el recuadro, o pulsa el recuadro y selecciona el archivo. Despues pulsa el boton Enviar.
- Usarla con la camara: activa la pestana de camara, permite el acceso cuando el navegador lo pida y haz la foto. Tambien hay un boton para pegar una imagen que hayas copiado antes.

En unos segundos veras el resultado: las cinco especies mas probables, cada una con su porcentaje de confianza, y una tabla con las mismas cifras.

Como interpretar los porcentajes

El porcentaje no significa que sea seguro que la flor es esa. Es la confianza que el modelo pone en cada una de las cinco candidatas. Se lee asi:

- Si la primera propuesta tiene un porcentaje alto y muy por encima de las demas, el modelo lo tiene claro.
- Si las cinco propuestas tienen porcentajes parecidos, el modelo duda: probablemente la foto no sea de buena calidad o la especie se parezca mucho a otras.
- La lista de cinco es la parte mas util. Aunque la primera no sea la correcta, la buena suele estar entre las cinco. En las pruebas del proyecto la especie correcta aparece entre las cinco propuestas en la gran mayoria de las fotos.

El aviso de confianza baja

Si la mejor propuesta no llega al 25 por ciento de confianza, la demostracion no te obliga a elegir: te avisa de que la confianza es baja y te recuerda que la foto podria no ser ninguna de las 102 especies conocidas. El modelo solo conoce esas 102; ante una flor de otro catalogo, lo honesto es avisar en lugar de inventarse un nombre.

Nota: un porcentaje bajo no siempre significa que la foto sea mala. En la corrida rapida los porcentajes son bajos por definicion, porque el modelo apenas ha entrenado. Compara siempre lo que ves con el resultado de la corrida completa.

Cómo recoger los resultados y dónde queda el informe

Al terminar la corrida completa, todo queda en dos sitios:

- En el archivo resultados_flores.zip que se descarga a tu ordenador. Dentro estan el modelo entrenado, que se llama best.pt; las cifras, en metrics.json; el historial de cada vuelta, en history.json; las figuras, que son las curvas de entrenamiento, la matriz de confusion y el ranking de confusiones; la lista de predicciones; y el resumen llamado resumen_informe.md.
- En el propio cuaderno, que imprime ese mismo resumen listo para copiar. Ese bloque es el que sustituye los huecos que hoy dicen pendiente de la corrida en la seccion de resultados del informe del proyecto.

Para montar el informe final: descomprime el archivo descargado, abre el resumen llamado resumen_informe.md, copia el bloque de cifras en la seccion de resultados y adjunta las tres figuras. No hay que calcular nada, porque las cifras salen directamente de la corrida.

Si conectaste tu Drive, tendras ademas una copia en la carpeta IA_flores de tu unidad de Drive, y el conjunto de imagenes quedara guardado para no volver a descargar los 330 MB la proxima vez.

Problemas frecuentes y cómo resolverlos

Que ves en pantalla	Que significa	Que haces
Aparece CUDA disponible: False o un aviso de que no hay GPU	La GPU no esta activada	Entra en Entorno de ejecucion, Cambiar tipo de entorno de ejecucion, elige T4 GPU, guarda y vuelve a ejecutar desde la primera celda
El entrenamiento va lentisimo y cada linea tarda muchisimo	Estas entrenando sin GPU, solo con el procesador	Lo mismo que en el caso anterior. La corrida completa en procesador no es razonable, porque tarda horas
Colab se desconecta y se pierde el progreso	La sesion gratuita tiene limite de tiempo y se cierra tambien por inactividad	Conecta tu Drive al empezar. El cuaderno copia el modelo y las cifras a Drive justo despues de entrenar, asi que al reconectar solo hay que recuperar esa carpeta
Un aviso dice que no se pudieron	La red de Colab no pudo bajar los	El sistema sigue funcionando, pero

descargar los pesos preentrenados	pesos de ImageNet	aprende desde cero y los resultados seran peores. Vuelve a ejecutar la celda y, si se repite, empieza una sesion nueva de Colab
La demo responde no reconocida	La confianza no llego al 25 por ciento	Prueba con una foto mas clara, centrada y con la flor grande. Si estas en la corrida rapida, es lo esperado: termina la corrida completa
Tarda mucho mas de lo previsto	Suele ser la descarga inicial de 330 MB o una sesion sin GPU	Espera la primera vez, que es la mas lenta. Las siguientes usan la copia local. Si sigue lento, comprueba la GPU
Error de memoria, con el texto out of memory	El lote de imagenes es demasiado grande para la memoria disponible	Baja el tamano de lote de 32 a 16 en la configuracion del cuaderno y reinicia el entorno para liberar memoria
La ultima celda avisa de que no hay resultados que comprimir	Se ejecuto antes de entrenar	Ejecuta primero la celda de entrenamiento y despues la de empaquetado
El enlace publico de la demo no aparece	No se activo la opcion de compartir	Es opcional. La demo funciona dentro de Colab sin enlace; el enlace solo sirve para abrirla desde el movil o mostrarla a otra persona

Advertencia importante sobre la corrida rápida

En modo rapido las respuestas no son fiables. Con solo 2 vueltas sobre un trozo minusculo del conjunto de imagenes, el modelo apenas ha aprendido nada: puede fallar la especie, dar porcentajes muy bajos o responder no reconocida ante una flor que conoce perfectamente. Eso es lo esperado y no significa que algo este roto.

La corrida rapida sirve para una sola cosa: comprobar que todo funciona. Las cifras que se copian en el informe, las que se ensenan y las que se usan para decidir si el sistema es bueno salen siempre de la corrida completa de 25 epocas.

Chuleta final

- Basta con una cuenta de Google, un navegador y el archivo IA_flores_colab.zip.
- Activa la GPU antes que nada.
- Ejecuta primero en modo rapido para comprobar que todo funciona.

- Apaga el modo rapido y lanza la corrida completa de 25 epocas.
- Sube una foto a la demo y lee las cinco propuestas con su porcentaje.
- Descarga resultados_flores.zip y copia el resumen en el informe.
- Si algo falla, lee el mensaje literal que aparece en pantalla antes de cambiar cualquier cosa: casi siempre dice exactamente que paso.

Parte II. El codigo explicado

Los cimientos: configuración y utilidades

Para qué sirve este archivo (en lenguaje llano)

Imagina que vas a montar una cocina. Antes de cocinar nada necesitas dos cosas: un lugar donde dejar escritas las cantidades de cada receta (cuanta harina, cuantos grados de horno, cuanto tiempo) y un cajon con las herramientas basicas que usaras en todas las recetas (el reloj, el termometro, el bote donde guardas las etiquetas).

Este capitulo explica esos dos cimientos del proyecto, que son dos archivos pequenos:

- `src/config.py` es la **libreta de ajustes**. No cocina nada: solo guarda los numeros y las rutas que usan todos los demas programas. Aqui estan escritas cosas como cuantas veces va a repasar el sistema las fotos, de cuantas en cuantas las va a mirar, o donde va a dejar el resultado.
- `src/utils.py` es el **cajon de herramientas**. Contiene siete funciones cortas y muy usadas: fijar el azar para que todo salga igual cada vez, elegir si el trabajo se hace con el procesador normal o con la tarjeta grafica, guardar y leer el resultado en un formato de texto sencillo, contar cuantas piezas ha aprendido el modelo y ponerle un nombre legible al motor que se esta usando.

Son los dos primeros archivos que se escribieron en el proyecto, y se escribieron antes que los demas a proposito. La razon es sencilla: si cada programa guardara sus propios ajustes, el dia que quisieras cambiar "25 repasos" por "10 repasos" tendrias que buscar ese numero por todo el proyecto. Con la libreta central solo cambias un sitio. Y como los demas archivos se apoyan en estas herramientas, sus nombres y su forma de usarlas quedaron fijados desde el primer dia y nadie los ha cambiado despues.

Qué hay dentro

Los dos archivos son cortos y faciles de recorrer de arriba abajo. `src/config.py` tiene 105 lineas y `src/utils.py` tiene otras 105. En el primero encontraras cuatro constantes numericas, una clase con veinticinco ajustes, siete atajos para construir rutas de archivos y tres funciones pequenas. En el segundo, las siete herramientas que acabamos de mencionar.

Conviene aclarar ya tres palabras que van a aparecer todo el rato, porque el resto del capitulo se apoya en ellas:

- Una **variable** es una caja con nombre donde se guarda un valor. Si escribes `epochs = 25`, acabas de crear una caja llamada `epochs` que contiene el numero 25. Cuando el programa necesita saber cuantos repasos hacer, mira dentro de esa caja en lugar de repetir el numero por todas partes.
- Una **clase** es un molde para fabricar objetos. El molde dice que todo objeto fabricado con el tendra los mismos apartados, aunque cada objeto puede tener valores distintos dentro de cada apartado. Un molde de "ficha de alumno" produciria fichas con nombre, apellidos y nota; cada ficha rellena esos tres apartados con datos diferentes.
- Una **funcion** es una tarea con nombre y con instrucciones dentro. En lugar de repetir esas instrucciones cada vez que hacen falta, se escribe la funcion una vez y luego se la llama por su nombre. Por ejemplo, una funcion llamada `hacer_cafe` contendra los pasos de moler, calentar agua y colar; cada vez que la llamas, ocurren esos pasos en orden.

Un cuarto termino, **importar**, aparece en las primeras lineas de los dos archivos. Importar significa pedir prestado: cuando el archivo escribe `import json`, esta diciendo "voy a usar la herramienta llamada json que ya viene con Python", y no hay que escribirla de nuevo.

Las constantes compartidas

Este es el primer bloque real de `src/config.py`, el que va justo despues de la linea `from dataclasses import asdict, dataclass`.

```
# ---------------------------------------------------------------- constantes
IMAGE_SIZE: int = 224
MEAN: tuple[float, float, float] = (0.485, 0.456, 0.406)  # ImageNet
STD: tuple[float, float, float] = (0.229, 0.224, 0.225)  # ImageNet
FLOWERS102_CLASSES: int = 102

@dataclass
class Config:
    """Hiperparametros y rutas de una corrida de entrenamiento/evaluacion."""
```

- Una **constante** es una variable que se escribe una sola vez y no se cambia durante el trabajo. Por convenio, en Python se escriben con mayusculas para que cualquiera las reconozca de un vistazo.
- `IMAGE_SIZE = 224` es el tamano de imagen que se usa en todo el proyecto: 224 pixeles de ancho por 224 de alto. Un **pixel** es el puntito de color mas pequeno del que se compone una foto; una imagen de 224 por 224 esta formada por 50176 puntitos.
- `MEAN` y `STD` son tres numeros cada uno. Significan media y desviacion tipica, y sirven para "normalizar" el color de las fotos, algo que explicamos con detalle en el capitulo de los datos. El comentario `# ImageNet` avisa de donde salen: del gran banco de imagenes ImageNet, porque la red que usamos se entreno primero con esas fotos y hay que hablarle en el mismo idioma de colores.
- `FLOWERS102_CLASSES = 102` dice cuantas especies distintas hay en el catalogo. En lugar de escribir el numero 102 repartido por el codigo, se escribe una vez aqui; el nombre de la constante deja claro de que 102 estamos hablando.
- La anotacion `: int` y `: float` no es un adorno: es una **anotacion de tipo**, una nota que dice "aqui va un numero entero" o "aqui va un numero con decimales". Python no obliga a cumplirla, pero sirve como documentacion para quien lee y como aviso automatico si algun editor de codigo la revisa.
- `tuple[float, float, float]` significa que el valor es una tupla de tres numeros decimales. Una **tupla** es una lista corta de valores que va junta y no se modifica; el orden importa, porque los tres numeros son la media del rojo, del verde y del azul, en ese orden.

La clase de configuracion Config

Una clase de configuracion es, en palabras llanas, **una libreta con apartados fijos**. El molde se llama `Config`, y cada vez que fabricas una libreta con ese molde, la libreta sale con los mismos apartados: uno para el numero de repasos, otro para el tamano de imagen, otro para la carpeta de salida, y asi hasta veinticinco. Lo unico que puede cambiar entre una libreta y otra son los valores escritos dentro de los apartados.

Esto es util porque el proyecto entero habla de "la configuracion" y siempre se refiere a ese objeto. El programa que entrena recibe una libreta, mira dentro y actua; el que evalua recibe una libreta, mira dentro y actua. Ninguno de los dos necesita saber como se llama la carpeta de salida, porque lo lee de la libreta.

Cada apartado de la libreta es un **atributo**: la palabra tecnica para designar un apartado con nombre dentro de un objeto. En total esta clase declara veinticinco.

La pieza que convierte esto en comodidad tiene nombre propio y conviene reconocerla:

```
from dataclasses import asdict, dataclass


@dataclass
class Config:
```

- `@dataclass` es un **decorador**: una etiqueta que se pone encima de una clase para anadirle comportamiento sin escribir codigo. La arroba es lo que la marca como decorador. En este caso, decirle `@dataclass` a Python significa "genera por mi todo el trabajo repetitivo de esta clase": el que fabrica las libretas, el que guarda cada valor en su apartado y el que permite comparar dos libretas entre si.
- Ese trabajo repetitivo es el que, sin el decorador, obligaria a escribir a mano una funcion por cada apartado y a copiar veinticinco lineas de asignaciones. Con `@dataclass` basta con declarar los apartados y su valor de partida, que es exactamente lo que ves en el archivo.
- `asdict` se importa en la misma linea porque se usa al final del archivo, en el metodo `to_dict`, para convertir la libreta en un diccionario. Un **diccionario** es una coleccion de parejas "etiqueta: valor"; por ejemplo `{"epochs": 25, "batch_size": 32}`. Es el formato ideal para guardar los ajustes en un archivo de texto y poder revisar despues con que numeros se hizo una corrida.
- Los **valores por defecto** son los valores de partida, los que trae la libreta recien fabricada. Si alguien escribe `Config()` sin mas, obtiene una libreta con 25 repasos, lotes de 32 imagenes y todo lo demas tal como esta escrito en el archivo. Solo hay que escribir un valor a mano cuando se quiere algo distinto de lo normal, por ejemplo `Config(epochs=4)` para una corrida corta de comprobacion.
- El metodo `to_dict()` es la costumbre de poder volcar la libreta entera de golpe a un diccionario, para guardarla en un archivo. Aqui es una sola linea: `return asdict(self)`. La palabra `self` es la forma que tiene una clase de referirse a si misma, es decir, a la libreta concreta que se esta usando en ese momento.

Los parametros reales, uno a uno

Estos son los veinticinco apartados tal como estan escritos en `src/config.py`, con su valor por defecto y su significado en lenguaje corriente. Un **parametro** es cada uno de esos ajustes que se puede tocar para cambiar el comportamiento del sistema.

Parametro	Valor por defecto	Que significa
`data_root`	`"data"`	Carpeta donde se guardan y se buscan las fotos.
`out_dir`	`"resultados"`	Carpeta donde se dejan los resultados del trabajo.
`num_classes`	`102`	Cuantas especies distintas puede llegar a distinguir.
`img_size`	`224`	Lado de la imagen en pixeles, ya recortada y preparada.
`backbone`	`"efficientnet_b0"`	Nombre de la red preentrenada que sirve de base.
`dropout`	`0.3`	Porcentaje de conexiones que se

		apagan al azar durante el entrenamiento.
pretrained	True	Si la red base empieza ya sabiendo de ImageNet o desde cero.
epochs	25	Cuantas veces se repasa el conjunto de entrenamiento completo.
batch_size	32	Cuantas fotos se miran juntas antes de corregir.
lr	3e-4	Ritmo de aprendizaje de la parte final de la red.
backbone_lr	3e-5	Ritmo de aprendizaje de la red base cuando se descongela.
weight_decay	1e-4	Cuanto se castiga que la red se aprenda los datos de memoria.
label_smoothing	0.1	Cuanto se rebaja la seguridad de las etiquetas durante el aprendizaje.
warmup_epochs	2	Cuantas epocas iniciales el ritmo de aprendizaje sube poco a poco.
freeze_epochs	3	Cuantas epocas iniciales la red base queda congelada.
early_stop_patience	6	Cuantas epocas sin mejorar se aguantan antes de parar.
amp	True	Si se usa calculo en precision mixta para ir mas rapido.
topk	5	Cuantas especies propone el sistema en cada respuesta.
min_conf	0.25	Por debajo de esta seguridad, responde "no reconocida".
seed	42	Numero que fija el azar para que todo salga igual cada vez.
num_workers	2	Cuantos ayudantes preparan fotos a la vez.
device	"auto"	Con que motor calcular: el que haya disponible, GPU o CPU.
debug	False	Modo de prueba rapida con muy

		pocas fotos.
`debug_per_class`	`4`	Cuantas fotos por especie se usan en ese modo de prueba.

Algunos de estos parametros merecen una explicacion mas larga, porque su nombre en ingles no ayuda nada a quien empieza.

`epochs`, los repasos. Una **epoca** es una vuelta completa por todas las fotos de entrenamiento. El sistema no aprende mirando una foto una sola vez; necesita repasar el conjunto muchas veces, corrigiendo su criterio en cada vuelta. Con 25 epocas, las 1020 fotos de entrenamiento se miran 25 veces, en orden distinto cada vez.

`batch_size`, el lote. Un **lote** es el grupo de fotos que se mira junto antes de corregir el criterio. El sistema no ajusta sus numeros despues de cada foto, sino despues de cada lote de 32. Es como estudiar: no cambias de metodo tras leer una sola frase, sino tras leer un parrafo. Un lote grande da una correccion mas estable pero exige mas memoria; uno pequeno es mas nervioso pero cabe en cualquier ordenador.

`lr` y `backbone_lr`, los ritmos. La abreviatura `lr` viene de *learning rate*, el ritmo al que el sistema corrige sus numeros. Un ritmo alto aprende rapido y puede pasarse de largo; uno bajo aprende fino pero tarda muchisimo. Aqui hay dos ritmos porque la red tiene dos partes: una parte final nueva, que empieza sin saber nada y necesita aprender deprisa (`3e-4`), y una parte base ya entrenada, que solo hay que retocar con suavidad (`3e-5`, diez veces mas lenta). La notacion `3e-4` es una forma compacta de escribir 0,0003.

`dropout`, apagar conexiones. Durante el entrenamiento, el sistema apaga al azar el 30 por ciento de las conexiones internas en cada lote. Suena a sabotearse a si mismo, y en parte lo es: es una tecnica deliberada para que la red no dependa de una sola conexion ni memorice las fotos de entrenamiento. Al evaluar, no se apaga nada.

`pretrained`, partir de algo ya sabido. Una red de este tipo tiene millones de numeros internos que hay que ajustar. Entrenarla desde cero con solo 1020 fotos de entrenamiento seria un desproposito. Con `pretrained = True` se parte de una red que ya fue entrenada con el enorme banco de imagenes ImageNet, y solo se le ensena el detalle nuevo: distinguir especies de flores.

`label_smoothing`, no jurar en falso. Cuando el sistema acierta una foto, la correccion normal le dice "esta es la respuesta correcta al cien por cien". Suavizar las etiquetas con `0.1` rebaja esa certeza a un 90 por ciento y reparte el resto. Asi se le ensena a no ser tan rotundo, lo que mejora su comportamiento con fotos nuevas y raras.

`warmup_epochs` y `freeze_epochs`, empezar con cuidado. Las dos primeras epocas el ritmo de aprendizaje sube poco a poco en lugar de arrancar de golpe, para no desbaratar los numeros heredados de ImageNet en el primer lote. Y las tres primeras epocas la red base queda **congelada**, es decir, sus numeros no se tocan y solo aprende la parte final nueva. Es prudente: primero se deja que la parte nueva se coloque, y despues se afina todo junto.

`early_stop_patience`, saber parar. Si pasan seis epocas seguidas sin que el resultado sobre las fotos de validacion mejore, el entrenamiento se detiene solo y se queda con la mejor version conseguida. La palabra *patience* significa paciencia, y aqui esta bien elegida: es el numero de oportunidades que se le dan antes de rendirse.

`amp`, calcular con menos decimales. La abreviatura viene de *automatic mixed precision*. Consiste en hacer parte de las cuentas con numeros de menos decimales, lo que ocupa menos memoria y va mas rapido, sin perder calidad apreciable. Se ignora automaticamente cuando no hay tarjeta grafica.

`topk` y `min_conf`, la respuesta que ve la persona. `topk = 5` significa que el sistema propone sus cinco mejores candidatas en lugar de una sola. `min_conf` es el minimo de seguridad exigido para contestar con una especie: si la mejor candidata no llega al 25 por ciento de confianza, el sistema responde "no reconocida", que es mucho mas honesto que inventarse un nombre.

`seed`, el numero que fija el azar. El sistema usa azar en varios sitios: al desordenar las fotos, al recortarlas y al apagar conexiones. `seed = 42` es la semilla que hace que ese azar sea siempre el mismo, y se explica con calma mas abajo, junto a la herramienta `set_seed`.

`num_workers`, los ayudantes. Preparar cada foto es trabajo pesado: hay que abrir el archivo, recortarlo y convertirlo. Con `num_workers = 2`, dos procesos ayudantes van preparando lotes mientras el principal calcula. En Windows esos ayudantes pueden dar problemas, y por eso existe la posibilidad de bajarlo a cero.

`device`, donde se calcula. El valor `"auto"` significa "usa la tarjeta grafica si la hay, y si no, el procesador". La explicacion completa esta en la herramienta `resolve_device`.

El modo debug y por que cambia cosas automaticamente

`debug` es el parametro con el nombre mas informal y, sin embargo, el mas util del dia a dia. La palabra viene del mundo de la informatica y significa "modo de depuracion": una version reducida del trabajo, pensada para comprobar que todo funciona sin esperar a que termine de verdad.

En modo debug se usan 408 fotos en lugar de las 8189 completas, como se detalla en el capitulo de los datos. Pero ademas, al fabricar la libreta, el propio archivo cambia otros ajustes por su cuenta. Esto ocurre en un metodo especial de las clases de datos, llamado `__post_init__`, cuyos cuatro guiones bajos a cada lado son la convencion de Python para los metodos internos.

```
def __post_init__(self) -> None:
    if self.debug:
        self.epochs = min(self.epochs, 2)
        self.freeze_epochs = 0
        self.warmup_epochs = 0
        self.amp = False
        self.num_workers = 0
    self.data_root = str(self.data_root)
    self.out_dir = str(self.out_dir)
```

- `def __post_init__(self)` es el metodo que la clase ejecuta automaticamente **justo despues** de rellenar los apartados. Es el sitio natural para retocar la libreta segun lo que se acaba de escribir en ella.
- `if self.debug:` significa "solo si se ha pedido el modo de prueba". En una corrida normal, todo lo que viene debajo se salta y nada cambia.
- `self.epochs = min(self.epochs, 2)` deja los repasos en dos como mucho. `min` elige el menor de los dos valores, asi que si alguien pide 25 repasos en modo debug, se queda con 2; si pide 1, se queda con 1. La idea es que la prueba dure minutos, no horas.
- `self.freeze_epochs = 0` y `self.warmup_epochs = 0` desactivan el cuidado inicial. Tiene sentido: con solo dos epocas, dedicar tres a congelar la red base o dos a calentar el ritmo no dejaria tiempo para aprender nada. Se quiere comprobar que el mecanismo gira, no obtener un buen resultado.

- `self.amp = False` desactiva el calculo de menos decimales. Es una simplificacion para que la prueba no dependa de capacidades especiales del motor de calculo.
- `self.num_workers = 0` deja de usar procesos ayudantes. Esto es puro pragmatismo: en una prueba de un minuto, arrancar y coordinar dos procesos auxiliares cuesta mas de lo que aporta, y ademas es la causa tipica de errores en Windows.
- Las dos ultimas lineas, `self.data_root = str(self.data_root)` y `self.out_dir = str(self.out_dir)`, envuelven esos valores en texto. Sirve para que la libreta funcione igual si alguien escribe la ruta como texto `"data"` o como objeto de tipo ruta. Convertirlo siempre a texto evita una familia entera de errores tontos mas adelante.

Nota: que el modo debug cambie los ajustes por su cuenta es comodo, pero tambien es la clase de magia que conviene conocer. Si algun dia una corrida se comporta de forma rara y el resultado no cuadra con lo pedido, lo primero que hay que mirar es si el modo debug quedo encendido.

Las propiedades de rutas

El final de `src/config.py` resuelve una molestia cotidiana: decidir donde va cada archivo de salida. En lugar de repartir rutas escritas a mano por todos los programas, la libreta las construye a partir de `out_dir`.

```
    # ------------------------------------------------------------ derivados
    @property
    def checkpoint_path(self) -> Path:
        return Path(self.out_dir) / "best.pt"

    @property
    def history_path(self) -> Path:
        return Path(self.out_dir) / "history.json"

    @property
    def metrics_path(self) -> Path:
        return Path(self.out_dir) / "metrics.json"

    @property
    def config_path(self) -> Path:
        return Path(self.out_dir) / "config_usada.json"

    def figures_dir(self) -> Path:
        return Path(self.out_dir) / "figuras"

    def predictions_dir(self) -> Path:
        return Path(self.out_dir) / "predicciones"

    def ensure_dirs(self) -> None:
        """Crea los directorios de salida. Idempotente."""
        Path(self.out_dir).mkdir(parents=True, exist_ok=True)
        self.figures_dir().mkdir(parents=True, exist_ok=True)
        self.predictions_dir().mkdir(parents=True, exist_ok=True)
        Path(self.data_root).mkdir(parents=True, exist_ok=True)

    def to_dict(self) -> dict:
        return asdict(self)
```

- Una **propiedad** es un apartado que no guarda un valor fijo, sino que lo calcula en el momento de pedirlo. Se reconoce por el decorador `@property` encima. La diferencia practica es esta: `cfg.out_dir` es un valor guardado, mientras que `cfg.checkpoint_path` se calcula al vuelo. Por eso las propiedades se usan sin parentesis, como si fueran un dato mas, y los metodos se usan con parentesis.

- `Path(self.out_dir) / "best.pt"` construye la ruta del archivo completando la carpeta de salida con el nombre del archivo. La barra inclinada entre las dos partes no es una division: en el modulo `Path` significa "y dentro de esta carpeta". El resultado es algo como `resultados\best.pt`. Se usa `Path` en lugar de pegar textos a mano porque `Path` sabe adaptarse al sistema operativo y coloca la barra correcta en Windows y en Linux.
- `checkpoint_path` apunta a `best.pt`, el archivo que guarda la mejor version del modelo conseguida durante el entrenamiento. La palabra inglesa *checkpoint* se usa mucho en este terreno y significa "punto de control": una copia del estado del trabajo que permite retomarlo o usarlo despues. Es, con diferencia, el archivo mas importante de todos, porque es lo que convierte el esfuerzo de entrenar en algo que luego se puede usar para reconocer flores.
- `history_path` apunta a `history.json`, el cuaderno de bitacora del entrenamiento. Guarda, epoca por epoca, como iba el sistema: cuanto se equivocaba, cuanto acertaba y con que ritmo estaba aprendiendo. Sirve para dibujar las curvas de aprendizaje y para justificar los resultados.
- `metrics_path` apunta a `metrics.json`, el resumen de la evaluacion final: porcentaje de acierto exacto, acierto entre las cinco primeras propuestas, y el detalle por especie. Es de donde salen las cifras que aparecen en los informes.
- `config_path` apunta a `config_usada.json`, la copia de la libreta tal como estaba en esa corrida. Este archivo es el que evita la pregunta incomoda "y esto, con que ajustes se hizo?". Abriendolo se sabe todo.
- `figures_dir()` y `predictions_dir()` son metodos, no propiedades, y por eso se llaman con parentesis. Devuelven dos carpetas: una para los graficos y otra para las predicciones volcadas a archivo. Son metodos y no propiedades por coherencia con `ensure_dirs`, ya que las tres cosas devuelven o crean carpetas y se leen mejor juntas.
- `ensure_dirs()` crea de una vez todas las carpetas de salida. El comentario final de su documentacion dice `Idempotente`, una palabra que conviene conocer: una operacion idempotente se puede repetir tantas veces como se quiera sin que el resultado empeore ni de error. Aqui, pedir "crea las carpetas" cuando ya existen no rompe nada.
- `Path(self.out_dir).mkdir(parents=True, exist_ok=True)` es la receta que hace posible esa repeticion tranquila. `parents=True` significa "crea tambien las carpetas intermedias que falten", y `exist_ok=True` significa "si ya existe, no te quejes y sigue".
- `to_dict()` cierra el archivo devolviendo la libreta entera convertida en diccionario, lista para guardarse en `config_usada.json`. Es la unica razon por la que se importaba `asdict` al principio.

set_seed: el dado de los numeros aleatorios

`src/utils.py` empieza con una funcion corta que se llama al arrancar cualquier trabajo y que existe por una razon muy concreta: que un experimento se pueda repetir.

```
  def set_seed(seed: int = 42) -> None:
      """Fija la semilla en random, numpy y torch (CPU y CUDA) para reproducibilidad."""
      import numpy as np

      random.seed(seed)
      np.random.seed(seed)
      os.environ["PYTHONHASHSEED"] = str(seed)
      try:
          import torch

          torch.manual_seed(seed)
          torch.cuda.manual_seed_all(seed)
      except ImportError:  # pragma: no cover - entorno sin torch
```

```
        pass
```

- La palabra **semilla** traduce el ingles *seed* y describe muy bien lo que hace. En un generador de numeros al azar, la semilla es el punto de partida: si le das siempre la misma, obtienes siempre la misma secuencia de numeros "al azar". Como los dados cargados, pero a proposito y de forma util.
- `def set_seed(seed: int = 42) -> None:` declara la funcion. Recibe un numero entero y, si no se le da ninguno, usa 42. El `-> None` avisa de que la funcion no devuelve nada: su tarea es dejar el entorno preparado, no calcular un valor.
- `random.seed(seed)` fija el azar del modulo `random`, el que usa Python de serie. Con esto queda fijado, por ejemplo, el desorden de las fotos.
- `np.random.seed(seed)` fija el azar de `numpy`, la libreria de calculo con listas de numeros. Hace falta repetirlo porque es un generador distinto del anterior: fijar uno no fija el otro.
- `os.environ["PYTHONHASHSEED"] = str(seed)` fija el azar interno que Python usa para ordenar ciertas colecciones. Aunque parezca un detalle minucioso, deja la ejecucion completamente determinista.
- `torch.manual_seed(seed)` y `torch.cuda.manual_seed_all(seed)` fijan el azar de PyTorch, la libreria de inteligencia artificial. Se fijan los dos porque los calculos hechos en el procesador y los hechos en la tarjeta grafica pueden usar generadores separados.
- El bloque `try` y `except ImportError` rodea a PyTorch y termina con `pass`, que significa "no hagas nada". Es una precaucion: si PyTorch no estuviera instalado en ese ordenador, la funcion fijaria el azar de lo demas y seguiria sin romperse.
- `str(seed)` convierte el numero en texto, porque las variables de entorno son siempre texto.

Por que importa la reproducibilidad. Dos razones, y las dos son practicas. La primera es la **confianza**: si dos ejecuciones con los mismos ajustes dan resultados distintos, es imposible saber si una mejora se debe a un cambio que hiciste o simplemente a la casualidad. Fijando la semilla, la unica diferencia entre dos corridas es lo que tu cambiaste a proposito. La segunda es la **comparacion honesta**: cuando en el capitulo de resultados se dice "esto mejoro", el lector puede repetir la corrida y obtener las mismas cifras. Sin semilla, cada cual obtendria las suyas y la afirmacion quedaria en el aire.

resolve_device: elegir el motor sin escribirlo a mano

La segunda herramienta decide donde se hacen las cuentas. Es la que evita que alguien escriba "cuda" en mitad del codigo y rompa el programa en cualquier ordenador que no tenga la tarjeta adecuada.

```
def resolve_device(spec: str = "auto"):
    """Devuelve un torch.device a partir de 'auto' | 'cuda' | 'cpu'.

    'auto' usa CUDA si esta disponible y si no cae a CPU (Colab sin GPU incluido).
    Si se pide 'cuda' y no hay GPU, lanza RuntimeError con un mensaje claro.
    """
    import torch

    spec = (spec or "auto").lower()
    if spec == "auto":
        return torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if spec.startswith("cuda") and not torch.cuda.is_available():
        raise RuntimeError(
            "Se pidio device='cuda' pero torch.cuda.is_available() es False. "
            "En Colab: Entorno de ejecucion > Cambiar tipo de entorno > GPU T4."
        )
    return torch.device(spec)
```

- Aqui aparece el termino central: el **dispositivo**, en ingles *device*. Es el motor de calculo que hace las multiplicaciones. Hay dos de uso comun: la **CPU**, el procesador normal del ordenador, que sirve para todo pero es lento en este tipo de cuentas, y la **GPU**, la tarjeta grafica, que hace muchisimas operaciones parecidas a la vez y por eso acelera enormemente el entrenamiento.
- La funcion recibe un texto, que puede ser `"auto"`, `"cuda"` o `"cpu"`. `cuda` es el nombre de la tecnologia de tarjetas graficas de NVIDIA. Devuelve un objeto del tipo `torch.device`, que es la etiqueta que PyTorch entiende para saber donde colocar cada calculo.
- `spec = (spec or "auto").lower()` limpia lo recibido antes de decidir. `spec or "auto"` significa "usa lo que te dieron, y si te dieron algo vacio, usa auto"; es una red de seguridad contra valores en blanco. El `.lower()` pasa todo a minusculas para que `"CUDA"` y `"cuda"` se traten igual.
- `torch.cuda.is_available()` es la pregunta clave: "tienes una tarjeta grafica utilizable?". Devuelve verdadero o falso. Con `"auto"`, la linea `torch.device("cuda" if ... else "cpu")` elige la tarjeta si la hay y el procesador si no. Ese `if` metido en medio de la linea es una forma compacta de escribir una eleccion entre dos valores.
- El bloque siguiente es el interesante: si alguien pidio `"cuda"` a proposito y no hay tarjeta, la funcion se niega a continuar y lanza un error con un mensaje que explica como conseguir una GPU en Colab. Negarse es mejor que la alternativa: si continuara en silencio, la corrida tardaria horas sin que nadie entendiera por que.
- La ultima linea, `return torch.device(spec)`, cubre el caso de `"cpu"` explicito. Aqui no hay comprobacion ninguna, y es correcto: cualquier ordenador tiene procesador.

Por que nunca se escribe "cuda" a mano. Porque quien escribe `cuda` a mano esta dando por hecho que hay tarjeta grafica, y ese supuesto solo se cumple en algunas maquinas. El proyecto tiene que funcionar en tres sitios muy distintos: un portatil normal sin tarjeta dedicada, un ordenador con tarjeta, y la nube gratuita de Google Colab, donde la tarjeta puede estar disponible o no segun la sesion. Escribir `cuda` fijo haria que el mismo codigo funcionara en un sitio y fallara en los otros. Pidiendo `"auto"` a esta funcion, el programa se adapta solo. Ademas, el mensaje de error esta escrito para un humano: no dice "CUDA no disponible" y se queda tan ancho, sino que indica el menu exacto donde activar la tarjeta.

describe_device: ponerle nombre al motor

```
def describe_device(device) -> str:
    """Texto legible para logs: 'cuda (NVIDIA T4)' o 'cpu'."""
    import torch

    if getattr(device, "type", str(device)) == "cuda":
        idx = getattr(device, "index", None) or 0
        return f"cuda ({torch.cuda.get_device_name(idx)})"
    return "cpu"
```

- Esta funcion recibe un dispositivo y devuelve un texto corto para escribir en el registro de la corrida, por ejemplo `cuda (NVIDIA T4)` o simplemente `cpu`. Un **registro** o *log* es el listado de mensajes que un programa va imprimiendo mientras trabaja, para que luego se pueda reconstruir lo que paso.
- `getattr(device, "type", str(device))` intenta leer el apartado `type` del dispositivo. `getattr` es una forma defensiva de preguntar "tienes este apartado?"; si no lo tiene, devuelve el tercer valor en lugar de fallar. Es una muestra del estilo del proyecto: preguntar antes de dar por hecho.
- `idx = getattr(device, "index", None) or 0` hace lo mismo con el numero de tarjeta, por si un ordenador tuviera varias. El `or 0` cubre el caso de que no venga ningun numero y asume la primera.

- `torch.cuda.get_device_name(idx)` pregunta el nombre comercial de la tarjeta, lo que permite que el registro diga si la corrida fue en una T4 de Google o en la tarjeta de casa.
- La ultima linea devuelve `"cpu"` para cualquier otra cosa. Esta funcion nunca acompana a un error: solo informa, y por eso su via de escape es tan simple.

Su utilidad es pequena pero real: cuando dentro de un mes alguien lea el historial de una corrida, sabra si aquel resultado se obtuvo con tarjeta grafica o con procesador. Es informacion que, en una comparacion de tiempos, cambia por completo la interpretacion.

save_json y load_json: guardar y leer resultados

```
def save_json(path: str | Path, obj: Any) -> None:
    """Guarda JSON serializable (convierte Path y tipos numpy/torch basicos)."""
    p = Path(path)
    ensure_dir(p.parent)

    def _default(o: Any):
        if isinstance(o, Path):
            return str(o)
        if hasattr(o, "item"):  # escalares numpy / torch
            return o.item()
        if isinstance(o, set):
            return sorted(o)
        raise TypeError(f"Objeto no serializable: {type(o)!r}")

    with open(p, "w", encoding="utf-8") as fh:
        json.dump(obj, fh, indent=2, ensure_ascii=False, default=_default)


def load_json(path: str | Path) -> Any:
    with open(Path(path), "r", encoding="utf-8") as fh:
        return json.load(fh)
```

- **JSON** es un formato de texto para guardar datos ordenados. Se lee a simple vista y lo entienden practicamente todos los programas, lo que lo hace ideal para los resultados: el historial, las metricas y los ajustes de una corrida quedan guardados en archivos de texto que se pueden abrir con el bloc de notas.
- `save_json` recibe una ruta y un objeto, y lo escribe en disco. `load_json` hace lo contrario: lee el archivo y devuelve el objeto reconstruido.
- `p = Path(path)` y `ensure_dir(p.parent)` preparan el terreno antes de escribir. `p.parent` es la carpeta que contiene al archivo, y `ensure_dir` se encarga de que exista. Asi es imposible el error clasico de intentar escribir en una carpeta que aun no se ha creado.
- La anotacion `str | Path` significa "acepta texto o un objeto de tipo ruta", y `Any` significa "cualquier objeto". La barra vertical se lee "o".
- La funcion interna `_default` es la que resuelve los problemas de traduccion. El formato JSON solo admite numeros sencillos, texto, listas y diccionarios; cualquier otra cosa hay que traducirla antes de escribirla, y esa es exactamente su tarea.
- El primer caso de esa traduccion, `if isinstance(o, Path): return str(o)`, convierte una ruta en texto. `isinstance` pregunta "es este objeto del tipo tal?".
- El segundo, `if hasattr(o, "item")`, cubre los numeros que vienen de PyTorch o de numpy. Estos numeros son objetos con minusculas diferencias respecto a un numero corriente de Python, y el metodo `item()` los devuelve como numero normal. Sin esta linea, guardar metricas fallaria con un error incomprensible, porque el formato JSON no sabe que hacer con un numero de PyTorch.

- El tercero, `if isinstance(o, set): return sorted(o)`, convierte un conjunto en una lista ordenada. Un **conjunto** es una coleccion sin orden y sin repetidos; como el formato JSON no tiene ese concepto, se guarda como lista. Ordenarlo hace que el archivo sea estable: dos corridas identicas producen exactamente el mismo texto.
- La ultima linea, `raise TypeError(...)`, es el caso de "no se como traducir esto". `raise` significa lanzar un error a proposito, y el mensaje incluye el tipo de objeto para que quien lo vea sepa que pasa. Fallar aqui y ahora es mucho mejor que escribir un archivo silenciosamente incompleto.
- `open(p, "w", encoding="utf-8")` abre el archivo para escritura, con la `w` de *write*. El detalle `encoding="utf-8"` merece mencion: es la tabla de caracteres que permite guardar acentos y enes correctamente. Sin ella, en Windows la palabra "validacion" podria guardarse con caracteres rotos. `fh` es simplemente el nombre que se le da al archivo abierto, abreviatura de *file handle*.
- `json.dump(..., indent=2)` escribe con dos espacios de sangrado en cada nivel, pensando en la persona que abra el archivo: asi se ve la estructura en lugar de una linea larguisima. `ensure_ascii=False` pide que los acentos se guarden tal cual y no como codigos numericos.
- `load_json` no necesita nada de todo eso porque leer es mas facil: abre el archivo en modo lectura, con la `r` de *read*, y devuelve su contenido. El `with` que aparece en las dos funciones garantiza que el archivo se cierre solo al terminar, incluso si algo falla por el camino.

count_params: contar lo que el modelo aprendio

```
def count_params(model) -> int:
    """Numero de parametros entrenables."""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

- Un **parametro** de un modelo es cada uno de los numeros internos que se ajustan durante el entrenamiento. Una red como la que usa este proyecto tiene millones, y ese numero es la mejor medida de su tamano.
- `model.parameters()` entrega la lista de todos los parametros del modelo. `sum(...)` los recorre y suma una cantidad por cada uno.
- `p.numel()` cuenta cuantos numeros hay dentro de ese parametro concreto. El nombre viene de *number of elements*, numero de elementos. Un parametro no tiene por que ser un solo numero: casi siempre es una tabla de muchos, y `numel` cuenta cuantos.
- El filtro `if p.requires_grad` es la parte importante: deja fuera los parametros **congelados**. `requires_grad` significa "necesita ajuste"; cuando vale falso, ese parametro esta bloqueado y no se toca. Contar solo los ajustables responde a la pregunta que de verdad interesa, que es cuanto tiene que aprender el sistema, no cuanto ocupa.
- La sintaxis `sum(... for p in ... if ...)` se llama comprension o generador: es una forma compacta de decir "suma tal cosa para cada elemento que cumpla tal condicion", sin escribir un bucle de tres lineas.

El resultado de esta funcion acaba en el historial de `train.py` y en los informes, donde sirve para explicar por que unos modelos tardan mas que otros.

ensure_dir: crear carpetas sin sobresaltos

```
def ensure_dir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p
```

- Convierte lo recibido en una ruta, crea la carpeta y devuelve la ruta ya construida. Es la version corta y reutilizable de lo que `ensure_dirs` hace dentro de la libreta de configuracion.
- `mkdir(parents=True, exist_ok=True)` es la llamada que lo hace todo: `mkdir` viene de *make directory*, crear carpeta; `parents=True` crea tambien las carpetas intermedias que falten, y `exist_ok=True` permite repetir la llamada sin que se queje si ya existe.
- Devuelve la ruta, y ese detalle tan pequeno es comodo: permite escribir `p = ensure_dir("resultados/figuras")` y seguir usando `p` sabiendo que la carpeta existe. La funcion se apoya en `save_json`, que la llama para asegurarse de que la carpeta del archivo esta lista antes de escribir.

Por qué se hizo asi

Este capitulo recoge las decisiones de diseno de los dos archivos y el motivo de cada una.

Por que centralizar los ajustes en lugar de repartirlos. Todos los numeros importantes viven en un solo archivo. La ventaja se nota al comparar: cambiar los repasos de 25 a 10 es editar una linea, y la libreta entera sigue siendo coherente. La segunda ventaja, menos evidente, es que `config_usada.json` guarda la libreta completa de cada corrida. Como consecuencia, un resultado siempre se puede reproducir, porque los ajustes con los que se obtuvo quedaron archivados junto a el.

Por que `config.py` no importa PyTorch. La cabecera del archivo lo declara como contrato: el modulo de configuracion no debe importar PyTorch y tiene que poder cargarse en cualquier entorno. El motivo es la comodidad y la robustez: asi una herramienta que solo quiera leer los ajustes no arrastra una libreria enorme, y un ordenador donde PyTorch este mal instalado todavia puede consultar la configuracion. Es la misma idea que aparece en `set_seed`, donde PyTorch se importa dentro de un `try`.

Por que los ajustes ambiguos tienen nombres explicitos. `lr` y `backbone_lr` en lugar de un solo ritmo; `debug_per_class` en lugar de un numero suelto de imagenes. Cada nombre dice a que parte del sistema afecta. Cuando dentro de seis meses alguien lea una linea de configuracion suelta, no tendra que adivinar que significa.

Por que el modo debug modifica ajustes por su cuenta. Porque la alternativa, pedirle a quien escribe el codigo que recuerde cambiar cinco parametros cada vez que quiere una prueba rapida, se olvida siempre. Un modo de prueba que no fuera coherente consigo mismo daria resultados enganosos: con dos epocas y tres de congelacion, por ejemplo, la red base no llegaria a descongelarse nunca y la prueba no comprobaria lo que debe. Que el propio archivo ajuste esos valores garantiza que cualquier prueba rapida sea internamente coherente. El precio es esa pizca de magia que ya avisamos en su seccion.

Por que las rutas de salida son propiedades calculadas y no valores guardados. Si `checkpoint_path` fuera un valor guardado, alguien podria cambiar `out_dir` y olvidarse de actualizar la ruta del archivo, con el resultado de escribir el modelo bueno en la carpeta antigua. Calculandola al vuelo a partir de `out_dir`, ese desajuste es imposible: hay una sola fuente de verdad y todo lo demas se deriva de ella.

Por que `set_seed` fija el azar en tres librerias distintas. Porque cada una tiene su propio generador de numeros al azar y fijar uno no fija los otros. Si solo se fijara el de PyTorch, quedaria suelto el desorden de las fotos, que lo decide otra libreria; y el resultado cambiaria de una corrida a otra. Para que la reproducibilidad sea real hay que cubrir todos los sitios donde se usa el azar.

Por que nunca se escribe "cuda" a mano. Porque el mismo codigo tiene que funcionar en un portatil sin tarjeta, en un ordenador con tarjeta y en Colab, donde la tarjeta aparece o no segun la sesion. Concentrar la

decision en `resolve_device` consigue que ningun otro archivo tenga que saber nada al respecto, y que un cambio de entorno no obligue a tocar veinte sitios. Esta regla esta escrita en el contrato del proyecto y se cumple en todos los modulos.

Por que `save_json` traduce objetos en lugar de exigir numeros corrientes. Porque las cifras que produce PyTorch y numpy no son numeros corrientes, y obligar a cada programa a convertirlos antes de guardarlos repartiria la misma conversion por todo el proyecto. Haciendola una sola vez en `save_json`, se elimina una familia entera de errores dificiles de diagnosticar. Y `load_json` existe para que cualquier programa, incluidas las pruebas automaticas, pueda leer un resultado y comprobarlo.

Por que se distinguen parametros entrenables y congelados en `count_params`. Porque el numero que interesa no es el tamano total del archivo, sino cuanto tiene que aprender el sistema. En este proyecto las primeras epocas transcurren con la red base congelada, y ese cambio se refleja de inmediato en el recuento. Contar solo los ajustables es la medida honesta del esfuerzo de entrenamiento.

Por que las funciones de utilidad son deliberadamente cortas. Hay siete herramientas pequenas y cada una hace una sola cosa. Son las piezas que usan todos los demas modulos, y precisamente por eso debian ser faciles de revisar y de probar por separado. Ademas, sus nombres y su forma de uso quedaron congelados el primer dia, de manera que el resto de programas puede apoyarse en ellas sin miedo a que cambien.

Los datos: cómo entran las fotos al sistema

Para qué sirve este archivo (en lenguaje llano)

Hasta ahora sabemos que el sistema aprende mirando fotos de flores y que las fichas tecnicas del proyecto estan escritas en una libreta de ajustes. Falta la pieza mas terrenal de todas: **de donde salen esas fotos y como se preparan antes de entrar**.

Este capitulo explica `src/data.py`, el archivo que se ocupa de esa puerta de entrada. Su trabajo es de portero y de cocinero a la vez. De portero, porque consigue las 8189 fotos del catalogo publico de la Universidad de Oxford y las reparte en los tres grupos de trabajo: las que sirven para aprender, las que sirven para vigilar el aprendizaje y las que se guardan para la medicion final. De cocinero, porque ninguna foto entra al sistema tal como salio de la camara: hay que cortarla al mismo tamano, ajustarle los colores y, durante el aprendizaje, variarla un poco para que el sistema no se acostumbre a verla siempre igual.

El archivo tiene tres funciones publicas, y son las tres que explican este capitulo. La primera, `build_transforms`, describe las operaciones que se le hacen a cada foto antes de usarla. La segunda, `build_dataloaders`, prepara el mecanismo que va sirviendo las fotos en tandas. La tercera, `get_class_names`, consigue la lista de los 102 nombres de especie para que el sistema pueda responder "petunia" y no "especie numero 53".

Una advertencia sobre las palabras, porque en este capitulo aparecen varios terminos nuevos y conviene no perderse. Llamamos **transformacion** a cada operacion que se aplica a una foto, como recortarla o cambiarle el brillo. Llamamos **lote** al grupo de fotos que se procesa de una vez. Y llamamos **catalogo** o **conjunto de datos** al total de fotos con sus etiquetas, que en ingles se dice *dataset*. La palabra **etiqueta** es simplemente el nombre correcto de la flor en cada foto: es lo que queremos que el sistema aprenda a deducir.

El archivo tiene 449 lineas, pero la mayor parte son funciones auxiliares y comentarios de seguridad. Las tres funciones que importan ocupan unas sesenta lineas en total.

Qué hay dentro

El catalogo de flores y sus tres particiones

Antes de mirar el codigo conviene entender que se esta manejando. **Oxford Flowers-102** es un catalogo publico de fotografias de flores publicado por la Universidad de Oxford y usado en investigacion desde hace anos. Contiene exactamente lo siguiente:

Dato	Valor
Fotos en total	8189
Especies distintas	102
Fotos de entrenamiento	1020
Fotos de validacion	1020
Fotos de prueba	6149

Esas tres particiones, en ingles *splits*, no las elegimos nosotros: vienen definidas en el propio catalogo y estan descritas en un archivo interno llamado `setid.mat`. El propio archivo de datos las declara asi:

```
#: Splits oficiales, en el orden que espera el contrato.
SPLITS: tuple[str, str, str] = ("train", "val", "test")
```

- `train` es el grupo de **entrenamiento**. Estas 1020 fotos son las que el sistema mira para aprender. Son 10 fotos por especie, que es muy poco, y esa escasez es intencionada en el diseno de este catalogo.
- `val` es el grupo de **validacion**. Estas otras 1020 fotos no se usan para aprender, sino para ir comprobando como va el aprendizaje. Como el sistema nunca aprende de ellas, sirven para saber si de verdad esta mejorando o solo memorizando.
- `test` es el grupo de **prueba**, el mas numeroso con 6149 fotos. Se guarda intacto hasta el final y sirve para la medicion definitiva. Es el examen de verdad, el que nunca se vio antes ni directa ni indirectamente.
- Una **tupla** es una lista corta de valores que van juntos y no se modifican. Aqui guarda los tres nombres en el orden en que el resto del programa los espera.

Por que se usan las particiones oficiales y no se reparten a mano. Hay tres motivos y los tres pesan. El primero es la **comparabilidad**: los resultados publicados sobre este catalogo se miden sobre estas mismas particiones, asi que usar otras haria que nuestras cifras no se pudieran comparar con ninguna referencia. El segundo es la **reproducibilidad**: cualquiera puede obtener exactamente la misma particion ejecutando el mismo programa, sin depender de una semilla de reparto ni de nuestras decisiones. El tercero, y el mas importante, es evitar la **fuga de datos**, una expresion que conviene entender bien.

La fuga de datos ocurre cuando una foto usada para aprender aparece tambien en el examen. El sistema la reconoceria de memoria y el resultado seria falsamente bueno: pareceria que acierta mucho, cuando en realidad esta haciendo trampa sin saberlo. Con flores el riesgo es real, porque una misma planta puede haberse fotografiado varias veces con pocos segundos de diferencia: si una foto de esa planta cae en entrenamiento y

su gemela en prueba, la medicion queda contaminada. Las particiones oficiales, que fueron construidas con cuidado por los autores del catalogo, reducen ese riesgo. Aqui conviene ser honestos: una comprobacion posterior por contenido encontro cuatro pares de imagenes identicas byte a byte, de los cuales dos cruzan particiones. Son dos imagenes sobre 8189, un efecto despreciable en las cifras finales, pero se documenta porque impide afirmar que la separacion sea absolutamente perfecta.

Que el grupo de prueba sea mas grande que el de entrenamiento sorprende a mucha gente, y es a proposito: en este catalogo la dificultad no esta en tener muchos datos para aprender, sino en aprender con muy pocos y demostrarlo sobre muchos.

build_transforms: preparar cada foto antes de usarla

Que es una transformacion. Una transformacion es una operacion que se le hace a una foto. Recortarla es una transformacion. Saltarla es una transformacion. Cambiarle el brillo es una transformacion. Se encadenan unas detras de otras para formar una **tuberia** o *pipeline*: un recorrido con estaciones, por el que cada foto pasa en orden y del que sale preparada.

Por que hay que recortar y normalizar. Por dos razones muy concretas. La primera es que el sistema exige siempre el mismo tamano: la red que usamos espera imagenes de 224 por 224 pixeles, tal como explica la libreta de ajustes, y las fotos del catalogo tienen tamanos muy distintos entre si. La segunda es que la red que usamos aprendio primero con el banco de imagenes ImageNet, donde las fotos estaban preparadas de una forma concreta; para aprovechar lo que ya sabe hay que hablarle en el mismo idioma, y ese idioma incluye el tamano y los colores. **Normalizar** es precisamente eso: dejar los colores en la escala que la red ya conoce, restableciendo la media y la desviacion tipica que tiene grabadas.

La funcion tiene dos caminos, segun se este entrenando o evaluando:

```
def build_transforms(cfg: Config, train: bool) -> Callable:
    transforms = _transforms()
    img_size = int(cfg.img_size)
    if img_size <= 0:
        raise ValueError(f"cfg.img_size debe ser positivo, se recibio {cfg.img_size!r}")

    ops: list[Any] = []
    if train:
        ops.append(transforms.RandomResizedCrop(img_size, scale=(0.6, 1.0)))
        ops.append(transforms.RandomHorizontalFlip(p=0.5))
        rand_augment = getattr(transforms, "RandAugment", None)
        if rand_augment is not None:
            ops.append(rand_augment(num_ops=2, magnitude=9))
        else:  # torchvision < 0.12: no hay RandAugment
            ops.append(transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05))
    else:
        ops.append(transforms.Resize(int(img_size * 1.14)))  # 224 -> 255
        ops.append(transforms.CenterCrop(img_size))

    ops.append(transforms.ToTensor())
    ops.append(transforms.Normalize(MEAN, STD))
    return transforms.Compose(ops)
```

- `def build_transforms(cfg: Config, train: bool) -> Callable:` declara la funcion. Recibe dos cosas: la libreta de ajustes, de donde lee el tamano de imagen y los valores de color, y un valor de si o no llamado `train` que dice si estamos entrenando. Devuelve un `Callable`, es decir, un objeto que se puede llamar como si fuera una funcion. Ese objeto es la tuberia ya montada y lista para aplicar a cualquier foto.

- `transforms = _transforms()` consigue la caja de herramientas de transformaciones de la libreria TorchVision. El nombre empieza por guion bajo, que es la convencion para las funciones internas del archivo, las que no se anuncian hacia fuera.
- `img_size = int(cfg.img_size)` copia el tamano de imagen de la libreta a una variable local, para no estar consultando la libreta en cada linea. El `int(...)` se asegura de que sea un numero entero.
- El bloque `if img_size <= 0: raise ValueError(...)` es un control de sentido comun: un tamano de imagen de cero o negativo es imposible, asi que la funcion se niega a seguir y explica por que. Detectar un disparate en este punto ahorra horas de busqueda despues. `raise` significa lanzar un error a proposito.
- `ops: list[Any] = []` crea una lista vacia donde se iran guardando las estaciones de la tuberia. Una **lista** es una coleccion ordenada de cosas; una **lista vacia** es una lista sin nada dentro todavia. Se empieza vacia y se van anadiendo estaciones con `ops.append(...)`, que significa "anade esto al final de la lista".
- `if train:` separa los dos caminos. Si estamos entrenando, la tuberia lleva variaciones; si no, lleva un recorrido fijo y predecible. Cada linea de este bloque y del siguiente se explica por separado mas abajo.
- Al final, las dos estaciones comunes se anaden siempre, tanto en entrenamiento como en evaluacion: convertir a numeros y normalizar. Es la parte que no admite variacion, porque es la que hace que la foto sea comprensible para la red.
- `return transforms.Compose(ops)` envuelve la lista en un objeto que la aplica de corrido, estacion tras estacion, en el orden en que se anadieron. `Compose` significa componer, y el orden importa: primero se recorta y se varia, y solo al final se convierte a numeros y se normaliza.

Ahora las estaciones una a una. Las cuatro primeras solo aparecen cuando se entrena.

`RandomResizedCrop`, recortar al azar. `transforms.RandomResizedCrop(img_size, scale=(0.6, 1.0))` elige un trozo de la foto al azar y lo estira hasta el tamano pedido. El parametro `scale=(0.6, 1.0)` dice que el trozo elegido tendra entre el 60 y el 100 por ciento del area de la imagen original; si sale un recorte pequeno, el sistema ve la flor de cerca y con menos contexto alrededor, y si sale grande, la ve mas entera. El trozo cambia en cada vuelta, asi que el sistema nunca ve la misma foto exactamente igual dos veces. Esto le ensena a reconocer la flor aunque este descentrada, recortada o mas cerca de lo habitual.

`RandomHorizontalFlip`, voltear como en un espejo. `transforms.RandomHorizontalFlip(p=0.5)` voltea la foto de izquierda a derecha en la mitad de los casos, porque `p=0.5` es una probabilidad de una entre dos. Una margarita fotografiada desde el otro lado sigue siendo una margarita, asi que el volteo no cambia la respuesta correcta pero si duplica las formas en que el sistema ve la misma flor. Conviene notar que solo se voltea en horizontal: girarla en vertical seria como mirarla boca abajo, algo que no ocurre en la naturaleza y que le ensenaria cosas falsas.

`RandAugment`, variaciones automaticas de aspecto. `rand_augment(num_ops=2, magnitude=9)` aplica dos variaciones al azar de una lista amplia de retoques, entre los que hay cambios de brillo, contraste, color o nitidez. `num_ops=2` significa dos variaciones por foto, y `magnitude=9` regula su intensidad. El nombre viene de *random augmentation*, aumento al azar. Su utilidad es que multiplica artificialmente la variedad del entrenamiento sin necesidad de fotos nuevas: una misma foto, retocada de mil maneras, vale por muchas. En este proyecto importa doblemente, porque solo hay 10 fotos por especie.

- La linea `rand_augment = getattr(transforms, "RandAugment", None)` pregunta si la version instalada de TorchVision trae esta herramienta. `getattr` es una forma defensiva de preguntar "tienes esto?"; si no lo tiene, devuelve `None`, que significa "nada".

- El bloque `if rand_augment is not None` la usa cuando existe. Si no existe, entra el `else` y se aplica `ColorJitter`, una version mas modesta que solo retoca brillo, contraste, saturacion y tono. Es una red de seguridad para que el proyecto funcione tambien con versiones antiguas de la libreria, en lugar de romperse: el sistema pierde algo de variedad, pero sigue funcionando.

En evaluacion y en prueba, en cambio, la tuberia es fija:

`Resize` y `CenterCrop`, el recorrido determinista. `ops.append(transforms.Resize(int(img_size * 1.14)))` agranda la foto un 14 por ciento, de modo que 224 pasa a ser 255, y despues `transforms.CenterCrop(img_size)` recorta el cuadrado central de 224. La combinacion equivale a un leve acercamiento al centro. Se hace asi, y no recortando directamente, para imitar la preparacion que se uso al entrenar la red original: primero se agranda un poco y luego se toma el centro.

- La palabra **determinista** es clave en este camino. Significa que la misma foto produce siempre exactamente el mismo resultado. Es obligatorio al medir: si cada evaluacion recortara la foto de una forma distinta, dos mediciones del mismo modelo darian cifras diferentes y no se sabria si la diferencia viene del modelo o del recorte.

`ToTensor`, convertir la foto en numeros. `transforms.ToTensor()` transforma la imagen en un **tensor**. Un tensor es la estructura de datos que usa PyTorch: una tabla de numeros con varias dimensiones. En este caso pasa de una imagen a una tabla de tres capas (rojo, verde y azul) de 224 por 224 numeros cada una. Ademas, la conversion cambia la escala: los pixeles, que venian como numeros enteros de 0 a 255, pasan a ser numeros decimales entre 0 y 1. Es el paso sin el cual la red no puede hacer nada, porque la red solo sabe calcular con numeros.

`Normalize`, dejar los colores en el idioma de la red. `transforms.Normalize(MEAN, STD)` resta la media y divide por la desviacion tipica, usando los tres valores de color que estan guardados en la libreta de ajustes. El resultado es que la mayoria de los valores quedan alrededor de cero. Suena a complicacion innecesaria, y tiene una explicacion clara: la red que usamos como base fue entrenada con el banco de imagenes ImageNet usando exactamente esta correccion. Si le entregamos las fotos sin corregir, los numeros le llegan en una escala distinta de la que aprendio a interpretar y su rendimiento baja. Normalizar es, literalmente, hablarle en el mismo idioma.

Por que en validacion y prueba no se aumenta nada. Porque ahi no queremos que el sistema aprenda, sino que queremos medirlo con honestidad. Si durante la evaluacion recortaramos y voltearamos las fotos al azar, cada medicion seria distinta y ademas estariamos midiendo sobre versiones deformadas de las fotos, no sobre las fotos reales. La aumentacion es una herramienta de entrenamiento y solo se usa entrenando. Esta regla esta escrita en el contrato del proyecto y se cumple en este archivo.

Nota: todas las estaciones de variacion son de la libreria TorchVision, que las aplica al azar en cada lectura de cada foto. Por eso el mismo catalogo produce, en la practica, una cantidad casi ilimitada de versiones distintas de las mismas 1020 fotos de entrenamiento.

build_dataloaders: servir las fotos en tandas

Que es un DataLoader. Un *DataLoader* es el mecanismo que sirve las fotos al sistema. Su nombre se traduce como cargador de datos y su tarea es triple: coger el catalogo, ir tomando las fotos de una en una, aplicarles su tuberia de transformaciones y entregarlas agrupadas en lotes. Sin el, el programa de entrenamiento tendria que ocuparse a mano de abrir archivos, ordenarlos y agruparlos.

Que son el lote y el barajado. El **lote** es el grupo de fotos que se entrega de una vez; su tamano lo fija la libreta de ajustes y vale 32. Como ya se explico en el capitulo de la configuracion, el sistema no corrige su criterio despues de cada foto, sino despues de cada lote. El **barajado** es desordenar las fotos antes de entregarlas. Si el catalogo se recorriera siempre en el mismo orden, el sistema veria durante un rato solo especies parecidas y luego solo otras, y ese orden acabaria influyendo en lo que aprende. Desordenar en cada vuelta rompe ese patron.

```
def build_dataloaders(cfg: Config) -> tuple["DataLoader", "DataLoader", "DataLoader", list[str]]:
    from torch.utils.data import DataLoader

    Flowers102 = _flowers102_class()
    root = Path(cfg.data_root)
    root.mkdir(parents=True, exist_ok=True)

    # pin_memory solo cuando hay CUDA (en CPU no aporta y consume RAM).
    pin_memory = bool(_torch().cuda.is_available())
    num_workers = int(cfg.num_workers)

    datasets: dict[str, Any] = {}
    loaders: dict[str, Any] = {}
    for split in SPLITS:
        dataset = Flowers102(
            root=str(root),
            split=split,
            transform=build_transforms(cfg, train=(split == "train")),
            download=True,
        )
        datasets[split] = dataset
        if cfg.debug:
            dataset = _debug_subset(dataset, per_class=cfg.debug_per_class, seed=int(cfg.seed) + 17 *
SPLITS.index(split))
        loaders[split] = DataLoader(
            dataset,
            batch_size=int(cfg.batch_size),
            shuffle=(split == "train"),  # val/test SIN barajar (contrato §3)
            num_workers=num_workers,
            pin_memory=pin_memory,
            drop_last=False,
        )

    class_names = _resolve_class_names(cfg, dataset=datasets["train"])
    return loaders["train"], loaders["val"], loaders["test"], class_names
```

- `Flowers102 = _flowers102_class()` consigue la clase que sabe leer el catalogo de Oxford. Entrega el catalogo ya organizado segun las particiones oficiales, de modo que no hay que repartir nada a mano.
- `root = Path(cfg.data_root)` y `root.mkdir(parents=True, exist_ok=True)` preparan la carpeta donde se guardaran las fotos. El detalle importante es que la carpeta no existe de antemano: el programa la crea la primera vez.
- `pin_memory = bool(_torch().cuda.is_available())` decide una optimizacion llamada **memoria fijada**. Consiste en dejar las fotos en una zona de memoria preparada para que el envio a la tarjeta grafica sea mas rapido. Se activa solo si hay tarjeta, por el motivo que se explica en la seccion de decisiones: sin tarjeta no aporta nada y consume memoria del ordenador. La palabra `bool` convierte la respuesta en un si o no limpio.
- `num_workers = int(cfg.num_workers)` lee cuantos procesos ayudantes se usaran para preparar las fotos. El valor normal es dos, y cero en modo de prueba.

- `datasets` y `loaders` son dos **diccionarios**, es decir, colecciones de parejas "nombre: valor". El primero guardara los tres catalogos ya preparados, y el segundo los tres cargadores. Se usan diccionarios para poder consultarlos por el nombre de la particion, por ejemplo `loaders["train"]`, en lugar de por su posicion en una lista.
- `for split in SPLITS:` recorre las tres particiones, `train`, `val` y `test`, y hace lo mismo con cada una. Un **bucle** es una instruccion que repite un bloque de codigo una vez por cada elemento de una lista.
- `Flowers102(root=..., split=split, transform=..., download=True)` crea el catalogo de esa particion. De los cuatro parametros, dos merecen atencion. `transform=build_transforms(cfg, train=(split == "train"))` es el que conecta este cargador con la funcion anterior: la comparacion `(split == "train")` da si solo cuando la particion es la de entrenamiento, de manera que la tuberia con variaciones se aplica unicamente ahi y las otras dos usan el recorrido fijo. Y `download=True` significa que, si las fotos no estan todavia en la carpeta, se descarguen automaticamente.
- `datasets[split] = dataset` guarda el catalogo recien creado bajo su nombre en el diccionario, para poder recuperarlo al final.
- El bloque `if cfg.debug:` sustituye el catalogo completo por una muestra reducida, segun se explica en su propia seccion mas abajo.
- `DataLoader(dataset, ...)` construye el cargador. Ahi estan los tres ajustes que definen su comportamiento: `batch_size=int(cfg.batch_size)` fija el lote en 32 fotos; `shuffle=(split == "train")` activa el barajado solo en entrenamiento, de modo que las fotos de validacion y de prueba se sirven siempre en el mismo orden, para que las mediciones sean comparables entre si; y `drop_last=False` ordena no descartar el ultimo lote aunque quede incompleto. Este ultimo detalle importa mas de lo que parece: con `num_workers` a cero, cada foto se lee y se transforma en el momento de usarla, sin ayuda de nadie. Es trabajo lento, pero es exactamente lo que se quiere en una prueba de un minuto y lo que evita los problemas descritos en la siguiente seccion.

Los tres cargadores que devuelve. El `return` final entrega cuatro cosas: el cargador de entrenamiento, el de validacion, el de prueba y la lista de nombres de las 102 especies. Los tres cargadores se comportan igual salvo en dos detalles, y los dos son deliberados: el de entrenamiento baraja las fotos y las varia; los otros dos no hacen ninguna de las dos cosas, porque su mision es medir y no ensenar. El orden del retorno esta fijado en el contrato del proyecto y todos los demas programas lo dan por hecho.

`num_workers` y por que en Windows puede fallar. Los **procesos ayudantes** son copias reducidas del programa que se lanzan para ir preparando fotos mientras el proceso principal calcula. Preparar una foto es trabajo pesado: hay que abrir el archivo, recortarlo, variarlo y convertirlo. Repartiendo esa tarea entre dos ayudantes, el sistema deja de esperar. En Colab y en Linux funciona sin complicaciones.

En Windows, en cambio, puede fallar, y conviene saber por que. Windows no crea procesos nuevos de la misma forma que Linux: en lugar de duplicar el programa en marcha, arranca una copia nueva del programa y le pide que se reconstruya. En ese arranque hay codigo que se ejecuta de nuevo, y a veces los ayudantes se quedan esperando al proceso principal mientras el proceso principal espera a los ayudantes. El resultado es un bloqueo o un error de permisos que desconcierta, porque el mismo programa funciona perfectamente en Colab. Por eso la libreta de ajustes usa dos ayudantes por defecto, que es lo correcto donde el proyecto se ejecuta de verdad, y por eso existe la posibilidad de bajarlo a cero para trabajar en Windows. El proyecto, ademas, tiene previsto que el programa de evaluacion reintente una vez con cero ayudantes si tropieza con este problema, avisando en el registro, en lugar de morir con un error incomprensible.

get_class_names: los nombres de las 102 especies

El sistema trabaja internamente con numeros: dice "especie 53", no "petunia". La ultima funcion del archivo se encarga de traducir esos numeros a nombres que una persona reconozca.

La dificultad es que ese nombre no esta garantizado en ningun sitio. La libreria TorchVision no siempre lo trae, el archivo oficial que lo contiene ha dejado de estar disponible en la direccion donde estaba, y el programa no puede permitirse el lujo de fallar por eso. La solucion es una **cadena de respaldos**: se intenta por un camino y, si no da resultado, se pasa al siguiente, hasta que alguno funcione. El ultimo recurso nunca falla porque son nombres inventados.

```
def get_class_names(cfg: Config) -> list[str]:
    num_classes = int(cfg.num_classes)

    if dataset is not None:
        names = _names_from_dataset(dataset, num_classes)
        if names:
            return names
    else:
        try:  # 1a) `Flowers102.classes`: atributo de CLASE, sin instanciar ni red
            Flowers102 = _flowers102_class()
        except Exception:
            Flowers102 = None
        names = _names_from_dataset(Flowers102, num_classes)
        if names:
            return names

        try:  # 1b) instancia real, por si esa version expone los nombres de otra forma
            opened = (
                Flowers102(root=str(cfg.data_root), split="train", download=False) if Flowers102 is not
None else None
            )
        except Exception:
            opened = None
        names = _names_from_dataset(opened, num_classes)
        if names:
            return names

    names = _names_from_cache(cfg)  # 2) cache local (offline-friendly)
    if names:
        return names

    names = _names_from_asset(num_classes)  # 3) respaldo offline versionado en el repo
    if names:
        return names

    names = _download_cat_to_name(cfg)  # 4) descarga de Oxford + cache
    if names:
        return names

    return _fallback_names(num_classes)  # 5) nunca fallar
```

Los cinco caminos, en el orden en que se intentan:

31. **Los nombres que trae la propia libreria.** `Flowers102.classes` es un apartado que en algunas versiones de TorchVision contiene la lista de especies. Es el camino preferido porque funciona sin internet y sin haber descargado nada. La anotacion del codigo lo llama **atributo de clase**, que significa que se puede consultar sobre el molde sin haber fabricado ningun objeto. Si esa lista existe pero contiene nombres genericos del estilo `class_7`, se descarta, porque no aporta nada a una persona.

32. **El archivo guardado en la carpeta de datos.** La primera vez que la descarga funciono, el archivo quedo copiado en la carpeta de datos del proyecto. Las siguientes veces se lee de ahi, sin usar la red.
33. **El respaldo que viaja con el proyecto.** En la carpeta `assets` hay una copia del archivo de nombres, guardada dentro del propio proyecto. Esta es la via mas fiable de todas: no depende de internet ni de la version de la libreria, y ha sido revisada. El propio codigo la prefiere a la descarga y lo deja escrito en un comentario.
34. **La descarga desde Oxford.** Esta es la direccion oficial del archivo de nombres, y hoy dia ya no responde. La peticion se hace con un limite de tiempo, y si falla, el programa lo apunta y sigue adelante sin protestar. Una vez intentado, no se vuelve a intentar en la misma corrida, para no perder tiempo esperando una respuesta que no va a llegar.
35. **Los nombres genericos.** El ultimo recurso fabrica nombres del tipo `especie_001`, `especie_002` y asi hasta `especie_102`. El codigo lo comenta con dos palabras que resumen su filosofia: "nunca fallar". Son nombres pobres pero honestos: el sistema sigue funcionando y avisa de que las especies no tienen nombre real, en lugar de interrumpir el trabajo.

El detalle de las etiquetas que empiezan en 1. Aqui esta la trampa mas fina de todo el archivo, y merece explicacion despacio porque es la clase de detalle que arruina resultados sin dar ningun error.

El catalogo original de Oxford numera sus 102 especies del 1 al 102. La libreria TorchVision, en cambio, tiene la costumbre de empezar a contar en cero, asi que a cada etiqueta le resta uno: donde Oxford dice 1, TorchVision dice 0, y donde Oxford dice 102, TorchVision dice 101. Como consecuencia, cuando el archivo de nombres asocia el nombre a la etiqueta interior 0, tiene que buscarlo en el archivo de nombres bajo la clave 1.

```
    for offset in (1, 0):  # primero 1-based (contrato), luego 0-based (asset)
        names: list[str] = []
        for i in range(int(num_classes)):
            value = mapping.get(str(i + offset))
```

- `for offset in (1, 0)` prueba primero con el desplazamiento de uno, que corresponde al archivo oficial de Oxford, y despues con el de cero, que corresponde a la copia guardada en el proyecto. Asi el mismo codigo sirve para las dos convenciones.
- `str(i + offset)` construye la clave que hay que buscar. Para la etiqueta interior 0 y desplazamiento 1, busca la clave `"1"`. El `str(...)` convierte el numero en texto porque las claves del archivo son texto.
- `mapping.get(...)` busca en el diccionario de nombres. Si no encuentra el nombre esperado, el bucle se corta con `break` y se prueba el desplazamiento siguiente. Si logra completar las 102 especies, se queda con ese resultado.
- La comprobacion adicional de que unas pocas muestras reales coincidan con los nombres, que aparece en las funciones auxiliares, es una defensa contra esta misma trampa: si una version de la libreria cambiara la convencion, el programa lo detectaria y volveria a contar a mano en lugar de devolver nombres desplazados una posicion, que seria un desastre silencioso: todas las respuestas equivocadas por un puesto, sin ningun error de por medio.

Los nombres genericos, reconocidos por su forma. Para descartar una lista que no sirve, el archivo usa un patron de texto que reconoce los nombres vacios de contenido.

```
_GENERIC_NAME_RE = re.compile(
    r"^(?:class|clase|label|etiqueta|category|categoria|species|especie|"
    r"n|img|image|idx|index)[_\- ]?\d+$",
    re.IGNORECASE,
```

```
)
```

- Una **expresion regular** es un patron que describe como tiene que ser un texto para coincidir con el. Aqui describe nombres compuestos por una palabra de relleno, un separador opcional y un numero: `class_7`, `n12`, `label 3`.
- `re.compile` prepara ese patron una sola vez, al cargar el archivo, para no reconstruirlo en cada comprobacion.
- La parte entre parentesis es la lista de palabras de relleno, en ingles y en espanol. La parte `[_\- ]?` admite un guion bajo, un guion o un espacio, y el signo de interrogacion significa que puede no haber nada. La parte final `\d+$` exige uno o mas digitos hasta el final del texto.
- `re.IGNORECASE` pide ignorar las mayusculas y minusculas, de modo que `Class_7` tambien se reconoce como nombre generico.
- Gracias a esta comprobacion, el programa distingue "un nombre de verdad" de "un relleno con pinta de nombre". Es la diferencia entre responder "petunia" y responder "clase 53", y ese detalle es el que hace util la funcion.

El modo debug: 408 fotos en lugar de 8189

En el capitulo anterior se explico que `debug` reduce el trabajo para poder comprobar en un minuto que todo funciona. Aqui esta la parte que lo hace realidad, y el numero concreto: en modo de prueba no se usan 8189 fotos, sino **408**, que son **4 por cada una de las 102 especies**.

```
    rng = random.Random(int(seed))
    selected: list[int] = []
    for label in sorted(by_class):
        indices = by_class[label]
        rng.shuffle(indices)
        selected.extend(sorted(indices[:per_class]))
    selected.sort()

    if not selected:  # dataset vacio: no fallar, devolver un Subset vacio
        return Subset(dataset, [])
    return Subset(dataset, selected)
```

- La funcion auxiliar que hace esto recibe el catalogo completo, el numero de fotos por especie y una semilla de azar. Devuelve un **subconjunto**, es decir, una vista sobre el catalogo original que solo deja ver las fotos elegidas. La palabra inglesa para esto es *Subset*. Es importante entender que no se copian fotos ni se reorganiza nada: se elige que fotos se miran.
- `rng = random.Random(int(seed))` crea un generador de azar propio, con su semilla. Se usa uno propio y no el general del programa para que la eleccion de fotos sea estable y no dependa de cuantas veces se haya usado el azar antes. La semilla sera la misma en cualquier ordenador, asi que las 408 fotos elegidas son siempre las mismas.
- El bucle recorre las especies una por una, en orden. Para cada una, desordena las fotos disponibles con `rng.shuffle(indices)` y se queda con las primeras `per_class`, que son cuatro.
- `selected.extend(...)` va acumulando los indices elegidos, y el `selected.sort()` final los pone en orden creciente. Ordenarlos es una precaucion: el resultado es siempre el mismo, independientemente del orden en que se recorrieran las especies.
- La proteccion `if not selected`: cubre el caso de un catalogo vacio y devuelve un subconjunto tambien vacio. Es coherente con la filosofia de "nunca fallar" que ya vimos en los nombres de especie.

- El detalle tecnico mas importante esta en la documentacion de la funcion: como solo se eligen indices del catalogo original, la correspondencia entre cada foto y su etiqueta se mantiene intacta. Esto parece obvio y no lo es: si el proceso de recorte mezclara las etiquetas, el modo de prueba estaria midiendo un sistema entrenado con respuestas falsas, y podria dar la impresion de que el proyecto esta roto cuando en realidad todo funciona.

Cada particion recibe su propia seleccion, calculada con una semilla ligeramente distinta, para que las fotos de entrenamiento y las de validacion no acaben siendo las mismas:

```
            dataset = _debug_subset(dataset, per_class=cfg.debug_per_class, seed=int(cfg.seed) + 17 *
SPLITS.index(split))
```

- `SPLITS.index(split)` devuelve la posicion de la particion en la lista: cero para entrenamiento, uno para validacion y dos para prueba.
- Multiplicar esa posicion por 17 y sumarla a la semilla base hace que cada particion use una semilla distinta. El numero 17 es arbitrario; lo unico que importa es que las tres semillas sean diferentes entre si.

Para que sirve. El modo de prueba permite comprobar en un minuto que toda la maquinaria gira: que las fotos se descargan, que las transformaciones no fallan, que el sistema aprende algo, que se guardan los resultados y que las mediciones se escriben. Con 408 fotos se cubren las 102 especies, de modo que ninguna queda fuera y el sistema no puede acertar por eliminacion. Lo que este modo no hace es dar un resultado presentable: sirve para comprobar que el mecanismo funciona, no para saber lo bueno que es.

Por qué se hizo asi

Por que los splits oficiales y no un reparto propio. Repartir a mano las fotos tiene una ventaja aparente: se podria decidir que hubiera mas fotos de entrenamiento. Tiene tres inconvenientes, y pesan mas. El primero es que las cifras dejarian de ser comparables con las de cualquier trabajo publicado sobre este catalogo. El segundo es que habria que fijar una semilla de reparto y documentarla, con lo que la reproducibilidad dependeria de un detalle nuestro. El tercero, y decisivo, es el riesgo de fuga de datos: en un catalogo donde la misma planta aparece en varias fotos, un reparto descuidado pondria fotos de la misma planta a los dos lados del examen y las cifras saldrian infladas. Las particiones oficiales fueron construidas por los autores del catalogo con ese cuidado, asi que se respetan tal cual.

Por que la aumentacion solo en entrenamiento. Porque la aumentacion es una herramienta para ensenar, no para medir. Variar las fotos mientras se aprende hace que el sistema sea mas robusto, porque le obliga a reconocer la flor en condiciones distintas. Aplicar esa misma variacion al medir seria un error por partida doble: cada medicion daria un resultado ligeramente distinto segun lo que tocara al azar, y ademas se estaria midiendo sobre fotos deformadas, no sobre las fotos reales que el sistema vera cuando se use. Por eso la particion de entrenamiento tiene su propia tuberia con variaciones, y las de validacion y prueba usan un recorrido fijo y predecible.

Por que los nombres de clase viven en un archivo propio del proyecto. Al principio se intentaba descargar el archivo de nombres desde la direccion oficial de Oxford cada vez que hiciera falta. Cuando esa direccion dejo de responder, quedo claro que un dato tan basico no podia depender de un servidor ajeno. Guardar una copia revisada dentro del proyecto convierte ese dato en algo que viaja con el codigo: funciona sin internet, funciona en cualquier ordenador y no cambia bajo los pies. La descarga se conserva como ultimo intento de conseguir

nombres reales, pero ya no es el camino principal. Detras queda todavia la red de seguridad de los nombres genericos, para que ni siquiera en el peor escenario el programa se detenga.

Por que `pin_memory` se activa solo con tarjeta grafica. La memoria fijada es una preparacion especial de la memoria del ordenador que acelera el envio de datos a la tarjeta grafica. Si no hay tarjeta, ese envio no existe, de modo que la preparacion no aporta ninguna velocidad y en cambio reserva memoria del ordenador que hace falta para otras cosas. Como las dos situaciones son la norma segun el entorno donde se ejecute, la decision se toma consultando si hay tarjeta disponible. Es el mismo criterio que sigue el proyecto en todo lo relacionado con el motor de calculo: adaptarse al entorno en lugar de dar por hecho cual es.

Por que no hay rutas absolutas ni ordenes de tarjeta grafica en este archivo. Dos reglas del contrato del proyecto se cumplen aqui de forma visible. Ninguna ruta empieza por la letra de una unidad de disco, porque todas se construyen a partir de la carpeta indicada en la libreta de ajustes; asi el proyecto funciona igual en Windows, en Linux y en Colab. Y ninguna linea ordena usar la tarjeta grafica, porque este archivo ni siquiera necesita saber donde se calcula: su tarea es servir los datos. La unica mencion a la tarjeta es la consulta de disponibilidad que decide la memoria fijada.

Por que las importaciones de la libreria de inteligencia artificial son perezosas. Cargar PyTorch y TorchVision ocupa tiempo y memoria. Este archivo lo hace dentro de las funciones, cuando de verdad hacen falta, y no en la primera linea. La consecuencia practica es valiosa: un programa que solo quiera preparar una imagen para la prediccion puede reutilizar la funcion de transformaciones sin cargar el catalogo de 8189 fotos ni la libreria que lee sus archivos internos. Ademas, cuando falta una pieza auxiliar llamada scipy, que sirve para leer los archivos con las etiquetas del catalogo, el programa no se limita a fallar: lo detecta y explica el comando exacto que hay que ejecutar para instalarla. Un mensaje util ahorra media hora de desconcierto.

Por que se comprueba la forma de los nombres en lugar de confiar en ellos. Porque un nombre generico que parece bueno es peor que no tener nombre. Si el sistema respondiera "clase 53" con toda naturalidad, nadie notaria que la traduccion fallo; en cambio, respondiendo "especie_053" queda claro de un vistazo que los nombres reales no se consiguieron. La comprobacion del patron de texto es la que permite distinguir una cosa de la otra antes de dar la lista por buena.

Por que una prueba rapida usa 408 fotos y no 8189. Porque el objetivo de esa prueba es ahorrar tiempo, y 8189 fotos exigen descargar cientos de megabytes y esperar. Con 408 fotos, cuatro por especie, la prueba recorre las 102 especies y comprueba toda la maquinaria en un minuto. Mantener la proporcion de cuatro por especie, en lugar de tomar las primeras 408 fotos del catalogo, es lo que evita que la prueba se centre en unas pocas especies y deje el resto sin comprobar: si se tomaran las primeras fotos de la lista, casi todas serian de las mismas especies y la prueba diria muy poco. Y para cerrar la idea con las cifras del proyecto: con solo cuatro epocas en la corrida completa, el sistema acierta el 37,8 por ciento de las 6149 fotos oficiales de prueba. Ese es el orden de magnitud de la prueba corta, y subraya que su papel es comprobar que todo funciona, no presumir de resultados.

El modelo: el cerebro que aprende a distinguir flores

Para qué sirve este archivo (en lenguaje llano)

El archivo **src/model.py** es la fábrica del cerebro del sistema. Aquí todavía no se aprende nada: lo que se hace es **construir** la estructura que después aprenderá, decidir qué partes de esa estructura se pueden modificar y guardar en el disco lo aprendido para poder reutilizarlo.

Tres palabras que se repiten en todo el documento:

- **Variable**: una caja con nombre donde se guarda un valor. Por ejemplo, la variable `dropout` guarda el número 0,3.
- **Función**: un bloque de instrucciones con nombre que recibe datos, hace un trabajo y entrega un resultado. Se usa escribiendo su nombre y un paréntesis: `build_model(...)`.
- **Clase**: un molde para fabricar objetos. Del molde llamado `FlowerClassifier` se pueden fabricar muchos cerebros distintos, cada uno con sus propios números internos.

Además de la clase, el archivo ofrece cinco funciones públicas —`build_model`, `set_backbone_trainable`, `build_optimizer`, `save_checkpoint` y `load_checkpoint`— que el resto del proyecto utiliza.

Qué es torchvision y por qué no se entrena una red desde cero

```
import torch
import torch.nn as nn
from torchvision import models as tv_models
```

- **torch** es la librería que hace los cálculos pesados: millones de multiplicaciones por segundo, en el procesador normal (CPU) o en una tarjeta gráfica (GPU).
- **torch.nn** trae las piezas con las que se arma una red: capas, funciones de error y demás.
- **torchvision** es la librería compañera especializada en **imágenes**. Incluye recetas de redes ya montadas y, sobre todo, redes que **ya fueron entrenadas** con millones de fotografías de la base de datos ImageNet.
- La parte `models as tv_models` significa "trae el apartado de modelos de torchvision y llámalo aquí `tv_models`": es solo un apodo para escribir menos.

Entrenar desde cero significa empezar con números al azar y enseñarle a ver desde el principio. Eso exige millones de fotos y días de cómputo. Este proyecto tiene **1020 fotos de entrenamiento** repartidas entre 102 especies de flores: es poquísimo para ese camino. Por eso se parte de una red que ya sabe reconocer bordes, texturas, pétalos y formas, y solo se le enseña la decisión final: "esta combinación de rasgos es una rosa, esta otra es un girasol".

*Nota: a esta técnica se le llama **aprendizaje por transferencia**. Se transfiere el conocimiento de una tarea enorme (reconocer mil objetos distintos en ImageNet) a una tarea pequeña (102 flores).*

La clase FlowerClassifier: el molde del cerebro

```
class FlowerClassifier(nn.Module):
    def __init__(
        self,
        backbone: nn.Module,
        in_features: int,
        num_classes: int = FLOWERS102_CLASSES,
        dropout: float = 0.3,
        backbone_name: str | None = None,
        pretrained: bool = False,
```

```
) -> None:
    super().__init__()
    self.backbone = backbone
    self.avgpool = nn.AdaptiveAvgPool2d(1)
    self.head = nn.Sequential(
        nn.Dropout(p=float(dropout)),
        nn.Linear(int(in_features), int(num_classes)),
    )
```

- `class FlowerClassifier(nn.Module)` define el molde. El paréntesis del final significa **heredar de nn.Module**: el molde nuevo es un caso particular de un molde que ya trae PyTorch y recibe gratis toda su maquinaria interna (registrar los números que se pueden aprender, moverlos a la tarjeta gráfica, guardarlos y cargarlos en disco, y alternar entre modo entrenamiento y modo examen).
- `def __init__(...)` es el **constructor**: las instrucciones que se ejecutan cada vez que se fabrica un cerebro nuevo. La palabra `self` significa "este objeto que estoy construyendo".
- `super().__init__()` llama al constructor del molde padre. Sin esta línea, la maquinaria heredada de `nn.Module` no queda preparada y el modelo no funciona.
- Los nombres entre paréntesis son los **parámetros** de la función: los datos que hay que darle para que trabaje. `in_features` es cuántos números entrega el extractor por cada foto; `num_classes` es cuántas especies distintas tiene que distinguir.
- Los valores escritos después del signo igual (`num_classes: int = FLOWERS102_CLASSES`) son **valores por defecto**: si quien llama a la función no indica nada, se usan esos.
- `self.backbone = backbone` y `self.head = ...` guardan las dos mitades del cerebro como **atributos**, es decir, como variables que viven dentro del objeto. Esos dos nombres son obligatorios: el resto del proyecto busca exactamente esos dos atributos para congelar, descongelar e inspeccionar el modelo.

El extractor de rasgos (backbone) y la cabeza (head)

El **backbone** es el ojo y la corteza visual: recibe la foto y devuelve una lista de números que describen lo que hay en ella. La **cabeza** es la parte que pone nombre: recibe esa lista y reparte puntos entre las 102 especies.

Backbone	Números que entrega por foto	Comentario
`efficientnet_b0`	1280	El elegido por el proyecto, es el valor por defecto
`resnet50`	2048	Alternativa clásica y mucho más pesada
`mobilenet_v3_large`	960	Alternativa ligera, pensada para teléfonos

- El proyecto eligió **EfficientNet-B0** porque da muy buen resultado con un coste bajo: una corrida completa cabe en 15 a 25 minutos de una GPU T4, y con 1020 fotos conviene no gastar en redes enormes, que se limitan a memorizar las imágenes.
- Las otras dos quedan disponibles por si se quiere comparar, pero piden más memoria y más tiempo.

```
    self.avgpool = nn.AdaptiveAvgPool2d(1)
    self.head = nn.Sequential(
        nn.Dropout(p=float(dropout)),
        nn.Linear(int(in_features), int(num_classes)),
```

```
        )
```

- `nn.AdaptiveAvgPool2d(1)` convierte el mapa de rasgos (una rejilla de números) en un solo número por canal, promediando. Así el resultado tiene siempre el mismo tamaño, sin importar el tamaño de la entrada.
- `nn.Sequential(...)` es una cadena: lo que entra pasa por la primera pieza y lo que sale de ella entra en la segunda.
- `nn.Linear(in_features, num_classes)` es la capa final de decisión. Tiene un peso por cada pareja formada por un rasgo y una especie, y produce los 102 números de salida, llamados **logits**, que son puntuaciones sin normalizar (no son porcentajes).

El Dropout de la cabeza

```
            nn.Dropout(p=float(dropout)),
```

- **Dropout** apaga al azar una fracción de los números durante el entrenamiento. Con `p=0.3` se apaga el 30 por ciento en cada pasada, y en la siguiente pasada se apagan otros distintos.
- Sirve para que la red no se apoye siempre en la misma pista. Si memorizara "pétalo rojo con mancha central", el dropout la obliga a buscar señales alternativas, y así aprende de verdad en lugar de aprenderse las fotos de memoria.
- Solo actúa mientras se entrena. Cuando el modelo ya está en modo examen, el dropout se desactiva solo.

forward: el recorrido de la foto por el cerebro

```
    def forward_features(self, x: torch.Tensor) -> torch.Tensor:
        """Vector de caracteristicas (B, in_features) antes de la cabeza lineal."""
        feats = self.backbone(x)
        if feats.dim() > 2:
            feats = self.avgpool(feats)
        return torch.flatten(feats, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """(B, 3, H, W) -> (B, num_classes) logits (sin softmax)."""
        return self.head(self.forward_features(x))
```

- **forward** define el camino que recorren los datos. Es la función que PyTorch ejecuta automáticamente cuando en el entrenamiento se escribe `model(imagenes)`.
- `x` es un **lote**: un bloque de imágenes que se procesa de una sola vez. Su forma es `(B, 3, H, W)`: B imágenes, 3 canales de color (rojo, verde y azul), H de alto y W de ancho. En este proyecto cada foto mide **224 por 224** píxeles y los lotes son de **32 imágenes**.
- La salida tiene la forma `(B, num_classes)`: para cada imagen del lote, 102 puntuaciones.
- La nota "sin softmax" aclara que esas puntuaciones todavía no son porcentajes; convertirlas en probabilidades es tarea de otra parte del sistema.
- `feats.dim() > 2` comprueba si el backbone devolvió una rejilla en lugar de una lista. Si es así, `avgpool` la reduce a un número por canal y `torch.flatten(feats, 1)` aplana todo en una lista larga por imagen. Gracias a esto, el mismo cerebro funciona igual sin importar cuál de los tres backbones se haya elegido.

build_model: la función que fabrica el cerebro

```
SUPPORTED_BACKBONES: tuple[str, ...] = (
    "efficientnet_b0",
    "resnet50",
    "mobilenet_v3_large",
```

```
)


def build_model(
    num_classes: int = FLOWERS102_CLASSES,
    backbone: str = "efficientnet_b0",
    pretrained: bool = True,
    dropout: float = 0.3,
) -> FlowerClassifier:
```

- Es la única puerta de entrada: nadie fabrica un `FlowerClassifier` a mano. Así se garantiza que todos los cerebros del proyecto se construyan igual.
- Antes de construir nada, se comprueba que el backbone pedido exista de verdad:

```
if not isinstance(backbone, str) or backbone not in _BACKBONE_BUILDERS:
    soportados = ", ".join(SUPPORTED_BACKBONES)
    raise ValueError(
        f"Backbone no soportado: {backbone!r}. Soportados: {soportados}."
    )
```

- `raise` lanza un error que detiene el programa y muestra un mensaje. Es mejor parar de inmediato, con la lista de nombres válidos, que entrenar 25 épocas con la red equivocada y descubrirlo al final.
- Un backbone **preentrenado** significa que sus números no empiezan al azar: ya traen los valores que aprendió en ImageNet. Se piden con `pretrained=True`.
- El respaldo automático si la descarga falla:

```
try:
    features, in_features = builder(used_pretrained)
except Exception as exc:  # noqa: BLE001 - cualquier fallo de descarga/carga
    if not pretrained:
        # El error es real (no de red): no lo enmascaramos.
        raise
    warnings.warn(
        f"No se pudieron cargar los pesos preentrenados de '{backbone}' "
        f"({type(exc).__name__}: {exc}). Se continua con pretrained=False "
        "(pesos aleatorios); el entrenamiento sigue siendo valido.",
        RuntimeWarning,
        stacklevel=2,
    )
    used_pretrained = False
    features, in_features = builder(False)
```

- `try` y `except` significan "intenta hacer esto; si falla, ejecuta aquello otro en lugar de morir".
- La primera vez que se usa una red preentrenada hay que **descargarla de internet**. Si no hay conexión, o si el servidor de descargas está bloqueado (algunas plataformas lo hacen), la descarga falla.
- En ese caso el programa no se detiene. Avisa con un mensaje visible, mediante `warnings.warn` con categoría `RuntimeWarning`, y continúa con **pesos aleatorios**: el extractor queda sin conocimiento previo.
- Ese aviso existe porque, aunque el entrenamiento sigue siendo válido, el resultado será peor y harán falta más épocas. Quien lea el informe debe saber que ocurrió, en vez de creer que el modelo rindió por debajo de lo esperado sin motivo.
- Detalle importante: si `pretrained` ya era falso y aun así hubo un error, el error **sí** se propaga con `raise`. Ese fallo no es de red, es un problema real y hay que verlo.

set_backbone_trainable: congelar y descongelar

```
def set_backbone_trainable(model: FlowerClassifier | nn.Module, trainable: bool) -> None:
    """Activa o desactiva ``requires_grad`` en **todos** los parametros del backbone.

    Se usa en las primeras epocas (``cfg.freeze_epochs``) para entrenar solo la
    cabeza. No cambia el modo train/eval del backbone.
    """
    backbone = getattr(model, "backbone", None)
    if backbone is None:
        raise AttributeError(
            "set_backbone_trainable espera un modelo con atributo .backbone "
            f"(recibido {type(model).__name__})."
        )
    for param in backbone.parameters():
        param.requires_grad = bool(trainable)
```

- Un **parámetro**, o peso, es cada uno de los números que la red puede aprender.
- `requires_grad` es una etiqueta que lleva cada parámetro y que significa "tengo permiso para que me modifiquen durante el aprendizaje".
- **Congelar** el backbone es poner esa etiqueta en falso en todos sus parámetros: la red los sigue usando para mirar, pero no los cambia. **Descongelar** es ponerla en verdadero y volver a permitir cambios.
- `for param in backbone.parameters()` recorre la lista completa de parámetros, de modo que la operación alcanza al backbone **entero**, no solo a sus últimos bloques. Algunos proyectos congelan únicamente el tramo final de la red; aquí se decidió descongelar todo (el motivo está en la última sección).
- `getattr(model, "backbone", None)` intenta leer el atributo y devuelve `None` si el objeto no lo tiene. Si falta, se lanza un `AttributeError` con un mensaje explicativo: es una protección contra usar la función con el objeto equivocado.
- La **analogía** del traductor: el extractor es como un traductor que ya domina un idioma. Al principio no se le cambia el idioma ni su manera de trabajar, solo se le enseña el vocabulario nuevo de las flores. Cuando ya maneja ese vocabulario, se le permite además ajustar su forma de traducir, poco a poco y con pasos pequeños.

build_optimizer: el corrector con dos velocidades

```
    head_params = [p for p in head.parameters()]
    backbone_params = [p for p in backbone.parameters()]

    param_groups: list[dict[str, Any]] = []
    if head_params:
        param_groups.append(
            {
                "params": head_params,
                "lr": float(cfg.lr),
                "weight_decay": float(cfg.weight_decay),
                "name": "head",
            }
        )
    if backbone_params:
        param_groups.append(
            {
                "params": backbone_params,
                "lr": float(cfg.backbone_lr),
                "weight_decay": float(cfg.weight_decay),
                "name": "backbone",
            }
        )
    if not param_groups:
        raise ValueError("El modelo no tiene parametros: no se puede crear el optimizador.")
```

```
    return torch.optim.AdamW(param_groups)
```

- Un **optimizador** es el mecanismo que decide cuánto y en qué dirección se corrige cada número después de ver el error. La corrección concreta la ordena el entrenamiento con `optimizer.step()`.
- **AdamW** es uno de esos mecanismos, muy usado porque se adapta solo a la escala de cada número. La letra W viene de **weight decay** (decaimiento de pesos): una fuerza suave que empuja los números hacia valores pequeños y evita que la red memorice las fotos en lugar de aprender. En este proyecto vale 1e-4, es decir, 0,0001.
- El **learning rate**, o velocidad de aprendizaje, es el tamaño del paso de cada corrección. Un paso grande aprende rápido pero puede destruir lo que ya se sabía; un paso pequeño es seguro pero lento.
- El **learning rate diferencial** consiste en usar dos velocidades a la vez dentro de la misma corrida, y para eso se crean dos **grupos de parámetros**. El grupo llamado `head` (la cabeza) usa 3e-4, es decir, 0,0003, porque es nueva y no sabe nada y necesita pasos grandes. El grupo llamado `backbone` (el extractor) usa 3e-5, es decir, 0,00003, diez veces más pequeño, porque ya sabe ver y solo hay que afinarlo con cuidado.
- Los dos grupos comparten el mismo `weight_decay` de 1e-4.
- Detalle de diseño: el grupo del backbone se crea **aunque el backbone esté congelado** en ese momento. El motivo está escrito en el propio código: AdamW ignora los parámetros que no tienen corrección pendiente, así que el mismo optimizador sigue sirviendo cuando más adelante se descongela el backbone, sin necesidad de fabricarlo otra vez.

save_checkpoint: guardar la partida

```
CHECKPOINT_KEYS: tuple[str, ...] = (
    "model_state",
    "class_names",
    "backbone",
    "num_classes",
    "img_size",
    "config",
    "epoch",
    "val_acc",
)
```

```
    payload: dict[str, Any] = {
        "model_state": model.state_dict(),
        "class_names": list(class_names),
        "backbone": backbone_name,
        "num_classes": num_classes,
        "img_size": int(getattr(cfg, "img_size", IMAGE_SIZE)),
        "config": cfg.to_dict() if hasattr(cfg, "to_dict") else dict(cfg),
        "epoch": int(extra.pop("epoch", -1)),
        "val_acc": float(extra.pop("val_acc", 0.0)),
    }
```

- Un **checkpoint** es como guardar la partida de un videojuego: se escribe en el disco el estado del modelo para poder retomarlo después sin volver a entrenar.
- `model.state_dict()` entrega el diccionario con todos los números aprendidos.
- Las ocho claves obligatorias del contrato son estas:

Clave	Qué guarda

`model_state`	Todos los números aprendidos, es decir, los pesos
`class_names`	La lista de las 102 especies, en el mismo orden que las salidas
`backbone`	Qué red se usó como extractor
`num_classes`	Cuántas especies distingue el modelo
`img_size`	De qué tamaño eran las fotos, 224 píxeles
`config`	La receta completa con la que se entrenó
`epoch`	En qué época se obtuvo este resultado
`val_acc`	Qué exactitud tenía en ese momento

- Se guardan los **nombres de las clases** porque el modelo no maneja nombres: maneja números del 0 al 101. La casilla 37 de la salida no significa nada por sí sola; el nombre es lo que le da sentido. Si los nombres se guardaran en otro archivo y alguien los reordenara por descuido, el sistema respondería con total seguridad la flor equivocada y no daría ningún error. Al viajar dentro del mismo archivo que los pesos, ese desastre es imposible.
- `torch.save(payload, target)` escribe el diccionario en el archivo. La línea anterior, `target.parent.mkdir(parents=True, exist_ok=True)`, crea la carpeta de destino si todavía no existe, para que el guardado no falle la primera vez.

load_checkpoint: retomar la partida

```
    faltantes = [k for k in CHECKPOINT_KEYS if k not in checkpoint]
    if faltantes:
        raise KeyError(
            f"Al checkpoint '{target}' le faltan claves obligatorias: {faltantes}."
        )


    model = build_model(
        num_classes=num_classes,
        backbone=backbone,
        pretrained=False,  # el state_dict ya trae los pesos: nunca hay descarga
        dropout=dropout,
    )
    model.load_state_dict(checkpoint["model_state"], strict=True)
```

- Lo primero es comprobar que el archivo exista. Si no está, el mensaje dice qué hacer: entrenar antes con `python -m src.train`.
- Después se verifica que estén las ocho claves. Si falta alguna, se lanza un `KeyError` que indica exactamente cuál. Un error inmediato y claro es preferible a un resultado mal calculado en silencio.
- El modelo se vuelve a fabricar con la misma receta (mismo backbone, mismo número de clases, mismo dropout) y se pide con `pretrained=False`, porque los números buenos ya vienen dentro del archivo: no hay que descargar nada.
- `load_state_dict(..., strict=True)` copia los números guardados dentro del modelo recién fabricado y exige que **todos** encajen en su sitio. Si la estructura no coincidiera, se detiene con un error en lugar de cargar a medias.

- `model.eval()` pone el modelo en modo examen: desactiva el dropout para que las respuestas sean estables y repetibles.
- La función devuelve tres cosas: el modelo listo para usar, la lista con los nombres de las clases y un diccionario llamado `meta` con el resto del contenido del archivo (la configuración, el tamaño de imagen, la época, la exactitud y cualquier dato adicional).

Por qué se hizo así

- **Se parte de una red ya entrenada y no de cero** porque hay 1020 fotos de entrenamiento. Con tan pocos ejemplos, una red que empieza al azar se limita a memorizar las fotos y falla con cualquier imagen nueva. Una red que ya sabe ver flores solo necesita aprender los nombres.
- **Los dos atributos `.backbone` y `.head` son obligatorios**, no un detalle estético. Todo el resto del proyecto depende de poder separar las dos mitades: para congelar y descongelar, para armar los dos grupos de parámetros y para inspeccionar qué mira el modelo cuando acierta.
- **Se congelan primero y se descongelan después las dos mitades** en lugar de entrenar todo a la vez. Si desde el primer día se corrigiera el extractor con las señales de una cabeza que todavía responde al azar, esas correcciones borrarían el conocimiento previo. Congelar al principio protege lo que ya se sabía.
- **El backbone se descongela entero, no solo sus últimos bloques.** Es cierto que cuesta más tiempo y memoria, pero las 102 especies de flores se parecen mucho entre sí y los rasgos que sirven para perros o coches no bastan. Permitir que todo el extractor se especialice en flores es lo que explica el salto de precisión de las últimas épocas, y se controla con una velocidad de aprendizaje diez veces menor que la de la cabeza.
- **El optimizador se construye una sola vez, al principio**, con los dos grupos ya dentro. Si se fabricara de nuevo al descongelar, se perdería el estado interno de AdamW y el entrenamiento sufriría un reinicio invisible.
- **El checkpoint se guarda con toda la información necesaria para reconstruir el modelo**, y no solo con los pesos. Un archivo con únicamente los números es inservible: no dice de qué tamaño eran las fotos, ni cuántas especies había, ni en qué orden. Guardando la receta completa, el archivo se explica a sí mismo y funciona meses después, en otro ordenador y sin recordar nada.
- **Los errores de verdad no se ocultan.** Solo se perdona el fallo de descarga de los pesos preentrenados, y aun así avisando. Cualquier otro problema detiene el programa con un mensaje claro, porque un modelo entrenado con la configuración equivocada produce resultados que parecen válidos y no lo son.
- **No hay ninguna orden que obligue a usar la tarjeta gráfica** (`cuda`). El dispositivo se recibe como argumento. Gracias a eso el mismo archivo funciona en un ordenador sin GPU, en una GPU de la nube y en el portátil de quien revisa el proyecto, sin tocar una sola línea.

El entrenamiento: las 25 pasadas que convierten el modelo en experto

Para qué sirve este archivo (en lenguaje llano)

El archivo **src/train.py** es el que enseña de verdad. Toma el cerebro construido en el archivo del modelo, las fotos preparadas en el archivo de datos y ejecuta la receta que convierte una red genérica en una especialista en flores.

La corrida completa del proyecto hace **25 pasadas** por las 1020 fotos de entrenamiento, con lotes de **32 imágenes** de **224 por 224** píxeles. En una GPU T4 eso tarda entre **15 y 25 minutos**. No hace falta tanto para ver que el sistema funciona: con solo **4 épocas** ya se alcanza un **37,8 por ciento** de acierto en la primera respuesta y un **67,4 por ciento** en las cinco primeras, medido sobre el conjunto oficial de prueba.

La función train: la receta completa

```
def train(cfg: Config) -> dict:
    set_seed(int(cfg.seed))
    device = resolve_device(cfg.device)
    cfg.ensure_dirs()
```

- `cfg` es el objeto de **configuración**: una caja con todos los ajustes de la corrida (cuántas épocas, de qué tamaño son los lotes, qué velocidades de aprendizaje, dónde se guardan los resultados).
- `set_seed(int(cfg.seed))` fija el punto de partida de todos los números al azar. En este proyecto la semilla es 42 y siempre la misma.
- `resolve_device(cfg.device)` decide dónde se calcula: en la tarjeta gráfica si está disponible, y si no en el procesador.
- `cfg.ensure_dirs()` crea las carpetas de salida antes de empezar, para que nada falle a mitad de camino.

Antes del bucle se arma todo lo necesario:

```
    model = build_model(
        num_classes=int(cfg.num_classes),
        backbone=str(cfg.backbone),
        pretrained=bool(cfg.pretrained),
        dropout=float(cfg.dropout),
    ).to(device)

    freeze_epochs = max(0, int(cfg.freeze_epochs))
    if freeze_epochs > 0:
        set_backbone_trainable(model, False)  # primeras epocas: solo la cabeza
    else:
        set_backbone_trainable(model, True)
```

- `.to(device)` mueve el modelo al dispositivo elegido.
- `freeze_epochs` vale 3 en la corrida completa: durante las **3 primeras épocas** el extractor de rasgos queda congelado y solo aprende la cabeza.
- El optimizador se construye una sola vez y se reutiliza durante todo el entrenamiento, incluso cuando el backbone se descongela.

El bucle de entrenamiento

```
    for epoch_index in range(epochs):
        epoch = epoch_index + 1  # 1-based (ver docstring del modulo)
        congelado = epoch <= freeze_epochs
        set_backbone_trainable(model, not congelado)

        train_loss, train_acc = _etapa_con_fallback("train", epoch)
        val_loss, val_acc, val_top5 = _etapa_con_fallback("val", epoch)
```

- El bucle se repite una vez por época.
- Las **épocas se cuentan desde 1**, no desde 0, para que los informes digan "época 3" de forma natural.

- En cada vuelta se decide si el backbone sigue congelado y se entrena, primero con las fotos de estudio y después con las de examen.
- Al final de cada época se guarda una línea en el registro con la pérdida, el acierto y la velocidad de aprendizaje usada.

Qué es una época y qué es una iteración

- Una **época** es una pasada completa por todas las fotos de entrenamiento. La corrida completa hace 25.
- Una **iteración**, o paso, es el trabajo con un solo lote. Con 1020 fotos y lotes de 32, cada época se recorre en unas 32 iteraciones.
- La **pérdida** (en inglés, **loss**) es un único número que mide cuánto se equivocó el modelo en ese lote: cuanto más pequeño, mejor.
- El **acierto**, o **accuracy**, es la fracción de imágenes bien clasificadas. Se mide en top-1 (la primera respuesta es la correcta) y en top-5 (la especie correcta está entre las cinco respuestas más probables del modelo).

El profesor corrigiendo exámenes: zero_grad, backward y step

```
    for batch in bar:
        images, targets = _move(batch, device)
        optimizer.zero_grad(set_to_none=True)
        with torch.amp.autocast("cuda", enabled=amp_enabled):
            logits = model(images)
            loss = criterion(logits, targets)


        if scaler is not None:
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        else:
            loss.backward()
            optimizer.step()
```

- `images` es el lote: 32 fotos convertidas en números. `targets` son las respuestas correctas de esas 32 fotos. `logits` son las 102 puntuaciones que el modelo da a cada imagen.
- `optimizer.zero_grad(set_to_none=True)` borra las notas de corrección del lote anterior. Sin este borrado, los errores se irían acumulando y las correcciones se volverían disparatadas. Es como limpiar la pizarra antes de empezar el examen siguiente.
- `loss.backward()` es el trabajo del profesor que busca culpables: calcula un **gradiente** para cada número del modelo. Un gradiente es una indicación de cuánto contribuyó ese número al error y hacia dónde hay que moverlo para reducirlo.
- `optimizer.step()` aplica las correcciones: mueve cada número un poquito en la dirección que reduce el error. El tamaño de ese movimiento es la velocidad de aprendizaje.
- La **analogía completa** es la del profesor corrigiendo exámenes: se reparte un examen (el lote), se calcula la nota del error (la pérdida), se averigua en qué se falló y cuánto pesa cada parte (el gradiente) y se anota la corrección que hay que estudiar (el paso del optimizador). Después, examen nuevo y pizarra limpia.
- La evaluación funciona distinto:

```
@torch.no_grad()
def _run_eval_epoch(
```

- `@torch.no_grad()` significa "aquí no hace falta calcular gradientes". En modo examen solo se quiere saber si acierta, no corregir nada, así que se ahorra muchísimo tiempo y memoria. Además se llama a `model.eval()`, que desactiva el dropout.

label_smoothing: no jurar que la respuesta es el cien por cien

```
criterion = nn.CrossEntropyLoss(label_smoothing=float(cfg.label_smoothing))
```

- La **función de pérdida** es la fórmula que convierte "cuánto te equivocaste" en un solo número. La elegida aquí es la entropía cruzada, la habitual cuando hay que elegir una sola clase entre muchas.
- Sin suavizado, la respuesta correcta vale 1 (el cien por cien) y todas las demás valen 0. Con `label_smoothing=0.1`, la respuesta correcta vale 0,9 y el 0,1 restante se reparte entre todas las demás.
- **Por qué se suaviza**: obligar al modelo a estar absolutamente seguro lo empuja a memorizar los casos raros, porque la única forma de no fallar nunca es aprenderse las fotos de memoria. Con el suavizado, el modelo aprende a ser prudente, duda un poco y se equivoca mejor cuando aparece una flor que nunca vio. En este proyecto el valor es 0,1.

El planificador de la velocidad de aprendizaje

```
def build_scheduler(optimizer: torch.optim.Optimizer, cfg: Config) -> Any:
    warmup = max(0, int(cfg.warmup_epochs))
    epochs = max(1, int(cfg.epochs))
    cosine_epochs = max(1, epochs - warmup)

    cosine = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=cosine_epochs, eta_min=0.0
    )
    if warmup <= 0:
        return cosine

    warmup_sched = torch.optim.lr_scheduler.LinearLR(
        optimizer, start_factor=0.1, end_factor=1.0, total_iters=warmup
    )
    return torch.optim.lr_scheduler.SequentialLR(
        optimizer, schedulers=[warmup_sched, cosine], milestones=[warmup]
    )
```

- Un **planificador** es el encargado de cambiar la velocidad de aprendizaje a lo largo del entrenamiento. No conviene usar siempre la misma: al principio estorba y al final sobra.
- La primera parte es el **calentamiento** (**warmup**), con `LinearLR`: durante las **2 primeras épocas** la velocidad sube desde el 10 por ciento de su valor hasta el valor completo. Se hace así porque al principio el modelo responde al azar y un paso grande con información tan mala podría destrozar los pesos preentrenados.
- La segunda parte es el **enfriamiento**, con `CosineAnnealingLR`: la velocidad baja de forma suave, siguiendo una curva de coseno, hasta casi cero en la última época. Así el modelo da pasos largos al principio, cuando está buscando la zona buena, y pasos diminutos al final, cuando solo queda afinar.
- `SequentialLR` encadena las dos partes: primero el calentamiento y luego el enfriamiento.
- El planificador actúa sobre **los dos grupos de parámetros a la vez**, y como aplica el mismo factor de escala a cada uno, la cabeza y el backbone conservan siempre su proporción de diez a uno (3e-4 frente a 3e-5).
- El valor que se anota en el historial es el de la cabeza, y se lee justo **antes** de aplicar el paso del planificador, para que la curva del informe muestre la velocidad que se usó de verdad en esa época y no la de la época siguiente:

```
        head_lr = float(optimizer.param_groups[0]["lr"])
        scheduler.step()
```

Precisión mixta (AMP)

```
    amp_enabled = bool(cfg.amp) and device.type == "cuda"
    scaler = _make_grad_scaler() if amp_enabled else None
    if bool(cfg.amp) and not amp_enabled:
        print("  AMP solicitado pero desactivado: el device no es CUDA (CPU).")
```

- Los números de la red se guardan normalmente con muchos decimales. La **precisión mixta** usa un formato más corto para la mayoría de los cálculos: ocupa la mitad de memoria y va aproximadamente al doble de velocidad.
- El inconveniente es que un formato corto pierde decimales y puede producir resultados inservibles. Para evitarlo entra en juego el **escalador** (`GradScaler`), que multiplica el error por un número grande antes de repartirlo y lo devuelve a su sitio después, de modo que los decimales importantes no se pierdan.
- **Solo se activa si hay GPU**, y esa es la razón de la condición `device.type == "cuda"`. Esa aceleración solo existe en las tarjetas gráficas de NVIDIA; en el procesador no aporta nada y solo añadiría riesgo. Por eso, cuando se pide en un ordenador sin tarjeta, el sistema lo desactiva y avisa con un mensaje, sin detenerse. Es la diferencia entre los 15 o 25 minutos de una T4 y varias horas de procesador.

El guardado del mejor modelo y la parada temprana

```
        mejora = val_acc > best_val_acc
        marca = ""
        if mejora:
            best_val_acc = float(val_acc)
            best_epoch = int(epoch)
            epochs_sin_mejora = 0
            save_checkpoint(
                cfg.checkpoint_path,
                model,
                class_names,
                cfg,
                epoch=epoch,
                val_acc=val_acc,
            )
            marca = " *mejor"
        else:
            epochs_sin_mejora += 1


        if epochs_sin_mejora >= max(1, int(cfg.early_stop_patience)):
            print(
                f"Early stopping: {epochs_sin_mejora} epocas sin mejorar val_acc "
                f"(paciencia {cfg.early_stop_patience}); mejor = {best_val_acc:.4f} "
                f"en la epoca {best_epoch}."
            )
            break
```

- La **exactitud de validación** (`val_acc`) se mide sobre fotos que el modelo nunca usa para aprender. Es el equivalente a un examen sorpresa: la nota de estudio se puede inflar memorizando, la del examen sorpresa no.
- El modelo solo se guarda en el disco cuando mejora esa marca. El archivo resultante es el mejor de toda la corrida, no el último. La línea del registro marca esas épocas con la etiqueta `*mejor`.

- La **parada temprana** vigila las épocas sin mejora. La paciencia es de **6 épocas**: si pasan seis seguidas sin superar el mejor resultado, el entrenamiento se detiene solo y se queda con lo mejor que había conseguido. Ahorra tiempo de GPU y, sobre todo, evita seguir entrenando un modelo que ya empezó a memorizar.

El historial: las 13 claves

```
HISTORY_LIST_KEYS: tuple[str, ...] = (
    "epochs",
    "train_loss",
    "train_acc",
    "val_loss",
    "val_acc",
    "val_top5",
    "lr",
)
HISTORY_SCALAR_KEYS: tuple[str, ...] = (
    "best_val_acc",
    "best_epoch",
    "epochs_run",
    "device",
    "num_params",
    "segundos",
)
```

- Un **historial** es el acta de la corrida. Se guarda como archivo `history.json`, que es un formato de texto que cualquier programa puede leer, y es la materia prima de las gráficas y de las tablas del informe.
- Las siete primeras claves son listas con un valor por época. Las seis últimas son valores sueltos que resumen toda la corrida.
- La clave `val_top5` mide si la respuesta correcta aparece entre las cinco primeras del modelo. Es una medida más indulgente que el top-1 y resulta útil para saber si el modelo se acerca aunque falle por poco.

Clave	Qué guarda
`epochs`	El número de cada época ejecutada
`train_loss`	El error medio sobre las fotos de entrenamiento
`train_acc`	La fracción de aciertos en entrenamiento
`val_loss`	El error medio sobre las fotos de validación
`val_acc`	La fracción de aciertos en validación, la nota honesta
`val_top5`	La fracción de veces que la respuesta correcta está entre las cinco primeras
`lr`	La velocidad de aprendizaje de la cabeza en esa época
`best_val_acc`	La mejor exactitud de validación de toda la corrida
`best_epoch`	En qué época se logró ese mejor resultado
`epochs_run`	Cuántas épocas se ejecutaron de verdad
`device`	En qué dispositivo se calculó

num_params	Cuántos números aprendibles tiene el modelo
segundos	Cuánto duró la corrida

La red de seguridad de num_workers

```
def _es_error_de_workers(exc: BaseException) -> bool:
    """True si la excepcion huele a fallo de los procesos hijo del DataLoader.

    En el sandbox de Windows de este entorno los workers no pueden abrir *named
    pipes* y revientan con ``PermissionError [WinError 5]``; en Linux/Colab
    ``num_workers > 0`` funciona bien. En vez de abortar un entrenamiento ya
    empezado, `train` reintenta UNA vez la etapa con ``num_workers=0``.
    """
    if not isinstance(exc, (OSError, RuntimeError, EOFError, MemoryError)):
        return False
    if isinstance(exc, PermissionError):  # el caso observado: WinError 5
        return True
    texto = f"{type(exc).__name__}: {exc}".lower()
    return any(pista in texto for pista in _PISTAS_WORKERS)


        try:
            return _etapa(etapa, epoch)
        except Exception as exc:  # noqa: BLE001 - se re-lanza si no es de workers
            if num_workers_efectivo <= 0 or not _es_error_de_workers(exc):
                raise
            mensaje = (
                f"La etapa '{etapa}' fallo con num_workers={num_workers_efectivo} "
                f"({type(exc).__name__}: {exc}). Se reintenta una sola vez con "
                "num_workers=0 (en Colab/Linux los workers funcionan; en el sandbox "
                "de Windows no pueden abrir named pipes)."
            )
            print("Aviso: " + mensaje)
            warnings.warn(mensaje, RuntimeWarning, stacklevel=2)

            data_cfg = replace(cfg, num_workers=0)
            train_dl, val_dl, _test_dl, class_names = build_dataloaders(data_cfg)
            num_workers_efectivo = 0
            return _etapa(etapa, epoch)  # unico reintento: si falla, propaga
```

- `num_workers` es el número de **procesos auxiliares** que van preparando las fotos (leerlas del disco, recortarlas, girarlas) mientras la tarjeta gráfica calcula con el lote anterior. Con ayudantes, la GPU no espera y todo va más rápido.
- El problema es que esos ayudantes necesitan un mecanismo del sistema operativo llamado **named pipe**. En algunos entornos de Windows restringidos no pueden abrirlo y revientan con un error de "acceso denegado". Es un fallo del entorno, no del código.
- En vez de abortar el entrenamiento a mitad, el sistema hace lo siguiente: detecta que el fallo huele a los procesos auxiliares, avisa con un mensaje visible, reconstruye los datos con `num_workers=0` (es decir, sin ayudantes: el programa principal prepara las fotos una a una) y repite **una sola vez** la etapa que falló. Si vuelve a fallar, entonces sí se detiene y muestra el error.
- **Por qué se añadió**: cuando la avería ocurría en la época 12 de 25, se perdía todo el trabajo hecho, que son minutos de GPU pagados o racionados. Esta red de seguridad convierte un desastre en una simple advertencia. En Linux y en la nube los ayudantes funcionan bien y la red de seguridad nunca se activa, así que allí no se pierde velocidad.

La interfaz de línea de comandos

Una **interfaz de línea de comandos** (en inglés, **command line interface**) es la forma de dar órdenes escribiendo texto en lugar de pulsar botones. Es útil porque deja escrito exactamente con qué ajustes se hizo cada corrida, y ese texto se puede copiar y repetir.

```
    parser.add_argument("--epochs", type=int, default=None, help="numero de epocas")
    parser.add_argument("--batch-size", type=int, default=None, dest="batch_size", help="tamano de
lote")
    parser.add_argument("--backbone", type=str, default=None, help="efficientnet_b0 | resnet50 |
mobilenet_v3_large")
    parser.add_argument("--device", type=str, default=None, choices=["auto", "cuda", "cpu"],
help="dispositivo")
    parser.add_argument("--debug", action="store_true", help="subconjunto pequeno (smoke test)")
```

- La orden se escribe así: `python -m src.train --epochs 5 --debug`. La parte `python -m src.train` significa "ejecuta el archivo de entrenamiento como un programa", y lo que sigue son las opciones.
- Cada opción tiene un valor por defecto, así que escribir `python -m src.train` a secas ya entrena con la receta completa del proyecto.
- `--debug` es el **modo de prueba rápida**: usa muy pocas fotos por especie y ajusta la configuración a dos épocas sin calentamiento ni precisión mixta, para comprobar en un minuto que todo funciona. Además, cambia la carpeta de salida a `resultados/debug` para no pisar los resultados definitivos que ya están guardados.

Opción	Qué controla
`--epochs N`	Cuántas pasadas completas por las fotos
`--batch-size N`	Cuántas imágenes por lote
`--backbone B`	Qué extractor de rasgos se usa
`--data-root D`	En qué carpeta están las fotos
`--out-dir O`	Dónde se guardan los resultados
`--device auto\`	cuda\
`--num-workers N`	Cuántos procesos auxiliares preparan las fotos
`--debug`	Prueba rápida con pocas fotos
`--seed N`	El punto de partida de los números al azar

Los archivos que deja la corrida

- `best.pt`: el modelo ganador, en el formato de checkpoint del proyecto.
- `history.json`: el acta con las 13 claves de la corrida.
- `config_usada.json`: la receta exacta que se empleó, para poder repetirla.
- `figuras/curvas.png`: tres gráficas con la evolución del error, del acierto y de la velocidad de aprendizaje.

Por qué se hizo así

- **Primero se congela el backbone y después se descongela.** Con 3 épocas congeladas, la cabeza (que empieza respondiendo al azar) aprende a leer los rasgos del extractor sin estropearlos. Solo entonces se permite que el extractor también se adapte a las flores. Si se hubiera descongelado desde el primer minuto, las correcciones llegarían con información sin sentido y borrarían el conocimiento previo, que es justo el tesoro que se quiere aprovechar.
- **El mejor modelo se guarda por exactitud de validación, no por el error de entrenamiento.** El rendimiento sobre las fotos de estudio siempre parece mejor de lo que es, porque el modelo puede memorizarlas. La exactitud de validación se mide sobre fotos que nunca vio, así que es la única nota que dice la verdad. Guardar por entrenamiento sería guardar al mejor memorizador, no al mejor clasificador.
- **La semilla está fija en 42.** Los números al azar intervienen en varias partes (el orden en que se reparten las fotos, los valores iniciales de la cabeza, el apagado del dropout). Si cambiaran en cada corrida, comparar dos experimentos sería imposible: no se sabría si la mejora vino del cambio que se hizo o de la casualidad. Con la semilla fija, cualquiera que repita la misma orden sobre los mismos datos obtiene el mismo resultado, y así el informe se puede comprobar.
- **Se guarda un acta de la corrida y no solo el modelo.** El historial es lo que permite dibujar las curvas, detectar que algo va mal (por ejemplo, que el error de validación empieza a subir mientras el de entrenamiento baja) y explicar en el informe de dónde sale cada número. Un modelo sin su acta no se puede defender.
- **El planificador calienta y luego enfría en lugar de mantener una velocidad fija.** Los primeros pasos son los más peligrosos, porque el modelo todavía no tiene ningún criterio; y los últimos son los más delicados, porque solo queda afinar. Subir al principio y bajar al final es la forma de aprovechar las dos situaciones.
- **La parada temprana y el suavizado de etiquetas combaten el mismo enemigo por dos caminos distintos**: que el modelo se aprenda las 1020 fotos de memoria en lugar de aprender a reconocer flores. El suavizado le quita la tentación de estar seguro al cien por cien, y la parada temprana corta el entrenamiento en cuanto deja de mejorar en el examen sorpresa.
- **La precisión mixta solo se activa en tarjeta gráfica porque solo ahí tiene sentido.** Aceptar un riesgo de precisión a cambio de velocidad es un buen negocio cuando el cálculo está en la GPU; en el procesador no hay ninguna ganancia que justifique ese riesgo, así que se apaga y se avisa.
- **El modo de prueba rápida escribe en su propia carpeta.** Un modo pensado para comprobar que todo arranca no puede sobrescribir el resultado definitivo que costó 25 minutos de GPU: si lo hiciera, una simple prueba de un minuto borraría el trabajo bueno.

Evaluar y predecir: cómo sabemos si acierta

Para qué sirve este archivo (en lenguaje llano)

Este archivo explica dos programas del proyecto: `src/evaluate.py` y `src/predict.py`. El primero sirve para poner a prueba al modelo ya entrenado y sacar notas ("de cada 100 fotos, cuantas acierta"); el segundo sirve para usar el modelo entrenado con una foto suelta y obtener una respuesta.

Antes de seguir, tres palabras que se repiten todo el rato y conviene tener claras desde el principio:

- **modelo**: el programa que ha aprendido a reconocer flores. Se guarda en un archivo llamado **checkpoint**, con extension `.pt`. Es como el "cerebro entrenado" guardado en un fichero.

- **conjunto de prueba** (`split` "test"): una lista de fotos con su respuesta correcta escrita al lado, que el modelo nunca ha usado para estudiar. Es el examen final.
- **prediccion**: la respuesta que da el modelo. Por ejemplo, "esta foto es una rosa".

Por qué "evaluar" y por que se hace sobre el conjunto de prueba

Evaluar significa pasarle al modelo un monton de fotos cuya respuesta correcta ya conocemos y comparar lo que responde con lo que deberia responder. El resultado de esa comparacion es la nota del modelo.

Se hace sobre el conjunto de prueba porque es el unico dato honesto. Si examinaramos al modelo con las mismas fotos que uso para estudiar, se pareceria a dejarle el examen copiado: sacaria una nota altisima y no sabriamos si de verdad reconoce flores nuevas. El proyecto usa los tres grupos oficiales del conjunto Oxford Flowers-102: `train` (estudiar), `val` (ajustar durante el estudio) y `test` (el examen final). El examen final tiene 6149 imagenes repartidas en 102 especies.

```
SPLITS_VALIDOS: tuple[str, str, str] = ("train", "val", "test")
MENSAJE_ENTRENA = "entrena primero: `python -m src.train`"
MAX_CONFUSIONES: int = 20
CABECERA_CSV: tuple[str, ...] = ("ruta", "etiqueta_real", "prediccion", "prob", "correcto")
```

- `SPLITS_VALIDOS` es la lista de los tres grupos permitidos; si alguien pide otro nombre, el programa se queja en vez de hacer algo raro.
- `MENSAJE_ENTRENA` es el texto unico de "todavia no hay modelo entrenado", para que siempre se explique lo mismo.
- `MAX_CONFUSIONES` pone un tope de 20 filas a la lista de confusiones, para no llenar los informes de cientos de casos.
- `CABECERA_CSV` son los cinco nombres de columna del archivo de resultados que se explica mas abajo.

La decision importante antes de escribir nada: trabajar con lo que trae el checkpoint

`evaluate` no da por hecho como es el modelo: lee del propio checkpoint el tamano de imagen y el numero de especies. Asi, si un checkpoint se entreno con imagenes de 224 pixels, se evalua siempre con 224, aunque quien lo llame se equivoque. Los nombres de las especies se recortan o se rellenan para que la posicion 1 sea la especie 1.

```
    n_clases = int(meta.get("num_classes") or len(nombres_ckpt) or getattr(cfg, "num_classes", 0))
    if n_clases <= 0:
        raise ValueError(
            f"El checkpoint '{ckpt}' no define num_classes y no se pudo inferir "
            f"(class_names={len(nombres_ckpt)})."
        )
    nombres = _ajustar_nombres(nombres_ckpt, n_clases)

    # El tamano de imagen del checkpoint manda (no asumir 224).
    img_size = int(meta.get("img_size") or getattr(cfg, "img_size", IMAGE_SIZE))
    cfg = _cfg_con_img_size(cfg, img_size)
```

- `meta` es la "etiqueta" que el entrenamiento guardo dentro del checkpoint: con cuantas especies se entreno, con que tamano de imagen y que ajustes se usaron.
- `_ajustar_nombres` garantiza que exista exactamente un nombre por especie; si el checkpoint trae menos nombres de los necesarios, inventa rellenos como `clase_007`.

- Si el checkpoint no dice cuantas especies hay y no se puede deducir, se levanta un error claro en vez de dar resultados sin sentido.

evaluate: el examen completo

`evaluate` es la funcion principal del archivo. Recibe la ruta del checkpoint, la configuracion, el grupo a examinar (`test` por defecto) y, opcionalmente, un cargador de datos ya hecho para las pruebas automaticas. Devuelve un diccionario con todas las notas y ademas escribe tres cosas en disco: las notas en `metrics.json`, las figuras y el archivo CSV con una fila por imagen.

```
def evaluate(
    checkpoint_path: str | Path,
    cfg: Config | None = None,
    split: str = "test",
    *,
    dataloader=None,
) -> dict:
```

- `checkpoint_path` es el archivo `.pt` con el modelo entrenado. Si no existe, se lanza `FileNotFoundError` con el recordatorio "entrena primero: `python -m src.train`".
- `cfg` son los ajustes de la corrida. Si no se pasa ninguno, se reconstruye leyendo el propio checkpoint, de modo que los resultados se guardan en la misma carpeta donde se entreno.
- `split` elige el grupo: `train`, `val` o `test`.
- `dataloader` es un parametro solo para pruebas automaticas. En uso normal no se toca.

_inferir: recorrer todas las fotos y apuntar la respuesta

`_inferir` es la parte que de verdad pasa las fotos por el modelo. Va lote a lote (grupos de imagenes, no una a una, para ir mas rapido) y apunta cinco listas: la respuesta correcta, la respuesta del modelo, la probabilidad que le dio, si acerto entre sus cinco mejores opciones y la ruta del archivo de imagen.

```
logits = modelo(imagenes)
probs = torch.softmax(logits.detach().float(), dim=1)
k_top5 = max(1, min(5, int(probs.shape[1])))
valores, indices = probs.topk(k_top5, dim=1)

reales.extend(int(v) for v in etiquetas.cpu().tolist())
predichas.extend(int(v) for v in indices[:, 0].cpu().tolist())
probabilidades.extend(float(v) for v in valores[:, 0].cpu().tolist())
aciertos = (indices == etiquetas.unsqueeze(1)).any(dim=1)
```

- Las **probabilidades** se obtienen con `softmax`, que convierte las salidas crudas del modelo en numeros entre 0 y 1 que suman 1. Es la forma de leer "cuanta confianza tiene" en cada especie.
- `topk` pide las cinco especies mas probables y las devuelve ya ordenadas de mayor a menor, con sus nombres de posicion.
- `modelo.eval()` y `torch.no_grad()` (unas lineas mas arriba) ponen el modelo en modo examen y le dicen que no hace falta calcular nada para entrenar: eso evita errores y ahorra tiempo.

Las metricas: como se leen las notas

`_calcular_metricas` es la funcion que traduce todas esas listas a numeros entendibles.

```
top1 = float(np.mean(y_true == y_pred))
top5 = float(np.mean(np.asarray(aciertos_top5, dtype=float))) if len(aciertos_top5) else top1
macro_f1 = float(
```

```
        f1_score(y_true, y_pred, labels=etiquetas, average="macro", zero_division=0)
    )
```

- **top1** (exactitud o acierto directo): de cada 100 fotos, cuantas acierta la especie correcta como primera respuesta. El sistema obtiene 37,8 % sobre las 6149 imagenes del conjunto de prueba oficial.
- **top5**: de cada 100 fotos, en cuantas aparece la especie correcta dentro de las cinco primeras propuestas. El sistema obtiene 67,4 %, mucho mas alto que el top-1 porque acertar "de entre cinco" es mas facil. Es util porque en la demo se muestran cinco candidatas: si la correcta esta en la lista, el usuario la ve.
- **macro_f1**: una nota que promedia el rendimiento de las 102 especies dando el mismo peso a todas, incluidas las que tienen pocas fotos. El sistema obtiene 0,354.
- `zero_division=0` evita que el programa falle cuando una especie no aparece nunca en las predicciones: en ese caso cuenta como cero, no como error.

```
precision, recall, f1, support = precision_recall_fscore_support(
    y_true, y_pred, labels=etiquetas, zero_division=0
)
per_class = {
    nombres[i]: {
        "precision": float(precision[i]),
        "recall": float(recall[i]),
        "f1": float(f1[i]),
        "support": int(support[i]),
    }
    for i in range(int(n_clases))
}
per_class_f1 = {nombres[i]: float(f1[i]) for i in range(int(n_clases))}
```

- **per_class** es el cuadro de rendimiento especie por especie, con un diccionario por especie que contiene `precision`, `recall`, `f1` y `support`.
- **precision** responde a "cuando el modelo dice que es una rosa, cuantas veces tenia razon".
- **recall** responde a "de todas las rosas que habia, cuantas encontro".
- **f1** combina las dos anteriores en un solo numero (0 es pesimo, 1 es perfecto).
- **support** no es una nota, es el numero de fotos de esa especie en el examen. Sirve para saber si la nota se apoya en 40 fotos o en 2.
- **per_class_f1** es lo mismo pero mas simple: solo el numero de f1 por especie, porque el cuaderno de analisis lo consume asi.

Por qué el macro-F1 puede ser mucho menor que la exactitud

No es un error de cuentas, es consecuencia de como se promedian:

- La exactitud (top-1, 37,8 %) se calcula sumando todos los aciertos y dividiendo entre 6149. Una especie con muchas fotos pesa mucho, y las especies frecuentes suelen ser las que mejor se reconocen. Esa nota sube sola.
- El macro-F1 (0,354) da exactamente el mismo peso a las 102 especies. Una especie rara con dos fotos, en la que el modelo falla, baja la media tanto como si fallara una especie con cien fotos.

Por eso los dos numeros difieren: la exactitud mide "que proporcion del examen apruebo" y el macro-F1 mide "como de bien me porto con todas las especies por igual". La diferencia entre 0,378 y 0,354 es la senal de que las especies con menos ejemplos son las que peor van.

La matriz de confusion: quien se confunde con quien

La **matriz de confusion** es una tabla cuadrada donde las filas son la respuesta correcta y las columnas lo que respondio el modelo. Cada casilla cuenta cuantas fotos de la fila acabaron en la columna. La diagonal (fila y columna iguales) son los aciertos; todo lo que queda fuera de la diagonal son confusiones. Es la forma de saber, de un vistazo, que el modelo confunde sobre todo petunia con wallflower (58 casos) y rosa con camelia (45 casos).

```
    matriz = confusion_matrix(y_true, y_pred, labels=etiquetas)
    return top1, top5, macro_f1, per_class, per_class_f1, matriz
```

- Con 102 especies la tabla es de 102 por 102, asi que tambien se genera una version corta y legible con solo las 20 confusiones mas frecuentes.

```
def _top_confusiones(matriz: np.ndarray, nombres: Sequence[str]) -> list[list]:
    pares: list[list] = []
    for i in range(matriz.shape[0]):
        for j in range(matriz.shape[1]):
            if i == j:
                continue
            n = int(matriz[i, j])
            if n > 0:
                pares.append([str(nombres[i]), str(nombres[j]), n])
    pares.sort(key=lambda fila: (-fila[2], fila[0], fila[1]))
    return pares[:MAX_CONFUSIONES]
```

- Se salta la diagonal (`if i == j`) porque ahi estan los aciertos, y aqui solo interesan los errores.
- Cada confusion se guarda como tres datos: especie real, especie predicha y numero de veces.
- Se ordenan de mas a menos casos y se cortan las 20 primeras. En una lista asi aparece arriba petunia hacia wallflower con 58.

Las figuras y el archivo de predicciones

El programa guarda dos imagenes dentro de `resultados/figuras/`. La primera, `confusion_matrix.png`, dibuja la tabla completa como un mapa de calor: cuanto mas oscura la casilla, mas imagenes. Se usa una escala logaritmica a proposito, porque sin ella la diagonal seria tan dominante que no se veria ninguna confusion.

```
    figura, eje = plt.subplots(figsize=(13, 11))
    norma = LogNorm(vmin=1, vmax=max(1, maximo)) if maximo > 0 else None
    imagen = eje.imshow(datos, interpolation="nearest", cmap="Blues", norm=norma)
```

- La escala `LogNorm` comprime las diferencias para que los valores pequenos sigan siendo visibles.
- El backend `Agg` se fija al principio del archivo. Significa que las figuras se dibujan directamente en el archivo, sin abrir ninguna ventana: asi el programa funciona igual en un servidor sin pantalla que en un ordenador normal.

La segunda figura, `top_confusions.png`, dibuja barras horizontales con las confusiones mas frecuentes, la mas repetida arriba, para poder citarlas en el informe.

```
        eje.barh(posiciones, valores, color="#c0392b")
        eje.set_yticks(posiciones)
        eje.set_yticklabels(etiquetas, fontsize=7)
        eje.set_xlabel("Numero de imagenes")
        eje.set_title("Confusiones mas frecuentes (real -> predicho)")
```

- Cada barra es una confusion y su etiqueta se lee "especie real -> especie predicha".
- La mas frecuente de todas, petunia confundida con wallflower (58 casos), sale arriba del todo.

Ademas se escribe un **CSV de predicciones** con una fila por cada imagen examinada:

```
CABECERA_CSV: tuple[str, ...] = ("ruta", "etiqueta_real", "prediccion", "prob", "correcto")
```

```
            escritor.writerow(
                [
                    str(ruta),
                    str(nombres[real]),
                    str(nombres[predicha]),
                    f"{float(prob):.6f}",
                    bool(real == predicha),
                ]
            )
```

- `ruta` es el archivo de imagen concreto, `etiqueta_real` la respuesta correcta, `prediccion` lo que dijo el modelo, `prob` la confianza de esa prediccion y `correcto` un si o un no.
- Para que sirve: permite auditar caso por caso. Si alguien pregunta por que la rosa se confunde con camelia, se abre el CSV, se filtran las filas donde `etiqueta_real` es rosa y `correcto` es falso, y se miran esas fotos concretas. Sin este archivo solo habria un numero global y no se podria investigar nada.
- El archivo se guarda como `resultados/predicciones/test_predicciones.csv`, con el nombre del grupo examinado delante para no pisar los resultados de `val`.

Si fallan los procesos auxiliares, el programa se reintenta solo

Para leer las fotos rapido se usan varios **procesos auxiliares** (`num_workers`), que son copias del programa trabajando en paralelo. En Windows esas copias a veces no pueden comunicarse y la evaluacion se cae con un error de permisos, aunque el modelo no tiene ninguna culpa. La solucion es no morir: se avisa y se repite el examen con cero procesos auxiliares, que siempre funciona.

```
        aviso = (
            f"El DataLoader con num_workers={workers} fallo ({type(exc).__name__}: {exc}). "
            "Se reintenta una sola vez con num_workers=0; esto es habitual en Windows "
            "(los workers de multiprocessing necesitan named pipes). Para evitar el "
            "reintento usa `--num-workers 0`."
        )
        warnings.warn(aviso, RuntimeWarning, stacklevel=2)
        print(f"[evaluate] aviso: {aviso}")
        return _intento(replace(cfg, num_workers=0))
```

- El reintento es **una sola vez** y solo si el error tiene pinta de venir de esos procesos auxiliares; si el fallo es de otra cosa, se deja ver el error de verdad. Esto ultimo es importante: si se ocultaran todos los errores, seria imposible enterarse de un problema real.
- El aviso se emite por dos vias (`warnings` y `print`) para que quede en el registro aunque nadie vea la consola.
- Si el cargador de datos lo habia puesto quien llamo a la funcion (pruebas automaticas), no se reintenta nada: ese fallo es de las pruebas y debe verse entero.
- En la misma idea, `--num-workers` es una opcion de la linea de comandos para pedirlo a mano y evitar el reintento.

```
    parser.add_argument(
```

```
        "--num-workers",
        dest="num_workers",
        type=int,
        default=None,
        help=(
            "workers del DataLoader (por defecto: los de Config, 2). Usa 0 en Windows "
            "o dentro de un sandbox donde los workers no puedan abrir named pipes"
        ),
    )
```

- Si no se indica nada, se respeta el valor normal de la configuracion, que es el correcto en Colab y Linux.

Como se lanza la evaluacion desde la consola

```
    parser = argparse.ArgumentParser(
        prog="python -m src.evaluate",
        description="Evalua un checkpoint de flores sobre el split val o test.",
    )
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="ruta del checkpoint (por defecto: resultados/best.pt)",
    )
```

- El comando completo es `python -m src.evaluate --checkpoint resultados/best.pt --split test`. `python -m` significa "ejecuta este archivo del proyecto como programa".
- `--split test` examina con el conjunto de prueba; con `--split val` se examina el grupo de validacion.
- `--out-dir` cambia la carpeta donde se escriben las notas y las figuras, `--device` elige si se usa la tarjeta grafica o el procesador, `--debug` usa un subconjunto pequeno de fotos.
- `main` devuelve 0 si todo fue bien y 1 si faltaba el checkpoint o hubo un error de datos. Ese numero es lo que usan los sistemas automaticos para saber si el comando funciono.

Predecir: dar una respuesta a una foto suelta

`src/predict.py` es el programa que usa el modelo ya entrenado con una imagen concreta. Aqui ya no hay notas ni examenes: hay una pregunta y una respuesta.

predict_image: que recibe y que devuelve

```
def predict_image(
    image_path,
    checkpoint_path,
    topk: int = 5,
    min_conf: float = 0.25,
    device: str = "auto",
) -> list[tuple[str, float]]:
```

- **Recibe** `image_path`: la foto. Puede ser la ruta de un archivo, una imagen ya abierta en memoria, un arreglo de numeros que representa pixeles, o directamente lo que entrega la ventana de la demo. Toda esa variedad se admite a proposito, para que quien llama no tenga que preocuparse por el formato.
- **Recibe** `checkpoint_path`: el archivo del modelo entrenado.
- **Recibe** `topk`: cuantas especies quiere de vuelta (5 por defecto).
- **Recibe** `min_conf`: el umbral de confianza, que se explica justo despues.

- **Devuelve** una lista de parejas ordenadas de mayor a menor probabilidad, por ejemplo `[("rosa", 0.31), ("camelia", 0.12), ...]`. Cada pareja es el nombre de la especie y un numero entre 0 y 1 que es su probabilidad. La primera de la lista es la apuesta principal.

```
    with torch.no_grad():
        logits = modelo(tensor)
    probabilidades = torch.softmax(logits.detach().float(), dim=1).flatten().cpu().numpy()

    n_clases = int(probabilidades.shape[0])
    nombres = _nombres_ajustados(nombres_ckpt, n_clases)
    mejor = float(probabilidades.max()) if n_clases else 0.0

    if mejor < float(min_conf):
        return [(NO_RECONOCIDA, mejor)]

    if n_clases == 0:
        return []

    k = max(1, min(int(topk), n_clases))
    indices = np.argsort(-probabilidades, kind="stable")[:k]
    return [(nombres[int(i)], float(probabilidades[int(i)])) for i in indices]
```

- `probabilidades` es la confianza en cada una de las 102 especies, y suman 1. `mejor` es la mas alta de todas.
- La linea `if mejor < float(min_conf)` es el umbral: si ni siquiera la mejor apuesta llega al minimo exigido, en vez de una lista de especies se responde con la frase `no reconocida / fuera del catalogo`. La idea es decir "esto no parece ninguna de las 102 flores que conozco" en lugar de dar una respuesta poco fiable.
- `NO_RECONOCIDA` es exactamente ese texto, definido una sola vez en todo el proyecto:

```
NO_RECONOCIDA = "no reconocida / fuera del catálogo"
```

- `k = max(1, min(int(topk), n_clases))` evita pedir mas especies de las que existen y garantiza que al menos se devuelva una.
- `np.argsort` ordena las probabilidades de mayor a menor y devuelve las posiciones; despues esas posiciones se traducen a nombres de especie.
- El tamano de imagen que se usa para preparar la foto se lee del checkpoint, no se da por hecho. Es la misma precaucion que en la evaluacion.

El umbral min_conf y su problema

El umbral `min_conf` vale 0,25 por defecto: si la mejor probabilidad no llega al 25 %, el resultado es "no reconocida". El razonamiento es sensato (mejor no responder que responder mal), pero en la practica tiene una trampa que se descubrio al probar la demo y que se explica en la seccion 09 de este documento. En resumen: con un modelo poco entrenado, el `softmax` queda muy plano (el maximo ronda el 2 %), y entonces el umbral borra predicciones que podian ser correctas. La conclusion practica es que este umbral hay que elegirlo mirando la confianza real del modelo entrenado, no copiarlo de un ejemplo.

La cache: no recargar el modelo en cada clic

Un detalle que no se ve pero se nota mucho: el modelo no se vuelve a leer de disco cada vez que se pide una prediccion. Se guarda en memoria con una **cache**.

Una **cache** es una memoria temporal: la primera vez que se necesita algo caro se guarda, y las siguientes veces se reutiliza en lugar de repetir el trabajo.

```
@functools.lru_cache(maxsize=_CACHE_SIZE)
def _modelo_cacheado(checkpoint_path: str, device: str):
    """Carga (una sola vez por ``checkpoint_path`` + ``device``) el modelo y sus metadatos.

    Returns:
        ``(modelo, nombres_de_clase, img_size, torch.device)``.
    """
    torch_device = resolve_device(device)
    modelo, class_names, meta = load_checkpoint(checkpoint_path, device=torch_device)
    img_size = int(meta.get("img_size") or getattr(modelo, "img_size", 224) or 224)
    return modelo, tuple(str(n) for n in class_names), img_size, torch_device
```

- `lru_cache` significa "guarda los ultimos resultados y reutilizalos". La clave de la cache es la pareja **ruta del checkpoint mas dispositivo**: si se pide el mismo modelo dos veces, se carga una sola vez.
- Por que se cachea: leer un checkpoint de unos 20 MB del disco tarda bastante, y la demo recibe una peticion por cada clic. Sin cache, cada clic volveria a leer el archivo y la interfaz se sentiria lenta y pesada.
- `maxsize=_CACHE_SIZE` (valor 8) pone un tope: como maximo se recuerdan 8 modelos a la vez, para no llenar la memoria del ordenador.
- Consecuencia practica de la cache, que hay que conocer: si se reentrena el modelo y se sobreescribe el mismo archivo, el programa puede seguir usando la version antigua que ya tenia en memoria. Para eso existe `limpiar_cache()`, que vacia esa memoria y obliga a leer de nuevo.

```
def limpiar_cache() -> None:
    """Vacia la cache de modelos cargados (util en tests y al reentrenar)."""
    _modelo_cacheado.cache_clear()
```

- Si no se vacia, despues de reentrenar se comparan resultados antiguos creyendo que son nuevos. Es la clase de error que hace perder una tarde entera.

predict_batch: varias fotos de golpe

`predict_batch` aplica `predict_image` a una lista de imagenes. Como el modelo ya esta en la cache, se carga una sola vez aunque la lista tenga cien fotos.

```
    return [
        predict_image(imagen, checkpoint_path, topk=topk, min_conf=DEFAULT_MIN_CONF, device=device)
        for imagen in image_paths
    ]
```

- Devuelve una lista de resultados, en el mismo orden en que se pidieron las imagenes.
- Aqui no se puede elegir el umbral: se usa siempre el valor por defecto `DEFAULT_MIN_CONF` de 0,25. Por eso algunas de las respuestas pueden ser "no reconocida".

Y como se manda una prediccion desde la consola

```
    parser.add_argument("--image", required=True, help="ruta de la imagen a clasificar")
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="ruta del checkpoint (por defecto: resultados/best.pt)",
    )
    parser.add_argument("--topk", type=int, default=5, help="numero de clases a mostrar (defecto: 5)")


    print(f"imagen:     {args.image}")
    print(f"checkpoint: {args.checkpoint}")
    for posicion, (nombre, probabilidad) in enumerate(resultados, start=1):
```

```
        print(f"{posicion:2d}. {nombre:<42s} {probabilidad * 100:6.2f}%")
    return 0
```

- El comando tipico es `python -m src.predict --image fotos/mi_flor.jpg --topk 5`.
- `--image` es obligatorio: sin foto no hay nada que predecir.
- `--topk 5` pide cinco candidatas; `--min-conf` cambia el umbral y `--device` elige tarjeta grafica o procesador.
- La salida en pantalla es una lista numerada con el nombre de la especie y el porcentaje, con dos decimales.
- Igual que en la evaluacion, si falta el checkpoint se imprime el error, se recuerda el comando de entrenamiento y se devuelve 1 en lugar de dejar un rastro de error incomprensible.

Por qué se hizo asi

- **Se evalua sobre el conjunto de prueba y no sobre el de entrenamiento** porque solo asi la nota es creible. Un modelo examinado con lo que ya estudio siempre parece mejor de lo que es.
- **Se guardan cuatro metricas distintas en lugar de una** porque cada una responde a una pregunta diferente: el top-1 dice la proporcion de aciertos directos, el top-5 dice si la respuesta correcta esta entre las candidatas que la demo ensena, el macro-F1 dice como de equilibrado es el rendimiento entre las 102 especies y el cuadro por especie dice donde falla exactamente. Con un solo numero no se podria explicar la diferencia entre 37,8 % de aciertos y 0,354 de macro-F1.
- **Se generan figuras y no solo numeros** porque una tabla de 102 por 102 no la lee nadie. Convertida en mapa de calor y en barras de las 20 confusiones mas frecuentes, la informacion se entiende de un vistazo y se puede citar en el informe con casos concretos.
- **Se guarda un CSV con una fila por imagen** para poder pasar del "el modelo falla un 62 % de las veces" al "estas cuatro fotos concretas de rosa se confundieron con camelia" y mirarlas.
- **El cargador de datos degrada solo a cero procesos auxiliares** porque el problema es del sistema operativo, no del proyecto. Abortar una evaluacion de varios minutos por un fallo de comunicacion entre procesos seria absurdo, pero ocultar cualquier error tambien: por eso se reintenta una sola vez, solo ante senales conocidas de ese fallo, y se avisa.
- **El tamano de imagen y el numero de especies se leen del checkpoint** en los dos archivos, porque es el unico sitio donde consta con que se entreno. Dar por hecho 224 pixels es la clase de suposicion que produce resultados malos sin dar ningun error.
- **El modelo se cachea** porque la demo y el uso interactivo repiten peticiones con el mismo modelo; recargar 20 MB por clic haria la interfaz inusable. El precio es que hay que vaciar la cache con `limpiar_cache()` al reentrenar, y ese precio esta documentado en el propio codigo.
- **Las predicciones devuelven probabilidades, no solo el ganador** porque la confianza es informacion util: permite al usuario decidir si se fia y permite que el resto del sistema aplique umbrales o avisos sin volver a calcular nada.
- **`predict_image` admite muchos formatos de entrada** para que la interfaz grafica no tenga que adaptar nada: lo que entrega la ventana de la demo se acepta tal cual.
- **Los errores se convierten en mensajes en lugar de excepciones sueltas** ("entrena primero: `python -m src.train`") porque cualquiera puede olvidar entrenar antes de predecir, y un texto claro ahorra tiempo a quien no programa.

La demo: la ventana donde cualquiera puede probarlo

Para qué sirve este archivo (en lenguaje llano)

Este archivo explica `src/app.py`, el programa que abre una **ventana con botones** donde cualquier persona puede subir una foto de una flor, pulsar un boton y ver que responde el modelo. No hace falta saber programar para usarla: solo hay que abrir la pagina, elegir la foto y mirar el resultado.

Dos palabras que se repiten y conviene entender antes de seguir:

- **interfaz grafica**: una pantalla con elementos visuales (cajas, botones, etiquetas) que se manejan con el raton, en lugar de escribir comandos de texto. El programa de la consola de la seccion anterior tambien funciona, pero exige escribir la ruta de la foto a mano cada vez.
- **Gradio**: la libreria (un conjunto de piezas ya hechas) que se usa para construir esa pantalla. Con Gradio se describe "quiero una caja para subir una imagen y una etiqueta de salida" y la libreria crea la pagina web, la publica en el ordenador y la deja lista para usar. Gracias a eso no hubo que escribir ni una linea de la parte web.

Decision de fondo: la demo nunca debe fallar

Todo el archivo esta escrito alrededor de una regla: **pase lo que pase, la ventana se abre y explica lo que ocurre en castellano**. Si falta el modelo entrenado, si falta una libreria, si la foto es ilegible o si la prediccion revienta, el usuario ve un mensaje comprensible. Nunca una pantalla en blanco ni un texto tecnico incomprensible.

La función que hace la predicción: predecir

Aunque el usuario solo ve botones, por debajo hay una funcion que hace el trabajo. Se llama `predecir` y es la que se ejecuta cada vez que se pulsa **Enviar**.

```
def predecir(
    imagen: Any,
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
) -> tuple[dict[str, float], str]:
```

- `imagen` es la foto que subio el usuario. Si no hay ninguna, se devuelve el aviso `MSG_SIN_IMAGEN` ("Sube una imagen primero") en vez de fallar.
- `checkpoint_path` es el archivo del modelo entrenado.
- `topk` son cuantas especies se muestran (5).
- `min_conf` es el umbral de confianza, que aqui tiene un papel especial que se explica mas abajo.
- Devuelve dos cosas a la vez: un diccionario `especie -> probabilidad` que alimenta la etiqueta de barras, y un texto en formato Markdown que alimenta el recuadro de detalle.

El aviso de "falta el modelo entrenado"

```
    if not Path(checkpoint_path).exists():
        return {}, mensaje_entrena_primero(checkpoint_path)


def mensaje_entrena_primero(checkpoint_path: str) -> str:
    """Mensaje en Markdown que se muestra cuando falta el checkpoint."""
    return (
```

```
        f"{AVISO_ENTRENA}\n\n"
        f"Ruta buscada: `{checkpoint_path}`"
    )
```

- La comprobacion se hace **antes** de intentar nada: si el archivo del modelo no esta, ni se intenta predecir.
- El mensaje no solo dice que falta: dice como arreglarlo (`python -m src.train`) y en que ruta se estaba buscando el archivo. Esa ultima parte ahorra el caso tipico de "si lo entrene, pero en otra carpeta".
- El texto esta en Markdown, el formato de texto sencillo que Gradio sabe mostrar con titulos y negritas.

La decision importante: avisar en vez de borrar la prediccion

Esta es la parte del archivo que mas veces se cambio, y merece una explicacion larga porque es una leccion practica y no un detalle tecnico.

El problema real, visto en las pruebas: con un modelo poco entrenado, el `softmax` queda **muy plano**. El `softmax` es la operacion que convierte la salida del modelo en porcentajes que suman 100 %. Cuando el modelo esta bien entrenado, la especie correcta se lleva un porcentaje alto (por ejemplo 70 %). Cuando esta poco entrenado, reparte la confianza casi a partes iguales entre las 102 especies y su maximo ronda el 2 %. En modo debug (2 epocas de entrenamiento) el maximo queda en torno al 2 % aunque la especie propuesta sea la correcta.

Con el umbral puesto antes de mirar la lista, la demo respondia "no reconocida / fuera del catalogo" incluso cuando estaba acertando la especie, porque un 2 % nunca llega al 25 %. El usuario recibia un "no la reconozco" que era falso.

La solucion que se adopto: **la demo pide siempre el top-5 con umbral 0 y ensena la respuesta; despues, si la confianza es baja, anade un AVISO encima de la lista en lugar de borrarla.**

```
try:
    crudos = predict_image(imagen, checkpoint_path, topk=topk, min_conf=0.0)


confianza = max(prob for _, prob in resultados)
if confianza < min_conf:
    tabla = (
        f"**AVISO: confianza baja ({confianza * 100:.1f} % < {min_conf * 100:.0f} %).** "
        "La lista de arriba es la mejor suposicion, no una certeza. Puede ocurrir "
        "por dos motivos: (a) la flor no esta entre las 102 especies del catalogo, "
        "o (b) el modelo esta poco entrenado (por ejemplo si entrenaste en "
        "MODO_DEBUG, o si aviso de que no pudo descargar los pesos preentrenados).\n\n"
    ) + tabla
```

- La llamada usa `min_conf=0.0`: se le pide al motor de prediccion que no filtre nada, que devuelva las cinco mejores apuestas aunque su confianza sea minima.
- Despues, `confianza` es el porcentaje de la primera candidata, y si no llega al umbral se antepone un texto de aviso que dice exactamente los dos motivos posibles: o la flor no esta en el catalogo de 102 especies, o el modelo esta poco entrenado.
- La consecuencia para el usuario es que **siempre ve una respuesta util**: la mejor suposicion, su porcentaje, y la advertencia de que hay que desconfiar. Antes veia "no reconocida" y no aprendia nada; ahora ve "creo que es una petunia, pero con un 2 % de confianza, asi que tomalo con pinzas".

- El umbral cambia de significado: deja de ser un interruptor que borra y pasa a ser una senal de calidad. Por eso `--min-conf` sigue existiendo, pero ahora sirve para decidir a partir de que porcentaje no hace falta el aviso.

Todo lo demas se convierte en un mensaje, no en un error

```
except FileNotFoundError as exc:
    return {}, mensaje_entrena_primero(checkpoint_path) + f"\n\nDetalle: `{_celda(exc)}`"
except Exception as exc:
    return {}, (
        "**Error al predecir la imagen.**\n\n"
        f"`{type(exc).__name__}: {_celda(exc)}`\n\n"
        "Revisa que la imagen sea valida (jpg/png) y que el checkpoint "
        "corresponda al mismo backbone."
    )
```

- `except` significa "si ocurre este problema, haz esto en lugar de romperte". Aqui se capturan dos casos: que falte el archivo del modelo y cualquier otro fallo.
- En los dos casos la funcion devuelve un texto, no lanza el error hacia arriba. Por eso la ventana sigue viva y muestra la explicacion.
- El mensaje incluye el nombre tecnico del error y una recomendacion concreta: revisar que la imagen sea un jpg o png valido y que el modelo corresponda al mismo tipo de red.
- `_celda` limpia el texto para que no rompa la tabla en Markdown.

La lista de resultados y la tabla

```
resultados = _normalizar(crudos, topk)
if not resultados:
    return {}, MSG_SIN_RESULTADOS

etiquetas = {nombre: prob for nombre, prob in resultados}
tabla = tabla_markdown(resultados)
```

- `_normalizar` ordena las candidatas de mayor a menor probabilidad, descarta las que vienen mal formadas y se queda con las `topk` primeras. Existe porque la prediccion puede llegar en varios formatos y aqui se unifican.
- `etiquetas` es el diccionario que Gradio dibuja como barras de porcentaje.
- `tabla_markdown` construye la tabla de texto:

```
filas = [
    "| Especie | Probabilidad |",
    "| :--- | ---: |",
]
for etiqueta, prob in resultados:
    filas.append(f"| {_celda(etiqueta)} | {_fmt_pct(prob)} |")

ganadora = _celda(resultados[0][0])
encabezado = f"**Prediccion principal:** {ganadora} ({_fmt_pct(resultados[0][1])})\n\n"
return encabezado + "\n".join(filas)
```

- Se genera una tabla con dos columnas: el nombre de la especie y su probabilidad en porcentaje con dos decimales.
- Antes de la tabla se escribe en negrita cual es la prediccion principal, porque es el dato que la gente busca primero.

- Que existan las dos salidas (barras y tabla) es a proposito: las barras dan la impresion rapida de la confianza, la tabla da los numeros exactos.

build_app: construir la ventana

`build_app` monta toda la pantalla. No abre el servidor: solo deja la ventana preparada y la devuelve, para que sea el llamador quien decida cuando encenderla.

```
def build_app(
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
) -> gr.Blocks:


    checkpoints = str(checkpoint_path)
    falta_checkpoint = not Path(checkpoints).exists()

    def _fn(imagen: Any):
        return predecir(imagen, checkpoints, topk, min_conf)

    ejemplo = _imagen_de_ejemplo()

    with gr.Blocks(title=TITULO, fill_width=True) as demo:
        gr.Markdown(f"# {TITULO}")
        gr.Markdown(DESCRIPCION)
        if falta_checkpoint:
            gr.Markdown(AVISO_CHECKPOINT_UI.format(ruta=checkpoints))
```

- `falta_checkpoint` se calcula una sola vez, al construir la pantalla, y se usa despues para decidir si se ensena el aviso.
- `_fn` es la funcion concreta que se ejecutara en cada clic: recuerda la ruta del modelo, cuantas especies y que umbral se pidieron al construir la app.
- `gr.Blocks` es la pieza de Gradio que permite describir una pantalla por partes y colocarlas como se quiera.
- `ejemplo` busca la primera imagen que haya en las carpetas `data` o `assets` para ofrecer un ejemplo de un clic. Si no hay ninguna, no se muestra ese panel.
- El aviso de "no hay modelo entrenado" se coloca **dentro del diseno de la pantalla**, no como respuesta a un clic. Asi el usuario lo ve nada mas abrir, sin tener que probar.

La caja para subir o fotografiar la imagen

```
        with gr.Row():
            with gr.Column(scale=1, min_width=320):
                entrada = gr.Image(
                    label="Imagen de la flor",
                    type="pil",  # PIL.Image: formato aceptado por predict_image
                    sources=["upload", "webcam", "clipboard"],
                    height=380,
                )
                with gr.Row():
                    boton_enviar = gr.Button("Enviar", variant="primary")
                    boton_limpiar = gr.Button("Limpiar")
                if ejemplo:
                    gr.Examples(examples=[ejemplo], inputs=entrada)
```

- `entrada` es la caja de la imagen. Se admiten tres formas de dar la foto: **upload** (subir un archivo del ordenador), **webcam** (hacer la foto con la camara) y **clipboard** (pegarla desde el portapapeles). Cuantas mas vias, menos gente se queda fuera.
- `type="pil"` indica en que formato llega la imagen al programa: una imagen ya abierta en memoria, que es justo lo que sabe interpretar `predict_image` de la seccion anterior.
- **Enviar** lanza la prediccion; **Limpiar** vacia la caja y los resultados para probar otra foto. Tener un boton de limpiar evita que el resultado anterior se confunda con el nuevo.
- `gr.Examples` coloca una fila de ejemplos ya cargados; al pulsarla, la imagen se rellena sola.
- La pantalla se organiza en dos columnas dentro de una fila: a la izquierda la entrada, a la derecha los resultados. Se fija un ancho minimo de 320 pixels por columna para que en una pantalla pequena las columnas se apilen en lugar de aplastarse.

La etiqueta con las cinco especies y la tabla de detalle

```
with gr.Column(scale=1, min_width=320):
    salida_etiquetas = gr.Label(
        label=f"Top-{topk} especies mas probables",
        num_top_classes=topk,
    )
    salida_tabla = gr.Markdown(
        value=(
            AVISO_CHECKPOINT_UI.format(ruta=checkpoints)
            if falta_checkpoint
            else "Sube una imagen para ver el detalle de las top-k."
        ),
        label="Detalle del top-k",
    )
```

- `gr.Label` es el recuadro de barras: muestra hasta `topk` especies con su porcentaje. De un vistazo se ve cual gana y por cuanto.
- `gr.Markdown` es el recuadro de texto: muestra el aviso de confianza baja, la prediccion principal en negrita y la tabla especie-probabilidad.
- La tabla arranca con un texto de instrucciones ("Sube una imagen para ver el detalle de las top-k") en lugar de vacia, para que el usuario sepa que hay que hacer. Si falta el modelo, arranca directamente con el aviso.

Los botones que disparan la prediccion

```
boton_enviar.click(fn=_fn, inputs=entradas, outputs=salidas, api_name="predict")
entrada.upload(fn=_fn, inputs=entradas, outputs=salidas, api_name=None)
boton_limpiar.click(
    fn=lambda: (None, {}, "Sube una imagen para ver el detalle de las top-k."),
    inputs=None,
    outputs=[entrada, salida_etiquetas, salida_tabla],
    api_name=None,
)
```

- La primera linea conecta el boton **Enviar** con la funcion de prediccion: `inputs` son las entradas y `outputs` las salidas que se rellenan.
- La segunda hace que la prediccion salga **tambien al subir la imagen**, sin esperar al clic. Es comodidad pura: subir y ver el resultado a la vez.
- El boton **Limpiar** no predice nada: deja la caja vacia, las barras vacias y el recuadro con el texto de instrucciones.

- `api_name="predict"` publica la funcion de prediccion con un nombre estable en la direccion web de la demo, de modo que se puede llamar desde fuera. `api_name=None` en los atajos evita publicar automatismos internos que no aportan nada a nadie.
- Nota de implementacion: en la version de Gradio instalada (6.30.0) el evento `submit` ya no existe en los componentes, por eso se usa el clic del boton y la subida de la imagen. Esta desviacion esta documentada en la cabecera del archivo, junto con el motivo de que `build_app` devuelva un `gr.Blocks` en lugar del `gr.Interface` que preveia el contrato original: con `gr.Interface` no se puede dejar el aviso de "falta el checkpoint" fijo en la pantalla, y ese aviso se considero mas importante que la forma exacta del objeto devuelto.

main y cli: encender la ventana

`build_app` construye; `main` enciende. La separacion permite probar la pantalla sin abrir un servidor.

```
def main(
    share: bool = False,
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
) -> None:


    if not Path(checkpoint_path).exists():
        print(
            f"[aviso] No se encontro el checkpoint: {checkpoint_path}\n"
            "        La demo se abrira igual, pero mostrara el aviso "
            "'entrena primero: python -m src.train'.\n"
            "        Entrena con:  python -m src.train",
            flush=True,
        )
```

- Si falta el modelo, `main` no se niega a arrancar: avisa por consola, abre la ventana igual y deja que sea la propia pantalla la que explique la situacion.
- `flush=True` obliga a que el mensaje salga inmediatamente en pantalla. Sin eso, en algunos entornos el texto se queda retenido y el usuario cree que el programa se colgo.

```
    app = build_app(checkpoint_path, topk, min_conf)

    # En notebooks/Colab no bloqueamos la celda; en script el servidor sigue vivo.
    kwargs: dict[str, Any] = {"share": share, "prevent_thread_lock": _es_notebook()}

    # `inbrowser` no aporta nada en Colab.
    if "inbrowser" in inspect.signature(app.launch).parameters:
        kwargs["inbrowser"] = not _es_notebook()


    if url_local:
        print(f"Demo local:   {url_local}", flush=True)
    if share and url_publica:
        print(f"Demo publica: {url_publica}", flush=True)
    elif share:
        print(
            "[aviso] Se pidio --share pero Gradio no devolvio enlace publico "
            "(revisa la conexion a internet).",
            flush=True,
        )
    print("Pulsa Ctrl+C para detener la demo.", flush=True)
```

- `_es_notebook` detecta si el programa corre dentro de un cuaderno tipo Colab. En ese caso no se bloquea la ejecucion de la celda; en un programa normal el servidor se queda encendido hasta que se interrumpe con **Ctrl+C**, y eso es lo que se avisa al final.
- La direccion local se imprime siempre, para poder copiarla y abrirla. En las pruebas se levanto un servidor real y quedo escuchando en `127.0.0.1:7860`; `127.0.0.1` significa "este mismo ordenador" y `7860` es el numero de puerta que usa Gradio.
- Si se pidio enlace publico y no llego, se avisa igual en lugar de quedarse en silencio: normalmente significa que no hay conexion a internet.

Las opciones de la linea de comandos

```
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="Ruta del checkpoint del modelo (por defecto: resultados/best.pt).",
    )
    parser.add_argument(
        "--topk",
        type=int,
        default=5,
        help="Cuantas especies mostrar como maximo (por defecto: 5).",
    )


    parser.add_argument(
        "--min-conf",
        type=float,
        default=0.25,
        help=(
            "Umbral de confianza por debajo del cual la demo muestra un AVISO de "
            "confianza baja (por defecto: 0.25). Bajalo si tu modelo esta poco "
            "entrenado y quieres ver la prediccion sin el aviso."
        ),
    )
    parser.add_argument(
        "--share",
        action="store_true",
        help=(
            "Genera un enlace publico temporal de Gradio (util en Google Colab; "
            "requiere salida a internet). Por defecto es False."
        ),
    )
```

- El comando tipico es `python -m src.app` y la ventana queda disponible en la direccion local que se imprime.
- `--checkpoint` apunta al archivo del modelo; sirve para usar uno entrenado en otra carpeta.
- `--topk 5` decide cuantas especies se ensenan.
- `--min-conf 0.25` es el umbral a partir del cual ya no aparece el aviso de confianza baja. El propio texto de ayuda recomienda bajarlo si el modelo esta poco entrenado.
- `--share` genera un **enlace publico temporal**: Gradio levanta un tunel hacia internet y devuelve una direccion que cualquiera puede abrir desde su navegador durante un rato, sin estar en la misma red ni instalar nada. En Google Colab es imprescindible, porque la maquina que ejecuta el programa esta en la nube y su `127.0.0.1` no lo ve nadie. El comando alli es `python -m src.app --share`.
- `--share` es opcional y por defecto esta apagado: `share=False`. La razon es que publicar una direccion accesible desde internet es una decision consciente, no algo que deba pasar por descuido.

```
def cli(argv: list[str] | None = None) -> None:
    """Punto de entrada de `python -m src.app`."""
    args = parse_args(argv)
    main(
        share=bool(args.share),
        checkpoint_path=str(args.checkpoint),
        topk=int(args.topk),
        min_conf=float(args.min_conf),
    )
```

- `parse_args` lee lo que se escribio en la consola y `cli` se lo pasa a `main`. Son dos funciones separadas para que se pueda probar la lectura de opciones sin abrir ninguna ventana.

Por qué se hizo asi

- **Se uso Gradio y no una pagina web propia** porque Gradio ya trae hecha toda la parte dificil: subir archivos, usar la webcam, pegar desde el portapapeles, dibujar las barras de porcentaje, mostrar Markdown y servir la pagina en el ordenador. Escribir eso a mano habria costado muchisimo mas tiempo y habria anadido mucho mas codigo que revisar, sin ninguna ventaja para el objetivo del proyecto, que es que se vea el modelo funcionando.
- **La demo muestra siempre el top-5 y el umbral pasa a ser un aviso** por lo que se aprendio en las pruebas: con un modelo poco entrenado el `softmax` es plano, el maximo ronda el 2 %, y filtrar por umbral producia "no reconocida" incluso acertando la especie. Ensenar la respuesta con una advertencia siempre es mas util que ocultarla: el usuario ve la mejor suposicion, ve su porcentaje y sabe si puede fiarse.
- **El aviso de confianza baja explica las dos causas posibles** ("no esta en el catalogo" o "el modelo esta poco entrenado") porque desde fuera las dos situaciones se ven igual: porcentajes bajos. Sin esa explicacion, quien prueba la demo concluiria que el sistema no sirve, cuando en realidad puede estar funcionando con un modelo a medio entrenar.
- **La app no debe fallar nunca** porque una demo que se rompe delante de alguien que no programa no se vuelve a probar. Por eso se comprueba el checkpoint antes de predecir, se capturan los errores y se traducen a mensajes con instrucciones concretas, y se importa el motor de prediccion de forma perezosa (dentro de la funcion), de modo que la ventana se pueda abrir aunque falte una libreria.
- **La aplicacion se abre igual aunque falte el checkpoint**, mostrando el aviso dentro de la pantalla. La alternativa (negarse a arrancar) dejaria al usuario sin saber que pasa; asi ve la interfaz y lee exactamente el comando que tiene que ejecutar.
- **`share=False` por defecto** porque abrir un enlace publico en internet es una decision con consecuencias y no debe ocurrir porque alguien haya olvidado una opcion. En Colab se pide de forma explicita con `--share`, que es donde de verdad hace falta.
- **`main` y `build_app` estan separadas** para poder construir y probar la pantalla sin levantar un servidor: es lo que permite verificar que la app se arma correctamente en unos segundos, sin dejar un proceso escuchando en un puerto.
- **Se imprime siempre la direccion local y el recordatorio de Ctrl+C** porque quien usa la demo necesita saber donde mirar y como pararla; son dos frases que evitan muchas preguntas.
- **La desviacion respecto al contrato original esta documentada en la cabecera del archivo**: la version instalada de Gradio es la 6.30.0, en la que el evento `submit` desaparecio, y con `gr.Interface` no se podia dejar el aviso de "falta el checkpoint" fijo en la pantalla. Se prefirio una desviacion explicada y justificada a

mantener la forma prevista perdiendo funcionalidad visible para el usuario, y el objeto devuelto sigue cumpliendo lo esencial: tiene `.launch(...)`.

- **Se cachea el modelo y se usa un unico motor de prediccion compartido con la consola** para que la demo no duplique logica: la misma funcion que responde en la ventana es la que responde en el comando `python -m src.predict`. Asi lo que se prueba en la pantalla y lo que se mide en la evaluacion son exactamente lo mismo.

El cuaderno de Colab, las pruebas automáticas y el empaquetado

Este capítulo explica las tres piezas del proyecto que no son el programa en si, pero que hacen que el programa se pueda ejecutar y se pueda confiar en el: el **cuaderno de Colab** (el archivo que pulsas para que todo ocurra), las **pruebas automáticas** (el mecanismo que detecta que algo se ha roto) y el **empaquetado** (el archivo comprimido que se sube a la nube). También se explica el archivo `requirements.txt`, que es la lista de programas auxiliares que necesita el proyecto.

No hace falta saber programar para leerlo: cada término técnico se define la primera vez que aparece.

Qué es un cuaderno (notebook) y qué es una celda

Un **notebook** (cuaderno) es un documento que mezcla texto explicativo y código ejecutable en la misma página. El formato se llama **Jupyter**, y **Google Colab** es un servicio gratuito de Google que abre estos cuadernos en el navegador y ejecuta el código en un ordenador suyo, no en el tuyo. Eso es lo importante para este proyecto: no hay que instalar nada en tu ordenador, y el cuaderno puede usar una **GPU**.

Una **GPU** (unidad de procesamiento gráfico) es un componente pensado para dibujar imágenes que, resulta, también sirve para hacer los cálculos matemáticos de la inteligencia artificial mucho más deprisa que el procesador normal. **Colab** regala horas de una GPU concreta, la **T4**, y por eso el proyecto entero está pensado para ejecutarse ahí.

El cuaderno se divide en **celdas** (casillas). Hay dos tipos:

- **Celda de texto.** Contiene explicaciones escritas, títulos y tablas. No se ejecuta: solo se lee.
- **Celda de código.** Contiene instrucciones para el ordenador. Se ejecuta al pulsar el botón de reproducir que aparece a su izquierda, y la respuesta del programa (los "mensajes que imprime") aparece justo debajo de la celda.

La diferencia práctica es está: el texto te cuenta **qué** se va a hacer y **por qué**; la celda de código lo **hace**. El cuaderno de este proyecto es `notebooks/flores_colab.ipynb`. La

extensión `.ipynb` significa "cuaderno de IPython" y es el formato estándar de Jupyter y de Colab. El cuaderno tiene 26 celdas, de las cuales 13 son de código y 13 son de texto. Se ejecutan **en orden**, de arriba abajo, porque cada celda aprovecha lo que hizo la anterior: si te saltas una, la siguiente puede no encontrar lo que necesita.

Una aclaración útil: **no hay nada que memorizar de las celdas de código**. Están escritas y probadas; tu trabajo es pulsar "ejecutar" y, si algo se queja, leer el mensaje. Todo el código lleva comentarios en español explicando cada bloque.

Recorrido del cuaderno, paso a paso

Las celdas de código están numeradas como pasos del 1 al 10. Esto es lo que hace cada uno, en lenguaje llano.

Pasó 1 - Comprobar la GPU e instalar solo lo que falte

Antes de gastar tiempo, el cuaderno mira que el entorno este en condiciones. Lo primero es comprobar que existe una GPU disponible, con una orden que se llama `torch.cuda.is_available()`. Si la respuesta es "no", el cuaderno **no se rompe**: avisa y sigue, pero entrenar sin GPU tarda muchisimo más, así que lo razonable es activarla en el menu de Colab antes de continuar.

Después viene una decisión importante que se explica sola en una tabla del propio cuaderno: los **programas auxiliares**. Un programa de inteligencia artificial no se escribe desde cero: se apoya en bibliotecas, que son colecciones de código ya hecho. La celda hace tres cosas:

36. **Comprueba** si ya están las bibliotecas pesadas (`torch` y `torchvision`, que son el motor de cálculo) y **no las instala nunca**. Esto es deliberado y se explica en el apartado del `requirements.txt`.
37. **Instala solo lo que falta** de cuatro bibliotecas más ligeras: `gradio` (la pantalla interactiva), `scikit-learn` (las métricas), `matplotlib` (los gráficos) y `scipy` (para leer los ficheros del catálogo de flores). Si ya estaban, no se llama al instalador.
38. Imprime un **resumen de versiones**: que motor tiene, si hay GPU, cuánta memoria tiene.

```
torch y torchvision ya estan disponibles: no se tocan (regla de Colab).
Todo lo necesario ya estaba instalado: no se llama a pip.
=== Resumen del entorno ===
torch          : 2.14.1+cpu
CUDA disponible: False
```

Pasó 2 - Montar Google Drive (opcional) y guardar el dataset

Google Drive es el disco duro en la nube de Google; **montar** Drive significa "conectar" esa carpeta para que el programa pueda leer y escribir en ella como si fuera una carpeta normal. Este

pasó es **opcional** y sirve para dos cosas:

- **Que el trabajo no se pierda.** Colab borra su disco cuando te desconectas. Si el resultado del entrenamiento (el archivo `best.pt`, que es el **checkpoint**, o sea la "foto" del modelo ya entrenado) está copiado en Drive, sobrevive a la desconexion.
- **No volver a descargar los datos.** El catálogo de fotos ocupa unos 330 MB y, sin esta caché, Colab lo vuelve a descargar en cada sesión. Aquí se guarda en Drive una vez y se restaura al empezar.

Todo el pasó está envuelto en un mecanismo de protección (`try/except`, que significa "intenta esto y, si falla, no te caigas"): si Drive no está disponible, el cuaderno avisa y continua. Hay un interruptor al principio de la celda para decidir si quieres usar Drive o no.

Pasó 3 - Obtener el código del proyecto

Aquí el cuaderno se hace con el programa. Hay cuatro maneras y se eligen con una variable llamada `MODO_CODIGO`:

`MODO_CODIGO`	**Cuándo usarlo**	**Qué hace**
`"auto"` (el valor por defecto)	Siempre	Si el proyecto ya está en la máquina, lo usa; si no, pide subir el archivo comprimido
`"zip"`	Estas en Colab y no tienes el proyecto allí	Pide subir el ZIP, lo descomprime y busca dentro la carpeta del proyecto
`"clone"`	Tienes el proyecto publicado en GitHub	Se lo descarga desde internet
`"drive"`	Ya tienes la carpeta del proyecto en Drive	La copia a la zona de trabajo

En la práctica se deja `MODO_CODIGO = "auto"`, que es inteligente: busca una señal concreta (un archivo llamado `src/config.py`) y, si la encuentra, no te pregunta nada. Solo si no la encuentra te pide el ZIP. Al final la celda comprueba que el archivo clave existe y, si no, se detiene con un mensaje claro en lugar de fallar tres pasos más adelante de forma confusa.

```
MODO_CODIGO='auto': proyecto detectado en /content/IA_flores
Raiz del proyecto: /content
src/config.py    : OK
```

Pasó 4 - Imports del proyecto

"Importar" significa traer al cuaderno las piezas escritas en otros archivos, para poder usarlas.

Esta celda carga la configuración del proyecto, la lista de las 102 especies, la función que fija el azar para que los resultados sean reproducibles y la función que decide si se calcula en GPU o en el procesador. No hay nada que configurar: se ejecuta y dice "Imports del proyecto OK".

Pasó 5 - Configuración: modo debug y modo completo

Aquí se decide si la corrida es un **ensayo rápido** o la **corrida de verdad**. El cuaderno arranca siempre en **modo debug** (ensayo), y esa palabra merece una explicación: no significa "con errores", sino "en pequeño, para comprobar que todo funciona". La tabla del cuaderno lo resume así:

	`MODO_DEBUG = True` **(por defecto)**	`MODO_DEBUG = False`
Imágenes	unas pocas por especie (102 x 4 x 3)	1020 de entrenamiento, 1020 de validación y 6149 de prueba
Épocas	2	25
Precisión mixta	desactivada	activada si hay GPU
Salida	carpeta `resultados/debug`	carpeta `resultados`
Tiempo en GPU T4	1 o 2 minutos	15 a 25 minutos

Una **época** es una pasada completa por todas las fotos de entrenamiento; se dan varias pasadas porque el modelo mejora poco a poco. La **precisión mixta** es un truco para hacer los cálculos con menos decimales y así ir más rápido sin perder calidad apreciable.

El detalle importante es que el modo debug escribe en una carpeta distinta, así que **no pisa** el resultado bueno. La rutina recomendada es: primero debug para verificar que todo funciona, luego cambiar la variable y repetir para la corrida definitiva.

Pasó 6 - Entrenamiento

Entrenar es el proceso de aprendizaje: se le muestran al modelo las fotos con su nombre correcto, se mide cuánto se equivoca y se corrige. Esta celda lanza ese proceso y avisa de todo lo que va generando:

- `best.pt`: el checkpoint, o sea la copia del modelo en su mejor momento.
- `history.json`: el diario de la corrida, con la pérdida (cuánto se equivoca), el acierto y la velocidad de aprendizaje, época por época.
- `config_usada.json`: los ajustes exactos que se usaron, por si hay que repetir la corrida.
- `figuras/curvas.png`: un gráfico con la evolución de todo lo anterior.

La primera vez se descarga el catálogo de flores (330 MB) dentro de la carpeta `data/`; después se

reutiliza. Además, si Drive está montado, la celda copia los resultados a Drive **justo después de entrenar**, de modo que si Colab se cae un minuto más tarde el trabajo está a salvo.

El cuaderno también traduce a lenguaje claro un aviso técnico que puede aparecer: si no consigue bajar los pesos preentrenados desde internet, lo dice y explica que el entrenamiento sigue siendo válido, solo que parte de cero. Este aviso tiene su propia historia, contada en el capítulo de decisiones y errores.

Pasó 7 - Evaluación sobre el conjunto de prueba oficial

Evaluar es poner a prueba al modelo con fotos que **nunca vio** mientras aprendía. Si se evaluara con las mismas fotos del entrenamiento, el resultado sería engañoso: sería como darle a alguien el examen con las respuestas ya vistas. El catálogo de flores trae sus propias particiones oficiales, y aquí se usa la de prueba (6149 fotos).

Esta celda escribe:

- `metrics.json`: el resumen numérico. Los dos datos principales son el **top-1** (porcentaje de veces que la especie más probable era la correcta) y el **top-5** (porcentaje de veces que la respuesta correcta estaba entre las cinco primeras propuestas). También guarda el **macro-F1**, una nota que reparte el merito entre las 102 especies por igual, para que una especie con muchas fotos no tape a otra con pocas.
- `figuras/confusion_matrix.png`: la "matriz de confusión", una tabla de 102 x 102 que muestra que especies se confunden entre si, y `figuras/top_confusions.png`, un resumen de las confusiones más frecuentes.
- `predicciones/test_predicciones.csv`: la lista completa de fotos con lo que se predijo para cada una.

La segunda celda de este pasó imprime el resumen y el ranking de las especies peor clasificadas, y enseña la imagen de la matriz de confusión.

Pasó 8 - Resumen listo para pegar en el informe

Esta celda no calcula nada nuevo: **recoge** las cifras de los dos archivos anteriores y las presenta como un bloque de texto ya formateado, entre dos marcas visuales. Ese bloque se copia y se pega directamente en la sección de resultados del informe. Lo importante es que **no hay ningún número escrito a mano**: la tabla se rellena sola con lo que mida tu corrida, y si la corrida fue en modo debug, el propio bloque lleva un aviso recordando que esas cifras son de un ensayo. El bloque también se guarda en un archivo para no perderlo.

Pasó 9 - Predicción de una foto suelta y demostración interactiva

Una **predicción** es pedirle al modelo que diga que flor es. Hay dos formas aquí:

- **Foto suelta.** Se elige una foto del catálogo (o de la carpeta de recursos del proyecto) y el programa imprime las cinco especies más probables con su porcentaje.
- **Demostración interactiva.** Se construye una pantalla web sencilla (hecha con la biblioteca `gradio`) donde puedes arrastrar una foto tuya, o usar la cámara, y ver el resultado. Si todavía no existe el modelo entrenado, la pantalla no se cae: muestra el aviso de que hay que entrenar primero.

Sobre está demo el cuaderno avisa de dos cosas: que se puede generar un enlace público temporal (que conviene no dejar abierto con fotos privadas) y que se lanza en segundo plano para no dejar la celda ocupada. Hay incluso una celda auxiliar para apagarla si quieres liberar el recurso.

Pasó 10 - Empaquetar los resultados, descargarlos y cachear los datos

El último pasó comprime toda la carpeta `resultados/` en un archivo ZIP llamado `resultados_flores.zip`, lo descarga a tu ordenador y deja una copia en Drive. Además copia el catálogo de fotos a Drive para que la próxima sesión no vuelva a descargar los 330 MB. Si no hay Drive, todo esto se omite sin errores.

Al final imprime un inventario: una lista con lo que existe y lo que falta de todo lo que debía generarse. Es la comprobación final de que la corrida produjo lo que tenía que producir.

Checklist final

La última celda del cuaderno es una **lista de comprobación** escrita, con casillas. Reúne en un solo sitio lo que hay que verificar antes de lanzar la corrida definitiva, donde queda cada archivo, como reanudar el trabajo si Colab se desconecta, como descargar los resultados y cuánto tarda cada escenario. Es la página que conviene leer antes de ponerse a ejecutar.

Las pruebas automáticas

Una **prueba automática** (o test) es un pequeño programa cuyo único trabajo es comprobar que otra parte del programa hace lo que promete. Se ejecuta solo, sin que nadie mire, y responde "correcto" o "falló". La idea es sencilla: si cambias algo y rompes otra cosa sin darte cuenta, la prueba lo detecta en segundos, en lugar de que el error aparezca tres semanas después delante de un tribunal.

Las pruebas de este proyecto viven en la carpeta `tests/`. Son **78 pruebas** repartidas en seis archivos:

Archivo	Que comprueba, en una frase
`tests/test_data.py`	Que las fotos se cargan y se preparan bien: que el

	tamaño de los lotes es el correcto, que las etiquetas van del 0 al 101, que el entrenamiento recibe fotos variadas y la validación siempre las mismas, y que la lista de las 102 especies se obtiene incluso sin internet.
`tests/test_model.py`	Que el modelo se construye bien, que recibe una imagen y devuelve 102 respuestas, que se puede guardar en un archivo y volver a cargarlo dando exactamente el mismo resultado, y que se puede congelar y descongelar su parte preentrenada.
`tests/test_train.py`	Que el entrenamiento termina, guarda sus archivos y sigue las reglas pactadas: la forma de la velocidad de aprendizaje, que se congela y se descongela la parte preentrenada en el momento previsto, que la parada temprana corta a tiempo y que dos corridas con la misma semilla dan el mismo resultado.
`tests/test_eval_predict.py`	Que la evaluación escribe las métricas, las figuras y el listado de predicciones, que falla con un mensaje claro si falta el modelo entrenado, y que la predicción devuelve resultados ordenados, recorta bien el número de propuestas y respeta el umbral de confianza.
`tests/test_smoke.py`	Que el recorrido completo funciona encadenado (cargar datos, modelo, entrenar, evaluar, predecir) sobre un conjunto de imágenes inventadas, sin internet y en unos segundos.
`tests/conftest.py`	No es una prueba, es el preparador común: crea las carpetas temporales que usan las demás pruebas, con un ajuste especial para está máquina concreta.

Para que sirven, dicho en una frase: **son el detector de humo del proyecto**. Si alguien toca el código y rompe la carga de datos, o el guardado del modelo, o el cálculo del umbral, lo sabrá en un minuto sin tener que revisar nada a mano. Y si una prueba se queda en verde después de un cambio, eso es la evidencia de que el cambió no rompió lo que ya funcionaba.

Se ejecutan con el comando siguiente, desde la carpeta del proyecto:

```
.venv\Scripts\python.exe -m pytest tests -q
```

La respuesta termina con una línea parecida a `78 passed`, que significa "78 pruebas superadas".

Las pruebas que necesitan el catálogo real de 330 MB (las más lentas) solo se ejecutan si se avisa antes con una variable de entorno; si no, se saltan para no hacer esperar. Su propio texto lo dice: si falta un programa auxiliar, algunas pruebas se saltan en lugar de fallar, de modo que un `78 passed` sin haber preparado el entorno no es exactamente lo mismo que un `78 passed` completo. Por eso el registró de verificación detalla con que ajustes se obtuvo cada recuento.

El empaquetado: empaquetar_colab.py

Un **ZIP** es un archivo comprimido: un solo archivo que contiene muchos otros, ocupando menos espacio, como una caja donde metes el proyecto entero para transportarlo. Es la forma práctica de llevar el proyecto a Colab, porque subir carpetas sueltas por la pantalla del navegador es lento y propenso a errores.

El programa `empaquetar_colab.py` construye ese archivo. Se ejecuta con:

```
.venv\Scripts\python.exe empaquetar_colab.py
```

Produce `IA_flores_colab.zip` en la carpeta del proyecto. Lo más interesante no es lo que incluye, sino **lo que deja fuera**:

- **La carpeta `data/` con las 330 MB de fotos.** No viaja en el ZIP: es enorme, es descargable y además no es nuestra (las fotos pertenecen a sus autores y se distribuyen para uso académico).
- **La carpeta `resultados/`.** Son resultados generados por una corrida concreta; el ZIP debe contener el programa, no sus resultados.
- **Los restos de ejecuciones anteriores**: las carpetas de caché del propio Python y los archivos que se regeneran solos.

Lo que si incluye es el código, las pruebas, el cuaderno, la documentación, los recursos del proyecto y los archivos de configuración.

Para decidir esto se usa una **lista blanca**: en lugar de recorrer todo el proyecto y quitar lo que no quieres, se escribe **explícitamente** la lista de lo que si quieres. Es una decisión de seguridad y de robustez, y tiene dos motivos que el propio programa documenta:

39. Con una lista blanca es imposible que se cuelen por descuido los 330 MB de fotos o los resultados de una corrida.
40. En el ordenador de desarrolló, algunas carpetas temporales no se pueden ni leer por una restricción del sistema; recorrerlo todo fallaría. La lista blanca ni las toca.

El programa no se limita a crear el ZIP: después **lo vuelve a abrir para comprobarlo**. Verifica que las piezas clave están dentro (el archivo de configuración, los módulos, el cuaderno, la lista

de dependencias) y que ninguna entrada está corrupta. Si falta algo, lo dice y termina con un aviso de error en lugar de dejarte un paquete incompleto.

```
ZIP creado: IA_flores_colab.zip  (160 KB)
Archivos incluidos: 28

Comprobacion de la estructura que espera el notebook:
  OK   src/config.py
  OK   src/utils.py
  OK   src/data.py
  OK   src/model.py
  OK   notebooks/flores_colab.ipynb
  OK   requirements.txt

RESULTADO: estructura correcta
```

Un dato curioso que merece mención: el proyecto entero, sin las fotos ni los resultados, cabe en unos 160 KB, es decir, menos de una décima parte de una sola foto sin comprimir de muchos móviles. Un informe de verificación anterior registró 25 archivos y 142 KB; la cifra creció después, porque el propio documento que estas leyendo viaja dentro del paquete. Los archivos de datos son los que pesan, no el programa.

El archivo requirements.txt

Un archivo `requirements.txt` es una **lista de la compra de programas auxiliares**. Cada línea indica un nombre y la versión mínima que hace falta, por ejemplo la primera línea dice que se necesita una versión de `torch` igual o posterior a la 2.4. Sirve para que cualquiera pueda preparar un ordenador nuevo con un solo comando y obtener las mismas piezas:

```
pip install -r requirements.txt
```

Ese comando se llama `pip` y es el instalador de programas auxiliares de Python. Aquí aparece la **trampa más importante de todo el proyecto**, y está escrita con mayúsculas en el propio archivo y repetida en el cuaderno:

*En Colab **no** se debe ejecutar ese comando. Colab ya trae `torch` y `torchvision` preparados para funcionar con la GPU. Si se reinstalan desde `pip`, es muy probable que se sustituyan por versiones para procesador normal y la GPU deje de servir. La solución es instalar únicamente lo que falte, que es exactamente lo que hace el pasó 1 del cuaderno. Si aún así se rompe, hay que reiniciar el entorno de Colab.*

El archivo cumple entonces dos papeles distintos según donde estés: en un ordenador propio es la lista de la compra completa; en Colab es una **referencia** de lo que el proyecto usa, pero no se ejecuta tal cual. Las anotaciones que acompañan a algunas líneas explican para que sirve cada pieza

y por que se exige una versión mínima concreta, por ejemplo que la biblioteca de la pantalla interactiva necesita una versión moderna porque en las antiguas faltaba una pieza que el proyecto usa.

Resumen del capítulo

- El **cuaderno** `notebooks/flores_colab.ipynb` es el guion: 26 celdas, 13 de ellas de código, que recorren el proyecto en 10 pasos, empiezan en modo debug para verificar rápido y terminan descargando los resultados.
- Las **78 pruebas automáticas** de `tests/` vigilan que nada se rompa en silencio, sin necesidad de revisar el programa a mano.
- El **empaquetado** `empaquetar_colab.py` construye un ZIP con lista blanca que deja fuera las 330 MB de fotos y los resultados, y que se autocomprueba después de crearse.
- El **`requirements.txt`** es la lista de dependencias, con el aviso crítico de no reinstalar `torch` dentro de Colab.

Parte III. Las decisiones y los errores

Por qué se hizo así: las decisiones y los errores del camino

Este capítulo no explica **que** hace el proyecto, sino **por que** se decidió cada cosa y **que salió mal** por el camino. Es el capítulo más útil de todo el documento, por una razón sencilla: los aciertos se cuentan solos, pero los errores solo enseñan si alguien los cuenta. Al final hay también una lista honesta de lo que **no** se ha podido verificar.

Cada decisión se presenta con el problema que había encima de la mesa, la alternativa que se descartó y el motivo.

1. Por qué se usaron las flores de Oxford y no fotos propias

El problema. Para que el programa aprenda a distinguir especies hace falta un montón de fotos ya etiquetadas con su nombre correcto. Una persona sola puede hacer, con mucha paciencia, cien o doscientas fotos de su jardín. Un programa de este tipo necesita miles.

La alternativa descartada. Hacer las fotos uno mismo. Se descartó por tres motivos que se refuerzan entre si:

- **Cantidad.** Con doscientas fotos de veinte especies el programa no aprende: memoriza. Aprender a distinguir 102 especies requiere miles de ejemplos.
- **Calidad de las etiquetas.** Cada foto habría que etiquetarla a mano, y equivocarse en la etiqueta de una sola foto engaña al programa. Existen flores muy parecidas entre si, y una persona sin formación botánica dudaría.
- **Comparabilidad.** Aunque se hicieran las fotos, no habría con que comparar el resultado: nadie más tendría ese conjunto de fotos, así que la cifra de acierto no diría nada a nadie.

La decisión. Usar **Oxford Flowers-102**, un catálogo público creado por investigadores de la Universidad de Oxford, con 8189 fotografías de 102 especies de flores comunes en el Reino Unido. Tiene tres virtudes que resuelven los tres problemas: es grande, viene ya etiquetado y es un catálogo conocido y usado en publicaciones científicas, así que los resultados se pueden comparar con los de otros.

El precio que se paga, y se dice en voz alta. El catálogo son fotos de catálogo, en su mayoría hechas en el Reino Unido, con fondos e iluminaciones parecidos. Fotos propias, con fondo cargado, de noche o de una variedad distinta, pueden salir mucho peor. Que el programa acierte mucho con este catálogo **no garantiza** que acierte con tus fotos. Validarlo con fotos propias quedó como

trabajo futuro, no se hizo.

2. Por qué PyTorch y no TensorFlow

El problema. Existen varias "cajas de herramientas" para construir inteligencia artificial. Las dos grandes son **PyTorch** y **TensorFlow**. Una caja de herramientas, en este contexto, es un conjunto de piezas ya escritas que te ahorra programar desde cero las cuentas matemáticas. Había que elegir una.

La decisión. Se eligió **PyTorch**. Los motivos, en orden de importancia práctica:

- **Tiene ya preparado el catálogo de flores y muchos más.** La biblioteca hermana de PyTorch, TorchVision, incluye una pieza que descarga Flowers-102 y reparte las fotos en sus particiones oficiales con una sola orden. Con la otra herramienta ese trabajo habría habido que hacerlo a mano.
- **Tiene ya preparado el modelo preentrenado** que se quería usar, con sus pesos ya entrenados descargables, y también modelos alternativos para comparar sin cambiar apenas código.
- **La pantalla interactiva de demostración (Gradio) se lleva muy bien con PyTorch**: se puede enseñar directamente un modelo en una página web sin escribir una aplicación entera.
- **Está pensado para leerlo.** Está escrito de forma más parecida al lenguaje normal, lo que importa mucho en un proyecto cuyo código tiene que entender otra persona, o el propio autor meses después.

Lo que no se hizo. No se comparó experimentalmente con TensorFlow. La elección se tomó por razones de utilidades disponibles y de facilidad de lectura, no por una medición de velocidad. Es una elección razonada, no un resultado experimental, y por eso se presenta como tal.

3. Por qué se usó una red ya entrenada y no una red construida desde cero

El problema. El conjunto de entrenamiento tiene 1020 fotos para 102 especies: **diez fotos por especie**. Una red de este tipo tiene que aprender a reconocer pétalos, texturas y formas, y diez ejemplos no alcanzan para aprenderlo todo desde el principio.

La alternativa descartada. Construir la red desde cero, con pesos iniciales al azar, y enseñarle todo con esas diez fotos por especie. Se descartó porque la experiencia dice que, con tan pocos datos, la red memoriza las diez fotos concretas en lugar de aprender el concepto de la especie. Aprende a reconocer "está foto", no "está flor". Además necesitaría muchisimo más tiempo de cálculo.

La decisión. Usar **aprendizaje por transferencia**, que se explica con una comparación: en

lugar de enseñar a alguien a mirar flores desde cero, se parte de una persona que ya sabe mirar imágenes en general, porque ha visto millones de ellas, y solo se le explica la parte de flores. En términos técnicos, se toma una red ya entrenada para clasificar un millón de imágenes de mil categorías distintas (un catálogo llamado **ImageNet**) y se aprovecha su partes iniciales, que detectan bordes, manchas y texturas: eso es útil para cualquier imagen, no solo para las del catálogo original.

La red elegida se llama **EfficientNet-B0**, una red relativamente pequeña y muy eficiente, adecuada para la GPU gratuita de Colab y para entrenar en minutos. Sobre ella se pone una **cabeza nueva**: una última capa, propia de este proyecto, que aprende a repartir lo que la red ve entre las 102 especies. La cabeza se inicializa al azar y es lo que realmente se aprende en este proyecto.

Traducción honesta. El proyecto no enseña a una red a ver flores desde cero: aprovecha una que ya sabe ver imágenes y le enseña la última parte del trabajo. Eso es una técnica estándar, no un atajo vergonzoso, y se documenta como tal.

4. Por qué congelar la parte preentrenada 3 épocas y luego descongelarla con una velocidad menor

El problema. Si desde el primer momento se permite ajustar **toda** la red, incluidas las partes preentrenadas, los primeros cálculos son muy malos (la cabeza nueva empieza al azar y da respuestas absurdas). Esos errores, aplicados sobre las partes que ya eran buenas, las estropean. Es como pedirle a un fotógrafo experto que se fije en las correcciones de alguien que acaba de empezar: acabaría dudando de lo que hacía bien.

La decisión, en dos etapas. Se llama **congelar** a impedir temporalmente que una parte de la red se modifique, y **descongelar** a volver a permitirlo.

41. **Las 3 primeras épocas se congelan** las partes preentrenadas: se usan tal cual, solo como "vista", y únicamente se entrena la cabeza nueva. Así la cabeza aprende a ordenar lo que la red ya sabe ver, sin estropear nada.
42. **A partir de la cuarta época se descongela todo** (la red completa, no solo sus últimos bloques) pero con una **velocidad de aprendizaje menor**. Esa velocidad (el "learning rate" o tasa de aprendizaje) es el tamaño del pasó con que la red corrige sus errores. La cabeza lleva un pasó de 0,0003 y las partes preentrenadas un pasó diez veces más pequeño, 0,00003: se afinan con cuidado, sin deshacer lo aprendido.

El cuaderno y el informe dejan escrito este número de épocas (3) y estas dos velocidades, para que la corrida se pueda repetir. También hay una prueba automática que comprueba que la secuencia real

es exactamente "congelado, congelado, congelado, libre, libre, libre": es uno de los pactos que vigilan las pruebas, porque es fácil equivocarse al programar esto.

Un matiz que se corrigió. En una versión inicial de la documentación se decía que al descongelar se liberaban "los últimos bloques". El código hace otra cosa: libera **toda** la parte preentrenada. Se detectó al revisar, y el texto se corrigió para que dijera la verdad. Es un ejemplo pequeño pero real de por que conviene leer el código antes de describirlo.

5. Por qué se aumentaron los datos y por qué se suavizaron las etiquetas

Aquí hay dos técnicas distintas que atacan el mismo problema: **con diez fotos por especie, la red tiende a memorizar**. Y dos medidas para evitarlo.

Aumentar los datos. Aumentar los datos (aumentó de datos, o "data augmentation") significa fabricar variantes de cada foto de entrenamiento: recortes, giros, cambios de color y de brillo. La especie no cambia, pero la red ve muchas más versiones de la misma flor. Sirve para dos cosas: hay más material con el que practicar y, sobre todo, la red se ve obligada a no depender de detalles accidentales (el fondo, el encuadre). Una flor vista de otra manera sigue siendo la misma flor, y eso es precisamente lo que se quiere que aprenda.

El detalle no es casual: estas variantes **solo se aplican al entrenamiento**. Para validar y para medir el resultado final, las fotos se preparan siempre igual, sin azar. Si se aplicaran variantes al evaluar, el resultado cambiaría de una vez a otra y no se podría comparar nada.

Suavizar las etiquetas. Suavizar las etiquetas ("label smoothing") consiste en no exigirle a la red que este **completamente segura**. Normalmente, cuando se le muestra una foto se le dice "la respuesta correcta es la especie 47 y todo lo demás está mal". Con el suavizado se le dice algo más realista: "es casi seguro la 47, pero deja un margen pequeñísimo a las demás". Eso tiene dos efectos buenos:

- La red se vuelve menos fanática y no se sobreajusta tanto, porque no puede alcanzar la perfección absoluta y sigue corrigiendo.
- Sus probabilidades son **más honestas**. Esto es decisivo para el umbral de confianza que se explica más abajo: si la red dijera "99 por ciento" incluso cuando duda, el umbral no serviría de nada.

El resultado de estas dos medidas, junto con la parada temprana (dejar de entrenar cuando el resultado de validación deja de mejorar durante 6 épocas seguidas, para no dar vueltas de más), es el conjunto de defensas contra el sobreajuste. Sobreajustar significa aprenderse de memoria el

material de entrenamiento y fallar con material nuevo: es el error clásico de este tipo de proyectos.

6. Por qué se usaron las particiones oficiales (y la salvedad de las fotos repetidas)

El problema. Las fotos hay que repartirlas en dos grupos: uno para entrenar y otro para evaluar. Si se evalua con las mismas fotos con las que se entreno, el resultado es un engaño: sería como aprobar un examen que ya tienes corregido.

La decisión. Usar las **particiones oficiales** del catálogo, que vienen ya definidas por sus autores: 1020 fotos para entrenar, 1020 para validar (que es un grupo intermedio para ir vigilando si mejora) y 6149 para la prueba final. Los motivos:

- **Comparabilidad.** Cualquier persona que use Flowers-102 usa estas mismas particiones, así que las cifras de este proyecto se pueden poner al lado de las publicadas por otros.
- **Reproducibilidad.** No hay que decidir ni sortear quien va a cada grupo: cualquiera obtiene exactamente el mismo reparto con el mismo comando.

La salvedad, contada sin adornos. Se hizo una comprobación por contenido: se cálculo una huella digital de cada una de las 8189 imágenes (una huella digital, o suma de verificación, es un código único que resume el contenido de un archivo: dos archivos idénticos dan el mismo código). Resultado: hay **4 pares de fotos idénticas**, y **2 de esos pares cruzan particiones** (uno pone la misma foto en entrenamiento y en prueba, y otro en validación y en prueba).

Qué significa. Durante la evaluación final, dos de las 6149 fotos de prueba son fotos que el modelo ya vio en el entrenamiento. Son 2 imágenes sobre 8189, aproximadamente un **0,02 %**, una cifra irrelevante para el resultado global. Pero la afirmación que se había escrito inicialmente (que las particiones separaban las fotos "por planta fotografiada", es decir, que ninguna planta aparecía en dos grupos) **resultó falsa** y se retiró.

Que se hizo. Documentarlo. El informe ahora dice literalmente que hay cuatro pares de imágenes idénticas byte a byte y que dos cruzan particiones, y que eso invalida cualquier afirmación de separación estricta. El defecto **no es del programa**: es del catálogo oficial, que publica sus particiones como listas de números sin comprobar repeticiones de contenido. Lo relevante para quien lea esto es que se encontró, se midió y se cuenta, en lugar de ocultarlo. Una fuga de datos silenciada sería un problema mucho más grave que dos fotos repetidas.

7. Por qué existe un umbral de confianza y por que acabó convertido en un aviso

El problema. El catálogo tiene 102 especies y el modelo solo conoce esas 102. Si le enseñas la

foto de un coche, o de una flor que no está en el catálogo, el modelo igualmente elegira una de sus 102 especies, porque es lo único que sabe hacer. Eso es peor que no responder: da una respuesta segura y equivocada.

La solución. Un **umbral de confianza**: si la probabilidad de la especie más probable no llega a 0,25 (un 25 %), el sistema no responde ninguna especie y dice **"no reconocida / fuera del catálogo"**. La idea es que, ante algo desconocido, las probabilidades se reparten entre muchas especies y la más alta queda baja: esa es la señal de duda que se aprovecha.

El giro inesperado. Más adelante se descubrió que este umbral, tal como estaba colocado, **estaba rompiendo la demostración**: respondía "no reconocida" casi siempre. El motivo está explicado en el apartado de errores; lo importante aquí es como se corrigió el diseño.

La decisión final. El umbral dejó de ser una censura para convertirse en un **aviso**. Ahora la demostración **siempre** enseña las cinco especies más probables con su porcentaje, y **además** añade un mensaje de advertencia cuando la confianza no llega al umbral: "confianza baja, puede que está flor no este en el catálogo, o que el modelo este poco entrenado". Es más útil así, porque el usuario ve toda la información y decide; antes, el sistema decidía por el y le ocultaba el resultado. El umbral sigue siendo configurable, así que quien quiera el comportamiento estricto puede activarlo.

8. Por qué todo se comprueba con pruebas automáticas y por que hubo una auditoría adversarial

Las pruebas automáticas. Cada pieza del programa tiene sus pruebas: se ejecutan solas, en minutos, y responden "correcto" o "falló". Son **78**. La razón de tenerlas es muy concreta: alguien que cambia una línea para arreglar una cosa rompe otra sin enterarse, y el falló aparece mucho después, en el peor momento. Con pruebas, ese falló aparece en segundos. Un detalle honesto: hay partes que las pruebas **no** cubren, y eso también está documentado. Por ejemplo, la pantalla interactiva de demostración no tiene ni una prueba automática, aunque se verificó a mano.

La auditoría adversarial. Aquí está la decisión más interesante del proyecto. Se creo un papel específico: una persona (un revisor independiente) cuyo trabajo **no** era escribir ni mejorar nada, sino **intentar demostrar que lo que afirmabamos era falso**. Es lo contrario de la actitud habitual, y precisamente por eso funciona.

Sus instrucciones no eran "revisa que esto este bien", sino "intenta refutar estas diez afirmaciones". Entre otras cosas:

- Reproducir el recorrido completo **desde cero y con otra configuración**, para comprobar que el

resultado no depende de una casualidad.

- Comprobar de forma independiente que no había **fuga de datos** (fotos compartidas entre el grupo de entrenamiento y el de prueba), sin fiarse de nuestro código: leyendo los ficheros oficiales del catálogo por su cuenta y calculando las huellas de las 8189 imágenes.
- Comprobar si cada modelo guardado había usado **realmente** los pesos preentrenados, comparando la primera capa de cada uno con los pesos originales de ImageNet.
- Ejecutar el cuaderno entero y comprobar que no cambiaba mientras se ejecutaba.
- Buscar rutas absolutas del ordenador del autor en todo lo que se entrega (un descuido clásico que rompe el proyecto en otra máquina y además revela información privada).

El resultado fue **valioso por incordioso**. De las diez afirmaciones, seis se confirmaron y cuatro quedaron refutadas o parcialmente refutadas. Dos de esas refutaciones eran **errores nuestros**, y están contados en el apartado de errores. También apareció algo que nadie había notado: ningún archivo guardaba si los pesos preentrenados se habían cargado de verdad.

La lección, dicha sin rodeos: revisar el propio trabajo con buena voluntad encuentra mucho menos que encargarle a alguien la tarea de destruirlo. La única condición es que después haya que hacerle caso, y en este proyecto se hizo.

9. Por qué el código "degrada" en lugar de fallar

El problema. El proyecto tiene que funcionar en sitios muy distintos: en Colab con GPU, en un ordenador normal sin GPU, sin conexión a internet, o en un ordenador con restricciones de seguridad como el que se usó para desarrollarlo. Hay cuatro cosas que **pueden** faltar: internet, GPU, los procesos auxiliares de carga de datos y el modelo entrenado. Si el programa se cayera cada vez que falta una de ellas, sería inútil la mitad de las veces.

La decisión. Se eligió **degradar**: cuando falta algo, el programa **avisa** y sigue funcionando como puede, en lugar de detenerse. Degradar es una palabra técnica que aquí significa "seguir funcionando con menos medios y diciéndolo". Los cuatro casos:

- **Sin internet.** No se pueden descargar las 102 versiones de los pesos. En lugar de caerse, el programa entrena igualmente partiendo de pesos al azar, **avisando de que lo está haciendo**. No es lo ideal, pero es infinitamente mejor que no hacer nada.
- **Sin GPU.** Todo el cálculo se hace en el procesador normal. Es mucho más lento, pero funciona y el resultado es el mismo. El código no tiene ninguna orden que exija GPU obligatoriamente.
- **Sin procesos auxiliares de carga de datos.** El programa reintenta una vez de otra manera y avisa. Se explica en el apartado de errores.

- **Sin modelo entrenado.** Si le pides predecir y no hay modelo, no se cae: responde con un mensaje claro que además dice **que comando ejecutar** para generarlo. Y la pantalla de demostración muestra el aviso "entrena primero" en lugar de quedarse en blanco.

El límite de esta idea, dicho con honestidad. Degradar en silencio es peligroso, porque uno puede creer que obtuvo el resultado bueno cuando en realidad obtuvo el resultado malo. Este proyecto intentó poner el aviso lo más visible posible, pero el apartado de errores cuenta un caso en el que el aviso **no se vio** y provocó un malentendido serio. La lección que se saca de ahí es que un aviso escondido entre el ruido de la pantalla no es un aviso.

10. Los errores reales que aparecieron y cómo se resolvieron

Estos son los errores de verdad, contados en el orden en que aparecieron, con el síntoma, la causa y la solución. Ninguno de ellos es un error de concepto: son los típicos tropiezos de llevarlo a la práctica.

10.a La instalación de librerías fallaba y se resolvió con otra herramienta

El síntoma. Al preparar el entorno del ordenador de desarrolló, la instalación de las bibliotecas fallaba con errores de permisos: el sistema decía "acceso denegado" y no dejaba continuar. Al principio parecía un problema del instalador.

La causa. Se investigó hasta el fondo y resultó ser una limitación del entorno de seguridad de esa máquina concreta. El instalador habitual (`pip`) crea una carpeta temporal con unos permisos que, en ese entorno, la dejan sin acceso después, y entonces falla al usarla. Curiosamente, otras herramientas crean la carpeta de forma distinta y no tropiezan con eso.

La solución. Instalar con **otra herramienta** que no usa ese tipo de carpeta temporal, y dejar el comando anotado en la documentación del proyecto para que nadie pierda una tarde con el mismo problema. **Esto no afecta a Colab**, donde el problema no existe. Es un ejemplo de un detalle que depende de la máquina, no del programa, y por eso está documentado aparte: quien lea el error en su ordenador sabrá que no ha hecho nada mal.

10.b Los procesos auxiliares de carga de datos fallaban en Windows

El síntoma. El entrenamiento se paraba de golpe, a mitad, con un mensaje de "acceso denegado" justo cuando empezaba a leer las fotos. En cambió, con las mismas fotos, a veces funcionaba.

La causa. Para leer las imágenes rápido, el programa puede usar **varios lectores a la vez** (fotogramas de trabajo paralelos, o "workers"). Esos lectores se comunican con el programa principal por un canal interno del sistema operativo, y en Windows, bajó determinadas condiciones de seguridad, ese canal está bloqueado. Al leer las fotos el programa moría.

La solución. En lugar de intentar adivinar de antemano si se puede, el programa **reintenta una sola vez con un único lector**. Es decir: intenta la vía rápida, y si falla, cambia sola a la vía lenta y sigue, avisando de que lo hizo. Lo importante es que **no se pierde el entrenamiento** que ya estaba en marcha. Además se programaron dos pruebas automáticas para esto: una comprueba que el reintento ocurre cuando fallan los lectores, y otra que **no** se reintenta cuando el error es de otro tipo (para no ocultar problemas reales tapándolos con un reintento). En Colab y en Linux los lectores funcionan bien y esta protección ni se activa.

10.c Los pesos preentrenados no se descargaban y el modelo entrenaba "en blanco"

Este fue el error más engañoso de todos.

El síntoma. Las corridas de prueba daban un acierto bajísimo, del orden del 1 %, que es exactamente lo que acertaría alguien que respondiera al azar (con 102 opciones, una entre 102 es aproximadamente 1 %). Parecía que el modelo **no aprendía nada**, y llevó a dudar del diseño entero: de si el aprendizaje por transferencia servía, de si el número de épocas era el adecuado, de si el problema era demasiado difícil.

La causa. En el ordenador de pruebas, la carpeta donde la biblioteca guarda los pesos descargados no se podía escribir. Por eso la descarga fallaba, y el programa, fiel a su política de degradar en lugar de caerse, **entrenaba partiendo de pesos al azar** ("en blanco"): exactamente la red desde cero que se había decidido evitar. Y aquí está lo grave: **el aviso existía, pero se perdía** entre los cientos de líneas de mensajes normales del entrenamiento. Además, los archivos que guardaba el programa declaraban "use pesos preentrenados: si", porque registraban lo que se había **pedido**, no lo que realmente había ocurrido. O sea: parecía que todo iba bien y el resultado era malo.

Como se descubrió. Lo encontró la auditoría adversarial, y de una forma muy elegante: comparó la primera capa de cada modelo guardado con los pesos originales de ImageNet. Si el modelo venía de ImageNet y solo se había ajustado unas épocas, la diferencia debía ser minúscula; si había empezado al azar, la diferencia tenía que ser del tamaño de los propios pesos. Los números fueron concluyentes: tres de los modelos guardados tenían **pesos aleatorios** mientras declaraban lo contrario.

La solución, en tres partes.

43. **Entender el alcance real.** El problema era del **entorno de pruebas**, no del programa, y en Colab la descarga de pesos funciona normalmente. Eso evito una falsa alarma mayor.
44. **Hacer el aviso imposible de perder.** El cuaderno ya no deja el mensaje técnico crudo: lo

detecta, lo traduce a lenguaje claro y lo imprime destacado, explicando que el entrenamiento sigue siendo válido pero que ha partido de cero.

45. **Registrar la verdad, no la intención.** Se anoto como mejora pendiente guardar en los archivos de resultado el valor **real** de si los pesos se cargaron, no el que se pidió, y mostrarlo en pantalla. Es la corrección de fondo: el error no fue que fallara la descarga, fue que el programa informaba de algo que no había comprobado.

La lección. Un aviso que nadie ve no es un aviso, y un dato que dice lo que pediste en lugar de lo que pasó es peor que no tener dato.

10.d El cuaderno no arrancaba por un permiso del sistema

El síntoma. Al intentar ejecutar el cuaderno **como cuaderno de verdad** (es decir, con el programa que abre un cuaderno y va ejecutando sus celdas una a una), el intentó fallaba antes de empezar, con un error de permisos: el sistema no dejaba crear el proceso interno que ejecuta el código.

La causa. Otra vez una restricción del entorno de seguridad del ordenador de desarrolló, no del proyecto. Ese proceso interno necesita un canal del sistema operativo que ahí está bloqueado.

La solución. Se construyó un **sustituto**: un programa propio que lee el cuaderno, extrae sus celdas de código y las ejecuta en el mismo orden, en el mismo espacio de trabajo, deteniendose en el primer error. Con el se comprobó que las 13 celdas de código se ejecutan de principio a fin, en poco más de minuto y medio (en modo debug), sin errores y sin modificar el cuaderno. La honestidad obliga a decir lo que ese sustituto **no** cubre: atajos propios de los cuadernos, reinicios del entorno interno ni las funciones exclusivas de Colab (subir archivos, descargar, conectar Drive). Por eso esto aparece en la lista de cosas no verificadas: el cuaderno se verificó celda a celda, no como cuaderno real.

10.e La demostración decía "no reconocida" casi siempre

El síntoma. La pantalla de demostración respondía "no reconocida / fuera del catálogo" una y otra vez, incluso con fotos del propio catálogo de flores, y aunque la especie correcta estuviera entre las propuestas. Parecía que el modelo no servía para nada.

La causa. El umbral de confianza (el 25 % que se explicó antes) se aplicaba **antes** de mostrar el resultado. Y con un modelo poco entrenado, o entrenado sin los pesos preentrenados, las probabilidades son muy planas: el modelo reparte su duda entre muchas especies y su respuesta más segura ronda el 2 %, muy por debajo del 25 %. Resultado: el umbral borraba la predicción y lo único que se veía era el mensaje de rechazo. La causa de fondo era doble: el umbral estaba mal colocado

(borraba información en lugar de advertir) y, en el entorno de pruebas, se sumaba el problema de los pesos preentrenados del apartado anterior.

La solución. Se separaron las dos cosas que el umbral estaba mezclando:

- **Predecir** y **juzgar la confianza** son decisiones distintas. Ahora la demostración pide siempre las cinco especies más probables sin filtrar y, **después**, añade un aviso visible si la confianza no llega al umbral. El usuario ve siempre el resultado y, además, la advertencia.
- En la celda de predicción del cuaderno se explica el mismo detalle en un comentario, para que nadie vuelva a caer en la trampa.

El detalle revelador. Este error se detectó precisamente porque alguien miró la pantalla y preguntó por que respondía siempre lo mismo. Los datos que el programa imprimía eran correctos: el error estaba en como se presentaba.

10.f Dos afirmaciones nuestras resultaron falsas y se corrigieron

La auditoría adversarial refuto dos afirmaciones que el proyecto daba por buenas. Ninguna de las dos rompía el programa, y las dos importaban, porque en un trabajo académico una afirmación sin comprobar es un defecto.

Falsa número 1: el recuento de pruebas. Circulaba una cifra de pruebas que no correspondía con la realidad. Los recuentos que se fueron anotando a lo largo del trabajo fueron cambiando (una revisión, otra, otra), porque las pruebas se estaban escribiendo **en paralelo** mientras se auditaba; el número final, medido con el comando y con todas las pruebas activadas, es **78**. La conclusión del auditor fue justa y merece citarse: no era una mentira, era un dato **caducado**.

La corrección no fue solo poner el número bueno, sino dejar por escrito que el número se movió y con que ajustes se obtuvo cada recuento, para que nadie compare cifras de revisiones distintas y crea que alguien miente.

Falsa número 2: la ausencia de duplicados. El informe afirmaba que las particiones oficiales separaban las fotos por planta fotografiada, es decir, que ninguna planta aparecía en dos grupos a la vez. La comprobación por contenido (huellas digitales de las 8189 imágenes) demostró que hay **cuatro pares de fotos idénticas** y que **dos cruzan particiones**. Es lo que se ha explicado en el apartado 6: un 0,02 % del total, irrelevante para las cifras, pero la afirmación tal como estaba escrita era falsa y se retiró, sustituyendola por el dato medido.

La lección común a las dos. Ambas eran afirmaciones hechas de buena fe y sin mala intención, y las dos se cayeron por el mismo motivo: se habían dado por buenas sin medirlas. Por eso el proyecto terminó con la costumbre de acompañar cada cifra importante de su fuente (el archivo y el dato

exacto de donde sale), en lugar de confiar en la memoria o en la documentación anterior.

11. Por qué se documenta lo que NO se ha podido verificar

Al final del informe de verificación hay una lista de cosas que **no** se comprobaron. Podría parecer una debilidad, y en realidad es lo contrario: es la parte que permite confiar en el resto.

El razonamiento es este.

Si un documento dice "todo verificado" y luego resulta que había cosas sin verificar, **todo** el documento queda en duda, incluidas las partes que si eran ciertas. En cambió, si dice "esto está comprobado, esto otro no, y este es el motivo", quien lo lee sabe exactamente sobre que puede apoyarse. La honestidad sobre los límites no resta crédito: lo reparte correctamente.

Las cosas que quedaron sin verificar son, sobre todo:

- **La corrida completa de 25 épocas en una GPU de Colab.** No había acceso a Colab y la GPU del ordenador de desarrolló no se podía usar. Es el único pasó que el proyecto declara pendiente, y corresponde a quien reciba el trabajo. Los tiempos (15 a 25 minutos) son una **estimación**, no una medición.
- **El uso real de la GPU y de la precisión mixta** (los cálculos con menos decimales que hacen el entrenamiento más rápido). Todo lo medido en el desarrolló se hizo en el procesador. Se revisó el código línea a línea, pero eso no es lo mismo que verlo funcionar.
- **Las funciones exclusivas de Colab:** conectar Drive, subir el archivo comprimido y descargar resultados. Aquí no existen, así que solo se pudo comprobar que están bien protegidas para que un falló no rompa nada.
- **La descarga real del catálogo desde cero** en Colab, que aquí ya estaba descargado.
- **La compatibilidad con versiones antiguas** de la biblioteca de la pantalla interactiva.

Conviene decir también lo que **si** se midió, porque es mucho: el recorrido completo entrenar, evaluar y predecir; una evaluación de las 6149 fotos de prueba, con 37,8 % de acierto exacto y 67,4 % entre las cinco primeras propuestas (con una corrida corta de 4 épocas, que sirve como suelo de partida y **no** como resultado final); el cuaderno entero ejecutado en modo debug; las 78 pruebas; y las comprobaciones independientes de particiones, pesos preentrenados, rutas y contenido del paquete.

Una última advertencia práctica, heredada de todo lo anterior. Los números de 37,8 % y 67,4 % **no son el resultado del proyecto**: son lo que se obtuvo con una corrida corta de 4 épocas, hecha para comprobar que la maquinaria media de verdad. El resultado bueno lo produce la corrida completa de 25 épocas en Colab, y esa corrida la tiene que lanzar quien reciba el trabajo. El propio cuaderno

imprime, al terminar, un bloque de texto con las cifras reales de esa corrida, listo para copiar en el informe: no hay que escribir ningún número a mano.

Resumen del capítulo

- Las **decisiones** (catálogo público, PyTorch, red preentrenada, congelar y descongelar, aumentar los datos, suavizar las etiquetas, particiones oficiales, umbral convertido en aviso, degradar en lugar de fallar) se tomaron para resolver problemas concretos: pocos datos, poco tiempo de cálculo, necesidad de comparar con otros trabajos y necesidad de no dar respuestas falsas.
- Los **errores** fueron reales y de todo tipo: permisos del sistema, procesos auxiliares que fallaban, un modelo que entrenaba en blanco sin que se notara, un cuaderno que no arrancaba, una demostración que decía siempre lo mismo y dos afirmaciones nuestras que resultaron falsas. Todos se investigaron hasta la causa, se corrigieron y se dejaron escritos.
- Lo que **no** se pudo verificar también está escrito, porque un documento que solo habla de sus éxitos no se puede usar para decidir en que confiar.

Parte IV. Diccionario

Diccionario: todos los términos en lenguaje llano

Cada fila explica un termino con palabras corrientes. La tercera columna dice donde aparece ese termino dentro de este proyecto concreto, no en general.

Termino	Que significa	Como se usa en este proyecto
AdamW	Es el mecanismo que decide como se corrige el modelo despues de cada error, con un freno incorporado que evita ajustes exagerados. Es como un entrenador que corrige la postura poco a poco y no de golpe.	Es el optimizador del proyecto. Se construye en src/model.py y se aplica en el entrenamiento con un freno de 1e-4 sobre el tamano de los ajustes.
AMP (precisión mixta)	Tecnica que hace los calculos pesados con numeros mas cortos, de 16 bits en vez de 32, para ir mas rapido y gastar menos memoria sin perder calidad donde importa.	Esta activada por defecto con el campo amp. El cuaderno avisa con la linea AMP activado y se desactiva sola cuando no hay GPU.
Aumentación de datos	Fabricar copias alteradas de cada foto de entrenamiento, con recortes, volteos y cambios de color, para que el modelo vea mas variedad sin tener mas fotos.	Se aplica solo al entrenamiento desde build_transforms en src/data.py, con recorte aleatorio, volteo horizontal y RandAugment.
backbone	La parte grande y ya entrenada de la red, que se encarga de mirar la imagen. Encima se coloca una parte nueva y pequena que decide la especie.	Es EfficientNet-B0 con pesos de ImageNet, definido en src/model.py. Empieza congelado y despues se afina.
batch (lote)	El grupo de fotos que el modelo mira antes de corregirse. En lugar de aprender de una en una, aprende a golpecitos de varias.	El tamano por defecto es 32 imagenes de 224 por 224 pixeles. Si aparece un error de memoria se baja a 16.
checkpoint	Una foto fija del modelo en un momento dado: sus ajustes internos y su configuracion, guardados en un archivo para poder volver a usarlos.	Se guarda en resultados/best.pt y solo cuando mejora la exactitud de validacion. Lo usan la evaluacion y la demo.
clase	Cada una de las categorias	Hay 102 clases, definidas en

	posibles entre las que el modelo elige. Aqui una clase es una especie de flor.	src/config.py. El checkpoint guarda ademas los nombres legibles de cada especie.
CLI	La forma de usar un programa escribiendo ordenes en una ventana de texto, en lugar de pulsar botones en una pantalla.	El proyecto incluye una CLI para entrenar, evaluar y predecir. En Colab no hace falta usarla: basta con ejecutar las celdas.
CNN (red convolucional)	Familia de redes pensadas para imagenes. En vez de mirar los pixeles uno a uno, peinan pequenas ventanas y detectan bordes, manchas y formas.	El modelo del proyecto es una CNN: EfficientNet-B0 mas una cabeza nueva que elige entre las 102 especies.
Colab	Google Colab es un cuaderno de trabajo en el navegador que se ejecuta en ordenadores de Google, con GPU gratuita. No hay que instalar nada en el propio equipo.	Es el entorno previsto para todo el flujo. El cuaderno notebooks/flores_colab.ipynb esta disenado para ejecutarse alli de principio a fin.
congelar y descongelar	Congelar una parte del modelo es prohibirle cambiar mientras se entrena el resto. Descongelar es volver a permitir que se ajuste.	El backbone empieza congelado durante las 3 primeras epocas y despues se descongela con una velocidad de aprendizaje mucho mas baja.
DataLoader	El encargado de servir las fotos al modelo: las busca, las prepara y las entrega por lotes mientras el modelo trabaja.	Se construye en src/data.py con build_dataloaders. Los de validacion y prueba no se barajan, para que la medicion sea siempre la misma.
dataset	El conjunto de datos: todas las fotos con su etiqueta que se usan para aprender y para medir el resultado.	Se usa Oxford Flowers-102, con 8189 fotos repartidas en 1020 de entrenamiento, 1020 de validacion y 6149 de prueba. Ocupa 330 MB y se descarga la primera vez.
dropout	Apagar al azar una parte de las conexiones durante el entrenamiento, para que el modelo no dependa siempre de las mismas pistas.	Se aplica en la cabeza nueva con probabilidad 0,3, configurada como dropout en src/config.py.
early stopping (parada temprana)	Dejar de entrenar cuando el	Esta activado con una paciencia de

	resultado en validacion deja de mejorar, en lugar de gastar vueltas inutiles.	6 epocas. Cuando salta, el cuaderno imprime una linea que empieza por Early stopping.
EfficientNet-B0	Un modelo de vision conocido y relativamente ligero, pensado para dar buen resultado con un costo de computo moderado.	Es el backbone por defecto y la razon de que la corrida completa quepa en 15 a 25 minutos de GPU T4.
época	Una vuelta completa del modelo por todas las fotos de entrenamiento. Diez epocas son diez pasadas por el mismo material.	La corrida rapida usa 2 epocas y la completa 25. El limite se configura con el campo epochs.
época de calentamiento (warmup)	Las primeras vueltas, en las que la velocidad de aprendizaje sube poco a poco en lugar de empezar a tope, para no desestabilizar el entrenamiento.	Son 2 epocas, configuradas como warmup_epochs. En la corrida rapida se desactivan.
etiqueta	El nombre correcto de cada foto. Es la respuesta que el modelo debe aprender a dar.	Cada imagen de Flowers-102 trae su especie. El checkpoint guarda ademas los nombres legibles para mostrarlos en la demo.
exactitud (accuracy)	La proporcion de aciertos: de cada 100 fotos, cuantas clasifica bien el sistema.	Se mide en la evaluacion como top1 y top5 sobre las 6149 fotos de prueba, y en cada epoca sobre las fotos de validacion.
fine-tuning (ajuste fino)	La segunda fase del aprendizaje por transferencia. En vez de dejar intacta la parte ya entrenada, se retoca con mucha suavidad para adaptarla a las flores.	Empieza despues de las 3 primeras epocas, con velocidad 3e-5 para el backbone frente a 3e-4 para la cabeza nueva.
Flowers-102 (Oxford Flowers-102)	El catalogo de fotos de flores que usa el proyecto, creado por la Universidad de Oxford. Reune 102 especies comunes en el Reino Unido.	Es el unico conjunto de datos del proyecto. Se cargan sus particiones oficiales, con solo 10 fotos de entrenamiento por especie.
función	Un bloque de ordenes con nombre que hace una tarea concreta y se puede reutilizar llamandola. Es como una receta guardada.	El proyecto esta lleno de funciones con nombre claro, como train, evaluate y predict_image. El cuaderno las llama, no las reescribe.

función de pérdida	La nota de castigo que mide cuanto se equivoca el modelo en cada lote. Entrenar consiste en hacer que ese castigo baje.	Es la que aparece como loss en las lineas de entrenamiento. Se combina con un suavizado de etiquetas de 0,1 para que el castigo no sea excesivamente severo.
GPU	Un procesador especializado en hacer muchisimos calculos a la vez. Es lo que convierte el entrenamiento de una red de imagenes en cuestion de minutos.	El proyecto espera una GPU T4 gratuita de Colab. El cuaderno comprueba al principio si esta disponible y avisa claramente si no lo esta.
GPU T4	El modelo concreto de tarjeta grafica gratuita que ofrece Google Colab, con 16 GB de memoria.	Es el acelerador que se elige en Entorno de ejecucion. Con ella, la corrida completa de 25 epocas tarda entre 15 y 25 minutos.
gradiente	La indicacion de en que direccion y con cuanta fuerza hay que retocar cada ajuste del modelo para equivocarse menos.	El entrenamiento calcula gradientes en cada lote. Durante las primeras epocas los del backbone no se usan, porque esa parte esta congelada.
Gradio	Una libreria que fabrica paginas web sencillas para probar un modelo: un recuadro para subir la imagen y un boton para enviar.	Con ella se construye la demo en src/app.py, con subida de imagen, camara y salida con las cinco especies mas probables.
hiperparámetro	Los ajustes que decide la persona antes de entrenar, como cuantas vueltas dar o a que velocidad aprender. El modelo no los aprende solo.	Estan todos reunidos en src/config.py. Cada corrida guarda los suyos en config_usada.json para poder repetirla.
historial	El registro de lo que paso en cada vuelta de entrenamiento: el castigo por los errores, la exactitud y la velocidad de aprendizaje.	Se guarda en history.json y sirve para dibujar las curvas y rellenar la tabla de resultados del informe.
ImageNet	Un catalogazo de mas de un millon de fotos repartidas en 1000 categorias, usado para entrenar de antemano modelos de vision.	El backbone del proyecto parte de pesos preentrenados en ImageNet. Si esa descarga falla, el sistema avisa y aprende desde cero con peores resultados.
import	La orden que trae a la memoria	El cuaderno importa las funciones

	una pieza de codigo escrita en otro archivo, para poder usarla.	del proyecto, por ejemplo la funcion train, en su celda de imports antes de entrenar.
learning rate (velocidad de aprendizaje)	Cuanto se atreve el modelo a cambiar en cada correccion. Si es muy alta da tumbos, y si es muy baja tarda una eternidad.	La cabeza usa 3e-4 y el backbone 3e-5. En el entrenamiento aparece como lr y baja sola siguiendo una curva de coseno.
lista	Una coleccion ordenada de elementos a la que se accede por su posicion. Es como una fila numerada.	Las predicciones se devuelven como una lista de parejas especie y probabilidad, ordenada de mayor a menor. Las confusiones frecuentes tambien son una lista.
logits	Los numeros en bruto que produce el modelo antes de convertirlos en porcentajes. Todavia no son probabilidades, solo puntuaciones.	La cabeza del modelo produce 102 logits, uno por especie. Despues se convierten en probabilidades para mostrar las cinco mejores.
macro-F1	Una nota que promedia el acierto por especie dando el mismo peso a cada una, de modo que una especie dificil no quede tapada por las faciles.	Se calcula sobre las fotos de prueba y aparece como macro_f1 en metrics.json y en el resumen que se copia al informe.
matriz de confusión	Una tabla cuadrada que cruza la especie real con la que dijo el modelo. La diagonal son los aciertos y todo lo demas son errores.	Es de 102 por 102 y se guarda como figura confusion_matrix.png al evaluar. Tambien se listan aparte las confusiones mas frecuentes.
métrica	Un numero que resume el desempeno, para poder comparar versiones y decidir si el sistema es bueno.	Las metricas principales son la exactitud top-1, la exactitud top-5 y la macro-F1, guardadas en metrics.json.
modelo	El cerebro entrenado: lo que queda despues de aprender y lo que responde cuando le das una foto.	El modelo final es EfficientNet-B0 con una cabeza nueva de 102 salidas, guardado como resultados/best.pt junto a la configuracion usada.
modo debug	El modo de prueba del cuaderno: usa un trozo pequeno de las fotos y solo 2 vueltas, para comprobar que todo funciona en minutos.	Viene activado de fabrica como MODO_DEBUG. Escribe en la carpeta resultados/debug y nunca pisa la corrida completa.

normalización	Reajustar los valores de los pixeles a una escala comun antes de que el modelo los mire, para que el aprendizaje sea estable.	Se aplica con las medias y desviaciones de ImageNet, definidas como constantes en src/config.py.
notebook (cuaderno)	Un documento que mezcla texto y celdas ejecutables, con los resultados guardados debajo de cada celda. Es el formato de trabajo de Colab.	El proyecto tiene un unico cuaderno, notebooks/flores_colab.ipynb, con 26 celdas que recorren todo el flujo de principio a fin.
num_workers	Cuantos ayudantes cargan las fotos del disco mientras el modelo calcula, para que nadie tenga que esperar.	Vale 2 en la corrida normal y 0 en modo debug, que es la configuracion segura para Colab en todos los casos.
optimizador	El mecanismo que aplica las correcciones al modelo usando los gradientes. Decide como se actualiza cada ajuste.	Es AdamW, construido en src/model.py con dos grupos de parametros: uno para la cabeza nueva y otro para el backbone.
overfitting (sobreajuste)	Cuando el modelo memoriza las fotos de entrenamiento en lugar de aprender la especie. Acierta mucho en lo ya visto y falla en lo nuevo.	Es el riesgo principal, porque solo hay 10 fotos de entrenamiento por especie. Se contiene con aumentacion de datos, dropout, suavizado de etiquetas y parada temprana.
peso preentrenado	Los valores ya guardados de un modelo que otras personas entrenaron antes, y que sirven como punto de partida.	El backbone arranca con pesos preentrenados en ImageNet. El cuaderno avisa con una nota si no se pudieron descargar.
píxel	El puntito de color mas pequeno de una imagen. Una foto es una rejilla de pixeles, con tres numeros por punto: rojo, verde y azul.	Las imagenes se preparan a 224 por 224 pixeles, que es el tamano que espera EfficientNet-B0.
precisión (y recall)	La precision mide, de todo lo que el modelo dijo de una especie, cuanto era cierto. El recall mide, de los casos reales de esa especie, cuantos encontro.	Ambas se calculan por especie en el informe detallado de la evaluacion y alimentan la macro-F1. Aparecen en el ranking de clases con peor resultado.
predicción	La respuesta que da el modelo ante una foto: la especie propuesta y su nivel de confianza.	La inferencia devuelve las cinco mejores predicciones con su probabilidad. Si ninguna llega al umbral de confianza, devuelve el

		aviso de no reconocida.
Python	El lenguaje de programacion en el que esta escrito el proyecto. Es de los mas legibles y se usa muchisimo en inteligencia artificial.	Todo el codigo vive en la carpeta src y en el cuaderno. Quien siga esta guia no necesita escribir ni una linea de Python.
PyTorch	La libreria de programacion mas usada para construir y entrenar redes neuronales. Aporta las piezas de calculo.	Es la base del proyecto: define el modelo, el entrenamiento, la precision mixta y la carga de datos. En Colab ya viene instalada con soporte de GPU.
RandAugment	Un metodo que aplica transformaciones aleatorias a las fotos de entrenamiento sin tener que elegir a mano cuales ni con que intensidad.	Forma parte de la aumentacion de datos del entrenamiento y esta definido en src/data.py.
reproducibilidad (semilla)	La capacidad de repetir una corrida y obtener lo mismo. Se consigue fijando un numero inicial, llamado semilla, que gobierna todos los sorteos.	El proyecto fija la semilla 42 al empezar. Aun asi, el resultado puede variar ligeramente entre equipos o versiones distintas.
requirements	La lista escrita de las librerias y versiones que necesita el proyecto para funcionar.	Esta en requirements.txt y sirve para montar el entorno en un ordenador. En Colab no se usa a proposito, porque Colab ya trae las piezas pesadas.
softmax	La operacion que convierte las puntuaciones en bruto en porcentajes que suman cien, faciles de leer.	Se aplica a los 102 logits para obtener las probabilidades de las cinco mejores propuestas y alimenta el umbral de confianza.
split (partición)	Cada uno de los trozos en que se divide el conjunto de fotos: uno para aprender, otro para vigilar el avance y otro para la prueba final.	Se usan las particiones oficiales: 1020 fotos de entrenamiento, 1020 de validacion y 6149 de prueba. No se inventan repartos propios.
suavizado de etiquetas (label smoothing)	En vez de exigir al modelo un cien por cien de seguridad en la respuesta correcta, se le pide algo menos, para que no se vuelva fanatico ni memorice.	Se aplica con valor 0,1. Hace que los porcentajes de la demo sean menos extremos, algo importante para que el umbral de confianza sea util.

tensor	Una caja de numeros ordenada por dimensiones: una lista es de una dimension, una tabla de dos y un bloque de imagenes de cuatro.	Las fotos se convierten en tensores con forma de lote, 3 canales y 224 por 224 antes de entrar al modelo.
top-1 y top-5	Top-1 es acertar a la primera. Top-5 es que la respuesta correcta este entre las cinco primeras propuestas.	Son las dos metricas de exactitud que reporta el proyecto. La demo muestra directamente las cinco propuestas.
transfer learning (aprendizaje por transferencia)	Aprovechar lo que un modelo ya aprendio con muchisimas imagenes y reutilizarlo para una tarea nueva con pocos datos.	Es la estrategia central del proyecto: EfficientNet-B0, entrenado antes en ImageNet, se adapta a las 102 especies con solo 10 fotos por clase.
umbral de confianza	El nivel minimo de seguridad que se le exige a una respuesta para darla por valida. Por debajo, el sistema prefiere avisar de que no lo sabe.	Vale 0,25, configurado como min_conf. En la evaluacion responde no reconocida y en la demo muestra un aviso de confianza baja.
validación y prueba	Validacion son las fotos que se miran durante el entrenamiento para elegir la mejor version. Prueba son las que se guardan hasta el final para medir de verdad.	Las 1020 fotos de validacion eligen el mejor checkpoint y las 6149 de prueba producen las cifras del informe. La prueba no influye en ninguna decision del entrenamiento.
variable	Un nombre que guarda un valor para usarlo despues. Es como una caja con etiqueta.	En el cuaderno las variables importantes son MODO_DEBUG, que enciende la corrida rapida, y la configuracion de la corrida, que reune todos los ajustes.
webcam	La camara del ordenador. La demo puede usarla para hacer la foto directamente, sin pasar por un archivo.	La interfaz Gradio de src/app.py incluye una pestana de camara, ademas de la subida de archivo y el pegado desde el portapapeles.
ZIP	Un archivo comprimido que reune muchos archivos en uno solo y ocupa menos. Se abre con doble clic.	IA_flores_colab.zip es el paquete que se sube a Colab, con el cuaderno dentro de la carpeta notebooks. resultados_flores.zip es el paquete de resultados que se descarga al final.

Anexo A. Código fuente completo

Los archivos que aparecen a continuación son exactamente los del proyecto, copiados sin ningún cambio. Se incluyen aquí para que este documento sea autosuficiente: quien lo lea no necesita abrir el ordenador para ver el código.

src/config.py

Todos los ajustes del proyecto en un solo sitio. (105 líneas)

```
"""Configuracion central del proyecto de reconocimiento de flores.

CONTRATO (Fase 0, propietario SA-00):
  - Este modulo NO importa torch: debe poder importarse en cualquier entorno.
  - Los subagentes NO deben modificar este archivo. Si necesitan un campo nuevo,
    lo reportan al orquestador en su informe final.
"""

from __future__ import annotations

from dataclasses import asdict, dataclass
from pathlib import Path

# ---------------------------------------------------------------- constantes
IMAGE_SIZE: int = 224
MEAN: tuple[float, float, float] = (0.485, 0.456, 0.406)  # ImageNet
STD: tuple[float, float, float] = (0.229, 0.224, 0.225)  # ImageNet
FLOWERS102_CLASSES: int = 102


@dataclass
class Config:
    """Hiperparametros y rutas de una corrida de entrenamiento/evaluacion."""

    # --- rutas (relativas a la raiz del repo; en Colab se sobreescriben) ---
    data_root: str = "data"
    out_dir: str = "resultados"

    # --- datos ---
    num_classes: int = FLOWERS102_CLASSES
    img_size: int = IMAGE_SIZE
    # Flowers102 trae splits oficiales (train/val/test), no hace falta repartir.

    # --- modelo ---
    backbone: str = "efficientnet_b0"
    dropout: float = 0.3
    pretrained: bool = True

    # --- entrenamiento ---
    epochs: int = 25
    batch_size: int = 32
    lr: float = 3e-4  # learning rate de la cabeza
    backbone_lr: float = 3e-5  # learning rate del backbone al descongelar
    weight_decay: float = 1e-4
    label_smoothing: float = 0.1
    warmup_epochs: int = 2
    freeze_epochs: int = 3  # epocas iniciales con el backbone congelado
    early_stop_patience: int = 6
    amp: bool = True  # mixed precision (se ignora automaticamente en CPU)

    # --- inferencia / demo ---
    topk: int = 5
    min_conf: float = 0.25  # por debajo de esto -> "no reconocida"

    # --- sistema ---
    seed: int = 42
```

```
num_workers: int = 2
device: str = "auto"  # "auto" | "cuda" | "cpu"

# --- modo debug (smoke test / verificar Colab en ~1 min) ---
debug: bool = False
debug_per_class: int = 4  # imagenes por clase en modo debug

def __post_init__(self) -> None:
    if self.debug:
        self.epochs = min(self.epochs, 2)
        self.freeze_epochs = 0
        self.warmup_epochs = 0
        self.amp = False
        self.num_workers = 0
    self.data_root = str(self.data_root)
    self.out_dir = str(self.out_dir)

# ------------------------------------------------------------ derivados
@property
def checkpoint_path(self) -> Path:
    return Path(self.out_dir) / "best.pt"

@property
def history_path(self) -> Path:
    return Path(self.out_dir) / "history.json"

@property
def metrics_path(self) -> Path:
    return Path(self.out_dir) / "metrics.json"

@property
def config_path(self) -> Path:
    return Path(self.out_dir) / "config_usada.json"

def figures_dir(self) -> Path:
    return Path(self.out_dir) / "figuras"

def predictions_dir(self) -> Path:
    return Path(self.out_dir) / "predicciones"

def ensure_dirs(self) -> None:
    """Crea los directorios de salida. Idempotente."""
    Path(self.out_dir).mkdir(parents=True, exist_ok=True)
    self.figures_dir().mkdir(parents=True, exist_ok=True)
    self.predictions_dir().mkdir(parents=True, exist_ok=True)
    Path(self.data_root).mkdir(parents=True, exist_ok=True)

def to_dict(self) -> dict:
    return asdict(self)
```

src/utils.py

Utilidades comunes: semillas, dispositivo, guardar JSON. (105 líneas)

```
"""Utilidades compartidas (semillas, dispositivo, JSON, conteo de parametros).

CONTRATO (Fase 0, propietario SA-00):
  - Firmas congeladas: todos los modulos pueden asumirlas.
  - Los subagentes NO deben modificar este archivo.
  - Las importaciones de torch son perezosas (dentro de la funcion) para que
    `load_json` / `save_json` funcionen sin torch instalado.
"""

from __future__ import annotations

import json
import os
import random
from pathlib import Path
from typing import Any

__all__ = [
    "set_seed",
    "resolve_device",
    "save_json",
    "load_json",
    "ensure_dir",
    "count_params",
    "describe_device",
]


def set_seed(seed: int = 42) -> None:
    """Fija la semilla en random, numpy y torch (CPU y CUDA) para reproducibilidad."""
    import numpy as np

    random.seed(seed)
    np.random.seed(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)
    try:
        import torch

        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    except ImportError:  # pragma: no cover - entorno sin torch
        pass


def resolve_device(spec: str = "auto"):
    """Devuelve un torch.device a partir de 'auto' | 'cuda' | 'cpu'.

    'auto' usa CUDA si esta disponible y si no cae a CPU (Colab sin GPU incluido).
    Si se pide 'cuda' y no hay GPU, lanza RuntimeError con un mensaje claro.
    """
    import torch

    spec = (spec or "auto").lower()
    if spec == "auto":
        return torch.device("cuda" if torch.cuda.is_available() else "cpu")
    if spec.startswith("cuda") and not torch.cuda.is_available():
        raise RuntimeError(
            "Se pidio device='cuda' pero torch.cuda.is_available() es False. "
            "En Colab: Entorno de ejecucion > Cambiar tipo de entorno > GPU T4."
        )
    return torch.device(spec)


def describe_device(device) -> str:
```

```
    """Texto legible para logs: 'cuda (NVIDIA T4)' o 'cpu'."""
    import torch

    if getattr(device, "type", str(device)) == "cuda":
        idx = getattr(device, "index", None) or 0
        return f"cuda ({torch.cuda.get_device_name(idx)})"
    return "cpu"


def ensure_dir(path: str | Path) -> Path:
    p = Path(path)
    p.mkdir(parents=True, exist_ok=True)
    return p


def save_json(path: str | Path, obj: Any) -> None:
    """Guarda JSON serializable (convierte Path y tipos numpy/torch basicos)."""
    p = Path(path)
    ensure_dir(p.parent)

    def _default(o: Any):
        if isinstance(o, Path):
            return str(o)
        if hasattr(o, "item"):  # escalares numpy / torch
            return o.item()
        if isinstance(o, set):
            return sorted(o)
        raise TypeError(f"Objeto no serializable: {type(o)!r}")

    with open(p, "w", encoding="utf-8") as fh:
        json.dump(obj, fh, indent=2, ensure_ascii=False, default=_default)


def load_json(path: str | Path) -> Any:
    with open(Path(path), "r", encoding="utf-8") as fh:
        return json.load(fh)


def count_params(model) -> int:
    """Numero de parametros entrenables."""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

src/data.py

Descarga el dataset, lo transforma y lo reparte en lotes. (449 líneas)

```
"""Datos de Oxford Flowers-102: transforms, DataLoaders y nombres de clase.

CONTRATO (Firmas congeladas, propietario SA-01 — no cambiar):
    build_transforms(cfg: Config, train: bool) -> Callable
    build_dataloaders(cfg: Config) -> tuple[DataLoader, DataLoader, DataLoader, list[str]]
    get_class_names(cfg: Config) -> list[str]

Notas de diseno:
  - Las importaciones de ``torch`` / ``torchvision`` son perezosas (dentro de las
    funciones). Asi ``import src.data`` no arrastra torch ni scipy, y modulos como
    ``predict.py`` pueden reutilizar ``build_transforms`` sin cargar el dataset.
    Ademas, ``torchvision.datasets.Flowers102`` necesita ``scipy`` (lee los .mat);
    el helper :func:`_flowers102_class` convierte ese ImportError en un mensaje util.
  - Nada de rutas absolutas ni de ``.cuda()``: este modulo no necesita device,
    ``pin_memory`` se activa solo si ``torch.cuda.is_available()``.
  - Pitfall documentado en CONTRATOS.md §3: las etiquetas originales de Flowers-102
    son 1-based (1..102) y torchvision resta 1; las claves de ``cat_to_name.json``
    son ``"1".."102"``, por eso se mapea ``class_names[i] = cat[str(i + 1)]``.
"""

from __future__ import annotations

import json
import random
import re
import urllib.error
import urllib.request
from pathlib import Path
from typing import TYPE_CHECKING, Any, Callable, Sequence

from src.config import MEAN, STD, Config

if TYPE_CHECKING:  # solo para anotaciones: mantiene el import de torch perezoso
    from torch.utils.data import DataLoader

__all__ = [
    "build_transforms",
    "build_dataloaders",
    "get_class_names",
    "CAT_TO_NAME_URL",
    "CAT_TO_NAME_FILENAME",
]

#: URL oficial (Oxford VGG) del mapa id de categoria -> nombre de especie.
CAT_TO_NAME_URL = "https://www.robots.ox.ac.uk/~vgg/data/flowers/102/cat_to_name.json"
#: Nombre del archivo de cache dentro de ``cfg.data_root``.
CAT_TO_NAME_FILENAME = "cat_to_name.json"
#: Respaldo offline versionado en el repo (claves 0-based "0".."101").
ASSET_CAT_TO_NAME = Path(__file__).resolve().parent.parent / "assets" / "cat_to_name.json"
#: Numero de clases de Flowers-102 (por si cfg no lo trae).
DEFAULT_NUM_CLASSES = 102
#: Splits oficiales, en el orden que espera el contrato.
SPLITS: tuple[str, str, str] = ("train", "val", "test")
#: Timeout de red (segundos) para la descarga de cat_to_name.json.
URL_TIMEOUT_S = 20.0

#: Nombres genericos tipo "class_0", "n12", "label_3"... no son nombres de especie.
_GENERIC_NAME_RE = re.compile(
    r"^(?:class|clase|label|etiqueta|category|categoria|species|especie|"
    r"n|img|image|idx|index)[_\- ]?\d+$",
    re.IGNORECASE,
)

#: Raices de datos donde ya se intento (y fallo) la descarga: evita reintentos lentos.
```

```
_REMOTE_ATTEMPTED: set[str] = set()


# ------------------------------------------------------------- import perezoso
def _torch():
    """Devuelve el modulo ``torch`` (import perezoso para no cargarlo al importar)."""
    import torch

    return torch


def _transforms():
    """Devuelve ``torchvision.transforms`` con un mensaje claro si falta scipy."""
    try:
        from torchvision import transforms
    except ImportError as exc:  # torchvision.datasets importa scipy.io de forma eager
        if "scipy" in str(exc).lower():
            raise ImportError(
                "torchvision no pudo importarse porque falta scipy "
                "(Flowers102 lee imagelabels.mat con scipy.io). Instala: pip install scipy"
            ) from exc
        raise
    return transforms


def _flowers102_class():
    """Devuelve la clase ``torchvision.datasets.Flowers102`` (import perezoso)."""
    try:
        from torchvision.datasets import Flowers102
    except ImportError as exc:
        if "scipy" in str(exc).lower():
            raise ImportError(
                "Flowers102 necesita scipy para leer imagelabels.mat / setid.mat. "
                "Instala: pip install scipy"
            ) from exc
        raise
    return Flowers102


# ------------------------------------------------------------------ transforms
def build_transforms(cfg: Config, train: bool) -> Callable:
    """Pipeline de preprocesado de una imagen.

    ``train=True``:  RandomResizedCrop(img_size, scale=(0.6, 1.0)) + flip horizontal
                     + RandAugment (o ColorJitter si RandAugment no existe)
                     + ToTensor + Normalize(MEAN, STD).
    ``train=False``: Resize(int(img_size * 1.14)) + CenterCrop(img_size)
                     + ToTensor + Normalize(MEAN, STD)  (determinista).

    Es valido para cualquier ``cfg.img_size`` (no asumir 224).
    """
    transforms = _transforms()
    img_size = int(cfg.img_size)
    if img_size <= 0:
        raise ValueError(f"cfg.img_size debe ser positivo, se recibio {cfg.img_size!r}")

    ops: list[Any] = []
    if train:
        ops.append(transforms.RandomResizedCrop(img_size, scale=(0.6, 1.0)))
        ops.append(transforms.RandomHorizontalFlip(p=0.5))
        rand_augment = getattr(transforms, "RandAugment", None)
        if rand_augment is not None:
            ops.append(rand_augment(num_ops=2, magnitude=9))
        else:  # torchvision < 0.12: no hay RandAugment
            ops.append(transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3, hue=0.05))
    else:
        ops.append(transforms.Resize(int(img_size * 1.14)))  # 224 -> 255
        ops.append(transforms.CenterCrop(img_size))
```

```
    ops.append(transforms.ToTensor())
    ops.append(transforms.Normalize(MEAN, STD))
    return transforms.Compose(ops)


# ------------------------------------------------------- etiquetas del dataset
def _sample_indices(total: int, k: int) -> list[int]:
    """Hasta ``k`` indices repartidos por todo el rango [0, total)."""
    if total <= 0 or k <= 0:
        return []
    if total <= k:
        return list(range(total))
    step = max(1, total // k)
    return list(range(0, total, step))[:k]


def _targets_from_attributes(dataset) -> list[int] | None:
    """Intenta leer las etiquetas de atributos conocidos (sin decodificar imagenes)."""
    for attr in ("targets", "labels", "_labels"):
        value = getattr(dataset, attr, None)
        if value is None or isinstance(value, (str, bytes)):
            continue
        try:
            seq = list(value)
        except TypeError:
            continue
        if len(seq) != len(dataset):
            continue
        try:
            return [int(t) for t in seq]
        except (TypeError, ValueError):
            continue
    return None


def _targets_by_scan(dataset, indices: Sequence[int] | None = None) -> list[int]:
    """Etiquetas leyendo ``dataset[i][1]`` (lento: decodifica imagenes)."""
    if indices is None:
        indices = range(len(dataset))
    return [int(dataset[i][1]) for i in indices]


def dataset_targets(dataset, verify: int = 5) -> list[int]:
    """Etiquetas enteras de ``dataset``, alineadas con ``dataset[i]``.

    Usa primero los atributos internos de torchvision (rapido). Como el formato de
    ``_labels`` cambio entre versiones (1-based vs 0-based), comprueba unas pocas
    muestras reales y, si no coinciden, hace un escaneo completo.
    """
    targets = _targets_from_attributes(dataset)
    if targets is None:
        return _targets_by_scan(dataset)
    for idx in _sample_indices(len(dataset), verify):
        try:
            real = int(dataset[idx][1])
        except Exception:  # no se pudo verificar (p. ej. imagen corrupta): confiamos
            return targets
        if real != targets[idx]:
            return _targets_by_scan(dataset)
    return targets


def _debug_subset(dataset, per_class: int, seed: int):
    """Subconjunto reproducible de ``per_class`` imagenes por clase.

    Mantiene la correspondencia etiqueta <-> nombre porque solo selecciona indices
    del dataset original (``Subset`` conserva los targets).
```

```
    """
    from torch.utils.data import Subset

    per_class = int(per_class)
    if per_class <= 0:
        raise ValueError(f"cfg.debug_per_class debe ser >= 1, se recibio {per_class!r}")

    targets = dataset_targets(dataset)
    by_class: dict[int, list[int]] = {}
    for idx, label in enumerate(targets):
        by_class.setdefault(label, []).append(idx)

    rng = random.Random(int(seed))
    selected: list[int] = []
    for label in sorted(by_class):
        indices = by_class[label]
        rng.shuffle(indices)
        selected.extend(sorted(indices[:per_class]))
    selected.sort()

    if not selected:  # dataset vacio: no fallar, devolver un Subset vacio
        return Subset(dataset, [])
    return Subset(dataset, selected)


# ------------------------------------------------------------ nombres de clase
def _looks_generic(name: str) -> bool:
    """True si el nombre no aporta informacion de especie ('class_7', '12', ...)."""
    text = str(name).strip()
    if not text:
        return True
    if text.isdigit():
        return True
    return bool(_GENERIC_NAME_RE.match(text))


def _names_from_dataset(dataset, num_classes: int = DEFAULT_NUM_CLASSES) -> list[str] | None:
    """Nombres propios del dataset si existen y NO son genericos; si no, ``None``."""
    if dataset is None:
        return None

    candidates: list[list[Any]] = []
    for attr in ("classes", "class_names", "_class_names", "categories"):
        value = getattr(dataset, attr, None)
        if isinstance(value, Sequence) and not isinstance(value, (str, bytes)):
            candidates.append(list(value))
    class_to_idx = getattr(dataset, "class_to_idx", None)
    if isinstance(class_to_idx, dict) and class_to_idx:
        candidates.append([name for name, _ in sorted(class_to_idx.items(), key=lambda kv: int(kv[1]))])

    for candidate in candidates:
        names = [str(x).strip() for x in candidate]
        if len(names) != int(num_classes):
            continue
        if any(_looks_generic(n) for n in names):
            continue
        return names
    return None


def _names_from_mapping(mapping: Any, num_classes: int) -> list[str] | None:
    """Mapea un JSON id->nombre a una lista ordenada por etiqueta 0-based.

    Se aceptan las dos convenciones vistas en la practica:
      - Oxford ``cat_to_name.json``: claves ``"1".."102"`` (1-based), por eso se
        prueba ``mapping[str(i + 1)]`` (pitfall documentado en CONTRATOS.md §3).
      - ``assets/cat_to_name.json`` del repo: claves ``"0".."101"`` (0-based).
    """
```

```
    if not isinstance(mapping, dict) or not mapping:
        return None
    for offset in (1, 0):  # primero 1-based (contrato), luego 0-based (asset)
        names: list[str] = []
        for i in range(int(num_classes)):
            value = mapping.get(str(i + offset))
            if value is None:
                value = mapping.get(i + offset)
            if not isinstance(value, str) or not value.strip():
                break
            names.append(value.strip())
        else:
            return names
    return None


def _names_from_cache(cfg: Config) -> list[str] | None:
    """Lee ``<data_root>/cat_to_name.json`` si existe y cubre las 102 clases."""
    path = Path(cfg.data_root) / CAT_TO_NAME_FILENAME
    if not path.is_file():
        return None
    try:
        with open(path, "r", encoding="utf-8") as fh:
            mapping = json.load(fh)
    except (OSError, ValueError):
        return None
    return _names_from_mapping(mapping, int(cfg.num_classes))


def _names_from_asset(num_classes: int) -> list[str] | None:
    """Respaldo offline versionado en el repo: ``assets/cat_to_name.json``.

    Es preferible a la descarga de red: no necesita internet y esta verificado.
    """
    path = Path(ASSET_CAT_TO_NAME)
    if not path.is_file():
        return None
    try:
        with open(path, "r", encoding="utf-8") as fh:
            mapping = json.load(fh)
    except (OSError, ValueError):
        return None
    return _names_from_mapping(mapping, int(num_classes))


def _download_cat_to_name(cfg: Config) -> list[str] | None:
    """Descarga (una sola vez por data_root) y cachea cat_to_name.json.

    Nunca lanza excepcion: sin internet devuelve ``None`` para que el llamador
    use los nombres de relleno.
    """
    root = str(Path(cfg.data_root).resolve())
    if root in _REMOTE_ATTEMPTED:
        return None
    _REMOTE_ATTEMPTED.add(root)

    try:
        with urllib.request.urlopen(CAT_TO_NAME_URL, timeout=URL_TIMEOUT_S) as response:
            payload = response.read()
        mapping = json.loads(payload.decode("utf-8"))
    except (urllib.error.URLError, OSError, ValueError, TypeError):
        return None

    names = _names_from_mapping(mapping, int(cfg.num_classes))
    if names is None:
        return None

    try:  # el cache es best-effort: si no se puede escribir, seguimos igual
```

```
        cache = Path(cfg.data_root) / CAT_TO_NAME_FILENAME
        cache.parent.mkdir(parents=True, exist_ok=True)
        with open(cache, "w", encoding="utf-8") as fh:
            json.dump(mapping, fh, ensure_ascii=False, indent=2)
    except OSError:
        pass
    return names


def _fallback_names(num_classes: int) -> list[str]:
    """Nombres de relleno: ['especie_001', ..., 'especie_102']."""
    return [f"especie_{i + 1:03d}" for i in range(int(num_classes))]


def _resolve_class_names(cfg: Config, dataset=None) -> list[str]:
    """Nombres de clase ordenados por etiqueta 0-based, con degradacion elegante."""
    num_classes = int(cfg.num_classes)

    if dataset is not None:
        names = _names_from_dataset(dataset, num_classes)
        if names:
            return names
    else:
        try:  # 1a) `Flowers102.classes`: atributo de CLASE, sin instanciar ni red
            Flowers102 = _flowers102_class()
        except Exception:
            Flowers102 = None
        names = _names_from_dataset(Flowers102, num_classes)
        if names:
            return names

        try:  # 1b) instancia real, por si esa version expone los nombres de otra forma
            opened = (
                Flowers102(root=str(cfg.data_root), split="train", download=False) if Flowers102 is not
None else None
            )
        except Exception:
            opened = None
        names = _names_from_dataset(opened, num_classes)
        if names:
            return names

    names = _names_from_cache(cfg)  # 2) cache local (offline-friendly)
    if names:
        return names

    names = _names_from_asset(num_classes)  # 3) respaldo offline versionado en el repo
    if names:
        return names

    names = _download_cat_to_name(cfg)  # 4) descarga de Oxford + cache
    if names:
        return names

    return _fallback_names(num_classes)  # 5) nunca fallar


def get_class_names(cfg: Config) -> list[str]:
    """102 nombres legibles en orden de etiqueta (``class_names[i]`` <-> etiqueta ``i``).

    Orden de intentos:
      1. nombres propios del dataset (``torchvision.datasets.Flowers102.classes``,
         es un atributo de clase: funciona sin datos y sin internet);
      2. cache ``<data_root>/cat_to_name.json``;
      3. respaldo offline ``assets/cat_to_name.json`` del repo (claves 0-based);
      4. descarga desde Oxford (se cachea en ``<data_root>/cat_to_name.json``);
      5. ``['especie_001', ..., 'especie_102']``.
```

```
    Nunca lanza excepcion por falta de datos o de internet.
    """
    return _resolve_class_names(cfg, dataset=None)


# ---------------------------------------------------------------- data loaders
def build_dataloaders(cfg: Config) -> tuple["DataLoader", "DataLoader", "DataLoader", list[str]]:
    """DataLoaders oficiales de Flowers-102 + nombres de clase.

    Devuelve ``(train_dl, val_dl, test_dl, class_names)``. Splits oficiales
    (1020 / 1020 / 6149); en ``cfg.debug`` se usan subconjuntos de
    ``cfg.debug_per_class`` imagenes por clase.
    """
    from torch.utils.data import DataLoader

    Flowers102 = _flowers102_class()
    root = Path(cfg.data_root)
    root.mkdir(parents=True, exist_ok=True)

    # pin_memory solo cuando hay CUDA (en CPU no aporta y consume RAM).
    pin_memory = bool(_torch().cuda.is_available())
    num_workers = int(cfg.num_workers)

    datasets: dict[str, Any] = {}
    loaders: dict[str, Any] = {}
    for split in SPLITS:
        dataset = Flowers102(
            root=str(root),
            split=split,
            transform=build_transforms(cfg, train=(split == "train")),
            download=True,
        )
        datasets[split] = dataset
        if cfg.debug:
            dataset = _debug_subset(dataset, per_class=cfg.debug_per_class, seed=int(cfg.seed) + 17 *
SPLITS.index(split))
        loaders[split] = DataLoader(
            dataset,
            batch_size=int(cfg.batch_size),
            shuffle=(split == "train"),  # val/test SIN barajar (contrato §3)
            num_workers=num_workers,
            pin_memory=pin_memory,
            drop_last=False,
        )

    class_names = _resolve_class_names(cfg, dataset=datasets["train"])
    return loaders["train"], loaders["val"], loaders["test"], class_names
```

src/model.py

El cerebro: backbone preentrenado, cabeza nueva y checkpoints. (395 líneas)

```
"""Modelo de clasificacion de flores (Flowers-102) con backbones de torchvision.

CONTRATO (seccion 3 de CONTRATOS.md, propietario SA-02):

    class FlowerClassifier(nn.Module):      # atributos OBLIGATORIOS: .backbone y .head
        def forward(self, x) -> Tensor      # (B,3,H,W) -> (B, num_classes) logits

    build_model(num_classes=102, backbone="efficientnet_b0",
                pretrained=True, dropout=0.3) -> FlowerClassifier
    set_backbone_trainable(model, trainable: bool) -> None
    build_optimizer(model, cfg: Config) -> torch.optim.Optimizer
    save_checkpoint(path, model, class_names: list[str], cfg: Config, **extra) -> None
    load_checkpoint(path, device="cpu") -> tuple[FlowerClassifier, list[str], dict]

Notas de diseno:
  - Todo el modulo funciona en CPU y en GPU: no hay ningun ``.cuda()`` duro, el
    device se pasa como argumento (``load_checkpoint(..., device=...)``) o se
    resuelve con ``src.utils.resolve_device`` cuando hace falta.
  - ``pretrained=True`` degrada con elegancia a ``pretrained=False`` si la
    descarga de pesos falla (Colab a veces bloquea ``download.pytorch.org``).
  - No hay rutas absolutas de Windows: las rutas se reciben como argumento.
"""

from __future__ import annotations

import warnings
from pathlib import Path
from typing import Any, Callable, Iterable

import torch
import torch.nn as nn
from torchvision import models as tv_models

try:  # import relativo (paquete `src`) con respaldo absoluto (cwd en sys.path)
    from .config import FLOWERS102_CLASSES, IMAGE_SIZE, Config
    from .utils import resolve_device
except ImportError:  # pragma: no cover - depende de como se importe el modulo
    from src.config import FLOWERS102_CLASSES, IMAGE_SIZE, Config
    from src.utils import resolve_device

__all__ = [
    "SUPPORTED_BACKBONES",
    "CHECKPOINT_KEYS",
    "FlowerClassifier",
    "build_model",
    "set_backbone_trainable",
    "build_optimizer",
    "save_checkpoint",
    "load_checkpoint",
]

#: Backbones soportados. El primero es el defecto del proyecto.
SUPPORTED_BACKBONES: tuple[str, ...] = (
    "efficientnet_b0",
    "resnet50",
    "mobilenet_v3_large",
)

#: Claves obligatorias del checkpoint (contrato compartido con SA-05 y SA-06).
CHECKPOINT_KEYS: tuple[str, ...] = (
    "model_state",
    "class_names",
    "backbone",
    "num_classes",
```

```
    "img_size",
    "config",
    "epoch",
    "val_acc",
)


# ---------------------------------------------------------------------------- #
# Modelo
# ---------------------------------------------------------------------------- #
class FlowerClassifier(nn.Module):
    """Clasificador de flores = extractor de caracteristicas (``backbone``) + ``head``.

    Los atributos ``.backbone`` y ``.head`` son obligatorios por contrato: SA-05
    los usa para congelar/descongelar y para armar los grupos de parametros, y
    SA-06 / SA-07 pueden inspeccionarlos (Grad-CAM, conteo de parametros, etc.).

    Args:
        backbone: red convolucional que devuelve un mapa (B, C, H, W).
        in_features: numero de canales de salida del backbone.
        num_classes: numero de clases de salida (102 para Flowers-102).
        dropout: probabilidad de dropout antes de la capa lineal final.
        backbone_name: nombre canonico del backbone (para el checkpoint).
        pretrained: si los pesos del backbone son preentrenados (informativo).
    """

    def __init__(
        self,
        backbone: nn.Module,
        in_features: int,
        num_classes: int = FLOWERS102_CLASSES,
        dropout: float = 0.3,
        backbone_name: str | None = None,
        pretrained: bool = False,
    ) -> None:
        super().__init__()
        self.backbone = backbone
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.head = nn.Sequential(
            nn.Dropout(p=float(dropout)),
            nn.Linear(int(in_features), int(num_classes)),
        )
        # Metadatos utiles (no son parametros, no entran en el state_dict).
        self.backbone_name = backbone_name
        self.in_features = int(in_features)
        self.num_classes = int(num_classes)
        self.dropout = float(dropout)
        self.pretrained = bool(pretrained)

    # ------------------------------------------------------------------ utils
    def forward_features(self, x: torch.Tensor) -> torch.Tensor:
        """Vector de caracteristicas (B, in_features) antes de la cabeza lineal."""
        feats = self.backbone(x)
        if feats.dim() > 2:
            feats = self.avgpool(feats)
        return torch.flatten(feats, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """(B, 3, H, W) -> (B, num_classes) logits (sin softmax)."""
        return self.head(self.forward_features(x))


# ---------------------------------------------------------------------------- #
# Construccion de backbones
# ---------------------------------------------------------------------------- #
def _build_efficientnet_b0(pretrained: bool) -> tuple[nn.Module, int]:
    weights = tv_models.EfficientNet_B0_Weights.DEFAULT if pretrained else None
    net = tv_models.efficientnet_b0(weights=weights)
```

```
    in_features = int(net.classifier[-1].in_features)  # 1280
    return net.features, in_features


def _build_resnet50(pretrained: bool) -> tuple[nn.Module, int]:
    weights = tv_models.ResNet50_Weights.DEFAULT if pretrained else None
    net = tv_models.resnet50(weights=weights)
    in_features = int(net.fc.in_features)  # 2048
    # Se conserva todo salvo la capa fc; incluye el avgpool global.
    backbone = nn.Sequential(*list(net.children())[:-1])
    return backbone, in_features


def _build_mobilenet_v3_large(pretrained: bool) -> tuple[nn.Module, int]:
    weights = tv_models.MobileNet_V3_Large_Weights.DEFAULT if pretrained else None
    net = tv_models.mobilenet_v3_large(weights=weights)
    in_features = int(net.classifier[0].in_features)  # 960
    return net.features, in_features


_BACKBONE_BUILDERS: dict[str, Callable[[bool], tuple[nn.Module, int]]] = {
    "efficientnet_b0": _build_efficientnet_b0,
    "resnet50": _build_resnet50,
    "mobilenet_v3_large": _build_mobilenet_v3_large,
}


def build_model(
    num_classes: int = FLOWERS102_CLASSES,
    backbone: str = "efficientnet_b0",
    pretrained: bool = True,
    dropout: float = 0.3,
) -> FlowerClassifier:
    """Crea un :class:`FlowerClassifier` con la cabeza adaptada a ``num_classes``.

    Args:
        num_classes: numero de clases de salida.
        backbone: uno de :data:`SUPPORTED_BACKBONES`.
        pretrained: si es ``True`` intenta descargar los pesos ImageNet; si la
            descarga falla (sin internet / Colab bloqueado) avisa por
            ``warnings.warn`` y continua con pesos aleatorios.
        dropout: dropout antes de la capa lineal final.

    Raises:
        ValueError: si ``backbone`` no esta soportado (el mensaje lista los validos).
    """
    if not isinstance(backbone, str) or backbone not in _BACKBONE_BUILDERS:
        soportados = ", ".join(SUPPORTED_BACKBONES)
        raise ValueError(
            f"Backbone no soportado: {backbone!r}. Soportados: {soportados}."
        )

    builder = _BACKBONE_BUILDERS[backbone]
    used_pretrained = bool(pretrained)

    try:
        features, in_features = builder(used_pretrained)
    except Exception as exc:  # noqa: BLE001 - cualquier fallo de descarga/carga
        if not pretrained:
            # El error es real (no de red): no lo enmascaramos.
            raise
        warnings.warn(
            f"No se pudieron cargar los pesos preentrenados de '{backbone}' "
            f"({type(exc).__name__}: {exc}). Se continua con pretrained=False "
            "(pesos aleatorios); el entrenamiento sigue siendo valido.",
            RuntimeWarning,
            stacklevel=2,
        )
```

```
        used_pretrained = False
        features, in_features = builder(False)

    return FlowerClassifier(
        backbone=features,
        in_features=in_features,
        num_classes=num_classes,
        dropout=dropout,
        backbone_name=backbone,
        pretrained=used_pretrained,
    )


# ---------------------------------------------------------------------------- #
# Congelar / descongelar
# ---------------------------------------------------------------------------- #
def set_backbone_trainable(model: FlowerClassifier | nn.Module, trainable: bool) -> None:
    """Activa o desactiva ``requires_grad`` en **todos** los parametros del backbone.

    Se usa en las primeras epocas (``cfg.freeze_epochs``) para entrenar solo la
    cabeza. No cambia el modo train/eval del backbone.
    """
    backbone = getattr(model, "backbone", None)
    if backbone is None:
        raise AttributeError(
            "set_backbone_trainable espera un modelo con atributo .backbone "
            f"(recibido {type(model).__name__})."
        )
    for param in backbone.parameters():
        param.requires_grad = bool(trainable)


# ---------------------------------------------------------------------------- #
# Optimizador con LR diferencial
# ---------------------------------------------------------------------------- #
def build_optimizer(model: FlowerClassifier | nn.Module, cfg: Config) -> torch.optim.Optimizer:
    """``AdamW`` con dos grupos: cabeza con ``cfg.lr`` y backbone con ``cfg.backbone_lr``.

    Ambos grupos usan ``cfg.weight_decay``. El grupo del backbone incluye sus
    parametros aunque esten congelados en el momento de crear el optimizador:
    ``AdamW`` ignora los que tienen ``grad is None`` y asi el mismo optimizador
    sigue sirviendo cuando SA-05 descongela el backbone.
    """
    backbone = getattr(model, "backbone", None)
    head = getattr(model, "head", None)
    if backbone is None or head is None:
        raise AttributeError(
            "build_optimizer espera un modelo con atributos .backbone y .head "
            f"(recibido {type(model).__name__})."
        )

    head_params = [p for p in head.parameters()]
    backbone_params = [p for p in backbone.parameters()]

    param_groups: list[dict[str, Any]] = []
    if head_params:
        param_groups.append(
            {
                "params": head_params,
                "lr": float(cfg.lr),
                "weight_decay": float(cfg.weight_decay),
                "name": "head",
            }
        )
    if backbone_params:
        param_groups.append(
            {
                "params": backbone_params,
```

```
                "lr": float(cfg.backbone_lr),
                "weight_decay": float(cfg.weight_decay),
                "name": "backbone",
            }
        )
    if not param_groups:
        raise ValueError("El modelo no tiene parametros: no se puede crear el optimizador.")

    return torch.optim.AdamW(param_groups)


# ---------------------------------------------------------------------------- #
# Checkpoints (contrato compartido con SA-05 y SA-06)
# ---------------------------------------------------------------------------- #
def _model_metadata(model: FlowerClassifier | nn.Module, cfg: Config) -> tuple[str, int]:
    """(nombre del backbone, num_classes) tomados del modelo, con respaldo en cfg."""
    backbone_name = getattr(model, "backbone_name", None) or getattr(cfg, "backbone", None)
    num_classes = getattr(model, "num_classes", None)
    if num_classes is None:
        head = getattr(model, "head", None)
        num_classes = getattr(head[-1], "out_features", None) if head is not None else None
    if num_classes is None:
        num_classes = getattr(cfg, "num_classes", FLOWERS102_CLASSES)
    return str(backbone_name), int(num_classes)


def save_checkpoint(
    path: str | Path,
    model: FlowerClassifier | nn.Module,
    class_names: Iterable[str],
    cfg: Config,
    **extra: Any,
) -> None:
    """Guarda el checkpoint con el formato congelado del contrato.

    Claves siempre presentes: ``model_state``, ``class_names``, ``backbone``,
    ``num_classes``, ``img_size``, ``config``, ``epoch``, ``val_acc``.
    ``**extra`` permite anadir/sobrescribir cualquier otra clave (por ejemplo
    ``epoch`` y ``val_acc`` que envia SA-05); los ``None`` no se escriben.
    """
    backbone_name, num_classes = _model_metadata(model, cfg)

    payload: dict[str, Any] = {
        "model_state": model.state_dict(),
        "class_names": list(class_names),
        "backbone": backbone_name,
        "num_classes": num_classes,
        "img_size": int(getattr(cfg, "img_size", IMAGE_SIZE)),
        "config": cfg.to_dict() if hasattr(cfg, "to_dict") else dict(cfg),
        "epoch": int(extra.pop("epoch", -1)),
        "val_acc": float(extra.pop("val_acc", 0.0)),
    }
    for key, value in extra.items():
        if value is not None:
            payload[key] = value

    target = Path(path)
    target.parent.mkdir(parents=True, exist_ok=True)
    torch.save(payload, target)


def load_checkpoint(
    path: str | Path,
    device: str | torch.device = "cpu",
) -> tuple[FlowerClassifier, list[str], dict]:
    """Carga un checkpoint y reconstruye el modelo en ``.eval()`` y en ``device``.

    Returns:
```

```
    ``(model, class_names, meta)`` donde ``meta`` es el checkpoint sin
    ``model_state`` (incluye ``config``, ``img_size``, ``epoch``, ``val_acc``
    y cualquier clave extra escrita por SA-05).

Raises:
    FileNotFoundError: si la ruta no existe (entrena primero).
    KeyError: si al checkpoint le faltan claves obligatorias.
"""
target = Path(path)
if not target.is_file():
    raise FileNotFoundError(
        f"No existe el checkpoint '{target}'. Entrena primero con "
        "`python -m src.train` para generarlo."
    )

try:
    checkpoint = torch.load(target, map_location="cpu", weights_only=False)
except TypeError:  # torch muy antiguo sin el argumento weights_only
    checkpoint = torch.load(target, map_location="cpu")

if not isinstance(checkpoint, dict) or "model_state" not in checkpoint:
    raise KeyError(
        f"El checkpoint '{target}' no tiene el formato esperado "
        f"(claves obligatorias: {', '.join(CHECKPOINT_KEYS)})."
    )

faltantes = [k for k in CHECKPOINT_KEYS if k not in checkpoint]
if faltantes:
    raise KeyError(
        f"Al checkpoint '{target}' le faltan claves obligatorias: {faltantes}."
    )

ckpt_cfg = checkpoint.get("config") or {}
backbone = str(checkpoint["backbone"])
num_classes = int(checkpoint["num_classes"])
dropout = float(ckpt_cfg.get("dropout", 0.3))

model = build_model(
    num_classes=num_classes,
    backbone=backbone,
    pretrained=False,  # el state_dict ya trae los pesos: nunca hay descarga
    dropout=dropout,
)
model.load_state_dict(checkpoint["model_state"], strict=True)

if isinstance(device, torch.device):
    torch_device = device
elif str(device).lower() == "auto":  # comodidad para SA-06 / SA-07
    torch_device = resolve_device("auto")
else:
    torch_device = torch.device(str(device))
model.to(torch_device)
model.eval()

class_names = [str(name) for name in checkpoint["class_names"]]
meta = {k: v for k, v in checkpoint.items() if k != "model_state"}
meta["img_size"] = int(checkpoint["img_size"])
meta["device"] = str(torch_device)
return model, class_names, meta
```

src/train.py

El bucle de entrenamiento. (661 líneas)

```
"""Entrenamiento de Flowers-102 (propietario: SA-05).

CONTRATO (CONTRATOS.md seccion 3, `src/train.py`):

    train(cfg: Config) -> dict          # entrena y devuelve el historial
    # CLI: python -m src.train --epochs 5 --debug --device cpu

Detalles congelados que respeta este modulo:
  * `freeze_epochs`: las primeras epocas entrenan SOLO la cabeza
    (`set_backbone_trainable(model, False)`); despues se descongela el backbone con
    el MISMO optimizador (el grupo "backbone" ya estaba en el AdamW de
    `build_optimizer`, que es [0] = cabeza con `cfg.lr` y [1] = backbone con
    `cfg.backbone_lr`).
  * Scheduler: warmup de `cfg.warmup_epochs` + coseno, aplicado a los dos grupos.
  * AMP (`torch.amp.autocast` + `torch.amp.GradScaler`) SOLO si el device es CUDA
    y `cfg.amp`; en CPU queda desactivado y sin avisos.
  * `cfg.checkpoint_path` se guarda SOLO cuando mejora `val_acc`, con `epoch=` y
    `val_acc=` como `**extra` (asi el checkpoint cumple las 8 claves del contrato).
  * Artefactos: `best.pt`, `history.json`, `config_usada.json` y `figuras/curvas.png`
    (matplotlib en backend "Agg": nunca hay `.cuda()` duro ni ventanas).

Claves del historial (listas por epoca salvo las seis ultimas):
    epochs, train_loss, train_acc, val_loss, val_acc, val_top5, lr,
    best_val_acc, best_epoch, epochs_run, device, num_params, segundos

Convenio de numeracion: `history["epochs"]` y `best_epoch` son **1-based** (la
primera epoca es 1) para que los informes y la celda de curvas del notebook de
SA-03 lean "epoca 3" de forma natural. El `epoch` del checkpoint usa el mismo
convenio. `history["lr"]` guarda el LR del grupo de la cabeza que se aplico
durante la epoca (se lee justo antes del `scheduler.step()`).

CLI congelada (seccion 3.b) mas `--num-workers N` (extension acordada con el
orquestador; el default de `Config` sigue siendo 2):
    python -m src.train [--epochs N] [--batch-size N] [--backbone B] [--data-root D]
                        [--out-dir O] [--device auto|cuda|cpu] [--debug] [--seed N]
                        [--num-workers N]

Robustez de datos: si la iteracion del DataLoader con `num_workers > 0` falla por
multiprocessing / permisos (sandbox de Windows: los workers no pueden abrir named
pipes), `train` avisa por `warnings.warn` y REINTENTA UNA VEZ esa etapa con
`num_workers=0`, en vez de abortar el entrenamiento a mitad. En Colab/Linux los
workers funcionan y esta red de seguridad no se activa.
"""

from __future__ import annotations

import argparse
import time
import warnings
from dataclasses import replace
from pathlib import Path
from typing import Any, Iterable

import torch
import torch.nn as nn

try:  # import relativo (paquete `src`) con respaldo absoluto (cwd en sys.path)
    from .config import Config
    from .data import build_dataloaders
    from .model import (
        build_model,
        build_optimizer,
        save_checkpoint,
        set_backbone_trainable,
```

```
    )
    from .utils import count_params, describe_device, resolve_device, save_json, set_seed
except ImportError:  # pragma: no cover - depende de como se importe el modulo
    from src.config import Config
    from src.data import build_dataloaders
    from src.model import (
        build_model,
        build_optimizer,
        save_checkpoint,
        set_backbone_trainable,
    )
    from src.utils import count_params, describe_device, resolve_device, save_json, set_seed

__all__ = [
    "HISTORY_LIST_KEYS",
    "HISTORY_SCALAR_KEYS",
    "HISTORY_KEYS",
    "train",
    "build_scheduler",
    "config_from_args",
    "build_argparser",
    "main",
]

#: Claves del historial que son listas con una entrada por epoca.
HISTORY_LIST_KEYS: tuple[str, ...] = (
    "epochs",
    "train_loss",
    "train_acc",
    "val_loss",
    "val_acc",
    "val_top5",
    "lr",
)
#: Claves del historial que son escalares (resumen de la corrida).
HISTORY_SCALAR_KEYS: tuple[str, ...] = (
    "best_val_acc",
    "best_epoch",
    "epochs_run",
    "device",
    "num_params",
    "segundos",
)
#: Todas las claves del historial, en el orden del contrato.
HISTORY_KEYS: tuple[str, ...] = HISTORY_LIST_KEYS + HISTORY_SCALAR_KEYS

#: Directorio de salida por defecto del CLI cuando se usa `--debug` (decision §5
#: del contrato: el modo debug NO debe pisar los resultados definitivos).
DEBUG_OUT_DIR = "resultados/debug"

#: Pistas de texto que identifican un fallo de los workers del DataLoader
#: (multiprocessing / named pipes / memoria compartida).
_PISTAS_WORKERS: tuple[str, ...] = (
    "winerror 5",
    "acceso denegado",
    "access is denied",
    "permission",
    "named pipe",
    "pipe",
    "dataloader",
    "data loader",
    "worker",
    "multiprocessing",
    "shared memory",
    "shared_memory",
    "semaphore",
    "queue",
    "eof",
```

```
)


# ---------------------------------------------------------------------------- #
# Utilidades internas
# ---------------------------------------------------------------------------- #
def _progress(iterable: Iterable, **kwargs: Any):
    """`tqdm` si esta disponible; si no, el iterable tal cual (sin romper nada)."""
    try:
        from tqdm.auto import tqdm
    except ImportError:  # pragma: no cover - tqdm esta en requirements.txt
        return iterable
    return tqdm(iterable, **kwargs)


def _topk_correct(logits: torch.Tensor, targets: torch.Tensor, k: int) -> int:
    """Numero de aciertos top-``k`` (``k`` se recorta a `num_classes`)."""
    num_classes = int(logits.shape[1])
    k = max(1, min(int(k), num_classes))
    _, pred = logits.topk(k, dim=1, largest=True, sorted=True)
    pred = pred.t()
    correct = pred.eq(targets.view(1, -1).expand_as(pred))
    return int(correct[:k].reshape(-1).sum().item())


def _move(batch: Any, device: torch.device) -> tuple[torch.Tensor, torch.Tensor]:
    """Lleva un lote (imagenes, etiquetas) al device resolviendo listas/tuplas."""
    images, targets = batch[0], batch[1]
    images = images.to(device, non_blocking=True)
    targets = targets.to(device, non_blocking=True).long()
    return images, targets


def _run_train_epoch(
    model: nn.Module,
    loader: Iterable,
    criterion: nn.Module,
    optimizer: torch.optim.Optimizer,
    scaler: Any,
    device: torch.device,
    amp_enabled: bool,
    epoch: int,
    epochs: int,
) -> tuple[float, float]:
    """Una epoca de entrenamiento. Devuelve ``(loss_media, acc_top1)``."""
    model.train()
    total_loss = 0.0
    total_correct = 0
    seen = 0

    bar = _progress(loader, desc=f"train {epoch}/{epochs}", leave=False)
    for batch in bar:
        images, targets = _move(batch, device)
        optimizer.zero_grad(set_to_none=True)
        with torch.amp.autocast("cuda", enabled=amp_enabled):
            logits = model(images)
            loss = criterion(logits, targets)
        if scaler is not None:
            scaler.scale(loss).backward()
            scaler.step(optimizer)
            scaler.update()
        else:
            loss.backward()
            optimizer.step()

        batch_size = int(targets.size(0))
        seen += batch_size
        total_loss += float(loss.detach()) * batch_size
```

```
        total_correct += _topk_correct(logits.detach(), targets, 1)
        if hasattr(bar, "set_postfix"):
            bar.set_postfix(
                loss=f"{total_loss / max(seen, 1):.4f}",
                acc=f"{total_correct / max(seen, 1):.3f}",
            )
    if hasattr(bar, "close"):
        bar.close()

    return total_loss / max(seen, 1), total_correct / max(seen, 1)


@torch.no_grad()
def _run_eval_epoch(
    model: nn.Module,
    loader: Iterable,
    criterion: nn.Module,
    device: torch.device,
    amp_enabled: bool,
    topk: int = 5,
) -> tuple[float, float, float]:
    """Evalua un split. Devuelve ``(loss_media, acc_top1, acc_top5)``."""
    model.eval()
    total_loss = 0.0
    total_top1 = 0
    total_top5 = 0
    seen = 0

    for batch in loader:
        images, targets = _move(batch, device)
        with torch.amp.autocast("cuda", enabled=amp_enabled):
            logits = model(images)
            loss = criterion(logits, targets)

        batch_size = int(targets.size(0))
        seen += batch_size
        total_loss += float(loss.detach()) * batch_size
        total_top1 += _topk_correct(logits, targets, 1)
        total_top5 += _topk_correct(logits, targets, topk)

    seen = max(seen, 1)
    return total_loss / seen, total_top1 / seen, total_top5 / seen


def _es_error_de_workers(exc: BaseException) -> bool:
    """True si la excepcion huele a fallo de los procesos hijo del DataLoader.

    En el sandbox de Windows de este entorno los workers no pueden abrir *named
    pipes* y revientan con ``PermissionError [WinError 5]``; en Linux/Colab
    ``num_workers > 0`` funciona bien. En vez de abortar un entrenamiento ya
    empezado, `train` reintenta UNA vez la etapa con ``num_workers=0``.
    """
    if not isinstance(exc, (OSError, RuntimeError, EOFError, MemoryError)):
        return False
    if isinstance(exc, PermissionError):  # el caso observado: WinError 5
        return True
    texto = f"{type(exc).__name__}: {exc}".lower()
    return any(pista in texto for pista in _PISTAS_WORKERS)


def _make_grad_scaler() -> Any:
    """`torch.amp.GradScaler("cuda")` (API del contrato) con respaldo para torch < 2.3.

    Solo se llama cuando AMP esta realmente activo (device CUDA + `cfg.amp`), asi
    que en CPU no se construye nunca y no hay avisos.
    """
    try:
        return torch.amp.GradScaler("cuda", enabled=True)
```

```
    except (AttributeError, TypeError):  # pragma: no cover - torch antiguo en Colab
        return torch.cuda.amp.GradScaler(enabled=True)


def build_scheduler(optimizer: torch.optim.Optimizer, cfg: Config) -> Any:
    """Scheduler warmup + coseno sobre TODOS los grupos de parametros.

    Con ``cfg.warmup_epochs > 0`` se usa
    ``SequentialLR([LinearLR, CosineAnnealingLR], milestones=[warmup_epochs])``;
    con ``warmup_epochs == 0`` se usa solo ``CosineAnnealingLR``. En ambos casos el
    factor de escala se aplica por grupo, asi que se respeta el LR diferencial
    cabeza/backbone que armo `build_optimizer`.
    """
    warmup = max(0, int(cfg.warmup_epochs))
    epochs = max(1, int(cfg.epochs))
    cosine_epochs = max(1, epochs - warmup)

    cosine = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=cosine_epochs, eta_min=0.0
    )
    if warmup <= 0:
        return cosine

    warmup_sched = torch.optim.lr_scheduler.LinearLR(
        optimizer, start_factor=0.1, end_factor=1.0, total_iters=warmup
    )
    return torch.optim.lr_scheduler.SequentialLR(
        optimizer, schedulers=[warmup_sched, cosine], milestones=[warmup]
    )


def plot_history(history: dict, path: str | Path) -> Path:
    """Guarda `curvas.png` (loss, accuracy y learning rate por epoca).

    Usa el backend "Agg" ANTES de importar `pyplot`, salvo que `pyplot` ya este
    importado (p. ej. dentro de un notebook, donde cambiar el backend a "Agg"
    romperia el `plt.show()` de las celdas siguientes).
    """
    import sys

    import matplotlib

    if "matplotlib.pyplot" not in sys.modules:  # no pisar el backend de Jupyter
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    target = Path(path)
    target.parent.mkdir(parents=True, exist_ok=True)

    epochs = list(history.get("epochs") or [])
    fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))

    if epochs:
        axes[0].plot(epochs, history["train_loss"], marker="o", label="train")
        axes[0].plot(epochs, history["val_loss"], marker="o", label="val")
        axes[1].plot(epochs, history["train_acc"], marker="o", label="train")
        axes[1].plot(epochs, history["val_acc"], marker="o", label="val")
        axes[1].plot(epochs, history["val_top5"], marker="s", alpha=0.6, label="val top-5")
        axes[2].plot(epochs, history["lr"], marker="o", color="tab:green")
        best_epoch = history.get("best_epoch")
        if best_epoch:
            axes[1].axvline(best_epoch, ls="--", lw=1, color="tab:red", alpha=0.7)
            axes[1].annotate(
                f"mejor val_acc={history.get('best_val_acc', 0):.3f}",
                xy=(best_epoch, history.get("best_val_acc", 0)),
                xytext=(5, 8),
                textcoords="offset points",
                fontsize=8,
```

```
                color="tab:red",
            )
    else:  # corrida de 0 epocas: la figura existe igualmente, pero vacia
        for axis in axes:
            axis.text(0.5, 0.5, "sin epocas registradas", ha="center", va="center")

    axes[0].set_xlabel("epoca")
    axes[0].set_ylabel("loss")
    axes[0].set_title("Perdida (label smoothing incluido)")
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    axes[1].set_xlabel("epoca")
    axes[1].set_ylabel("accuracy")
    axes[1].set_title("Accuracy top-1 / top-5")
    axes[1].legend()
    axes[1].grid(alpha=0.3)

    axes[2].set_xlabel("epoca")
    axes[2].set_ylabel("learning rate")
    axes[2].set_title("LR de la cabeza (warmup + coseno)")
    axes[2].grid(alpha=0.3)

    fig.suptitle(f"Entrenamiento Flowers-102 ({history.get('device', '?')})")
    fig.tight_layout()
    fig.savefig(target, dpi=110)
    plt.close(fig)
    return target


# ---------------------------------------------------------------------------- #
# Entrenamiento
# ---------------------------------------------------------------------------- #
def train(cfg: Config) -> dict:
    """Entrena el clasificador y devuelve el historial (JSON serializable).

    Args:
        cfg: hiperparametros y rutas (ver `src/config.py`).

    Returns:
        Diccionario con las claves de :data:`HISTORY_KEYS`. Las listas tienen una
        entrada por epoca ejecutada; `best_val_acc`, `best_epoch`, `epochs_run`,
        `device`, `num_params` y `segundos` resumen la corrida.

    Efectos secundarios: crea `cfg.out_dir` (y `figuras/`, `predicciones/`) y
    escribe `best.pt`, `history.json`, `config_usada.json` y `figuras/curvas.png`.
    """
    set_seed(int(cfg.seed))
    device = resolve_device(cfg.device)
    cfg.ensure_dirs()

    print("=" * 72)
    print("Entrenamiento Flowers-102")
    print(f"  dispositivo : {describe_device(device)}")
    print(f"  backbone    : {cfg.backbone} (pretrained={cfg.pretrained}, dropout={cfg.dropout})")
    print(
        f"  epocas      : {cfg.epochs} | batch: {cfg.batch_size} | img: {cfg.img_size}"
        f" | clases: {cfg.num_classes}"
    )
    print(
        f"  lr cabeza/backbone: {cfg.lr}/{cfg.backbone_lr} | warmup: {cfg.warmup_epochs}"
        f" | freeze: {cfg.freeze_epochs} | amp: {cfg.amp}"
    )
    print(f"  debug       : {cfg.debug} | salida: {Path(cfg.out_dir)}")
    print("=" * 72)

    # ------------------------------------------------------------------ datos
    train_dl, val_dl, _test_dl, class_names = build_dataloaders(cfg)
```

```
num_workers_efectivo = int(cfg.num_workers)
print(
    f"  lotes: train={len(train_dl)} val={len(val_dl)} | "
    f"imagenes: train={len(train_dl.dataset)} val={len(val_dl.dataset)} | "
    f"nombres de clase: {len(class_names)} | num_workers: {num_workers_efectivo}"
)

# ------------------------------------------------------------------ modelo
model = build_model(
    num_classes=int(cfg.num_classes),
    backbone=str(cfg.backbone),
    pretrained=bool(cfg.pretrained),
    dropout=float(cfg.dropout),
).to(device)

freeze_epochs = max(0, int(cfg.freeze_epochs))
if freeze_epochs > 0:
    set_backbone_trainable(model, False)  # primeras epocas: solo la cabeza
else:
    set_backbone_trainable(model, True)

# El optimizador SIEMPRE se construye una sola vez, con los dos grupos:
# [0] cabeza (cfg.lr) y [1] backbone (cfg.backbone_lr), incluso si el backbone
# esta congelado ahora mismo. Al descongelar se reutiliza el mismo objeto.
optimizer = build_optimizer(model, cfg)
scheduler = build_scheduler(optimizer, cfg)

criterion = nn.CrossEntropyLoss(label_smoothing=float(cfg.label_smoothing))
topk = max(1, min(int(cfg.topk), int(cfg.num_classes)))

amp_enabled = bool(cfg.amp) and device.type == "cuda"
scaler = _make_grad_scaler() if amp_enabled else None
if bool(cfg.amp) and not amp_enabled:
    print("  AMP solicitado pero desactivado: el device no es CUDA (CPU).")
if amp_enabled:
    print("  AMP activado (autocast fp16 + GradScaler).")

# ---------------------------------------------------------------- historial
history: dict[str, Any] = {key: [] for key in HISTORY_LIST_KEYS}
best_val_acc = -1.0
best_epoch = -1
start = time.perf_counter()
epochs = max(0, int(cfg.epochs))
epochs_sin_mejora = 0

def _etapa(etapa: str, epoch: int) -> tuple[float, ...]:
    """Corre una etapa de la epoca leyendo los loaders ACTUALES del scope."""
    if etapa == "train":
        return _run_train_epoch(
            model, train_dl, criterion, optimizer, scaler, device, amp_enabled, epoch, epochs
        )
    return _run_eval_epoch(model, val_dl, criterion, device, amp_enabled, topk=topk)

def _etapa_con_fallback(etapa: str, epoch: int) -> tuple[float, ...]:
    """Como `_etapa`, pero si fallan los workers del DataLoader reintenta UNA vez.

    El reintento reconstruye los loaders con ``num_workers=0`` (mismo
    `data_root`, mismos subconjuntos) y vuelve a correr SOLO esa etapa, de modo
    que no se pierde el entrenamiento ya hecho. En Colab/Linux no se activa
    nunca: alli ``num_workers=2`` funciona y acelera.
    """
    nonlocal num_workers_efectivo, train_dl, val_dl, _test_dl, class_names
    try:
        return _etapa(etapa, epoch)
    except Exception as exc:  # noqa: BLE001 - se re-lanza si no es de workers
        if num_workers_efectivo <= 0 or not _es_error_de_workers(exc):
            raise
        mensaje = (
```

```
            f"La etapa '{etapa}' fallo con num_workers={num_workers_efectivo} "
            f"({type(exc).__name__}: {exc}). Se reintenta una sola vez con "
            "num_workers=0 (en Colab/Linux los workers funcionan; en el sandbox "
            "de Windows no pueden abrir named pipes)."
        )
        print("Aviso: " + mensaje)
        warnings.warn(mensaje, RuntimeWarning, stacklevel=2)

        data_cfg = replace(cfg, num_workers=0)
        train_dl, val_dl, _test_dl, class_names = build_dataloaders(data_cfg)
        num_workers_efectivo = 0
        return _etapa(etapa, epoch)  # unico reintento: si falla, propaga

for epoch_index in range(epochs):
    epoch = epoch_index + 1  # 1-based (ver docstring del modulo)
    congelado = epoch <= freeze_epochs
    set_backbone_trainable(model, not congelado)

    train_loss, train_acc = _etapa_con_fallback("train", epoch)
    val_loss, val_acc, val_top5 = _etapa_con_fallback("val", epoch)

    # LR del grupo de la cabeza (indice 0) USADO en esta epoca: se lee antes
    # del `scheduler.step()` para que la curva de `history["lr"]` muestre el LR
    # que realmente se aplico (warmup subiendo, coseno bajando) en vez del que
    # se aplicara en la epoca siguiente.
    head_lr = float(optimizer.param_groups[0]["lr"])
    scheduler.step()

    history["epochs"].append(int(epoch))
    history["train_loss"].append(float(train_loss))
    history["train_acc"].append(float(train_acc))
    history["val_loss"].append(float(val_loss))
    history["val_acc"].append(float(val_acc))
    history["val_top5"].append(float(val_top5))
    history["lr"].append(head_lr)

    mejora = val_acc > best_val_acc
    marca = ""
    if mejora:
        best_val_acc = float(val_acc)
        best_epoch = int(epoch)
        epochs_sin_mejora = 0
        save_checkpoint(
            cfg.checkpoint_path,
            model,
            class_names,
            cfg,
            epoch=epoch,
            val_acc=val_acc,
        )
        marca = " *mejor"
    else:
        epochs_sin_mejora += 1

    estado = "backbone congelado" if congelado else "backbone entrenando"
    print(
        f"[{epoch:>3}/{epochs}] {estado} | "
        f"train loss {train_loss:.4f} acc {train_acc:.4f} | "
        f"val loss {val_loss:.4f} acc {val_acc:.4f} top5 {val_top5:.4f} | "
        f"lr {head_lr:.3e}{marca}"
    )

    if epochs_sin_mejora >= max(1, int(cfg.early_stop_patience)):
        print(
            f"Early stopping: {epochs_sin_mejora} epocas sin mejorar val_acc "
            f"(paciencia {cfg.early_stop_patience}); mejor = {best_val_acc:.4f} "
            f"en la epoca {best_epoch}."
        )
```

```
            break

    segundos = time.perf_counter() - start

    # Al terminar, el backbone queda entrenable: `num_params` refleja el modelo
    # completo tal como se guardo en el checkpoint.
    set_backbone_trainable(model, True)
    num_params = int(count_params(model))

    history["best_val_acc"] = float(best_val_acc)
    history["best_epoch"] = int(best_epoch)
    history["epochs_run"] = int(len(history["epochs"]))
    history["device"] = describe_device(device)
    history["num_params"] = num_params
    history["segundos"] = float(segundos)

    # ------------------------------------------------------------ artefactos
    save_json(cfg.history_path, history)
    save_json(cfg.config_path, cfg.to_dict())
    curvas = plot_history(history, Path(cfg.figures_dir()) / "curvas.png")

    print("-" * 72)
    print(
        f"Listo en {segundos:.1f} s ({segundos / 60.0:.1f} min) | epocas ejecutadas: "
        f"{history['epochs_run']}/{epochs}"
    )
    print(f"Mejor val_acc: {best_val_acc:.4f} (epoca {best_epoch})")
    print(f"Checkpoint   : {cfg.checkpoint_path} (existe: {Path(cfg.checkpoint_path).is_file()})")
    print(f"Historial    : {cfg.history_path}")
    print(f"Config usada : {cfg.config_path}")
    print(f"Curvas       : {curvas}")
    print(f"Parametros   : {num_params:,}")
    print("=" * 72)

    return history


# ---------------------------------------------------------------------------- #
# CLI (seccion 3.b del contrato)
# ---------------------------------------------------------------------------- #
def build_argparser() -> argparse.ArgumentParser:
    """Argumentos de `python -m src.train` (CLI congelada)."""
    parser = argparse.ArgumentParser(
        prog="python -m src.train",
        description=(
            "Entrena el clasificador de flores (Flowers-102) segun CONTRATOS.md seccion 3.b. "
            "Sin argumentos usa los valores por defecto de src/config.py."
        ),
        epilog=(
            "Ejemplos:\n"
            "  python -m src.train --debug --device cpu\n"
            "  python -m src.train --epochs 5 --backbone resnet50 --out-dir resultados/prueba\n"
        ),
        formatter_class=argparse.RawDescriptionHelpFormatter,
    )
    parser.add_argument("--epochs", type=int, default=None, help="numero de epocas")
    parser.add_argument("--batch-size", type=int, default=None, dest="batch_size", help="tamano de
lote")
    parser.add_argument("--backbone", type=str, default=None, help="efficientnet_b0 | resnet50 |
mobilenet_v3_large")
    parser.add_argument("--data-root", type=str, default=None, dest="data_root", help="carpeta del
dataset")
    parser.add_argument("--out-dir", type=str, default=None, dest="out_dir", help="carpeta de
resultados")
    parser.add_argument("--device", type=str, default=None, choices=["auto", "cuda", "cpu"],
help="dispositivo")
    parser.add_argument(
        "--num-workers",
```

```
        type=int,
        default=None,
        dest="num_workers",
        help=(
            "procesos del DataLoader (default de Config: 2, lo correcto en Colab). "
            "Usa 0 en entornos donde multiprocessing este bloqueado"
        ),
    )
    parser.add_argument("--debug", action="store_true", help="subconjunto pequeno (smoke test)")
    parser.add_argument("--seed", type=int, default=None, help="semilla")
    return parser


def config_from_args(args: argparse.Namespace) -> Config:
    """`Config` base + solo los argumentos informados (`dataclasses.replace`).

    Comodidad documentada (no cambia ningun default de `Config`): si se pide
    `--debug` y no se indica `--out-dir`, la salida va a `resultados/debug` para no
    pisar los resultados definitivos de `resultados/` (decision §5 del contrato).
    """
    overrides: dict[str, Any] = {}
    for campo in ("epochs", "batch_size", "backbone", "data_root", "out_dir", "device", "seed",
"num_workers"):
        valor = getattr(args, campo, None)
        if valor is not None:
            overrides[campo] = valor
    if getattr(args, "debug", False):
        overrides["debug"] = True
        if "out_dir" not in overrides:
            overrides["out_dir"] = DEBUG_OUT_DIR
            print(
                f"Aviso: --debug sin --out-dir -> la salida va a '{DEBUG_OUT_DIR}' "
                "para no pisar los resultados definitivos de 'resultados/'."
            )

    cfg = replace(Config(), **overrides)

    # Avisos utiles: los valores que Config.__post_init__ ajusta por el modo debug.
    if cfg.debug:
        for campo in ("epochs", "freeze_epochs", "warmup_epochs", "amp", "num_workers"):
            informado = overrides.get(campo)
            if informado is not None and informado != getattr(cfg, campo):
                print(
                    f"Aviso: modo debug ajusto {campo}: {informado} -> {getattr(cfg, campo)}"
                )
    return cfg


def main(argv: list[str] | None = None) -> int:
    """Punto de entrada del CLI. Devuelve el codigo de salida."""
    parser = build_argparser()
    args = parser.parse_args(argv)
    cfg = config_from_args(args)
    train(cfg)
    return 0


if __name__ == "__main__":  # pragma: no cover - entrada del CLI
    raise SystemExit(main())
```

src/evaluate.py

Mide el acierto sobre el conjunto de prueba y dibuja la matriz de confusión. (724 líneas)

```
"""Evaluacion de un checkpoint sobre un split de Oxford Flowers-102.

CONTRATO (seccion 3 de CONTRATOS.md, propietario SA-06):

    evaluate(checkpoint_path: str | Path, cfg: Config | None = None,
             split: str = "test") -> dict
    # CLI: python -m src.evaluate [--checkpoint resultados/best.pt] [--split test|val]
    #                             [--out-dir O] [--device auto|cuda|cpu] [--debug]
    #                             [--num-workers N]   (extension autorizada por SA-00)

Que escribe (todo dentro de ``cfg.out_dir``):

  * ``metrics.json``            metricas completas (ver :func:`evaluate`);
  * ``figuras/confusion_matrix.png``  matriz de confusion NxN (102x102 con el catalogo real);
  * ``figuras/top_confusions.png``    barras con las confusiones mas frecuentes;
  * ``predicciones/<split>_predicciones.csv``  una fila por imagen evaluada.

Notas de diseno:
  - El tamano de imagen se toma SIEMPRE del checkpoint (``meta["img_size"]``), nunca
    se asume 224: si el usuario pasa un ``cfg`` con otro ``img_size`` se respeta el
    del checkpoint para construir los transforms de val/test.
  - El numero de clases tambien se toma del checkpoint (``meta["num_classes"]``); los
    nombres de clase se recortan o rellenan para que ``class_names[i]`` sea la clase
    ``i`` aunque el checkpoint venga de una corrida con otro ``num_classes``.
  - Todo funciona en CPU y en GPU: el device se resuelve con
    :func:`src.utils.resolve_device` a partir de ``cfg.device`` (nada de ``.cuda()``).
  - Matplotlib se usa con el backend ``Agg`` (sin ventanas ni display).
  - No hay rutas absolutas de Windows: todas las rutas derivan de ``Config`` o del
    argumento ``checkpoint_path``.
"""

from __future__ import annotations

import csv
import time
import warnings
from dataclasses import fields, replace
from pathlib import Path
from typing import Any, Sequence

import numpy as np
import torch

try:  # import relativo (paquete `src`) con respaldo absoluto (cwd en sys.path)
    from .config import IMAGE_SIZE, Config
    from .data import build_dataloaders
    from .model import load_checkpoint
    from .utils import resolve_device, save_json
except ImportError:  # pragma: no cover - depende de como se importe el modulo
    from src.config import IMAGE_SIZE, Config
    from src.data import build_dataloaders
    from src.model import load_checkpoint
    from src.utils import resolve_device, save_json

__all__ = [
    "evaluate",
    "main",
    "SPLITS_VALIDOS",
    "MENSAJE_ENTRENA",
    "MAX_CONFUSIONES",
    "CABECERA_CSV",
]

#: Splits oficiales aceptados (los mismos de ``src.data``).
```

```
SPLITS_VALIDOS: tuple[str, str, str] = ("train", "val", "test")
#: Mensaje unico para "todavia no hay modelo entrenado" (texto de CONTRATOS.md seccion 3.b).
MENSAJE_ENTRENA = "entrena primero: `python -m src.train`"
#: Maximo de confusiones guardadas/graficadas (contrato: 20).
MAX_CONFUSIONES: int = 20
#: Encabezado exacto del CSV de predicciones (en espanol, congelado por contrato).
CABECERA_CSV: tuple[str, ...] = ("ruta", "etiqueta_real", "prediccion", "prob", "correcto")

#: Senales de que el fallo viene de los procesos hijo del DataLoader (no del modelo).
#: En el sandbox de Windows de este entorno (y en algunos Windows corporativos) los
#: workers de ``multiprocessing`` no pueden abrir *named pipes*, que es el IPC que usa
#: PyTorch en Windows, y la iteracion revienta con ``PermissionError [WinError 5]``.
_SENALES_FALLO_WORKERS: tuple[str, ...] = (
    "permissionerror",
    "winerror 5",
    "acceso denegado",
    "access is denied",
    "errno 13",
    "errno 22",
    "named pipe",
    "multiprocessing",
    "dataloader worker",
    "worker",
    "spawn",
    "sharing strategy",
    "broken pipe",
    "too many open files",
)


def _es_fallo_de_workers(exc: BaseException) -> bool:
    """True si la excepcion huele a problema de los workers del DataLoader."""
    if isinstance(exc, PermissionError):
        return True
    texto = f"{type(exc).__name__}: {exc}".lower()
    return any(senal in texto for senal in _SENALES_FALLO_WORKERS)


# ---------------------------------------------------------------------------- #
# Helpers de configuracion
# ---------------------------------------------------------------------------- #
def _config_desde_meta(meta: dict) -> Config:
    """Reconstruye ``Config(**meta["config"])`` ignorando claves desconocidas.

    Un checkpoint escrito por una version futura podria traer campos que este
    ``Config`` no conoce: se descartan en vez de reventar.
    """
    valores = dict(meta.get("config") or {})
    validos = {campo.name for campo in fields(Config)}
    limpios = {clave: valor for clave, valor in valores.items() if clave in validos}
    cfg = Config(**limpios)
    if not str(cfg.out_dir).strip():
        cfg = replace(cfg, out_dir="resultados")
    return cfg


def _cfg_con_img_size(cfg: Config, img_size: int) -> Config:
    """Copia de ``cfg`` con el ``img_size`` del checkpoint (no asumir 224)."""
    img_size = int(img_size)
    if int(getattr(cfg, "img_size", IMAGE_SIZE)) == img_size:
        return cfg
    return replace(cfg, img_size=img_size)


def _ajustar_nombres(class_names: Sequence[str], n_clases: int) -> list[str]:
    """Lista de exactamente ``n_clases`` nombres (recorta o rellena si hace falta)."""
    nombres = [str(nombre) for nombre in class_names]
    if len(nombres) < n_clases:
```

```
        nombres += [f"clase_{i + 1:03d}" for i in range(len(nombres), n_clases)]
    return nombres[:n_clases]


def _dataloader_de_split(cfg: Config, split: str):
    """DataLoader del split pedido reutilizando :func:`src.data.build_dataloaders`.

    ``build_dataloaders`` devuelve ``(train_dl, val_dl, test_dl, class_names)``; aqui
    solo se reutiliza el loader del split solicitado (los transforms ya se construyen
    con el ``img_size`` corregido del checkpoint).
    """
    resultado = build_dataloaders(cfg)
    if not isinstance(resultado, (tuple, list)) or len(resultado) < 3:
        raise RuntimeError(
            "build_dataloaders no devolvio (train_dl, val_dl, test_dl, class_names); "
            f"se recibio {type(resultado).__name__}."
        )
    return resultado[SPLITS_VALIDOS.index(split)]


def _inferir_robusto(
    modelo,
    cfg: Config,
    split: str,
    dataloader,
    device,
    n_clases: int,
    nombres: Sequence[str],
):
    """Itera el split reintentando UNA vez con ``num_workers=0`` si fallan los workers.

    En Windows (y en particular en el sandbox de este entorno) los procesos hijo del
    ``DataLoader`` no siempre pueden abrir *named pipes*; PyTorch los usa como IPC y la
    iteracion muere con ``PermissionError [WinError 5]`` (o con un error de
    ``multiprocessing``/``spawn``). Como el modelo y las metricas no tienen nada que ver
    con eso, se avisa y se repite la inferencia con ``num_workers=0`` (que siempre
    funciona) en lugar de abortar la evaluacion.

    Si el ``dataloader`` fue inyectado por el llamador (uso de tests) no se reconstruye:
    un fallo suyo se propaga tal cual.
    """

    def _intento(cfg_actual: Config):
        cargador = (
            dataloader if dataloader is not None else _dataloader_de_split(cfg_actual, split)
        )
        return _inferir(modelo, cargador, device, n_clases, nombres)

    try:
        return _intento(cfg)
    except Exception as exc:  # noqa: BLE001 - se decide con _es_fallo_de_workers
        workers = int(getattr(cfg, "num_workers", 0) or 0)
        if dataloader is not None or workers <= 0 or not _es_fallo_de_workers(exc):
            raise
        aviso = (
            f"El DataLoader con num_workers={workers} fallo ({type(exc).__name__}: {exc}). "
            "Se reintenta una sola vez con num_workers=0; esto es habitual en Windows "
            "(los workers de multiprocessing necesitan named pipes). Para evitar el "
            "reintento usa `--num-workers 0`."
        )
        warnings.warn(aviso, RuntimeWarning, stacklevel=2)
        print(f"[evaluate] aviso: {aviso}")
        return _intento(replace(cfg, num_workers=0))


# ---------------------------------------------------------------------------- #
# Rutas de las imagenes del dataset
# ---------------------------------------------------------------------------- #
```

```
def _rutas_dataset(dataset) -> list[str] | None:
    """Rutas de las imagenes de ``dataset`` en el mismo orden que ``dataset[i]``.

    Soporta ``Subset`` (uno o varios niveles anidados, como los del modo debug de
    ``src.data``) y los atributos que exponen los datasets de torchvision
    (``_image_files``, ``samples``, ``imgs``, ...). Devuelve ``None`` si no se puede
    determinar (por ejemplo en un dataset sintetico de tests): el llamador usa
    entonces nombres de relleno.
    """
    if dataset is None:
        return None

    base = dataset
    indices: list[int] | None = None
    while True:  # desanida Subset(Subset(...))
        sub_indices = getattr(base, "indices", None)
        sub_dataset = getattr(base, "dataset", None)
        if sub_indices is None or sub_dataset is None:
            break
        sub_indices = list(sub_indices)
        indices = sub_indices if indices is None else [sub_indices[i] for i in indices]
        base = sub_dataset

    for atributo in ("_image_files", "image_files", "image_paths", "imgs", "samples", "_images"):
        valor = getattr(base, atributo, None)
        if valor is None or isinstance(valor, (str, bytes)):
            continue
        try:
            elementos = list(valor)
        except TypeError:
            continue
        if not elementos:
            continue

        rutas: list[str] = []
        for elemento in elementos:
            if isinstance(elemento, (str, Path)):
                rutas.append(str(elemento))
            elif (
                isinstance(elemento, (tuple, list))
                and elemento
                and isinstance(elemento[0], (str, Path))
            ):
                rutas.append(str(elemento[0]))
            else:
                rutas = []
                break
        if not rutas:
            continue
        if indices is not None:
            if any(i < 0 or i >= len(rutas) for i in indices):
                return None
            rutas = [rutas[i] for i in indices]
        return rutas
    return None


# ---------------------------------------------------------------------------- #
# Inferencia
# ---------------------------------------------------------------------------- #
def _inferir(
    modelo,
    dataloader,
    device,
    n_clases: int,
    nombres: Sequence[str],
) -> tuple[list[int], list[int], list[float], list[bool], list[str]]:
    """Recorre ``dataloader`` y devuelve listas por imagen (no tensores).
```

```
    Returns:
        ``(etiquetas_reales, predicciones, probabilidades, aciertos_top5, rutas)``
        donde ``probabilidades[i]`` es la probabilidad softmax de la clase predicha
        para la imagen ``i``.
    """
    reales: list[int] = []
    predichas: list[int] = []
    probabilidades: list[float] = []
    aciertos_top5: list[bool] = []
    rutas = _rutas_dataset(getattr(dataloader, "dataset", None))

    modelo.eval()
    with torch.no_grad():
        for lote in dataloader:
            if isinstance(lote, dict):  # por comodidad con loaders "tipo dict"
                imagenes, etiquetas = lote.get("image"), lote.get("label")
            else:
                imagenes, etiquetas = lote[0], lote[1]
            imagenes = imagenes.to(device)
            etiquetas = etiquetas.long().to(device)

            logits = modelo(imagenes)
            probs = torch.softmax(logits.detach().float(), dim=1)
            k_top5 = max(1, min(5, int(probs.shape[1])))
            valores, indices = probs.topk(k_top5, dim=1)

            reales.extend(int(v) for v in etiquetas.cpu().tolist())
            predichas.extend(int(v) for v in indices[:, 0].cpu().tolist())
            probabilidades.extend(float(v) for v in valores[:, 0].cpu().tolist())
            aciertos = (indices == etiquetas.unsqueeze(1)).any(dim=1)
            aciertos_top5.extend(bool(v) for v in aciertos.cpu().tolist())

    total = len(reales)
    if rutas is None or len(rutas) != total:
        rutas = [f"imagen_{i:05d}" for i in range(total)]
    return reales, predichas, probabilidades, aciertos_top5, rutas


# ---------------------------------------------------------------------------- #
# Metricas
# ---------------------------------------------------------------------------- #
def _metricas_vacias(n_clases: int, nombres: Sequence[str]):
    """Metricas neutras para un split sin imagenes (evita que sklearn reviente)."""
    per_class = {
        nombres[i]: {"precision": 0.0, "recall": 0.0, "f1": 0.0, "support": 0}
        for i in range(n_clases)
    }
    per_class_f1 = {nombres[i]: 0.0 for i in range(n_clases)}
    matriz = np.zeros((n_clases, n_clases), dtype=np.int64)
    return 0.0, 0.0, 0.0, per_class, per_class_f1, matriz


def _calcular_metricas(
    reales: Sequence[int],
    predichas: Sequence[int],
    aciertos_top5: Sequence[bool],
    n_clases: int,
    nombres: Sequence[str],
):
    """top-1, top-5, macro-F1, reporte por clase y matriz de confusion."""
    from sklearn.metrics import confusion_matrix, f1_score, precision_recall_fscore_support

    if not len(reales):
        return _metricas_vacias(n_clases, nombres)

    y_true = np.asarray(reales, dtype=int)
    y_pred = np.asarray(predichas, dtype=int)
```

```
    etiquetas = list(range(int(n_clases)))

    top1 = float(np.mean(y_true == y_pred))
    top5 = float(np.mean(np.asarray(aciertos_top5, dtype=float))) if len(aciertos_top5) else top1
    macro_f1 = float(
        f1_score(y_true, y_pred, labels=etiquetas, average="macro", zero_division=0)
    )

    precision, recall, f1, support = precision_recall_fscore_support(
        y_true, y_pred, labels=etiquetas, zero_division=0
    )
    per_class = {
        nombres[i]: {
            "precision": float(precision[i]),
            "recall": float(recall[i]),
            "f1": float(f1[i]),
            "support": int(support[i]),
        }
        for i in range(int(n_clases))
    }
    # SA-03 (notebook) consume per_class_f1 como dict nombre -> float.
    per_class_f1 = {nombres[i]: float(f1[i]) for i in range(int(n_clases))}

    matriz = confusion_matrix(y_true, y_pred, labels=etiquetas)
    return top1, top5, macro_f1, per_class, per_class_f1, matriz


def _top_confusiones(matriz: np.ndarray, nombres: Sequence[str]) -> list[list]:
    """``[[real, pred, n], ...]`` ordenada por ``n`` descendente (maximo 20).

    Solo incluye errores (``real != pred``) con ``n >= 1``.
    """
    pares: list[list] = []
    for i in range(matriz.shape[0]):
        for j in range(matriz.shape[1]):
            if i == j:
                continue
            n = int(matriz[i, j])
            if n > 0:
                pares.append([str(nombres[i]), str(nombres[j]), n])
    pares.sort(key=lambda fila: (-fila[2], fila[0], fila[1]))
    return pares[:MAX_CONFUSIONES]


# ---------------------------------------------------------------------------- #
# Figuras
# ---------------------------------------------------------------------------- #
def _pyplot():
    """``matplotlib.pyplot`` con backend ``Agg`` (requisito del contrato)."""
    import matplotlib

    matplotlib.use("Agg", force=True)
    import matplotlib.pyplot as plt

    return plt


def _etiquetas_cortas(nombres: Sequence[str], maximo: int = 22) -> list[str]:
    """Nombres recortados a ``maximo`` caracteres (para los ejes de las figuras)."""
    cortas = []
    for nombre in nombres:
        texto = str(nombre)
        cortas.append(texto if len(texto) <= maximo else texto[: maximo - 1] + ".")
    return cortas


def _guardar_matriz_confusion(matriz: np.ndarray, nombres: Sequence[str], destino: Path) -> None:
    """Heatmap NxN (102x102 con el catalogo real) con escala logaritmica."""
```

```
    plt = _pyplot()
    from matplotlib.colors import LogNorm

    n_clases = int(matriz.shape[0])
    datos = np.ma.masked_where(matriz == 0, matriz.astype(float))
    maximo = int(matriz.max()) if matriz.size else 0

    figura, eje = plt.subplots(figsize=(13, 11))
    norma = LogNorm(vmin=1, vmax=max(1, maximo)) if maximo > 0 else None
    imagen = eje.imshow(datos, interpolation="nearest", cmap="Blues", norm=norma)
    figura.colorbar(imagen, ax=eje, fraction=0.046, pad=0.04, label="imagenes")

    if n_clases <= 20:
        marcas = list(range(n_clases))
    else:
        paso = max(1, n_clases // 20)
        marcas = list(range(0, n_clases, paso))
    etiquetas = _etiquetas_cortas(nombres)
    eje.set_xticks(marcas)
    eje.set_yticks(marcas)
    eje.set_xticklabels([etiquetas[i] for i in marcas], rotation=90, fontsize=6)
    eje.set_yticklabels([etiquetas[i] for i in marcas], fontsize=6)
    eje.set_xlabel("Prediccion")
    eje.set_ylabel("Etiqueta real")
    eje.set_title(f"Matriz de confusion ({n_clases} clases)")

    figura.tight_layout()
    destino.parent.mkdir(parents=True, exist_ok=True)
    figura.savefig(destino, dpi=150, bbox_inches="tight")
    plt.close(figura)


def _guardar_top_confusiones(confusiones: Sequence[Sequence], destino: Path) -> None:
    """Barras horizontales con las confusiones mas frecuentes."""
    plt = _pyplot()

    figura, eje = plt.subplots(figsize=(9, 6))
    if confusiones:
        etiquetas = [
            f"{fila[0]} -> {fila[1]}" if len(str(fila[0]) + str(fila[1])) <= 44
            else f"{str(fila[0])[:20]}... -> {str(fila[1])[:20]}..."
            for fila in confusiones
        ]
        valores = [int(fila[2]) for fila in confusiones]
        posiciones = np.arange(len(valores))[::-1]  # la mas frecuente arriba
        eje.barh(posiciones, valores, color="#c0392b")
        eje.set_yticks(posiciones)
        eje.set_yticklabels(etiquetas, fontsize=7)
        eje.set_xlabel("Numero de imagenes")
        eje.set_title("Confusiones mas frecuentes (real -> predicho)")
        eje.grid(axis="x", alpha=0.3)
        for posicion, valor in zip(posiciones, valores):
            eje.text(valor, posicion, f" {valor}", va="center", fontsize=7)
    else:
        eje.text(
            0.5,
            0.5,
            "Sin confusiones: todas las imagenes\nse clasificaron correctamente",
            ha="center",
            va="center",
            fontsize=12,
        )
        eje.set_axis_off()
        eje.set_title("Confusiones mas frecuentes (real -> predicho)")

    figura.tight_layout()
    destino.parent.mkdir(parents=True, exist_ok=True)
    figura.savefig(destino, dpi=150, bbox_inches="tight")
```

```
    plt.close(figura)


# ---------------------------------------------------------------------------- #
# CSV de predicciones
# ---------------------------------------------------------------------------- #
def _guardar_csv(
    destino: Path,
    rutas: Sequence[str],
    reales: Sequence[int],
    predichas: Sequence[int],
    probabilidades: Sequence[float],
    nombres: Sequence[str],
) -> Path:
    """CSV con encabezado ``ruta, etiqueta_real, prediccion, prob, correcto``."""
    destino.parent.mkdir(parents=True, exist_ok=True)
    with open(destino, "w", encoding="utf-8", newline="") as fh:
        escritor = csv.writer(fh)
        escritor.writerow(CABECERA_CSV)
        for ruta, real, predicha, prob in zip(rutas, reales, predichas, probabilidades):
            escritor.writerow(
                [
                    str(ruta),
                    str(nombres[real]),
                    str(nombres[predicha]),
                    f"{float(prob):.6f}",
                    bool(real == predicha),
                ]
            )
    return destino


# ---------------------------------------------------------------------------- #
# API publica
# ---------------------------------------------------------------------------- #
def evaluate(
    checkpoint_path: str | Path,
    cfg: Config | None = None,
    split: str = "test",
    *,
    dataloader=None,
) -> dict:
    """Evalua un checkpoint sobre ``split`` y escribe metricas, figuras y CSV.

    Args:
        checkpoint_path: ruta al ``.pt`` guardado por ``src.train``/``src.model``.
        cfg: configuracion de salida. Si es ``None`` se reconstruye desde el propio
            checkpoint con ``Config(**meta["config"])`` (y por tanto se escribe en el
            ``out_dir`` con el que se entreno). El ``img_size`` de los transforms y el
            ``num_classes`` salen siempre del checkpoint.
        split: ``"train"``, ``"val"`` o ``"test"`` (por defecto ``"test"``). Solo se usa
            para elegir el DataLoader, para nombrar el CSV y para el campo ``split``.
        dataloader: **parametro opcional de test** (keyword-only). Si se inyecta un
            ``DataLoader`` ya construido, ``evaluate`` lo usa en lugar de llamar a
            ``src.data.build_dataloaders``; sirve para evaluar sin dataset real y sin
            red (tests). En uso normal debe dejarse en ``None``.

    Robustez de los workers: si el ``DataLoader`` construido con ``cfg.num_workers > 0``
    falla por un problema de *multiprocessing*/*permisos* (tipico en Windows, donde los
    workers necesitan *named pipes*; en el sandbox de este entorno da
    ``PermissionError [WinError 5]``), ``evaluate`` avisa por ``warnings.warn`` y por
    consola y **reintenta una sola vez con ``num_workers=0``** en vez de abortar. El
    reintento solo aplica al loader que construye ``evaluate`` (no a un ``dataloader``
    inyectado).

    Returns:
        El mismo diccionario que se escribe en ``metrics.json``, con las claves
        ``split, num_images, top1, top5, macro_f1, per_class, per_class_f1,
```

```
    top_confusions, ruta_confusion, segundos`` (y algunos metadatos extra del
    checkpoint). ``per_class[clase]`` es ``{precision, recall, f1, support}`` y
    ``per_class_f1[clase]`` es directamente el float de F1 (lo consume SA-03).
    ``top_confusions`` es ``[[real, pred, n], ...]`` ordenada por ``n`` descendente
    (maximo 20).

Raises:
    FileNotFoundError: si el checkpoint no existe (hay que entrenar primero).
    ValueError: si ``split`` no es valido o el checkpoint trae un ``cfg`` invalido.
"""
inicio = time.perf_counter()

ckpt = Path(checkpoint_path)
if not ckpt.is_file():
    raise FileNotFoundError(
        f"No existe el checkpoint '{ckpt}'. Para generarlo, {MENSAJE_ENTRENA}."
    )

split = str(split).lower()
if split not in SPLITS_VALIDOS:
    raise ValueError(
        f"split invalido: {split!r}. Validos: {', '.join(SPLITS_VALIDOS)}."
    )

device = resolve_device(getattr(cfg, "device", "auto") or "auto")
modelo, nombres_ckpt, meta = load_checkpoint(ckpt, device=device)

if cfg is None:
    cfg = _config_desde_meta(meta)

n_clases = int(meta.get("num_classes") or len(nombres_ckpt) or getattr(cfg, "num_classes", 0))
if n_clases <= 0:
    raise ValueError(
        f"El checkpoint '{ckpt}' no define num_classes y no se pudo inferir "
        f"(class_names={len(nombres_ckpt)})."
    )
nombres = _ajustar_nombres(nombres_ckpt, n_clases)

# El tamano de imagen del checkpoint manda (no asumir 224).
img_size = int(meta.get("img_size") or getattr(cfg, "img_size", IMAGE_SIZE))
cfg = _cfg_con_img_size(cfg, img_size)
cfg.ensure_dirs()

reales, predichas, probabilidades, aciertos_top5, rutas = _inferir_robusto(
    modelo, cfg, split, dataloader, device, n_clases, nombres
)
top1, top5, macro_f1, per_class, per_class_f1, matriz = _calcular_metricas(
    reales, predichas, aciertos_top5, n_clases, nombres
)

confusiones = _top_confusiones(matriz, nombres)
ruta_confusion = cfg.figures_dir() / "confusion_matrix.png"
_guardar_matriz_confusion(matriz, nombres, ruta_confusion)
_guardar_top_confusiones(confusiones, cfg.figures_dir() / "top_confusions.png")
ruta_csv = _guardar_csv(
    cfg.predictions_dir() / f"{split}_predicciones.csv",
    rutas,
    reales,
    predichas,
    probabilidades,
    nombres,
)

metricas: dict[str, Any] = {
    "split": split,
    "num_images": int(len(reales)),
    "top1": float(top1),
    "top5": float(top5),
```

```
        "macro_f1": float(macro_f1),
        "per_class": per_class,
        "per_class_f1": per_class_f1,
        "top_confusions": confusiones,
        "ruta_confusion": str(ruta_confusion),
        "segundos": float(round(time.perf_counter() - inicio, 3)),
        # --- metadatos extra (utiles para el INFORME; no rompen el contrato) ---
        "checkpoint": str(ckpt),
        "backbone": str(meta.get("backbone", "")),
        "num_classes": int(n_clases),
        "img_size": int(img_size),
        "device": str(device),
        "epoch": int(meta.get("epoch", -1) or -1),
        "val_acc": float(meta.get("val_acc", 0.0) or 0.0),
        "ruta_predicciones": str(ruta_csv),
    }

    save_json(cfg.metrics_path, metricas)

    print(
        f"[evaluate] split={split} imagenes={metricas['num_images']} "
        f"top1={metricas['top1']:.4f} top5={metricas['top5']:.4f} "
        f"macro_f1={metricas['macro_f1']:.4f} ({metricas['segundos']:.2f}s)"
    )
    print(f"[evaluate] metricas: {cfg.metrics_path}")
    print(f"[evaluate] figuras:  {ruta_confusion} | {cfg.figures_dir() / 'top_confusions.png'}")
    print(f"[evaluate] csv:      {ruta_csv}")

    return metricas


# ---------------------------------------------------------------------------- #
# CLI  (python -m src.evaluate --help)
# ---------------------------------------------------------------------------- #
def build_parser():
    """Parser de la CLI congelada de la seccion 3.b."""
    import argparse

    parser = argparse.ArgumentParser(
        prog="python -m src.evaluate",
        description="Evalua un checkpoint de flores sobre el split val o test.",
    )
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="ruta del checkpoint (por defecto: resultados/best.pt)",
    )
    parser.add_argument(
        "--split",
        default="test",
        choices=("val", "test"),
        help="split a evaluar (por defecto: test)",
    )
    parser.add_argument(
        "--out-dir",
        dest="out_dir",
        default=None,
        help="directorio de salida (por defecto: resultados)",
    )
    parser.add_argument(
        "--device",
        default="auto",
        help="auto|cuda|cpu (por defecto: auto)",
    )
    parser.add_argument(
        "--debug",
        action="store_true",
        help="usa el subconjunto debug (cfg.debug_per_class imagenes por clase)",
```

```
    )
    parser.add_argument(
        "--num-workers",
        dest="num_workers",
        type=int,
        default=None,
        help=(
            "workers del DataLoader (por defecto: los de Config, 2). Usa 0 en Windows "
            "o dentro de un sandbox donde los workers no puedan abrir named pipes"
        ),
    )
    return parser


def main(argv: Sequence[str] | None = None) -> int:
    """Entrada de la CLI. Devuelve 0 si todo fue bien y 1 si falta el checkpoint."""
    args = build_parser().parse_args(argv)

    ckpt = Path(args.checkpoint)
    if not ckpt.is_file():
        print(f"[error] No existe el checkpoint '{ckpt}'. Para generarlo, {MENSAJE_ENTRENA}.")
        print("[error] Comando: python -m src.train")
        return 1

    if args.num_workers is not None and int(args.num_workers) < 0:
        print(f"[error] --num-workers debe ser >= 0, se recibio {args.num_workers}.")
        return 1

    # Solo se pasa num_workers si el usuario lo pidio: el default sigue siendo el de
    # Config (2), que es el correcto en Colab/Linux.
    extra = {} if args.num_workers is None else {"num_workers": int(args.num_workers)}
    cfg = Config(
        out_dir=args.out_dir or "resultados",
        device=args.device,
        debug=bool(args.debug),
        pretrained=False,  # nunca se descargan pesos al evaluar
        **extra,
    )
    try:
        evaluate(ckpt, cfg, split=args.split)
    except (FileNotFoundError, ValueError, RuntimeError, OSError) as exc:
        print(f"[error] {type(exc).__name__}: {exc}")
        return 1
    return 0


if __name__ == "__main__":  # pragma: no cover - entrada de la CLI
    raise SystemExit(main())
```

src/predict.py

Predice la especie de una foto suelta. (337 líneas)

```
"""Inferencia (demo y CLI) sobre un checkpoint entrenado de Flowers-102.

CONTRATO (seccion 3 de CONTRATOS.md, propietario SA-06):

    predict_image(image_path, checkpoint_path, topk=5,
                  min_conf=0.25, device="auto") -> list[tuple[str, float]]
    predict_batch(image_paths, checkpoint_path, topk=5,
                  device="auto") -> list[list[tuple[str, float]]]
    # CLI: python -m src.predict --image ruta.jpg --checkpoint resultados/best.pt
    #                            --topk 5 [--min-conf 0.25] [--device auto|cuda|cpu]

Notas de diseno:
  - ``image_path`` acepta ``str``/``Path``, ``PIL.Image.Image``, ``numpy.ndarray``,
    dicts con ``path``/``name``/``data`` y la tupla/valor que devuelve Gradio.
  - El modelo se cachea con :func:`functools.lru_cache` por
    ``(checkpoint_path, device)``: la demo de Gradio no puede releer 20 MB en cada
    clic. :func:`limpiar_cache` vacia esa cache (tests y reentrenamientos).
  - Si ``max(prob) < min_conf`` se devuelve
    ``[("no reconocida / fuera del catálogo", max_prob)]``.
  - Si el checkpoint no existe se lanza ``FileNotFoundError`` con el recordatorio de
    entrenar primero (``python -m src.train``).
  - El ``img_size`` de los transforms viene del propio checkpoint: no se asume 224.
  - CPU y GPU por igual: el device se resuelve con :func:`src.utils.resolve_device`.
"""

from __future__ import annotations

import functools
from pathlib import Path
from typing import Sequence

import numpy as np
import torch

try:  # import relativo (paquete `src`) con respaldo absoluto (cwd en sys.path)
    from .config import Config
    from .data import build_transforms
    from .model import load_checkpoint
    from .utils import resolve_device
except ImportError:  # pragma: no cover - depende de como se importe el modulo
    from src.config import Config
    from src.data import build_transforms
    from src.model import load_checkpoint
    from src.utils import resolve_device

__all__ = ["predict_image", "predict_batch", "limpiar_cache", "main", "NO_RECONOCIDA"]

#: Etiqueta que se devuelve cuando la confianza no llega a ``min_conf``.
#: NOTA: es el unico texto acentuado del proyecto (CONTRATOS.md seccion 5 pide el
#: codigo sin acentos, pero la tarea de SA-06 fija esta cadena literal). Si se
#: cambia aqui hay que cambiarla tambien en ``tests/test_eval_predict.py`` y en
#: ``src/app.py`` (SA-07), que debe importar esta constante en vez de copiarla.
NO_RECONOCIDA = "no reconocida / fuera del catálogo"
#: Confianza minima por defecto de ``predict_batch`` (la firma no expone ``min_conf``).
DEFAULT_MIN_CONF: float = 0.25
#: Tamano de la cache de modelos (checkpoint, device).
_CACHE_SIZE: int = 8


def _mensaje_entrena(checkpoint_path: Path) -> str:
    return (
        f"No existe el checkpoint '{checkpoint_path}'. Para generarlo, entrena primero: "
        "`python -m src.train` (comando: python -m src.train)."
    )
```

```
# ---------------------------------------------------------------------------- #
# Cache del modelo
# ---------------------------------------------------------------------------- #
@functools.lru_cache(maxsize=_CACHE_SIZE)
def _modelo_cacheado(checkpoint_path: str, device: str):
    """Carga (una sola vez por ``checkpoint_path`` + ``device``) el modelo y sus metadatos.

    Returns:
        ``(modelo, nombres_de_clase, img_size, torch.device)``.
    """
    torch_device = resolve_device(device)
    modelo, class_names, meta = load_checkpoint(checkpoint_path, device=torch_device)
    img_size = int(meta.get("img_size") or getattr(modelo, "img_size", 224) or 224)
    return modelo, tuple(str(n) for n in class_names), img_size, torch_device


def limpiar_cache() -> None:
    """Vacia la cache de modelos cargados (util en tests y al reentrenar)."""
    _modelo_cacheado.cache_clear()


def _nombres_ajustados(class_names: Sequence[str], n_clases: int) -> list[str]:
    """``n_clases`` nombres (recorta o rellena si el checkpoint no cuadra)."""
    nombres = [str(nombre) for nombre in class_names]
    if len(nombres) < n_clases:
        nombres += [f"clase_{i + 1:03d}" for i in range(len(nombres), n_clases)]
    return nombres[:n_clases]


# ---------------------------------------------------------------------------- #
# Normalizacion de la entrada
# ---------------------------------------------------------------------------- #
def _a_uint8(arreglo: np.ndarray) -> np.ndarray:
    """Convierte cualquier arreglo de imagen a ``uint8`` en rango 0..255."""
    if arreglo.dtype == np.uint8:
        return arreglo
    datos = arreglo.astype(np.float32)
    if np.issubdtype(arreglo.dtype, np.floating):
        maximo = float(np.nanmax(datos)) if datos.size else 0.0
        if maximo <= 1.0 + 1e-6:  # imagenes float normalizadas a [0, 1]
            datos = datos * 255.0
    return np.clip(datos, 0.0, 255.0).round().astype(np.uint8)


def _ndarray_a_pil(arreglo: np.ndarray):
    """``numpy.ndarray`` (H,W), (H,W,C) o (C,H,W) -> ``PIL.Image`` RGB."""
    from PIL import Image

    datos = np.asarray(arreglo)
    if datos.ndim == 2:  # escala de grises
        return Image.fromarray(_a_uint8(datos)).convert("RGB")
    if datos.ndim == 3 and datos.shape[0] in (1, 3, 4) and datos.shape[0] < datos.shape[-1]:
        datos = np.transpose(datos, (1, 2, 0))  # channel-first -> channel-last
    if datos.ndim != 3 or datos.shape[-1] not in (1, 3, 4):
        raise ValueError(
            f"no se pudo interpretar un arreglo de forma {datos.shape!r} como imagen "
            "(se esperaba (H,W), (H,W,3) o (3,H,W))."
        )
    return Image.fromarray(_a_uint8(datos)).convert("RGB")


def _a_pil(imagen):
    """Normaliza cualquiera de las entradas aceptadas a ``PIL.Image`` RGB.

    Acepta ``str``/``Path`` (ruta de archivo), ``PIL.Image.Image``, ``numpy.ndarray``,
    ``dict`` con ``path``/``name``/``data``/``image`` y tuplas/listas (por ejemplo
```

```
    ``(imagen, ruta)`` de Gradio).
    """
    from PIL import Image

    if imagen is None:
        raise ValueError("no se recibio ninguna imagen (imagen=None).")

    if isinstance(imagen, Image.Image):
        return imagen.convert("RGB")

    if isinstance(imagen, np.ndarray):
        return _ndarray_a_pil(imagen)

    if isinstance(imagen, (str, Path)):
        ruta = Path(imagen)
        if not ruta.is_file():
            raise FileNotFoundError(f"No existe la imagen '{ruta}'.")
        with Image.open(ruta) as abierta:
            return abierta.convert("RGB")

    if isinstance(imagen, dict):
        for clave in ("path", "name", "data", "image", "orig_name"):
            if clave in imagen and imagen[clave] is not None:
                return _a_pil(imagen[clave])
        raise ValueError(
            f"el diccionario recibido no tiene ninguna clave de imagen util "
            f"(claves: {sorted(imagen)})."
        )

    if isinstance(imagen, (tuple, list)):
        error: Exception | None = None
        for elemento in imagen:
            try:
                return _a_pil(elemento)
            except (TypeError, ValueError, FileNotFoundError) as exc:
                error = exc
        raise ValueError(
            f"no se pudo interpretar la tupla/lista recibida como imagen: {error}"
        )

    # Objetos tipo Gradio (FileData, NamedString, ...): intenta atributos de ruta.
    for atributo in ("path", "name", "orig_name", "value"):
        valor = getattr(imagen, atributo, None)
        if isinstance(valor, (str, Path, np.ndarray)):
            return _a_pil(valor)
    if hasattr(imagen, "convert"):  # cualquier cosa compatible con PIL
        return imagen.convert("RGB")

    raise TypeError(
        f"tipo de imagen no soportado: {type(imagen)!r}. Usa str, Path, PIL.Image, "
        "numpy.ndarray o el valor de Gradio."
    )


# ---------------------------------------------------------------------------- #
# API publica
# ---------------------------------------------------------------------------- #
def predict_image(
    image_path,
    checkpoint_path,
    topk: int = 5,
    min_conf: float = 0.25,
    device: str = "auto",
) -> list[tuple[str, float]]:
    """Clasifica una imagen y devuelve ``[(clase, probabilidad), ...]``.

    Args:
        image_path: ``str``/``Path``, ``PIL.Image.Image``, ``numpy.ndarray``, dict con
```

```
            ``path``/``name``/``data`` o la tupla/valor que devuelve Gradio.
        checkpoint_path: ruta del ``.pt`` entrenado.
        topk: numero de clases a devolver (se recorta a ``num_classes`` si es mayor).
        min_conf: confianza minima. Si ``max(prob) < min_conf`` se devuelve
            ``[("no reconocida / fuera del catálogo", max_prob)]``.
        device: ``"auto"`` | ``"cuda"`` | ``"cpu"`` (se resuelve con ``resolve_device``).

    Returns:
        Lista de ``(nombre_de_clase, probabilidad)`` con probabilidades **decrecientes**
        en ``[0, 1]`` (softmax). El modelo y los nombres de clase se leen de una cache
        ``lru_cache`` por ``(checkpoint_path, device)``.

    Raises:
        FileNotFoundError: si no existe el checkpoint (entrena primero) o la imagen.
        ValueError: si la imagen no se puede interpretar.
    """
    ckpt = Path(checkpoint_path)
    if not ckpt.is_file():
        raise FileNotFoundError(_mensaje_entrena(ckpt))

    imagen = _a_pil(image_path)

    modelo, nombres_ckpt, img_size, device_real = _modelo_cacheado(str(ckpt), str(device))

    # Config auxiliar: solo aporta img_size (y debug para no crear workers) a los
    # transforms; el modelo NO se construye aqui (viene del checkpoint).
    cfg_inferencia = Config(
        num_classes=max(1, len(nombres_ckpt)),
        img_size=int(img_size),
        pretrained=False,
    )
    transform = build_transforms(cfg_inferencia, train=False)
    tensor = transform(imagen).unsqueeze(0).to(device_real)

    with torch.no_grad():
        logits = modelo(tensor)
    probabilidades = torch.softmax(logits.detach().float(), dim=1).flatten().cpu().numpy()

    n_clases = int(probabilidades.shape[0])
    nombres = _nombres_ajustados(nombres_ckpt, n_clases)
    mejor = float(probabilidades.max()) if n_clases else 0.0

    if mejor < float(min_conf):
        return [(NO_RECONOCIDA, mejor)]

    if n_clases == 0:
        return []

    k = max(1, min(int(topk), n_clases))
    indices = np.argsort(-probabilidades, kind="stable")[:k]
    return [(nombres[int(i)], float(probabilidades[int(i)])) for i in indices]


def predict_batch(
    image_paths,
    checkpoint_path,
    topk: int = 5,
    device: str = "auto",
) -> list[list[tuple[str, float]]]:
    """``predict_image`` sobre varias imagenes (el modelo se carga una sola vez).

    Args:
        image_paths: iterable de imagenes aceptadas por :func:`predict_image`.
        checkpoint_path: ruta del ``.pt`` entrenado.
        topk: numero de clases por imagen.
        device: ``"auto"`` | ``"cuda"`` | ``"cpu"``.

    Returns:
```

```
        Una lista de resultados (mismo orden que ``image_paths``); usa el ``min_conf``
        por defecto de 0.25 y por tanto algunas entradas pueden ser
        ``[("no reconocida / fuera del catálogo", prob)]``.
    """
    return [
        predict_image(imagen, checkpoint_path, topk=topk, min_conf=DEFAULT_MIN_CONF, device=device)
        for imagen in image_paths
    ]


# ---------------------------------------------------------------------------- #
# CLI  (python -m src.predict --help)
# ---------------------------------------------------------------------------- #
def build_parser():
    """Parser de la CLI congelada de la seccion 3.b."""
    import argparse

    parser = argparse.ArgumentParser(
        prog="python -m src.predict",
        description="Clasifica una imagen de flor con un checkpoint entrenado.",
    )
    parser.add_argument("--image", required=True, help="ruta de la imagen a clasificar")
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="ruta del checkpoint (por defecto: resultados/best.pt)",
    )
    parser.add_argument("--topk", type=int, default=5, help="numero de clases a mostrar (defecto: 5)")
    parser.add_argument(
        "--min-conf",
        dest="min_conf",
        type=float,
        default=0.25,
        help="confianza minima; por debajo se responde 'no reconocida' (defecto: 0.25)",
    )
    parser.add_argument("--device", default="auto", help="auto|cuda|cpu (por defecto: auto)")
    return parser


def main(argv: Sequence[str] | None = None) -> int:
    """Entrada de la CLI. Devuelve 0 si todo fue bien, 1 si falta el checkpoint."""
    args = build_parser().parse_args(argv)

    try:
        resultados = predict_image(
            args.image,
            args.checkpoint,
            topk=args.topk,
            min_conf=args.min_conf,
            device=args.device,
        )
    except FileNotFoundError as exc:
        print(f"[error] {exc}")
        if not Path(args.checkpoint).is_file():
            print("[error] Entrena primero: python -m src.train")
        return 1
    except (OSError, ValueError, TypeError, RuntimeError) as exc:
        print(f"[error] {type(exc).__name__}: {exc}")
        return 1

    print(f"imagen:     {args.image}")
    print(f"checkpoint: {args.checkpoint}")
    for posicion, (nombre, probabilidad) in enumerate(resultados, start=1):
        print(f"{posicion:2d}. {nombre:<42s} {probabilidad * 100:6.2f}%")
    return 0


if __name__ == "__main__":  # pragma: no cover - entrada de la CLI
```

```
raise SystemExit(main())
```

src/app.py

La demo interactiva de Gradio. (448 líneas)

```
"""Demo interactiva (Gradio) para clasificar especies de flores.

CONTRATO (Fase 0, propietario SA-07):
    build_app(checkpoint_path="resultados/best.pt", topk=5) -> gr.Interface
    main(share=False, checkpoint_path="resultados/best.pt") -> None
    CLI: python -m src.app [--checkpoint resultados/best.pt] [--topk 5] [--share]

Notas de implementacion:
  - La demo NUNCA debe crashear si falta el checkpoint: en ese caso la funcion
    interna devuelve un aviso de texto ("entrena primero: python -m src.train").
  - `src.predict` se importa de forma perezosa (dentro de la funcion) para que la
    app se pueda construir aunque el modulo de inferencia o `torch` fallen al
    importar. Esto mantiene el criterio de aceptacion de `build_app`.
  - Se escribio contra la API realmente instalada: gradio 6.30.0.
  - DESVIACION (justificada): `build_app` devuelve un `gr.Blocks`, no un
    `gr.Interface`. En Gradio 6 el evento `submit` desaparecio de los componentes
    (solo quedan `click`/`change`/`upload`), y con `gr.Interface` no se puede
    mostrar el aviso de "falta el checkpoint" de forma permanente en el layout.
    El objeto devuelto cumple el contrato funcional: tiene `.launch(...)`.
    Para volver a `gr.Interface` bastaria reconstruirlo con `fn`, `inputs`,
    `outputs`, `title`, `description` y `article` (ya estan todos definidos).
"""

from __future__ import annotations

import argparse
import html
import inspect
import sys
from pathlib import Path
from typing import Any

import gradio as gr

# ---------------------------------------------------------------------------- #
# Textos de la interfaz (sin acentos, como el resto del codigo del proyecto)
# ---------------------------------------------------------------------------- #
TITULO = "Clasificador de flores (Flowers102)"

DESCRIPCION = (
    "Sube una foto de una flor o tomala con la webcam y el modelo mostrara las "
    "especies mas probables con su nivel de confianza. "
    "Modelo: EfficientNet-B0 entrenado sobre Oxford Flowers102 (102 especies)."
)

ARTICULO = (
    "### Como usarlo\n"
    "1. Sube una imagen (arrastrar y soltar) **o** activa la pestana de camara.\n"
    "2. Pulsa **Enviar**.\n"
    "3. Revisa las etiquetas con su probabilidad y la tabla de las top-k.\n\n"
    "Si aparece *no reconocida / fuera del catalogo*, la confianza maxima quedo por "
    "debajo del umbral (`min_conf`) y la imagen probablemente no sea una de las 102 "
    "especies del dataset.\n\n"
    "En Google Colab puedes lanzar la demo con: "
    "`python -m src.app --share` (genera un enlace publico temporal)."
)

AVISO_ENTRENA = (
    "**No hay modelo entrenado todavia.**\n\n"
    "Entrena primero:\n\n"
    "```bash\n"
    "python -m src.train\n"
    "```\n\n"
    "Si el checkpoint esta en otra ruta, indicala con "
```

```
    "`python -m src.app --checkpoint RUTA/al/best.pt`."
)

MSG_SIN_IMAGEN = (
    "**Sube una imagen primero.** Usa el panel de la izquierda "
    "(subida o webcam) y vuelve a pulsar Enviar."
)

MSG_SIN_RESULTADOS = "No se pudo obtener ninguna prediccion para esa imagen."

# Aviso que se muestra de forma permanente en la propia app cuando el
# checkpoint no existe: asi el usuario lo ve sin necesidad de enviar una imagen.
AVISO_CHECKPOINT_UI = (
    "### No hay modelo entrenado\n\n"
    "No se encontro el checkpoint en `{ruta}`. Entrena primero con:\n\n"
    "```bash\n"
    "python -m src.train\n"
    "```\n\n"
    "La demo se abre igual, pero al enviar una imagen mostrara este mismo aviso "
    "en lugar de la prediccion."
)


# ---------------------------------------------------------------------------- #
# Utilidades
# ---------------------------------------------------------------------------- #
def mensaje_entrena_primero(checkpoint_path: str) -> str:
    """Mensaje en Markdown que se muestra cuando falta el checkpoint."""
    return (
        f"{AVISO_ENTRENA}\n\n"
        f"Ruta buscada: `{checkpoint_path}`"
    )


def _fmt_pct(valor: Any) -> str:
    """Formatea una probabilidad como porcentaje; tolera None y valores raros."""
    try:
        return f"{float(valor) * 100:.2f} %"
    except (TypeError, ValueError):
        return "s/d"


def _celda(texto: Any) -> str:
    """Sanea un valor para que no rompa una celda de tabla Markdown."""
    plano = " ".join(str(texto).split())
    return html.escape(plano, quote=False).replace("|", "\\|") or "s/d"


def _normalizar(resultados: Any, topk: int) -> list[tuple[str, float]]:
    """Normaliza la salida de `predict_image` a una lista de (etiqueta, prob).

    Acepta listas de tuplas, listas de listas o diccionarios; descarta entradas
    mal formadas y ordena de mayor a menor probabilidad.
    """
    if resultados is None:
        return []

    if isinstance(resultados, dict):
        candidatos: list[Any] = list(resultados.items())
    else:
        candidatos = list(resultados)

    normalizados: list[tuple[str, float]] = []
    for item in candidatos:
        if isinstance(item, (tuple, list)) and len(item) >= 2:
            etiqueta, prob = item[0], item[1]
        else:
            continue
```

```
        try:
            prob = float(prob)
        except (TypeError, ValueError):
            continue
        normalizados.append((str(etiqueta), prob))

    normalizados.sort(key=lambda par: par[1], reverse=True)
    return normalizados[:topk] if topk > 0 else normalizados


def tabla_markdown(resultados: list[tuple[str, float]]) -> str:
    """Construye la tabla Markdown (especie | probabilidad %)."""
    if not resultados:
        return MSG_SIN_RESULTADOS

    filas = [
        "| Especie | Probabilidad |",
        "| :--- | ---: |",
    ]
    for etiqueta, prob in resultados:
        filas.append(f"| {_celda(etiqueta)} | {_fmt_pct(prob)} |")

    ganadora = _celda(resultados[0][0])
    encabezado = f"**Prediccion principal:** {ganadora} ({_fmt_pct(resultados[0][1])})\n\n"
    return encabezado + "\n".join(filas)


def _cargar_predict_image():
    """Importa `predict_image` de forma perezosa.

    Devuelve `(funcion, error)`: exactamente uno de los dos es None.
    """
    try:
        from src.predict import predict_image  # import perezoso a proposito
    except Exception as exc:  # ImportError, fallos de torch, etc.
        return None, exc
    if not callable(predict_image):
        return None, AttributeError("src.predict no define predict_image()")
    return predict_image, None


def _imagen_de_ejemplo() -> str | None:
    """Primera imagen disponible de `data/` o `assets/` para el panel de ejemplos."""
    for carpeta in ("data", "assets"):
        raiz = Path(carpeta)
        if not raiz.is_dir():
            continue
        for patron in ("*.jpg", "*.jpeg", "*.png", "*.webp"):
            for ruta in sorted(raiz.rglob(patron)):
                if ruta.is_file():
                    return str(ruta.resolve())
    return None


def _es_notebook() -> bool:
    """True si estamos dentro de Jupyter/Colab (para no bloquear el hilo)."""
    return "ipykernel" in sys.modules or "google.colab" in sys.modules


# ---------------------------------------------------------------------------- #
# Funcion interna de la demo (visible para verificacion directa)
# ---------------------------------------------------------------------------- #
def predecir(
    imagen: Any,
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
) -> tuple[dict[str, float], str]:
```

```
    """Funcion que ejecuta la demo.

    Devuelve `(etiquetas_para_gr_Label, tabla_markdown)` y **nunca** lanza
    excepciones: cualquier fallo se traduce en un mensaje claro para el usuario.

    Sobre `min_conf`: se pide la lista top-k con `min_conf=0.0` y el umbral se
    aplica DESPUES, como AVISO, no como borrado de la prediccion. Motivo: con un
    modelo poco entrenado (por ejemplo en MODO_DEBUG, o entrenado sin pesos
    preentrenados) el softmax es muy plano y su maximo ronda el 2 %, de modo que
    aplicar el umbral antes hacia que la demo respondiera siempre "no reconocida"
    aunque la especie propuesta fuera la correcta. Asi el usuario ve la mejor
    suposicion y, ademas, sabe si hay que desconfiar de ella.
    """
    if imagen is None:
        return {}, MSG_SIN_IMAGEN

    if not Path(checkpoint_path).exists():
        return {}, mensaje_entrena_primero(checkpoint_path)

    predict_image, error_import = _cargar_predict_image()
    if predict_image is None:
        return {}, (
            "**No se pudo cargar el modulo de inferencia** (`src.predict`).\n\n"
            f"Detalle: `{type(error_import).__name__}: {error_import}`"
        )

    try:
        crudos = predict_image(imagen, checkpoint_path, topk=topk, min_conf=0.0)
    except FileNotFoundError as exc:
        return {}, mensaje_entrena_primero(checkpoint_path) + f"\n\nDetalle: `{_celda(exc)}`"
    except Exception as exc:
        return {}, (
            "**Error al predecir la imagen.**\n\n"
            f"`{type(exc).__name__}: {_celda(exc)}`\n\n"
            "Revisa que la imagen sea valida (jpg/png) y que el checkpoint "
            "corresponda al mismo backbone."
        )

    resultados = _normalizar(crudos, topk)
    if not resultados:
        return {}, MSG_SIN_RESULTADOS

    etiquetas = {nombre: prob for nombre, prob in resultados}
    tabla = tabla_markdown(resultados)

    confianza = max(prob for _, prob in resultados)
    if confianza < min_conf:
        tabla = (
            f"**AVISO: confianza baja ({confianza * 100:.1f} % < {min_conf * 100:.0f} %).** "
            "La lista de arriba es la mejor suposicion, no una certeza. Puede ocurrir "
            "por dos motivos: (a) la flor no esta entre las 102 especies del catalogo, "
            "o (b) el modelo esta poco entrenado (por ejemplo si entrenaste en "
            "MODO_DEBUG, o si aviso de que no pudo descargar los pesos preentrenados).\n\n"
        ) + tabla
    return etiquetas, tabla


# Alias estable para verificaciones/test manuales.
app_fn = predecir


# ---------------------------------------------------------------------------- #
# Construccion de la interfaz
# ---------------------------------------------------------------------------- #
def build_app(
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
```

```
) -> gr.Blocks:
    """Construye la demo de Gradio (no lanza el servidor).

    Devuelve un `gr.Blocks` con `.launch(...)` (ver la desviacion documentada en
    el docstring del modulo: en Gradio 6 `gr.Interface` no permite mostrar el
    aviso de checkpoint ausente dentro del layout).

    Si `checkpoint_path` no existe, la app se construye igual, muestra un aviso
    visible en el propio layout y, al predecir, devuelve ese aviso en lugar de
    lanzar una excepcion.
    """
    checkpoints = str(checkpoint_path)
    falta_checkpoint = not Path(checkpoints).exists()

    def _fn(imagen: Any):
        return predecir(imagen, checkpoints, topk, min_conf)

    ejemplo = _imagen_de_ejemplo()

    with gr.Blocks(title=TITULO, fill_width=True) as demo:
        gr.Markdown(f"# {TITULO}")
        gr.Markdown(DESCRIPCION)
        if falta_checkpoint:
            gr.Markdown(AVISO_CHECKPOINT_UI.format(ruta=checkpoints))

        with gr.Row():
            with gr.Column(scale=1, min_width=320):
                entrada = gr.Image(
                    label="Imagen de la flor",
                    type="pil",  # PIL.Image: formato aceptado por predict_image
                    sources=["upload", "webcam", "clipboard"],
                    height=380,
                )
                with gr.Row():
                    boton_enviar = gr.Button("Enviar", variant="primary")
                    boton_limpiar = gr.Button("Limpiar")
                if ejemplo:
                    gr.Examples(examples=[ejemplo], inputs=entrada)

            with gr.Column(scale=1, min_width=320):
                salida_etiquetas = gr.Label(
                    label=f"Top-{topk} especies mas probables",
                    num_top_classes=topk,
                )
                salida_tabla = gr.Markdown(
                    value=(
                        AVISO_CHECKPOINT_UI.format(ruta=checkpoints)
                        if falta_checkpoint
                        else "Sube una imagen para ver el detalle de las top-k."
                    ),
                    label="Detalle del top-k",
                )

        gr.Markdown(ARTICULO)

        entradas = [entrada]
        salidas = [salida_etiquetas, salida_tabla]
        # Gradio 6 ya no expone el evento `submit` en los componentes: se usa el
        # boton (y ademas `Image.upload`, para predecir al subir la imagen).
        # `api_name=None` evita publicar los handlers auxiliares en la API HTTP.
        boton_enviar.click(fn=_fn, inputs=entradas, outputs=salidas, api_name="predict")
        entrada.upload(fn=_fn, inputs=entradas, outputs=salidas, api_name=None)
        boton_limpiar.click(
            fn=lambda: (None, {}, "Sube una imagen para ver el detalle de las top-k."),
            inputs=None,
            outputs=[entrada, salida_etiquetas, salida_tabla],
            api_name=None,
        )
```

```
    return demo


# ---------------------------------------------------------------------------- #
# CLI
# ---------------------------------------------------------------------------- #
def parse_args(argv: list[str] | None = None) -> argparse.Namespace:
    """Argumentos de linea de comandos segun el contrato (seccion 3.b)."""
    parser = argparse.ArgumentParser(
        prog="python -m src.app",
        description=(
            "Abre la demo interactiva (Gradio) del clasificador de flores. "
            "No requiere el checkpoint para abrirse: si falta, la demo lo avisa."
        ),
    )
    parser.add_argument(
        "--checkpoint",
        default="resultados/best.pt",
        help="Ruta del checkpoint del modelo (por defecto: resultados/best.pt).",
    )
    parser.add_argument(
        "--topk",
        type=int,
        default=5,
        help="Cuantas especies mostrar como maximo (por defecto: 5).",
    )
    parser.add_argument(
        "--min-conf",
        type=float,
        default=0.25,
        help=(
            "Umbral de confianza por debajo del cual la demo muestra un AVISO de "
            "confianza baja (por defecto: 0.25). Bajalo si tu modelo esta poco "
            "entrenado y quieres ver la prediccion sin el aviso."
        ),
    )
    parser.add_argument(
        "--share",
        action="store_true",
        help=(
            "Genera un enlace publico temporal de Gradio (util en Google Colab; "
            "requiere salida a internet). Por defecto es False."
        ),
    )
    return parser.parse_args(argv)


def main(
    share: bool = False,
    checkpoint_path: str = "resultados/best.pt",
    topk: int = 5,
    min_conf: float = 0.25,
) -> None:
    """Construye y lanza la demo.

    Imprime la URL local y (si `share=True`) la publica. En Jupyter/Colab no
    bloquea el hilo; en un script normal si (hasta que se interrumpa con Ctrl+C).
    """
    if not Path(checkpoint_path).exists():
        print(
            f"[aviso] No se encontro el checkpoint: {checkpoint_path}\n"
            "        La demo se abrira igual, pero mostrara el aviso "
            "'entrena primero: python -m src.train'.\n"
            "        Entrena con:  python -m src.train",
            flush=True,
        )
```

```
    app = build_app(checkpoint_path, topk, min_conf)

    # En notebooks/Colab no bloqueamos la celda; en script el servidor sigue vivo.
    kwargs: dict[str, Any] = {"share": share, "prevent_thread_lock": _es_notebook()}

    # `inbrowser` no aporta nada en Colab.
    if "inbrowser" in inspect.signature(app.launch).parameters:
        kwargs["inbrowser"] = not _es_notebook()

    resultado = app.launch(**kwargs)

    url_local: str | None = None
    url_publica: str | None = None
    if isinstance(resultado, tuple) and len(resultado) >= 3:
        url_local, url_publica = resultado[1], resultado[2]
    elif isinstance(resultado, str):
        url_local = resultado

    if url_local:
        print(f"Demo local:   {url_local}", flush=True)
    if share and url_publica:
        print(f"Demo publica: {url_publica}", flush=True)
    elif share:
        print(
            "[aviso] Se pidio --share pero Gradio no devolvio enlace publico "
            "(revisa la conexion a internet).",
            flush=True,
        )
    print("Pulsa Ctrl+C para detener la demo.", flush=True)


def cli(argv: list[str] | None = None) -> None:
    """Punto de entrada de `python -m src.app`."""
    args = parse_args(argv)
    main(
        share=bool(args.share),
        checkpoint_path=str(args.checkpoint),
        topk=int(args.topk),
        min_conf=float(args.min_conf),
    )


if __name__ == "__main__":  # pragma: no cover
    cli()
```

empaquetar_colab.py

Crea el archivo comprimido que se sube a Colab. (112 líneas)

```
"""Empaqueta el proyecto en un ZIP listo para subir a Google Colab.

El notebook (`notebooks/flores_colab.ipynb`) tiene un modo "zip" que pide subir
este archivo y luego busca `src/config.py` en la carpeta extraida.

Uso:
    .venv\\Scripts\\python.exe empaquetar_colab.py

Salida: `IA_flores_colab.zip` en la raiz del proyecto.

Se usa una LISTA BLANCA explicita (no se recorre todo el proyecto) por dos razones:
  1. `data/` (330 MB de imagenes) y `resultados/` no deben viajar en el ZIP.
  2. En este equipo el sandbox deja directorios temporales ilegibles
     (`.venv`, `.piptmp`, `.pytmp`, `pytest-cache-files-*`); recorrerlos falla.
"""

from __future__ import annotations

import sys
import zipfile
from pathlib import Path

RAIZ = Path(__file__).resolve().parent
DESTINO = RAIZ / "IA_flores_colab.zip"

# ------------------------------------------------------------------ lista blanca
ARCHIVOS = [
    ".gitignore",
    "CONTRATOS.md",
    "INFORME.md",
    "PLAN_TRABAJO.md",
    "README.md",
    "pytest.ini",
    "requirements.txt",
]
CARPETAS = ["src", "tests", "notebooks", "assets", "docs"]

EXCLUIR_NOMBRES = {"__pycache__", ".ipynb_checkpoints", ".pytest_cache"}
EXCLUIR_SUFIJOS = {".pyc", ".pyo"}


def _incluir(p: Path) -> bool:
    if any(part in EXCLUIR_NOMBRES for part in p.parts):
        return False
    return p.suffix not in EXCLUIR_SUFIJOS


def _recolectar() -> list[Path]:
    encontrados: list[Path] = []
    faltantes: list[str] = []

    for nombre in ARCHIVOS:
        p = RAIZ / nombre
        if p.is_file():
            encontrados.append(p)
        else:
            faltantes.append(nombre)

    for carpeta in CARPETAS:
        base = RAIZ / carpeta
        if not base.is_dir():
            faltantes.append(carpeta + "/")
            continue
        for p in sorted(base.rglob("*")):
```

```
            if p.is_file() and _incluir(p):
                encontrados.append(p)

    if faltantes:
        print("AVISO: no existen todavia (se omiten):", ", ".join(faltantes))
    return encontrados


def main() -> int:
    archivos = _recolectar()
    if not archivos:
        print("ERROR: no se encontro ningun archivo que empaquetar")
        return 1

    with zipfile.ZipFile(DESTINO, "w", zipfile.ZIP_DEFLATED, compresslevel=6) as z:
        for p in archivos:
            z.write(p, p.relative_to(RAIZ).as_posix())

    # Verificacion: leer el ZIP y comprobar la estructura esperada por el notebook.
    with zipfile.ZipFile(DESTINO) as z:
        nombres = z.namelist()
        malos = z.testzip()

    print(f"\nZIP creado: {DESTINO.name}  ({DESTINO.stat().st_size / 1024:.0f} KB)")
    print(f"Archivos incluidos: {len(nombres)}")
    if malos:
        print("ERROR: entrada corrupta:", malos)
        return 1

    requeridos = ["src/config.py", "src/utils.py", "src/data.py", "src/model.py",
                  "notebooks/flores_colab.ipynb", "requirements.txt"]
    print("\nComprobacion de la estructura que espera el notebook:")
    todo_ok = True
    for req in requeridos:
        ok = req in nombres
        todo_ok &= ok
        print(f"  {'OK ' if ok else 'FALTA'}  {req}")

    print("\nContenido:")
    for n in sorted(nombres):
        print("  ", n)

    print("\nRESULTADO:", "estructura correcta" if todo_ok else "FALTAN ARCHIVOS")
    return 0 if todo_ok else 1


if __name__ == "__main__":
    sys.exit(main())
```

tests/conftest.py

Arreglo de los directorios temporales de las pruebas. (58 líneas)

```
"""Infraestructura de tests del proyecto (propietario: orquestador / SA-00).

MOTIVO (peculiaridad de esta maquina, no del proyecto):
  `tempfile.mkdtemp()` y la fixture `tmp_path` de pytest crean directorios con
  modo 0o700. Bajo el sandbox de Windows de este entorno, esos directorios NO se
  pueden listar despues (WinError 5 / PermissionError), asi que cualquier test
  que use `tmp_path` falla con un error de permisos que no tiene nada que ver con
  el codigo probado.

  Solucion: se sustituyen `tmp_path` y `tmp_path_factory` por versiones que crean
  los directorios con el modo POR DEFECTO (que si se pueden listar y recorrer).
  Los tests existentes que piden `tmp_path` siguen funcionando sin cambios.

  NO uses `tempfile.mkdtemp()` en el codigo de `src/` ni en los tests: pide
  `tmp_path` o crea el directorio con `Path.mkdir()` sin modo explicito.
"""

from __future__ import annotations

import shutil
import uuid
from pathlib import Path

import pytest

BASE = Path(__file__).resolve().parent.parent / ".pytmp"
BASE.mkdir(parents=True, exist_ok=True)


def _nuevo_dir(prefijo: str = "case") -> Path:
    """Crea un directorio temporal con modo por defecto (listable)."""
    destino = BASE / f"{prefijo}-{uuid.uuid4().hex[:10]}"
    destino.mkdir()  # sin mode=: a proposito
    return destino


class _FactoryTemporal:
    """Sustituto compatible (lo justo) de `tmp_path_factory`."""

    def mktemp(self, basename: str = "case", numbered: bool = True) -> Path:
        return _nuevo_dir(basename or "case")

    def getbasetemp(self) -> Path:
        return BASE


@pytest.fixture
def tmp_path_factory() -> _FactoryTemporal:
    return _FactoryTemporal()


@pytest.fixture
def tmp_path(tmp_path_factory) -> Path:
    directorio = tmp_path_factory.mktemp("case")
    try:
        yield directorio
    finally:
        shutil.rmtree(directorio, ignore_errors=True)
```

tests/test_data.py

Pruebas de la carga de datos. (561 líneas)

```
"""Tests de ``src/data.py`` (propietario SA-01).

Estructura:
  - Tests rapidos y offline (PNG sinteticos + dataset falso): formas de lote,
    rango de etiquetas, determinismo de val/test y estrategia de nombres.
  - ``test_real_flowers102_debug_mode`` (marcado ``slow``): dataset real en modo
    debug. Intenta la descarga como maximo una vez con timeout y hace
    ``pytest.skip`` si no hay red.

Comando de verificacion:
    .venv\\Scripts\\python.exe -m pytest tests\\test_data.py -q
"""

from __future__ import annotations

import json
import threading
import urllib.error
from pathlib import Path

import pytest

from src import data as data_mod
from src.config import Config

IMG_SIZE = 224
NUM_CLASSES = 102
DOWNLOAD_TIMEOUT_S = 600.0

# NOTA: la fixture ``tmp_path`` que usan estos tests la sustituye
# ``tests/conftest.py`` (crea los directorios con permisos por defecto, porque en
# este sandbox los directorios 0o700 de pytest quedan inaccesibles).


# ---------------------------------------------------------------------- helpers
def _require_torch():
    """Salta el test si el entorno no tiene torch/torchvision (venv incompleto)."""
    pytest.importorskip("torch", reason="torch no disponible en el entorno")
    pytest.importorskip("torchvision", reason="torchvision no disponible en el entorno")
    try:
        data_mod._transforms()
    except ImportError as exc:  # p. ej. falta scipy y torchvision lo importa de forma eager
        pytest.skip(f"torchvision.transforms no disponible: {exc}")


def _make_pngs(root: Path, n_classes: int = 3, per_class: int = 4, size: int = 32, seed: int = 0) ->
None:
    """Crea PNGs temporales repartidos en una carpeta por clase."""
    import numpy as np
    from PIL import Image

    rng = np.random.default_rng(seed)
    for class_id in range(n_classes):
        folder = root / f"class_{class_id:03d}"
        folder.mkdir(parents=True, exist_ok=True)
        for k in range(per_class):
            array = rng.integers(0, 256, size=(size, size, 3), dtype="uint8")
            Image.fromarray(array).save(folder / f"img_{k:02d}.png")


def _make_fake_flowers(root: Path, n_classes: int = NUM_CLASSES, per_class: int = 3, size: int = 32) ->
None:
    """PNGs planos ``cNNN_kk.png`` (etiqueta 0-based codificada en el nombre)."""
    import numpy as np
```

```
    from PIL import Image

    folder = root / "imgs"
    folder.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(7)
    for label in range(n_classes):
        for k in range(per_class):
            array = rng.integers(0, 256, size=(size, size, 3), dtype="uint8")
            Image.fromarray(array).save(folder / f"c{label:03d}_{k:02d}.png")


def _tiny_dataset(root: Path, transform):
    """Dataset minimo (sin depender de torchvision.datasets, que exige scipy)."""
    from PIL import Image
    from torch.utils.data import Dataset

    class _TinyImages(Dataset):
        def __init__(self, root: Path, transform):
            self.transform = transform
            self.samples: list[tuple[Path, int]] = []
            for class_dir in sorted(p for p in Path(root).iterdir() if p.is_dir()):
                for img in sorted(class_dir.glob("*.png")):
                    self.samples.append((img, int(class_dir.name.split("_")[-1])))

        def __len__(self) -> int:
            return len(self.samples)

        def __getitem__(self, index: int):
            path, label = self.samples[index]
            image = Image.open(path).convert("RGB")
            return self.transform(image), label

    return _TinyImages(root, transform)


class _FakeFlowers102:
    """Imita la parte de ``torchvision.datasets.Flowers102`` que usa src/data.py."""

    def __init__(self, root, split: str = "train", transform=None, target_transform=None, download: bool
= False):
        self.root = Path(root)
        self.split = split
        self.transform = transform
        self.download = download
        self._image_files = sorted((self.root / "imgs").glob("*.png"))
        # etiquetas 0-based, como las entrega torchvision
        self._labels = [int(path.stem.split("_")[0][1:]) for path in self._image_files]

    def __len__(self) -> int:
        return len(self._image_files)

    def __getitem__(self, index: int):
        from PIL import Image

        image = Image.open(self._image_files[index]).convert("RGB")
        if self.transform is not None:
            image = self.transform(image)
        return image, self._labels[index]


def _write_cache(root: Path, prefix: str = "sp") -> Path:
    """Escribe un ``cat_to_name.json`` falso con claves 1-based ('1'..'102')."""
    root = Path(root)  # acepta tambien el ``cfg.data_root`` (str)
    root.mkdir(parents=True, exist_ok=True)
    path = root / data_mod.CAT_TO_NAME_FILENAME
    path.write_text(
        json.dumps({str(i + 1): f"{prefix}{i + 1:03d}" for i in range(NUM_CLASSES)}),
        encoding="utf-8",
```

```
    )
    return path


def _cfg(tmp_path: Path, **kwargs) -> Config:
    kwargs.setdefault("data_root", str(tmp_path / "data"))
    kwargs.setdefault("out_dir", str(tmp_path / "out"))
    return Config(**kwargs)


def _raise_no_network(*args, **kwargs):
    raise urllib.error.URLError("sin red (test offline)")


def _hide_dataset_names(monkeypatch):
    """Simula un torchvision que NO expone nombres propios del dataset.

    Necesario para probar de forma determinista la cache, el respaldo del repo, la
    descarga y el fallback: si torchvision esta instalado,
    ``Flowers102.classes`` tiene prioridad y ganaria siempre.
    """

    def _raise_import_error():
        raise ImportError("nombres del dataset no disponibles (test)")

    monkeypatch.setattr(data_mod, "_flowers102_class", _raise_import_error)


def _hide_offline_backups(monkeypatch):
    """Desactiva ``assets/cat_to_name.json`` para poder probar la red y el fallback."""
    monkeypatch.setattr(data_mod, "ASSET_CAT_TO_NAME", Path("no_existe_cat_to_name.json"))


#: Estado de la descarga real: garantiza como maximo UN intento fallido por sesion.
_DOWNLOAD_STATE: dict[str, str | None] = {"failed": None}


def _run_with_timeout(fn, timeout_s: float):
    """Ejecuta ``fn`` en un hilo y falla con TimeoutError si excede ``timeout_s``."""
    box: dict = {}

    def target() -> None:
        try:
            box["value"] = fn()
        except BaseException as exc:  # se re-lanza en el hilo principal
            box["error"] = exc

    thread = threading.Thread(target=target, daemon=True)
    thread.start()
    thread.join(timeout_s)
    if thread.is_alive():
        raise TimeoutError(f"la operacion supero {timeout_s:.0f}s")
    if "error" in box:
        raise box["error"]
    return box["value"]


# ---------------------------------------------------- transforms (offline)
def test_build_transforms_train_batch_shape_and_label_range(tmp_path):
    """Un lote de train sintetico: (B, 3, 224, 224) normalizado y etiquetas validas."""
    _require_torch()
    import torch
    from torch.utils.data import DataLoader

    _make_pngs(tmp_path / "imgs")
    cfg = _cfg(tmp_path, debug=True, batch_size=6)
    dataset = _tiny_dataset(tmp_path / "imgs", data_mod.build_transforms(cfg, train=True))
    assert len(dataset) == 12
```

```
    batch_x, batch_y = next(iter(DataLoader(dataset, batch_size=6, shuffle=False, num_workers=0)))
    assert tuple(batch_x.shape) == (6, 3, IMG_SIZE, IMG_SIZE)
    assert batch_x.dtype == torch.float32
    assert bool(torch.isfinite(batch_x).all())
    assert int(batch_y.min()) >= 0
    assert int(batch_y.max()) < 3
    # Normalize(MEAN, STD) aplicado: hay pixeles negativos (no queda en [0, 1])
    assert float(batch_x.min()) < 0.0
    assert float(batch_x.max()) <= 3.0


def test_build_transforms_eval_is_deterministic_and_uses_img_size(tmp_path):
    """Val/test: sin aleatoriedad y respetando ``cfg.img_size`` (no asumir 224)."""
    _require_torch()
    import torch

    _make_pngs(tmp_path / "imgs")
    cfg = _cfg(tmp_path, img_size=96, debug=True)
    dataset = _tiny_dataset(tmp_path / "imgs", data_mod.build_transforms(cfg, train=False))

    first = dataset[0][0]
    second = dataset[0][0]
    assert tuple(first.shape) == (3, 96, 96)
    assert torch.equal(first, second)

    default_cfg = _cfg(tmp_path, debug=True)
    default_dataset = _tiny_dataset(tmp_path / "imgs", data_mod.build_transforms(default_cfg,
train=False))
    assert tuple(default_dataset[0][0].shape) == (3, IMG_SIZE, IMG_SIZE)


def test_build_transforms_train_is_random(tmp_path):
    """La augmentacion de train debe variar entre dos lecturas de la misma imagen."""
    _require_torch()
    import torch

    _make_pngs(tmp_path / "imgs", size=64, seed=3)
    cfg = _cfg(tmp_path, debug=True)
    dataset = _tiny_dataset(tmp_path / "imgs", data_mod.build_transforms(cfg, train=True))
    assert not torch.equal(dataset[0][0], dataset[0][0])


# ------------------------------------ build_dataloaders en debug (offline)
def test_build_dataloaders_debug_subset_shapes_and_names(tmp_path, monkeypatch):
    """Criterio de aceptacion: 3 loaders, 102 nombres, lote (B,3,224,224), etiquetas en [0,102)."""
    _require_torch()
    import torch
    from torch.utils.data import RandomSampler, SequentialSampler

    data_root = tmp_path / "data"
    _make_fake_flowers(data_root, n_classes=NUM_CLASSES, per_class=3)
    _write_cache(data_root)
    monkeypatch.setattr(data_mod, "_flowers102_class", lambda: _FakeFlowers102)

    cfg = _cfg(tmp_path, debug=True, debug_per_class=1, batch_size=8)
    train_dl, val_dl, test_dl, class_names = data_mod.build_dataloaders(cfg)

    assert len(class_names) == NUM_CLASSES
    assert class_names[0] == "sp001"
    assert class_names[NUM_CLASSES - 1] == "sp102"

    assert len(train_dl.dataset) == NUM_CLASSES  # 1 imagen por clase
    assert len(val_dl.dataset) == NUM_CLASSES
    assert len(test_dl.dataset) == NUM_CLASSES
    assert train_dl.batch_size == 8
    assert isinstance(train_dl.sampler, RandomSampler)
    assert isinstance(val_dl.sampler, SequentialSampler)
```

```
    assert isinstance(test_dl.sampler, SequentialSampler)

    batch_x, batch_y = next(iter(train_dl))
    assert tuple(batch_x.shape) == (8, 3, IMG_SIZE, IMG_SIZE)
    assert int(batch_y.min()) >= 0
    assert int(batch_y.max()) < NUM_CLASSES

    # Correspondencia etiqueta <-> nombre (pitfall 1-based de cat_to_name.json)
    for index in range(len(train_dl.dataset)):
        _, label = train_dl.dataset[index]
        assert class_names[label] == f"sp{label + 1:03d}"


def test_debug_subset_is_reproducible(tmp_path, monkeypatch):
    """El subconjunto de debug no debe cambiar entre corridas con la misma semilla."""
    _require_torch()

    data_root = tmp_path / "data"
    _make_fake_flowers(data_root, n_classes=NUM_CLASSES, per_class=5)
    _write_cache(data_root)
    monkeypatch.setattr(data_mod, "_flowers102_class", lambda: _FakeFlowers102)

    cfg = _cfg(tmp_path, debug=True, debug_per_class=2)
    first = data_mod.build_dataloaders(cfg)[0].dataset.indices
    second = data_mod.build_dataloaders(cfg)[0].dataset.indices
    assert list(first) == list(second)
    assert len(first) == NUM_CLASSES * 2


def test_debug_subset_handles_classes_with_few_samples(tmp_path):
    """Si una clase tiene menos imagenes que debug_per_class, no debe fallar."""
    _require_torch()
    from torch.utils.data import Subset

    data_root = tmp_path / "data"
    _make_fake_flowers(data_root, n_classes=4, per_class=2)
    dataset = _FakeFlowers102(data_root)
    subset = data_mod._debug_subset(dataset, per_class=10, seed=0)
    assert isinstance(subset, Subset)
    assert len(subset) == 8  # 4 clases x 2 disponibles


# ---------------------------------------------------------- class names
def test_names_from_dataset_prefers_real_names_and_rejects_generic():
    """Los nombres propios del dataset ganan; los genericos se descartan."""

    class _WithRealNames:
        classes = ["pink primrose", "globe thistle"] + [f"especie real {i}" for i in range(100)]

        def __len__(self) -> int:
            return NUM_CLASSES

    class _WithGenericNames:
        classes = [f"class_{i}" for i in range(NUM_CLASSES)]

        def __len__(self) -> int:
            return NUM_CLASSES

    class _WithClassToIdx:
        class_to_idx = {f"flor_{i:03d}": i for i in range(NUM_CLASSES)}

        def __len__(self) -> int:
            return NUM_CLASSES

    class _WithWrongLength:
        classes = ["a", "b"]

        def __len__(self) -> int:
```

```
            return 2

    assert data_mod._names_from_dataset(_WithRealNames(), NUM_CLASSES)[0] == "pink primrose"
    assert data_mod._names_from_dataset(_WithGenericNames(), NUM_CLASSES) is None
    assert data_mod._names_from_dataset(_WithWrongLength(), NUM_CLASSES) is None
    assert data_mod._names_from_dataset(_WithClassToIdx(), NUM_CLASSES)[7] == "flor_007"
    assert data_mod._names_from_dataset(None, NUM_CLASSES) is None


def test_get_class_names_prefers_dataset_class_attribute(tmp_path, monkeypatch):
    """Prioridad 1 del contrato: los nombres propios del dataset ganan al cache."""
    _require_torch()
    try:
        flowers = data_mod._flowers102_class()
    except ImportError as exc:
        pytest.skip(f"torchvision.datasets.Flowers102 no disponible: {exc}")
    if not getattr(flowers, "classes", None):
        pytest.skip("esta version de torchvision no expone Flowers102.classes")

    cfg = _cfg(tmp_path)
    _write_cache(cfg.data_root, prefix="cache")  # el cache NO debe tener prioridad
    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _raise_no_network)

    assert data_mod.get_class_names(cfg) == list(flowers.classes)


def test_get_class_names_from_cache_uses_1based_keys(tmp_path, monkeypatch):
    """``class_names[i]`` debe salir de ``cat[str(i + 1)]`` (claves 1-based)."""
    _hide_dataset_names(monkeypatch)
    cfg = _cfg(tmp_path)
    _write_cache(cfg.data_root)

    names = data_mod.get_class_names(cfg)
    assert len(names) == NUM_CLASSES
    assert names[0] == "sp001"
    assert names[NUM_CLASSES - 1] == "sp102"


def test_get_class_names_downloads_and_caches(tmp_path, monkeypatch):
    """Sin cache y sin respaldo offline: descarga (simulada) y deja el JSON."""
    _hide_dataset_names(monkeypatch)
    _hide_offline_backups(monkeypatch)
    cfg = _cfg(tmp_path)
    payload = json.dumps({str(i + 1): f"flor_{i + 1:03d}" for i in range(NUM_CLASSES)}).encode("utf-8")
    calls: list[str] = []

    class _FakeResponse:
        def __enter__(self):
            return self

        def __exit__(self, *exc_info):
            return False

        def read(self):
            return payload

    def _fake_urlopen(url, timeout=None):
        calls.append(str(url))
        return _FakeResponse()

    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _fake_urlopen)

    names = data_mod.get_class_names(cfg)
    assert calls == [data_mod.CAT_TO_NAME_URL]
    assert names[0] == "flor_001"
    assert names[NUM_CLASSES - 1] == "flor_102"
```

```
    cached = json.loads((Path(cfg.data_root) /
data_mod.CAT_TO_NAME_FILENAME).read_text(encoding="utf-8"))
    assert cached["1"] == "flor_001"
    assert cached["102"] == "flor_102"

    # Segunda llamada: usa el cache, sin volver a la red
    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _raise_no_network)
    assert data_mod.get_class_names(cfg) == names


def test_get_class_names_without_internet_returns_fallback(tmp_path, monkeypatch):
    """Sin red, sin cache y sin respaldo: ``['especie_001', ..., 'especie_102']``."""
    _hide_dataset_names(monkeypatch)
    _hide_offline_backups(monkeypatch)
    cfg = _cfg(tmp_path)
    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _raise_no_network)

    names = data_mod.get_class_names(cfg)
    assert len(names) == NUM_CLASSES
    assert names[0] == "especie_001"
    assert names[NUM_CLASSES - 1] == "especie_102"
    assert not (Path(cfg.data_root) / data_mod.CAT_TO_NAME_FILENAME).exists()


def test_get_class_names_ignores_incomplete_cache(tmp_path, monkeypatch):
    """Un cache parcial o corrupto se descarta (y no rompe la carga)."""
    _hide_dataset_names(monkeypatch)
    _hide_offline_backups(monkeypatch)
    cfg = _cfg(tmp_path)
    data_root = Path(cfg.data_root)
    data_root.mkdir(parents=True, exist_ok=True)
    (data_root / data_mod.CAT_TO_NAME_FILENAME).write_text(json.dumps({"1": "solo una"}),
encoding="utf-8")
    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _raise_no_network)

    names = data_mod.get_class_names(cfg)
    assert len(names) == NUM_CLASSES
    assert names[0] == "especie_001"

    (data_root / data_mod.CAT_TO_NAME_FILENAME).write_text("{no es json", encoding="utf-8")
    assert len(data_mod.get_class_names(cfg)) == NUM_CLASSES


def test_names_from_mapping_accepts_1based_and_0based_keys():
    """Acepta las dos convenciones: Oxford ("1".."102") y asset del repo ("0".."101")."""
    one_based = {str(i + 1): f"sp{i + 1:03d}" for i in range(NUM_CLASSES)}
    zero_based = {str(i): f"z{i:03d}" for i in range(NUM_CLASSES)}

    assert data_mod._names_from_mapping(one_based, NUM_CLASSES)[0] == "sp001"
    assert data_mod._names_from_mapping(one_based, NUM_CLASSES)[101] == "sp102"
    assert data_mod._names_from_mapping(zero_based, NUM_CLASSES)[0] == "z000"
    assert data_mod._names_from_mapping(zero_based, NUM_CLASSES)[101] == "z101"
    assert data_mod._names_from_mapping({"1": "solo una"}, NUM_CLASSES) is None
    assert data_mod._names_from_mapping({}, NUM_CLASSES) is None


def test_names_from_asset_backup_matches_dataset_order():
    """El respaldo del repo debe coincidir con el orden de etiqueta de torchvision."""
    if not Path(data_mod.ASSET_CAT_TO_NAME).is_file():
        pytest.skip("assets/cat_to_name.json no esta en el repo")

    names = data_mod._names_from_asset(NUM_CLASSES)
    assert names is not None
    assert len(names) == NUM_CLASSES
    assert names[0] == "pink primrose"  # etiqueta 0
    assert names[101] == "blackberry lily"  # etiqueta 101
```

```
def test_get_class_names_uses_offline_asset_backup(tmp_path, monkeypatch):
    """Sin nombres del dataset, sin cache y sin red: usa ``assets/cat_to_name.json``."""
    if not Path(data_mod.ASSET_CAT_TO_NAME).is_file():
        pytest.skip("assets/cat_to_name.json no esta en el repo")

    _hide_dataset_names(monkeypatch)
    monkeypatch.setattr(data_mod.urllib.request, "urlopen", _raise_no_network)

    names = data_mod.get_class_names(_cfg(tmp_path))
    assert len(names) == NUM_CLASSES
    assert names[0] == "pink primrose"
    assert names[101] == "blackberry lily"


# ---------------------------------------------------- dataset real (lento)
def _build_or_skip(cfg: Config):
    """Construye los loaders reales; si la red falla, ``pytest.skip``.

    La descarga real se intenta como maximo UNA vez por sesion (con timeout).

    Nota de entorno (solo Windows + sandbox): ``tarfile`` crea los directorios del
    ``.tgz`` con ``os.mkdir(..., 0o700)`` y en este sandbox un directorio 0o700
    queda inaccesible, asi que la extraccion falla con PermissionError. Solucion:
    crear antes ``data/flowers-102/jpg`` con permisos normales (en Colab/Linux no
    hace falta nada de esto).
    """
    _require_torch()
    pytest.importorskip("scipy", reason="Flowers102 necesita scipy (imagelabels.mat / setid.mat)")

    if _DOWNLOAD_STATE["failed"]:
        pytest.skip(f"intento previo de descarga fallido: {_DOWNLOAD_STATE['failed']}")
    try:
        return _run_with_timeout(lambda: data_mod.build_dataloaders(cfg), DOWNLOAD_TIMEOUT_S)
    except TimeoutError as exc:
        _DOWNLOAD_STATE["failed"] = str(exc)
        pytest.skip(f"descarga de Flowers-102 no terminada: {exc}")
    except PermissionError as exc:
        _DOWNLOAD_STATE["failed"] = f"{type(exc).__name__}: {exc}"
        pytest.skip(
            f"extraccion bloqueada por permisos ({exc}). En este sandbox, crea antes "
            "data/flowers-102/jpg con mkdir normal y repite (tarfile usa mode 0o700 "
            "para los directorios del .tgz)"
        )
    except Exception as exc:  # sin red, URL caida, zip corrupto...
        _DOWNLOAD_STATE["failed"] = f"{type(exc).__name__}: {exc}"
        pytest.skip(f"dataset real no disponible ({type(exc).__name__}: {exc})")


@pytest.fixture(scope="session")
def real_debug_loaders():
    """Loaders reales en modo debug (data_root por defecto: 'data')."""
    return _build_or_skip(Config(debug=True, debug_per_class=4, batch_size=8))


@pytest.mark.slow
def test_real_flowers102_debug_mode(real_debug_loaders):
    """Dataset real en debug: 408 imagenes por split, lote (B,3,224,224), etiquetas en [0,102)."""
    import torch

    train_dl, val_dl, test_dl, class_names = real_debug_loaders

    assert len(class_names) == NUM_CLASSES
    assert all(isinstance(name, str) and name.strip() for name in class_names)
    # Si salieron nombres reales (dataset/cache/descarga), la etiqueta 0 debe ser
    # la especie 1 oficial: valida el pitfall 1-based de cat_to_name.json.
    if not any(name.startswith("especie_") for name in class_names):
        assert class_names[0] == "pink primrose"
        assert len(set(class_names)) == NUM_CLASSES
```

```
    expected = NUM_CLASSES * 4  # debug_per_class=4
    assert len(train_dl.dataset) == expected
    assert len(val_dl.dataset) == expected
    assert len(test_dl.dataset) == expected

    batch_x, batch_y = next(iter(train_dl))
    assert batch_x.ndim == 4
    assert tuple(batch_x.shape[1:]) == (3, IMG_SIZE, IMG_SIZE)
    assert int(batch_y.min()) >= 0
    assert int(batch_y.max()) < NUM_CLASSES
    assert bool(torch.isfinite(batch_x).all())

    # etiqueta -> nombre consistente para todas las muestras del subconjunto
    for index in range(len(train_dl.dataset)):
        _, label = train_dl.dataset[index]
        assert 0 <= int(label) < NUM_CLASSES
        assert class_names[int(label)]


@pytest.mark.slow
def test_real_flowers102_official_sizes():
    """Los splits oficiales deben ser 1020 / 1020 / 6149 (contrato §3)."""
    train_dl, val_dl, test_dl, class_names = _build_or_skip(Config(debug=False, num_workers=0,
batch_size=4))

    assert (len(train_dl.dataset), len(val_dl.dataset), len(test_dl.dataset)) == (1020, 1020, 6149)
    assert len(class_names) == NUM_CLASSES
```

tests/test_model.py

Pruebas del modelo y de los checkpoints. (273 líneas)

```
"""Pruebas de `src/model.py` (SA-02).

Criterio de aceptacion del contrato:
  * `build_model(102)` con `pretrained=False` -> forward (1,3,224,224) -> (1,102).
  * `save_checkpoint` + `load_checkpoint` reproducen EXACTAMENTE las mismas salidas.

Todas las pruebas usan `pretrained=False` para no depender de la red.
Se ejecutan con:  .venv\\Scripts\\python.exe -m pytest tests\\test_model.py -q
"""

from __future__ import annotations

import sys
from pathlib import Path

import pytest

# Permite `import src.model` aun si se ejecuta pytest desde otro directorio
# (el comando del contrato ya funciona porque `python -m pytest` agrega el cwd).
ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import torch  # noqa: E402

torch.set_num_threads(2)  # CPU: evita que resnet50 acapare todos los nucleos

from src.config import FLOWERS102_CLASSES, Config  # noqa: E402
from src.model import (  # noqa: E402
    CHECKPOINT_KEYS,
    SUPPORTED_BACKBONES,
    FlowerClassifier,
    build_model,
    build_optimizer,
    load_checkpoint,
    save_checkpoint,
    set_backbone_trainable,
)

BACKBONES = ["efficientnet_b0", "resnet50", "mobilenet_v3_large"]
CLASS_NAMES = [f"especie_{i + 1:03d}" for i in range(FLOWERS102_CLASSES)]


def _input(batch: int = 1, seed: int = 0) -> torch.Tensor:
    """Tensor de entrada deterministico (misma entrada al comparar salidas)."""
    generator = torch.Generator().manual_seed(seed)
    return torch.randn(batch, 3, 224, 224, generator=generator)


def _trainable_params(module: torch.nn.Module) -> int:
    return sum(p.numel() for p in module.parameters() if p.requires_grad)


# ---------------------------------------------------------------------------- #
# Estructura y forward
# ---------------------------------------------------------------------------- #
def test_supported_backbones_declarados() -> None:
    assert set(SUPPORTED_BACKBONES) == set(BACKBONES)


@pytest.mark.parametrize("backbone", BACKBONES)
def test_forward_shape_y_atributos(backbone: str) -> None:
    model = build_model(102, backbone=backbone, pretrained=False)
    assert isinstance(model, FlowerClassifier)
```

```
    # Atributos obligatorios del contrato.
    assert isinstance(model.backbone, torch.nn.Module)
    assert isinstance(model.head, torch.nn.Module)
    assert model.backbone is not model.head

    model.eval()
    with torch.no_grad():
        out = model(_input())
    assert out.shape == (1, 102)
    assert torch.isfinite(out).all()


def test_pretrained_degrada_a_no_pretrained_con_warning(monkeypatch: pytest.MonkeyPatch) -> None:
    """Si la descarga de pesos falla (Colab sin internet) avisa y sigue sin fallar."""
    import src.model as model_module

    def _builder_sin_red(pretrained: bool):
        if pretrained:
            raise RuntimeError("descarga de pesos bloqueada (simulada)")
        return model_module._build_efficientnet_b0(False)

    monkeypatch.setitem(model_module._BACKBONE_BUILDERS, "efficientnet_b0", _builder_sin_red)

    with pytest.warns(RuntimeWarning, match="pretrained"):
        model = build_model(102, backbone="efficientnet_b0", pretrained=True)

    assert model.pretrained is False
    model.eval()
    with torch.no_grad():
        assert model(_input()).shape == (1, 102)


def test_error_real_sin_pretrained_se_propaga(monkeypatch: pytest.MonkeyPatch) -> None:
    """Con pretrained=False no se enmascara un fallo de construccion del backbone."""
    import src.model as model_module

    def _builder_roto(pretrained: bool):
        raise RuntimeError("fallo real del backbone")

    monkeypatch.setitem(model_module._BACKBONE_BUILDERS, "efficientnet_b0", _builder_roto)
    with pytest.raises(RuntimeError, match="fallo real"):
        build_model(102, backbone="efficientnet_b0", pretrained=False)


@pytest.mark.parametrize("backbone", ["efficientnet_b0", "resnet50"])
def test_forward_batch_y_num_classes_configurable(backbone: str) -> None:
    model = build_model(10, backbone=backbone, pretrained=False, dropout=0.0).eval()
    with torch.no_grad():
        out = model(_input(batch=2, seed=1))
    assert out.shape == (2, 10)


def test_dropout_se_aplica_en_la_cabeza() -> None:
    dropout = build_model(102, backbone="efficientnet_b0", pretrained=False, dropout=0.5)
    assert isinstance(dropout.head[0], torch.nn.Dropout)
    assert dropout.head[0].p == pytest.approx(0.5)
    assert dropout.head[-1].out_features == 102


# ---------------------------------------------------------------------------- #
# Congelar / descongelar
# ---------------------------------------------------------------------------- #
@pytest.mark.parametrize("backbone", BACKBONES)
def test_set_backbone_trainable(backbone: str) -> None:
    model = build_model(102, backbone=backbone, pretrained=False)
    total_backbone = sum(p.numel() for p in model.backbone.parameters())
    assert total_backbone > 0
```

```
    head_antes = _trainable_params(model.head)
    set_backbone_trainable(model, False)
    assert _trainable_params(model.backbone) == 0
    # Solo se toca el backbone: la cabeza sigue entrenable.
    assert _trainable_params(model.head) == head_antes > 0

    set_backbone_trainable(model, True)
    assert _trainable_params(model.backbone) == total_backbone


def test_set_backbone_trainable_sin_backbone() -> None:
    with pytest.raises(AttributeError):
        set_backbone_trainable(torch.nn.Linear(3, 3), False)


# ---------------------------------------------------------------------------- #
# Optimizador con LR diferencial
# ---------------------------------------------------------------------------- #
def test_build_optimizer_dos_grupos() -> None:
    cfg = Config()
    model = build_model(cfg.num_classes, backbone=cfg.backbone, pretrained=False)
    optimizer = build_optimizer(model, cfg)

    assert isinstance(optimizer, torch.optim.AdamW)
    assert len(optimizer.param_groups) == 2

    grupo_cabeza, grupo_backbone = optimizer.param_groups
    assert grupo_cabeza["lr"] == pytest.approx(cfg.lr)
    assert grupo_backbone["lr"] == pytest.approx(cfg.backbone_lr)
    assert grupo_cabeza["weight_decay"] == pytest.approx(cfg.weight_decay)
    assert grupo_backbone["weight_decay"] == pytest.approx(cfg.weight_decay)

    ids_cabeza = {id(p) for p in grupo_cabeza["params"]}
    ids_backbone = {id(p) for p in grupo_backbone["params"]}
    assert ids_cabeza == {id(p) for p in model.head.parameters()}
    assert ids_backbone == {id(p) for p in model.backbone.parameters()}
    assert not (ids_cabeza & ids_backbone)

    # El optimizador sigue siendo valido tras descongelar el backbone.
    set_backbone_trainable(model, False)
    set_backbone_trainable(model, True)
    x = _input()
    torch.nn.functional.cross_entropy(model(x), torch.zeros(1, dtype=torch.long)).backward()
    optimizer.step()


# ---------------------------------------------------------------------------- #
# Checkpoints
# ---------------------------------------------------------------------------- #
@pytest.mark.parametrize("backbone", ["efficientnet_b0", "resnet50", "mobilenet_v3_large"])
def test_save_load_checkpoint_reproduce_salidas(tmp_path: Path, backbone: str) -> None:
    cfg = Config(img_size=224, backbone=backbone, dropout=0.2, pretrained=False)
    model = build_model(cfg.num_classes, backbone=backbone, pretrained=False, dropout=cfg.dropout)
    model.eval()

    x = _input()
    with torch.no_grad():
        salida_original = model(x).clone()

    ckpt_path = tmp_path / "best.pt"
    save_checkpoint(ckpt_path, model, CLASS_NAMES, cfg, epoch=7, val_acc=0.8123)
    assert ckpt_path.is_file()

    cargado, class_names, meta = load_checkpoint(ckpt_path, device="cpu")

    # Contrato del checkpoint.
    assert isinstance(cargado, FlowerClassifier)
    assert class_names == CLASS_NAMES
```

```
    assert len(class_names) == 102
    # Contrato del checkpoint: las 8 claves deben estar en el ARCHIVO guardado.
    crudo = torch.load(ckpt_path, map_location="cpu", weights_only=False)
    for clave in CHECKPOINT_KEYS:
        assert clave in crudo, f"falta la clave {clave!r} en el checkpoint"
    # `meta` es el checkpoint SIN los pesos (decision documentada de SA-02).
    for clave in set(CHECKPOINT_KEYS) - {"model_state"}:
        assert clave in meta, f"falta la clave {clave!r} en meta"
    assert meta["backbone"] == backbone
    assert meta["num_classes"] == 102
    assert meta["img_size"] == 224
    assert meta["epoch"] == 7
    assert meta["val_acc"] == pytest.approx(0.8123)
    assert isinstance(meta["config"], dict) and meta["config"]["backbone"] == backbone

    # El modelo cargado queda en eval() y en el device pedido.
    assert cargado.training is False
    assert next(cargado.parameters()).device.type == "cpu"

    cargado.eval()
    with torch.no_grad():
        salida_cargada = cargado(x)
    assert salida_cargada.shape == salida_original.shape
    assert torch.allclose(salida_original, salida_cargada, atol=1e-6, rtol=1e-5)
    assert (salida_original - salida_cargada).abs().max().item() == 0.0


def test_checkpoint_crea_directorios_y_defaults(tmp_path: Path) -> None:
    cfg = Config(pretrained=False)
    model = build_model(cfg.num_classes, backbone=cfg.backbone, pretrained=False)
    ckpt_path = tmp_path / "sub" / "carpeta" / "sin_extra.pt"  # no existe todavia
    save_checkpoint(ckpt_path, model, CLASS_NAMES, cfg)
    assert ckpt_path.is_file()

    _, _, meta = load_checkpoint(ckpt_path)
    assert meta["epoch"] == -1
    assert meta["val_acc"] == pytest.approx(0.0)


def test_load_checkpoint_device_auto(tmp_path: Path) -> None:
    from src.utils import resolve_device

    cfg = Config(pretrained=False)
    model = build_model(cfg.num_classes, backbone=cfg.backbone, pretrained=False)
    ckpt_path = tmp_path / "auto.pt"
    save_checkpoint(ckpt_path, model, CLASS_NAMES, cfg, epoch=0, val_acc=0.1)

    cargado, _, _ = load_checkpoint(ckpt_path, device="auto")
    esperado = resolve_device("auto")
    assert next(cargado.parameters()).device == esperado


def test_load_checkpoint_archivo_inexistente(tmp_path: Path) -> None:
    with pytest.raises(FileNotFoundError):
        load_checkpoint(tmp_path / "no_existe.pt")


# ---------------------------------------------------------------------------- #
# Entradas invalidas
# ---------------------------------------------------------------------------- #
def test_backbone_invalido_lanza_valueerror() -> None:
    with pytest.raises(ValueError) as excinfo:
        build_model(102, backbone="vgg16", pretrained=False)

    mensaje = str(excinfo.value)
    for soportado in SUPPORTED_BACKBONES:
        assert soportado in mensaje
    assert "vgg16" in mensaje
```

```
def test_backbone_invalido_tambien_con_pretrained_true() -> None:
    with pytest.raises(ValueError):
        build_model(102, backbone="alexnet", pretrained=True)
```

tests/test_train.py

Pruebas del entrenamiento. (620 líneas)

```
"""Pruebas de `src/train.py` (SA-05).

Criterio de aceptacion del contrato (§4):
    `train(Config(debug=True, epochs=2, device="cpu"))` termina y escribe
    `best.pt`, `history.json` y `curvas.png`.

Estrategia:
  * Pruebas rapidas: se monkeypatchea `src.train.build_dataloaders` para devolver
    3 `DataLoader` con tensores aleatorios. No hay red, ni dataset, ni descarga de
    pesos (`pretrained=False`).
  * Prueba lenta (dataset real, 330 MB): `test_train_dataset_real_end_to_end` se
    salta salvo que se exporte `IA_FLORES_SLOW=1`. No usa ningun marcador de pytest
    a proposito (pytest.ini es del orquestador y no registra marcadores):
        $env:IA_FLORES_SLOW = "1"
        .\\.venv\\Scripts\\python.exe -m pytest tests\\test_train.py -q

Ejecutar:  .venv\\Scripts\\python.exe -m pytest tests\\test_train.py -q
"""

from __future__ import annotations

import os
import sys
import time
import warnings
from pathlib import Path

import pytest

# Permite `import src.train` aun si se ejecuta pytest desde otro directorio.
ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import torch  # noqa: E402

torch.set_num_threads(2)  # CPU: no acaparar nucleos mientras otros subagentes trabajan

from torch.utils.data import DataLoader, TensorDataset  # noqa: E402

import src.train as train_module  # noqa: E402
from src.config import Config  # noqa: E402
from src.model import CHECKPOINT_KEYS, load_checkpoint  # noqa: E402
from src.train import (  # noqa: E402
    HISTORY_KEYS,
    HISTORY_LIST_KEYS,
    HISTORY_SCALAR_KEYS,
    build_argparser,
    config_from_args,
    main,
    train,
)
from src.utils import load_json  # noqa: E402

#: Claves que la celda de curvas del notebook (SA-03) indexa DIRECTO.
CLAVES_DEL_NOTEBOOK = ("epochs", "train_loss", "val_loss", "train_acc", "val_acc")

IMG_SIZE_SINTETICO = 32  # imagenes diminutas: los tests deben durar segundos


# ---------------------------------------------------------------------------- #
# Infraestructura sintetica (sin red, sin dataset)
# ---------------------------------------------------------------------------- #
def _split(n: int, num_classes: int, seed: int) -> TensorDataset:
```

```
    """Subconjunto sintetico reproducible: tensores aleatorios + etiquetas."""
    generator = torch.Generator().manual_seed(seed)
    x = torch.randn(n, 3, IMG_SIZE_SINTETICO, IMG_SIZE_SINTETICO, generator=generator)
    y = torch.randint(0, int(num_classes), (n,), generator=generator)
    return TensorDataset(x, y)


def build_dataloaders_falso(n_train: int = 16, n_val: int = 8, n_test: int = 8):
    """Sustituto de `src.train.build_dataloaders` con 3 DataLoaders sinteticos.

    Respeta `cfg.batch_size` y `cfg.num_classes` para que el resto de `train()`
    (criterio, checkpoint, historial) se comporte como con datos reales.
    """

    def _fake(cfg: Config):
        batch_size = int(cfg.batch_size)
        class_names = [f"especie_{i + 1:03d}" for i in range(int(cfg.num_classes))]
        loaders = (
            DataLoader(_split(n_train, cfg.num_classes, 1), batch_size=batch_size, shuffle=True),
            DataLoader(_split(n_val, cfg.num_classes, 2), batch_size=batch_size, shuffle=False),
            DataLoader(_split(n_test, cfg.num_classes, 3), batch_size=batch_size, shuffle=False),
        )
        return loaders[0], loaders[1], loaders[2], class_names

    return _fake


def config_sintetica(tmp_path: Path, **overrides) -> Config:
    """`Config` minimo y rapido para los tests (CPU, 4 clases, pretrained=False)."""
    base = dict(
        num_classes=4,
        debug=True,
        epochs=2,
        batch_size=8,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    base.update(overrides)
    return Config(**base)


class LoaderQueFalla:
    """DataLoader falso que revienta como los workers del sandbox de Windows.

    Simula `PermissionError [WinError 5]` al iterar con `num_workers > 0` (los
    procesos hijo no pueden abrir named pipes). Expone `.dataset` y `__len__`
    porque `train()` los usa para el log.
    """

    def __init__(self, dataset, error: BaseException | None = None) -> None:
        self.dataset = dataset
        self.error = error or PermissionError(
            "[WinError 5] Acceso denegado: no se puede abrir el named pipe del worker"
        )

    def __len__(self) -> int:
        return 1

    def __iter__(self):
        raise self.error


def build_dataloaders_con_fallo_de_workers(n_train: int = 8, n_val: int = 8, n_test: int = 8):
    """Sustituto que falla con `num_workers > 0` y funciona con `num_workers=0`.

    Devuelve `(fake, llamadas)`; `llamadas` registra el `num_workers` de cada
    invocacion para comprobar que hubo exactamente un reintento.
```

```
    """
    llamadas: list[int] = []

    def _fake(cfg: Config):
        llamadas.append(int(cfg.num_workers))
        class_names = [f"especie_{i + 1:03d}" for i in range(int(cfg.num_classes))]
        if int(cfg.num_workers) > 0:
            roto = LoaderQueFalla(_split(2, int(cfg.num_classes), 1))
            return roto, roto, roto, class_names

        real = build_dataloaders_falso(n_train, n_val, n_test)(cfg)
        return real

    return _fake, llamadas


# ---------------------------------------------------------------------------- #
# End-to-end sintetico
# ---------------------------------------------------------------------------- #
def test_train_sintetico_termina_y_escribe_artefactos(tmp_path: Path, monkeypatch) -> None:
    """`train()` termina, devuelve el historial completo y escribe los 4 artefactos."""
    cfg = config_sintetica(tmp_path)
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    history = train(cfg)

    # --- contrato del historial -------------------------------------------
    assert set(history) == set(HISTORY_KEYS)
    assert list(history) == list(HISTORY_KEYS), "orden de claves del contrato §3"
    for clave in CLAVES_DEL_NOTEBOOK:
        assert clave in history, f"el notebook de SA-03 indexa {clave!r} directamente"
    for clave in HISTORY_LIST_KEYS:
        assert isinstance(history[clave], list)
        assert len(history[clave]) == 2, f"{clave} debe tener una entrada por epoca"
    for clave in HISTORY_SCALAR_KEYS:
        assert not isinstance(history[clave], list)

    assert history["epochs"] == [1, 2]
    assert history["epochs_run"] == 2
    assert history["device"] == "cpu"
    assert history["num_params"] > 0
    assert history["segundos"] > 0.0
    assert 0.0 <= history["best_val_acc"] <= 1.0
    assert history["best_epoch"] in (1, 2)
    assert history["best_val_acc"] == pytest.approx(max(history["val_acc"]))
    assert history["best_val_acc"] == pytest.approx(history["val_acc"][history["best_epoch"] - 1])
    assert all(0.0 <= v <= 1.0 for v in history["val_acc"])
    assert all(0.0 <= v <= 1.0 for v in history["val_top5"])
    assert all(v > 0 for v in history["lr"])
    assert history["lr"][0] == pytest.approx(cfg.lr)  # sin warmup arranca en cfg.lr

    # --- artefactos en disco ----------------------------------------------
    assert cfg.checkpoint_path.is_file(), "falta best.pt"
    assert cfg.history_path.is_file(), "falta history.json"
    assert cfg.config_path.is_file(), "falta config_usada.json"
    curvas = cfg.figures_dir() / "curvas.png"
    assert curvas.is_file(), "falta figuras/curvas.png"
    assert curvas.read_bytes()[:8] == b"\x89PNG\r\n\x1a\n", "curvas.png no es un PNG valido"

    # history.json == historial devuelto (JSON serializable de verdad)
    guardado = load_json(cfg.history_path)
    assert set(guardado) == set(HISTORY_KEYS)
    assert guardado["epochs"] == [1, 2]
    assert guardado["best_epoch"] == history["best_epoch"]
    assert guardado["num_params"] == history["num_params"]

    # config_usada.json refleja la config efectiva de la corrida
    config_guardada = load_json(cfg.config_path)
```

```
    assert config_guardada["num_classes"] == 4
    assert config_guardada["debug"] is True
    assert config_guardada["out_dir"] == str(cfg.out_dir)


def test_checkpoint_cumple_contrato_y_guarda_la_mejor_epoca(tmp_path: Path, monkeypatch) -> None:
    """`best.pt` solo se escribe al mejorar `val_acc` y trae las 8 claves del contrato."""
    cfg = config_sintetica(tmp_path)
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    history = train(cfg)

    crudo = torch.load(cfg.checkpoint_path, map_location="cpu", weights_only=False)
    for clave in CHECKPOINT_KEYS:
        assert clave in crudo, f"falta la clave {clave!r} en el checkpoint"
    assert crudo["epoch"] == history["best_epoch"]
    assert crudo["val_acc"] == pytest.approx(history["best_val_acc"])
    assert crudo["num_classes"] == 4
    assert crudo["backbone"] == cfg.backbone
    assert len(crudo["class_names"]) == 4
    assert crudo["config"]["num_classes"] == 4

    # El checkpoint es cargable por SA-02 / SA-06 y reproduce inferencia.
    modelo, class_names, meta = load_checkpoint(cfg.checkpoint_path, device="cpu")
    assert class_names == [f"especie_{i + 1:03d}" for i in range(4)]
    assert meta["val_acc"] == pytest.approx(history["best_val_acc"])
    assert modelo.training is False
    with torch.no_grad():
        salida = modelo(torch.randn(2, 3, IMG_SIZE_SINTETICO, IMG_SIZE_SINTETICO))
    assert salida.shape == (2, 4)


def test_train_registra_lr_warmup_coseno_y_descongela_backbone(tmp_path: Path, monkeypatch) -> None:
    """Warmup + coseno (SequentialLR) y descongelado del backbone en `freeze_epochs`."""
    cfg = Config(
        num_classes=4,
        debug=False,  # debug forzaria warmup=freeze=0: aqui queremos probar ambos
        epochs=4,
        batch_size=8,
        warmup_epochs=2,
        freeze_epochs=2,
        early_stop_patience=50,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    llamadas: list[bool] = []
    real = train_module.set_backbone_trainable

    def _spy(model, trainable):
        llamadas.append(bool(trainable))
        return real(model, trainable)

    monkeypatch.setattr(train_module, "set_backbone_trainable", _spy)

    history = train(cfg)

    # Congelado en las 2 primeras epocas y descongelado despues, con el MISMO
    # optimizador (no se reconstruye en ningun momento).
    assert False in llamadas, "el backbone nunca se congelo"
    assert True in llamadas, "el backbone nunca se descongelo"
    # [antes del bucle, epoca 1, epoca 2, epoca 3, epoca 4, al terminar]
    assert llamadas == [False, False, False, True, True, True]

    lrs = history["lr"]
    assert len(lrs) == 4
```

```
    assert len(set(lrs)) > 1, "el scheduler no cambio el LR"
    assert lrs[0] < lrs[1] < lrs[2], "el warmup debe subir el LR epoca a epoca"
    assert lrs[-1] < max(lrs), "el coseno debe bajar el LR"
    assert max(lrs) == pytest.approx(cfg.lr)  # pico del warmup = lr de la cabeza
    assert lrs[0] == pytest.approx(cfg.lr * 0.1)  # arranque del warmup
    assert lrs[-1] >= 0.0


def test_train_warmup_cero_usa_solo_coseno(tmp_path: Path, monkeypatch) -> None:
    """Con `warmup_epochs=0` no hay SequentialLR: el LR solo decrece (coseno)."""
    cfg = Config(
        num_classes=4,
        debug=False,
        epochs=3,
        batch_size=8,
        warmup_epochs=0,
        freeze_epochs=0,
        early_stop_patience=50,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    history = train(cfg)

    lrs = history["lr"]
    assert len(lrs) == 3
    assert lrs[0] == pytest.approx(cfg.lr)
    assert lrs[0] > lrs[1] > lrs[2]


def test_early_stopping_corta_el_entrenamiento(tmp_path: Path, monkeypatch) -> None:
    """Con `early_stop_patience=1` y val_acc que nunca mejora, para tras la 2a epoca."""
    cfg = Config(
        num_classes=4,
        debug=False,
        epochs=10,
        batch_size=8,
        warmup_epochs=0,
        freeze_epochs=0,
        early_stop_patience=1,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    # Serie determinista de val_acc decreciente: nunca hay mejora despues de la 1a.
    serie = [0.50, 0.40, 0.30, 0.20, 0.10]

    def _eval_falso(model, loader, criterion, device, amp_enabled, topk=5):
        val_acc = serie.pop(0)
        return 1.0 - val_acc, val_acc, min(1.0, val_acc + 0.1)

    monkeypatch.setattr(train_module, "_run_eval_epoch", _eval_falso)

    history = train(cfg)

    assert history["epochs_run"] == 2, "debio parar en la 2a epoca (paciencia 1)"
    assert history["epochs"] == [1, 2]
    assert history["best_epoch"] == 1
    assert history["best_val_acc"] == pytest.approx(0.50)
    assert history["val_acc"] == pytest.approx([0.50, 0.40])
    assert cfg.checkpoint_path.is_file()
    assert len(serie) == 3, "el bucle siguio consumiendo epocas tras el corte"
```

```
def test_amp_desactivado_en_cpu_sin_avisos(tmp_path: Path, monkeypatch) -> None:
    """En CPU, `amp=True` no construye GradScaler ni activa autocast, y no avisa."""
    cfg = Config(
        num_classes=4,
        debug=False,
        epochs=1,
        batch_size=8,
        warmup_epochs=0,
        freeze_epochs=0,
        amp=True,  # se debe ignorar en CPU
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    escalares: list[tuple] = []
    real_scaler = torch.amp.GradScaler

    def _spy_scaler(*args, **kwargs):
        escalares.append((args, kwargs))
        return real_scaler(*args, **kwargs)

    flags_autocast: list[bool] = []
    real_autocast = torch.amp.autocast

    def _spy_autocast(*args, **kwargs):
        flags_autocast.append(bool(kwargs.get("enabled", True)))
        return real_autocast(*args, **kwargs)

    monkeypatch.setattr(torch.amp, "GradScaler", _spy_scaler)
    monkeypatch.setattr(torch.amp, "autocast", _spy_autocast)

    with warnings.catch_warnings(record=True) as capturados:
        warnings.simplefilter("always")
        history = train(cfg)

    assert escalares == [], "no se debe crear GradScaler en CPU"
    assert flags_autocast, "autocast no se llamo"
    assert not any(flags_autocast), "autocast debe ir con enabled=False en CPU"
    assert history["epochs_run"] == 1

    mensajes = [str(w.message).lower() for w in capturados]
    assert not [m for m in mensajes if "gradscaler" in m or "cuda" in m], (
        f"AMP en CPU no debe avisar: {mensajes}"
    )


def test_train_es_reproducible_con_la_misma_semilla(tmp_path: Path, monkeypatch) -> None:
    """`set_seed(cfg.seed)` hace que dos corridas identicas den el mismo historial."""
    monkeypatch.setattr(train_module, "build_dataloaders", build_dataloaders_falso())

    primera = train(config_sintetica(tmp_path / "a", seed=123))
    segunda = train(config_sintetica(tmp_path / "b", seed=123))

    assert primera["train_loss"] == pytest.approx(segunda["train_loss"])
    assert primera["val_acc"] == pytest.approx(segunda["val_acc"])


def test_train_reintenta_con_num_workers_0_si_fallan_los_workers(tmp_path: Path, monkeypatch) -> None:
    """Fallo de multiprocessing/named pipes: aviso + UN reintento con num_workers=0."""
    fake, llamadas = build_dataloaders_con_fallo_de_workers()
    monkeypatch.setattr(train_module, "build_dataloaders", fake)

    cfg = Config(
        num_classes=4,
        debug=False,  # debug forzaria num_workers=0 y no probaria el fallback
        epochs=2,
```

```
        batch_size=8,
        num_workers=4,
        warmup_epochs=0,
        freeze_epochs=0,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )

    with pytest.warns(RuntimeWarning, match="num_workers=0"):
        history = train(cfg)

    assert llamadas == [4, 0], f"debe reconstruir los loaders una sola vez: {llamadas}"
    assert history["epochs_run"] == 2
    assert cfg.checkpoint_path.is_file()
    assert history["num_params"] > 0


def test_train_no_reintenta_errores_que_no_son_de_workers(tmp_path: Path, monkeypatch) -> None:
    """Un error real del dataset NO se enmascara con el reintento."""

    def _fake(cfg: Config):
        raise ValueError("el dataset tiene un formato inesperado")

    monkeypatch.setattr(train_module, "build_dataloaders", _fake)

    cfg = Config(
        num_classes=4,
        debug=False,
        epochs=1,
        num_workers=2,
        device="cpu",
        pretrained=False,
        out_dir=str(tmp_path / "salida"),
    )
    with pytest.raises(ValueError, match="formato inesperado"):
        train(cfg)


def test_es_error_de_workers_clasifica_bien() -> None:
    """El detector de fallos de workers no confunde errores reales."""
    from src.train import _es_error_de_workers

    assert _es_error_de_workers(PermissionError("[WinError 5] Acceso denegado"))
    assert _es_error_de_workers(RuntimeError("DataLoader worker (pid 123) is killed by signal"))
    assert _es_error_de_workers(EOFError())
    assert not _es_error_de_workers(ValueError("backbone no soportado"))
    assert not _es_error_de_workers(FileNotFoundError("imagen.jpg no existe"))


# ---------------------------------------------------------------------------- #
# CLI (§3.b)
# ---------------------------------------------------------------------------- #
def test_cli_help_no_crashea() -> None:
    with pytest.raises(SystemExit) as excinfo:
        main(["--help"])
    assert excinfo.value.code == 0


def test_cli_sin_argumentos_usa_los_defaults_de_config() -> None:
    cfg = config_from_args(build_argparser().parse_args([]))
    assert cfg.to_dict() == Config().to_dict()


def test_cli_aplica_los_overrides_informados() -> None:
    args = build_argparser().parse_args(
        [
            "--epochs", "7",
```

```
            "--batch-size", "16",
            "--backbone", "resnet50",
            "--data-root", "datos-x",
            "--out-dir", "salida-x",
            "--device", "cpu",
            "--seed", "7",
        ]
    )
    cfg = config_from_args(args)
    assert cfg.epochs == 7
    assert cfg.batch_size == 16
    assert cfg.backbone == "resnet50"
    assert cfg.data_root == "datos-x"
    assert cfg.out_dir == "salida-x"
    assert cfg.device == "cpu"
    assert cfg.seed == 7
    # Lo no informado conserva los defaults (dataclasses.replace sobre Config()).
    base = Config()
    assert cfg.lr == base.lr
    assert cfg.dropout == base.dropout
    assert cfg.early_stop_patience == base.early_stop_patience
    assert cfg.debug is False


def test_cli_num_workers_default_de_config_y_override() -> None:
    """`--num-workers` es una extension acordada; el default de Config sigue siendo 2."""
    base = Config()
    assert base.num_workers == 2, "el default de Config no debe cambiar (Colab)"

    cfg = config_from_args(build_argparser().parse_args([]))
    assert cfg.num_workers == 2

    cfg = config_from_args(build_argparser().parse_args(["--num-workers", "0"]))
    assert cfg.num_workers == 0

    cfg = config_from_args(build_argparser().parse_args(["--num-workers", "6"]))
    assert cfg.num_workers == 6

    # Con --debug, Config fuerza num_workers=0 (subset en memoria, sin workers).
    cfg = config_from_args(build_argparser().parse_args(["--debug", "--num-workers", "6"]))
    assert cfg.num_workers == 0


def test_cli_debug_usa_resultados_debug_y_no_pisa_la_corrida_completa() -> None:
    cfg = config_from_args(build_argparser().parse_args(["--debug"]))
    assert cfg.debug is True
    assert cfg.out_dir == "resultados/debug"
    assert cfg.checkpoint_path == Path("resultados/debug") / "best.pt"

    # `--out-dir` explicito siempre gana.
    cfg = config_from_args(build_argparser().parse_args(["--debug", "--out-dir", "otra/salida"]))
    assert cfg.out_dir == "otra/salida"

    # Modo debug: Config.__post_init__ recorta epocas y desactiva freeze/warmup/amp.
    cfg = config_from_args(build_argparser().parse_args(["--debug", "--epochs", "7"]))
    assert cfg.epochs == 2
    assert cfg.freeze_epochs == 0
    assert cfg.warmup_epochs == 0
    assert cfg.amp is False
    assert cfg.num_workers == 0


# ---------------------------------------------------------------------------- #
# End-to-end con el dataset REAL (lento, opt-in)
# ---------------------------------------------------------------------------- #
def _parece_error_de_red(exc: BaseException) -> bool:
    """True si la excepcion huele a falta de internet (para saltar, no fallar)."""
    texto = f"{type(exc).__name__}: {exc}".lower()
```

```
    pistas = (
        "download",
        "descarga",
        "url",
        "http",
        "https",
        "connection",
        "conexion",
        "timed out",
        "timeout",
        "socket",
        "network",
        "unreachable",
        "ssl",
        "name or service",
        "getaddrinfo",
    )
    return any(pista in texto for pista in pistas)


def test_train_dataset_real_end_to_end(tmp_path: Path) -> None:
    """2 epocas REALES con Flowers-102 en CPU (lento: ~minutos).

    Se salta salvo que se exporte `IA_FLORES_SLOW=1`:

        $env:IA_FLORES_SLOW = "1"
        .\\.venv\\Scripts\\python.exe -m pytest tests\\test_train.py -q -s

    Si la descarga del dataset (o de los pesos) falla por red, se hace `skip`.

    NOTA (peculiaridad de esta maquina, no del proyecto): el sandbox bloquea
    `~/.cache/torch`, asi que para que esta prueba use pesos preentrenados DE
    VERDAD hay que apuntar TORCH_HOME a una carpeta escribible del workspace:

        $env:TORCH_HOME = (Resolve-Path .piptmp\\torchhome).Path

    Sin eso, `src/model.py` degrada a `pretrained=False` con un `RuntimeWarning`
    (comportamiento correcto y ya probado por SA-02) y el test sigue pasando.
    """
    if os.environ.get("IA_FLORES_SLOW") != "1":
        pytest.skip("lento (dataset real): exporta IA_FLORES_SLOW=1 para ejecutarlo")

    cfg = Config(
        debug=True,
        epochs=2,
        device="cpu",
        backbone="mobilenet_v3_large",
        out_dir=str(tmp_path / "real"),
    )

    inicio = time.perf_counter()
    try:
        history = train(cfg)
    except Exception as exc:  # noqa: BLE001 - red caida -> skip, bug real -> raise
        if _parece_error_de_red(exc):
            pytest.skip(f"sin red para el dataset/pesos reales: {exc}")
        raise
    segundos = time.perf_counter() - inicio

    print(
        f"\n[SLOW] Flowers-102 real, mobilenet_v3_large, CPU: 2 epocas en "
        f"{segundos:.1f} s ({segundos / 60.0:.2f} min)"
    )

    assert history["epochs_run"] == 2
    assert history["epochs"] == [1, 2]
    assert history["num_params"] > 300_000
    assert all(0.0 <= v <= 1.0 for v in history["val_acc"])
```

```
assert cfg.checkpoint_path.is_file()
assert cfg.history_path.is_file()
assert cfg.config_path.is_file()
assert (cfg.figures_dir() / "curvas.png").is_file()

# El dataset real trae las 102 especies y el checkpoint lo refleja.
crudo = torch.load(cfg.checkpoint_path, map_location="cpu", weights_only=False)
assert crudo["num_classes"] == 102
assert len(crudo["class_names"]) == 102
assert crudo["backbone"] == "mobilenet_v3_large"
for clave in CHECKPOINT_KEYS:
    assert clave in crudo
```

tests/test_eval_predict.py

Pruebas de la evaluación y de la predicción. (567 líneas)

```
"""Pruebas de `src/evaluate.py` y `src/predict.py` (SA-06).

Criterio de aceptacion del contrato (seccion 4):
  * `evaluate(checkpoint, Config(debug=True), split="test")` escribe `metrics.json`
    con `top1` y `top5` numericos, las dos figuras y el CSV de predicciones.
  * `predict_image(imagen_sintetica, checkpoint)` devuelve pares `(str, float)` con
    probabilidades decrecientes, recorta `topk` a `num_classes` y respeta `min_conf`.

Nada de red y nada del dataset real: el checkpoint es sintetico (4 clases,
`mobilenet_v3_large` con `pretrained=False`) y el DataLoader se inyecta con el
parametro opcional `dataloader` de `evaluate`.
Se ejecuta con:
    .venv\\Scripts\\python.exe -m pytest tests\\test_eval_predict.py -q
"""

from __future__ import annotations

import csv
import json
import sys
from pathlib import Path

import numpy as np
import pytest

# Permite `import src...` aunque pytest se ejecute desde otro directorio.
ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import torch  # noqa: E402
from PIL import Image  # noqa: E402
from torch.utils.data import DataLoader, Dataset  # noqa: E402

torch.set_num_threads(2)  # no saturar la maquina mientras otros subagentes trabajan

import src.evaluate as evaluate_mod  # noqa: E402
import src.predict as predict_mod  # noqa: E402
from src.config import Config  # noqa: E402
from src.evaluate import MENSAJE_ENTRENA, evaluate  # noqa: E402
from src.model import build_model, save_checkpoint  # noqa: E402
from src.predict import NO_RECONOCIDA, limpiar_cache, predict_batch, predict_image  # noqa: E402

N_CLASES = 4
NOMBRES = ["rosa", "tulipan", "girasol", "margarita"]
NOMBRES8 = [f"especie_{i + 1:02d}" for i in range(8)]
IMG_SIZE = 64  # pequeno a proposito: los tests tienen que ser rapidos en CPU
N_IMAGENES = 40  # 10 imagenes por clase
BACKBONE = "mobilenet_v3_large"  # el mas rapido de los tres soportados


# ---------------------------------------------------------------------------- #
# Utilidades de test
# ---------------------------------------------------------------------------- #
class _DatasetSintetico(Dataset):
    """`N_IMAGENES` tensores aleatorios con etiquetas ciclicas 0..n_clases-1."""

    def __init__(self, n: int = N_IMAGENES, n_clases: int = N_CLASES, img_size: int = IMG_SIZE, seed:
int = 7):
        generador = torch.Generator().manual_seed(seed)
        self.imagenes = torch.rand(n, 3, img_size, img_size, generator=generador)
        self.etiquetas = [i % n_clases for i in range(n)]

    def __len__(self) -> int:
```

```
        return len(self.etiquetas)

    def __getitem__(self, indice: int):
        return self.imagenes[indice], self.etiquetas[indice]


def _loader_sintetico(batch_size: int = 16) -> DataLoader:
    return DataLoader(_DatasetSintetico(), batch_size=batch_size, shuffle=False, num_workers=0,
drop_last=False)


def _crear_checkpoint(
    destino: Path,
    n_clases: int = N_CLASES,
    nombres: list[str] | None = None,
    img_size: int = IMG_SIZE,
) -> tuple[Path, Config]:
    """Checkpoint sintetico real (sin red) + la `Config` que lo describe."""
    nombres = list(nombres if nombres is not None else NOMBRES[:n_clases])
    out_dir = destino / "resultados"
    cfg = Config(
        num_classes=n_clases,
        img_size=img_size,
        debug=True,  # num_workers=0, amp=False: no descarga nada ni abre workers
        out_dir=str(out_dir),
        data_root=str(destino / "data"),
        pretrained=False,
    )
    modelo = build_model(n_clases, BACKBONE, pretrained=False).eval()
    ckpt = out_dir / "best.pt"
    save_checkpoint(ckpt, modelo, nombres, cfg, epoch=1, val_acc=1.0)
    return ckpt, cfg


@pytest.fixture(autouse=True)
def _cache_limpia():
    """Cada test arranca con la cache de modelos vacia (y la deja vacia)."""
    limpiar_cache()
    yield
    limpiar_cache()


# ---------------------------------------------------------------------------- #
# evaluate
# ---------------------------------------------------------------------------- #
def test_evaluate_escribe_metricas_figuras_y_csv(tmp_path: Path) -> None:
    ckpt, cfg = _crear_checkpoint(tmp_path)

    metricas = evaluate(ckpt, cfg, split="test", dataloader=_loader_sintetico())

    assert isinstance(metricas, dict)
    for clave in (
        "split",
        "num_images",
        "top1",
        "top5",
        "macro_f1",
        "per_class",
        "per_class_f1",
        "top_confusions",
        "ruta_confusion",
        "segundos",
    ):
        assert clave in metricas, f"falta la clave {clave!r} en el dict devuelto"

    assert metricas["split"] == "test"
    assert metricas["num_images"] == N_IMAGENES
    for clave in ("top1", "top5", "macro_f1"):
```

```
        assert isinstance(metricas[clave], float), f"{clave} debe ser float"
        assert 0.0 <= metricas[clave] <= 1.0
    assert metricas["top5"] >= metricas["top1"] - 1e-9
    assert isinstance(metricas["segundos"], float) and metricas["segundos"] >= 0.0

    # per_class: dict clase -> {precision, recall, f1, support}
    assert set(metricas["per_class"]) == set(NOMBRES)
    for valores in metricas["per_class"].values():
        assert set(valores) == {"precision", "recall", "f1", "support"}
        assert all(isinstance(valores[k], float) for k in ("precision", "recall", "f1"))
        assert valores["support"] == N_IMAGENES // N_CLASES
        assert 0.0 <= valores["f1"] <= 1.0

    # per_class_f1: dict nombre -> float (lo consume el notebook de SA-03)
    assert set(metricas["per_class_f1"]) == set(NOMBRES)
    assert all(isinstance(v, float) for v in metricas["per_class_f1"].values())
    for nombre, f1 in metricas["per_class_f1"].items():
        assert f1 == pytest.approx(metricas["per_class"][nombre]["f1"])

    # top_confusions: [[real, pred, n], ...] con n descendente y maximo 20
    confusiones = metricas["top_confusions"]
    assert isinstance(confusiones, list)
    assert len(confusiones) <= 20
    assert len(confusiones) <= N_CLASES * (N_CLASES - 1)  # solo pares real != pred
    for fila in confusiones:
        assert isinstance(fila, list) and len(fila) == 3
        assert isinstance(fila[0], str) and isinstance(fila[1], str)
        assert isinstance(fila[2], int) and fila[2] >= 1
        assert fila[0] != fila[1]
    assert [fila[2] for fila in confusiones] == sorted((f[2] for f in confusiones), reverse=True)

    # metrics.json
    assert cfg.metrics_path.is_file()
    datos = json.loads(cfg.metrics_path.read_text(encoding="utf-8"))
    assert datos["split"] == "test"
    assert datos["num_images"] == N_IMAGENES
    for clave in ("top1", "top5", "macro_f1"):
        assert isinstance(datos[clave], (int, float))
    assert set(datos["per_class_f1"]) == set(NOMBRES)
    assert datos["ruta_confusion"] == str(cfg.figures_dir() / "confusion_matrix.png")

    # figuras (PNG reales, no vacias)
    matriz_png = cfg.figures_dir() / "confusion_matrix.png"
    confusiones_png = cfg.figures_dir() / "top_confusions.png"
    assert matriz_png.is_file() and matriz_png.stat().st_size > 1000
    assert confusiones_png.is_file() and confusiones_png.stat().st_size > 1000
    assert Path(metricas["ruta_confusion"]) == matriz_png

    # CSV: encabezado exacto + una fila por imagen
    ruta_csv = cfg.predictions_dir() / "test_predicciones.csv"
    assert ruta_csv.is_file()
    with open(ruta_csv, newline="", encoding="utf-8") as fh:
        filas = list(csv.reader(fh))
    assert filas[0] == ["ruta", "etiqueta_real", "prediccion", "prob", "correcto"]
    assert len(filas) == N_IMAGENES + 1, "debe haber 40 filas + encabezado"
    for fila in filas[1:]:
        assert len(fila) == 5
        assert fila[1] in NOMBRES and fila[2] in NOMBRES
        assert 0.0 <= float(fila[3]) <= 1.0
        assert fila[4] in ("True", "False")


def test_evaluate_reconstruye_la_config_del_checkpoint_si_cfg_es_none(tmp_path: Path) -> None:
    ckpt, cfg = _crear_checkpoint(tmp_path)

    metricas = evaluate(ckpt, None, split="val", dataloader=_loader_sintetico())

    assert metricas["split"] == "val"
```

```
    # out_dir viene del `config` guardado en el checkpoint (no de `resultados/` del cwd)
    assert Path(metricas["checkpoint"]) == ckpt
    ruta_metricas = cfg.metrics_path
    assert ruta_metricas.is_file()
    datos = json.loads(ruta_metricas.read_text(encoding="utf-8"))
    assert datos["split"] == "val"
    assert (cfg.predictions_dir() / "val_predicciones.csv").is_file()


def test_evaluate_usa_el_img_size_del_checkpoint(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) ->
None:
    """Aunque el `cfg` del usuario diga 224, mandan los 64 px del checkpoint."""
    ckpt, cfg_ckpt = _crear_checkpoint(tmp_path, img_size=IMG_SIZE)
    capturado: dict = {}

    def _build_dataloaders_falso(cfg: Config):
        capturado["img_size"] = cfg.img_size
        cargador = _loader_sintetico()
        return cargador, cargador, cargador, list(NOMBRES)

    monkeypatch.setattr(evaluate_mod, "build_dataloaders", _build_dataloaders_falso)

    cfg_usuario = Config(
        num_classes=N_CLASES,
        img_size=224,  # distinto al del checkpoint a proposito
        debug=True,
        out_dir=str(cfg_ckpt.out_dir),
        data_root=str(tmp_path / "data"),
        pretrained=False,
    )
    metricas = evaluate(ckpt, cfg_usuario, split="test")

    assert capturado["img_size"] == IMG_SIZE
    assert metricas["img_size"] == IMG_SIZE
    assert metricas["num_images"] == N_IMAGENES


def test_evaluate_checkpoint_inexistente(tmp_path: Path) -> None:
    with pytest.raises(FileNotFoundError) as excinfo:
        evaluate(tmp_path / "no_existe.pt", Config(debug=True))
    mensaje = str(excinfo.value)
    assert "entrena" in mensaje
    assert "python -m src.train" in mensaje
    assert MENSAJE_ENTRENA in mensaje


def test_evaluate_split_invalido(tmp_path: Path) -> None:
    ckpt, cfg = _crear_checkpoint(tmp_path)
    with pytest.raises(ValueError):
        evaluate(ckpt, cfg, split="entrenamiento")


# ---------------------------------------------------------------------------- #
# Robustez de los workers del DataLoader (sandbox/Windows: named pipes bloqueados)
# ---------------------------------------------------------------------------- #
class _LoaderQueFalla:
    """DataLoader falso: iterar con workers lanza PermissionError como en Windows."""

    dataset = None

    def __iter__(self):
        raise PermissionError("[WinError 5] Acceso denegado")


def _cfg_con_workers(cfg_base: Config, num_workers: int) -> Config:
    """Copia de ``cfg_base`` SIN debug (debug fuerza num_workers=0) y con N workers."""
    return Config(
        num_classes=cfg_base.num_classes,
```

```
        img_size=cfg_base.img_size,
        debug=False,
        num_workers=num_workers,
        out_dir=str(cfg_base.out_dir),
        data_root=str(cfg_base.data_root),
        pretrained=False,
    )


def test_evaluate_reintenta_con_num_workers_0(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
    """Con `num_workers>0` que falla por permisos, `evaluate` reintenta con 0 y termina."""
    ckpt, cfg_base = _crear_checkpoint(tmp_path)
    cfg = _cfg_con_workers(cfg_base, 2)
    intentos: list[int] = []

    def _build_dataloaders_falso(cfg_actual: Config):
        intentos.append(int(cfg_actual.num_workers))
        cargador = _LoaderQueFalla() if int(cfg_actual.num_workers) > 0 else _loader_sintetico()
        return cargador, cargador, cargador, list(NOMBRES)

    monkeypatch.setattr(evaluate_mod, "build_dataloaders", _build_dataloaders_falso)

    with pytest.warns(RuntimeWarning, match="num_workers"):
        metricas = evaluate(ckpt, cfg, split="test")

    assert intentos == [2, 0], "debe reintentar una sola vez con num_workers=0"
    assert metricas["num_images"] == N_IMAGENES
    assert cfg.metrics_path.is_file()
    assert (cfg.figures_dir() / "confusion_matrix.png").is_file()
    assert (cfg.predictions_dir() / "test_predicciones.csv").is_file()


def test_evaluate_no_enmascara_errores_ajenos_a_los_workers(
    tmp_path: Path, monkeypatch: pytest.MonkeyPatch
) -> None:
    """Un error real del modelo no se reintenta ni se oculta."""
    ckpt, cfg_base = _crear_checkpoint(tmp_path)
    cfg = _cfg_con_workers(cfg_base, 2)

    class _LoaderRoto:
        dataset = None

        def __iter__(self):
            raise ValueError("forma de tensor inesperada")

    def _build_dataloaders_falso(cfg_actual: Config):
        cargador = _LoaderRoto()
        return cargador, cargador, cargador, list(NOMBRES)

    monkeypatch.setattr(evaluate_mod, "build_dataloaders", _build_dataloaders_falso)

    with pytest.raises(ValueError, match="forma de tensor"):
        evaluate(ckpt, cfg, split="test")


def test_evaluate_con_num_workers_0_no_reintenta(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) ->
None:
    """El reintento solo aplica cuando cfg.num_workers > 0."""
    ckpt, cfg_base = _crear_checkpoint(tmp_path)
    cfg = _cfg_con_workers(cfg_base, 0)
    assert cfg.num_workers == 0

    def _build_dataloaders_falso(cfg_actual: Config):
        cargador = _LoaderQueFalla()
        return cargador, cargador, cargador, list(NOMBRES)

    monkeypatch.setattr(evaluate_mod, "build_dataloaders", _build_dataloaders_falso)
```

```
    with pytest.raises(PermissionError):
        evaluate(ckpt, cfg, split="test")


def test_cli_evaluate_acepta_num_workers(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    capturado: dict = {}

    def _build_dataloaders_falso(cfg: Config):
        capturado["num_workers"] = cfg.num_workers
        cargador = _loader_sintetico()
        return cargador, cargador, cargador, list(NOMBRES)

    monkeypatch.setattr(evaluate_mod, "build_dataloaders", _build_dataloaders_falso)

    assert evaluate_mod.main(
        ["--checkpoint", str(ckpt), "--out-dir", str(tmp_path / "s1"), "--device", "cpu", "--num-
workers", "0"]
    ) == 0
    assert capturado["num_workers"] == 0

    # Sin el flag sigue mandando el default de Config (2): el flag es opcional.
    assert evaluate_mod.main(
        ["--checkpoint", str(ckpt), "--out-dir", str(tmp_path / "s2"), "--device", "cpu"]
    ) == 0
    assert capturado["num_workers"] == Config().num_workers == 2

    # Valor invalido: mensaje claro, sin traza.
    assert evaluate_mod.main(["--checkpoint", str(ckpt), "--num-workers", "-1"]) == 1


# ---------------------------------------------------------------------------- #
# predict_image
# ---------------------------------------------------------------------------- #
def _imagen_roja() -> Image.Image:
    return Image.new("RGB", (300, 220), "red")


def test_predict_image_devuelve_pares_decrecientes(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)

    resultados = predict_image(_imagen_roja(), ckpt, topk=3, min_conf=0.0, device="cpu")

    assert len(resultados) == 3, "debe devolver exactamente topk pares"
    assert all(isinstance(nombre, str) for nombre, _ in resultados)
    assert all(isinstance(prob, float) for _, prob in resultados)

    probabilidades = [prob for _, prob in resultados]
    assert probabilidades == sorted(probabilidades, reverse=True), "deben ser decrecientes"
    assert all(0.0 <= p <= 1.0 for p in probabilidades)
    assert sum(probabilidades) <= 1.0001, "suma de probabilidades softmax <= 1"
    assert all(nombre in NOMBRES for nombre, _ in resultados)


def test_predict_image_min_conf_imposible(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)

    resultados = predict_image(_imagen_roja(), ckpt, topk=5, min_conf=1.1, device="cpu")

    assert len(resultados) == 1
    assert resultados[0][0] == NO_RECONOCIDA == "no reconocida / fuera del catálogo"
    assert isinstance(resultados[0][1], float)
    assert 0.0 <= resultados[0][1] <= 1.0


def test_predict_topk_mayor_que_num_classes(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path, n_clases=N_CLASES, nombres=NOMBRES)
```

```
    resultados = predict_image(_imagen_roja(), ckpt, topk=999, min_conf=0.0, device="cpu")

    assert len(resultados) == N_CLASES, "topk se recorta a num_classes sin fallar"
    assert len({nombre for nombre, _ in resultados}) == N_CLASES


def test_predict_topk_no_supera_ocho_clases(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path, n_clases=8, nombres=NOMBRES8)

    resultados = predict_image(_imagen_roja(), ckpt, topk=999, min_conf=0.0, device="cpu")

    assert len(resultados) == 8


def test_predict_acepta_todos_los_formatos(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    roja = _imagen_roja()
    ruta = tmp_path / "roja.png"
    roja.save(ruta)
    arreglo = np.asarray(roja)

    referencia = predict_image(roja, ckpt, topk=2, min_conf=0.0, device="cpu")

    entradas = [
        ruta,  # Path
        str(ruta),  # str
        roja,  # PIL.Image
        arreglo,  # numpy uint8 (H, W, 3)
        (arreglo.astype("float32") / 255.0),  # numpy float en [0, 1]
        arreglo.transpose(2, 0, 1),  # numpy channel-first uint8
        {"path": str(ruta)},  # dict tipo Gradio
        (arreglo, str(ruta)),  # tupla que devuelve Gradio
        [str(ruta)],  # lista de un elemento
    ]
    for entrada in entradas:
        resultados = predict_image(entrada, ckpt, topk=2, min_conf=0.0, device="cpu")
        assert [nombre for nombre, _ in resultados] == [nombre for nombre, _ in referencia], (
            f"resultado distinto para {type(entrada)!r}"
        )
        assert [round(p, 6) for _, p in resultados] == [round(p, 6) for _, p in referencia]


def test_predict_checkpoint_inexistente(tmp_path: Path) -> None:
    with pytest.raises(FileNotFoundError) as excinfo:
        predict_image(_imagen_roja(), tmp_path / "no_existe.pt", device="cpu")
    assert "entrena" in str(excinfo.value)


def test_predict_imagen_inexistente(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    with pytest.raises(FileNotFoundError):
        predict_image(tmp_path / "no_existe.jpg", ckpt, device="cpu")


def test_predict_usa_la_cache(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)

    contador = {"llamadas": 0}
    load_real = predict_mod.load_checkpoint

    def _load_contado(*args, **kwargs):
        contador["llamadas"] += 1
        return load_real(*args, **kwargs)

    monkeypatch.setattr(predict_mod, "load_checkpoint", _load_contado)
    limpiar_cache()

    primero = predict_image(_imagen_roja(), ckpt, topk=2, min_conf=0.0, device="cpu")
```

```
    segundo = predict_image(_imagen_roja(), ckpt, topk=2, min_conf=0.0, device="cpu")

    assert contador["llamadas"] == 1, "la segunda llamada debe salir de la cache"
    assert primero == segundo

    # El modelo cacheado es literalmente el mismo objeto.
    modelo_a = predict_mod._modelo_cacheado(str(ckpt), "cpu")[0]
    modelo_b = predict_mod._modelo_cacheado(str(ckpt), "cpu")[0]
    assert modelo_a is modelo_b
    assert predict_mod._modelo_cacheado.cache_info().currsize == 1

    # limpiar_cache() vacia la cache y fuerza una recarga.
    limpiar_cache()
    assert predict_mod._modelo_cacheado.cache_info().currsize == 0
    predict_image(_imagen_roja(), ckpt, topk=2, min_conf=0.0, device="cpu")
    assert contador["llamadas"] == 2


def test_predict_batch(tmp_path: Path) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    rutas = []
    for indice, color in enumerate(("red", "green", "blue")):
        ruta = tmp_path / f"imagen_{indice}.png"
        Image.new("RGB", (120, 90), color).save(ruta)
        rutas.append(ruta)

    lotes = predict_batch(rutas, ckpt, topk=2, device="cpu")

    assert len(lotes) == 3
    for lote in lotes:
        assert isinstance(lote, list) and lote
        for nombre, probabilidad in lote:
            assert isinstance(nombre, str)
            assert isinstance(probabilidad, float)
            assert 0.0 <= probabilidad <= 1.0
    assert all(len(lote) <= 2 for lote in lotes)


# ---------------------------------------------------------------------------- #
# CLI (se prueban las funciones main/build_parser: sin subprocess ni tuberias)
# ---------------------------------------------------------------------------- #
def test_cli_evaluate_help_y_checkpoint_ausente(tmp_path: Path, capsys: pytest.CaptureFixture) -> None:
    with pytest.raises(SystemExit) as excinfo:
        evaluate_mod.main(["--help"])
    assert excinfo.value.code == 0
    assert "--split" in capsys.readouterr().out

    codigo = evaluate_mod.main(["--checkpoint", str(tmp_path / "no_existe.pt")])
    salida = capsys.readouterr().out
    assert codigo == 1
    assert "entrena" in salida.lower()
    assert "python -m src.train" in salida

    assert evaluate_mod.build_parser().parse_args(["--split", "val"]).split == "val"


def test_cli_evaluate_ejecuta_sobre_checkpoint(tmp_path: Path, monkeypatch: pytest.MonkeyPatch, capsys)
-> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    monkeypatch.setattr(evaluate_mod, "build_dataloaders", lambda cfg: (_loader_sintetico(),) * 3 +
(list(NOMBRES),))

    codigo = evaluate_mod.main(
        ["--checkpoint", str(ckpt), "--split", "test", "--out-dir", str(tmp_path / "salida"), "--
device", "cpu", "--debug"]
    )

    assert codigo == 0
```

```
    salida = capsys.readouterr().out
    assert "top1=" in salida
    assert (tmp_path / "salida" / "metrics.json").is_file()


def test_cli_predict_help_y_checkpoint_ausente(tmp_path: Path, capsys: pytest.CaptureFixture) -> None:
    with pytest.raises(SystemExit) as excinfo:
        predict_mod.main(["--help"])
    assert excinfo.value.code == 0
    assert "--image" in capsys.readouterr().out

    ruta = tmp_path / "roja.png"
    _imagen_roja().save(ruta)
    codigo = predict_mod.main(["--image", str(ruta), "--checkpoint", str(tmp_path / "no_existe.pt")])
    salida = capsys.readouterr().out
    assert codigo == 1
    assert "entrena" in salida.lower()
    assert "python -m src.train" in salida


def test_cli_predict_imprime_el_topk(tmp_path: Path, capsys: pytest.CaptureFixture) -> None:
    ckpt, _ = _crear_checkpoint(tmp_path)
    ruta = tmp_path / "roja.png"
    _imagen_roja().save(ruta)

    codigo = predict_mod.main(
        ["--image", str(ruta), "--checkpoint", str(ckpt), "--topk", "4", "--min-conf", "0.0", "--
device", "cpu"]
    )

    salida = capsys.readouterr().out
    assert codigo == 0
    assert "imagen:" in salida and "checkpoint:" in salida
    assert sum(1 for linea in salida.splitlines() if linea.strip().startswith(("1.", "2.", "3.", "4.")))
== 4
    for nombre in NOMBRES:
        assert nombre in salida
```

tests/test_smoke.py

Prueba de humo: el pipeline entero de principio a fin. (121 líneas)

```
"""Smoke test end-to-end del pipeline completo (dataset sintetico, sin red).

Ejercita la cadena REAL de modulos -- `data` (transforms) -> `model` -> `train` ->
`evaluate` -> `predict` -- sobre un dataset de imagenes sinteticas en disco, con un
modelo diminuto de 4 clases. No descarga nada y tarda unos segundos en CPU.

Es el test que anuncian `README.md` (seccion 4) y `CONTRATOS.md` (seccion 2) como
"pipeline end-to-end con dataset sintetico".

Ejecutar:
    .venv\\Scripts\\python.exe -m pytest tests\\test_smoke.py -q
"""

from __future__ import annotations

import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parents[1]
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))

import torch  # noqa: E402
from PIL import Image  # noqa: E402
from torch.utils.data import DataLoader  # noqa: E402
from torchvision.datasets import ImageFolder  # noqa: E402

torch.set_num_threads(2)

from src import evaluate as evaluate_mod  # noqa: E402
from src import train as train_mod  # noqa: E402
from src.config import Config  # noqa: E402
from src.data import build_transforms  # noqa: E402
from src.predict import predict_image  # noqa: E402
from src.utils import load_json  # noqa: E402

CLASES = 4
POR_CLASE = 6


def _crear_dataset_sintetico(base: Path) -> Path:
    """Crea `base/clase_i/i_j.png` con colores distintos por clase."""
    for i in range(CLASES):
        carpeta = base / f"clase_{i}"
        carpeta.mkdir(parents=True, exist_ok=True)
        for j in range(POR_CLASE):
            color = ((i * 61) % 256, (j * 37) % 256, (i * 17 + j * 11) % 256)
            Image.new("RGB", (96, 96), color).save(carpeta / f"{i}_{j}.png")
    return base


def _fabrica_loaders(base: Path, cfg: Config):
    """Devuelve una funcion compatible con `build_dataloaders` sobre el dataset falso."""

    def _build(_cfg=None):
        dataset = ImageFolder(str(base), transform=build_transforms(cfg, train=False))
        loader = DataLoader(dataset, batch_size=8, shuffle=False, num_workers=0)
        return loader, loader, loader, list(dataset.classes)

    return _build
```

```
@pytest.fixture
def cfg(tmp_path: Path) -> Config:
    config = Config(
        num_classes=CLASES,
        debug=True,  # fuerza num_workers=0, amp=False, freeze_epochs=0, epochs<=2
        epochs=1,
        backbone="mobilenet_v3_large",
        pretrained=False,  # sin red
        out_dir=str(tmp_path / "salida"),
        data_root=str(tmp_path / "datos"),
    )
    config.ensure_dirs()
    return config


def test_pipeline_completo_end_to_end(tmp_path: Path, cfg: Config, monkeypatch) -> None:
    """data -> model -> train -> evaluate -> predict, con artefactos reales en disco."""
    base = _crear_dataset_sintetico(tmp_path / "imagenes")
    fabrica = _fabrica_loaders(base, cfg)
    monkeypatch.setattr(train_mod, "build_dataloaders", fabrica)
    monkeypatch.setattr(evaluate_mod, "build_dataloaders", fabrica)

    # --- 1) ENTRENAMIENTO ----------------------------------------------------
    historial = train_mod.train(cfg)

    assert cfg.checkpoint_path.is_file(), "el entrenamiento no guardo el checkpoint"
    assert cfg.history_path.is_file()
    for clave in ("epochs", "train_loss", "train_acc", "val_loss", "val_acc", "val_top5"):
        assert clave in historial and len(historial[clave]) == 1, f"historial incompleto: {clave}"
    assert (cfg.figures_dir() / "curvas.png").is_file()

    # --- 2) EVALUACION -------------------------------------------------------
    metricas = evaluate_mod.evaluate(cfg.checkpoint_path, cfg, split="test")

    assert metricas["num_images"] == CLASES * POR_CLASE
    assert 0.0 <= metricas["top1"] <= 1.0
    assert 0.0 <= metricas["top5"] <= 1.0
    assert metricas["top5"] >= metricas["top1"], "top-5 nunca puede ser menor que top-1"
    assert cfg.metrics_path.is_file()
    assert (cfg.figures_dir() / "confusion_matrix.png").is_file()
    csv = cfg.predictions_dir() / "test_predicciones.csv"
    assert csv.is_file()
    assert len(csv.read_text(encoding="utf-8").strip().splitlines()) == CLASES * POR_CLASE + 1

    # --- 3) INFERENCIA -------------------------------------------------------
    imagen = Image.new("RGB", (200, 150), (30, 140, 90))
    top = predict_image(imagen, cfg.checkpoint_path, topk=3, min_conf=0.0, device="cpu")

    assert len(top) == 3
    assert all(isinstance(n, str) and isinstance(p, float) for n, p in top)
    assert all(0.0 <= p <= 1.0 for _, p in top)
    assert top == sorted(top, key=lambda par: par[1], reverse=True), "no estan ordenadas"
    assert {n for n, _ in top} <= set(load_json(cfg.metrics_path)["per_class_f1"].keys())

    # --- 4) EL UMBRAL DE CONFIANZA RESPONDE "NO RECONOCIDA" ------------------
    estricto = predict_image(imagen, cfg.checkpoint_path, topk=3, min_conf=1.1, device="cpu")
    assert estricto[0][0].lower().startswith("no reconocida")
```

pytest.ini

Configuración de las pruebas automáticas. (9 líneas)

```
[pytest]
testpaths = tests
# no:cacheprovider evita que pytest intente crear su cache con mkdtemp(0o700),
# que el sandbox de este entorno no puede listar (ver tests/conftest.py).
addopts = -p no:cacheprovider -ra
markers =
    slow: pruebas que usan el dataset real o tardan mas de ~30 s
filterwarnings =
    ignore::DeprecationWarning
```

requirements.txt

Lista de librerías necesarias. (17 líneas)

```
# Entorno local de verificacion (venv, CPU).
# IMPORTANTE para Colab: NO ejecutar `pip install -r requirements.txt` tal cual,
# porque Colab ya trae torch/torchvision con CUDA y pip los reemplazaria por
# ruedas CPU (se pierde la GPU). En Colab instala solo: gradio scikit-learn matplotlib

torch>=2.4
torchvision>=0.19
numpy>=1.26
Pillow>=10.2
scipy>=1.11          # requerido por torchvision.datasets.Flowers102 (scipy.io.loadmat)
matplotlib>=3.8
scikit-learn>=1.4
pandas>=2.1
tqdm>=4.66
gradio>=6.0          # src/app.py usa la API de Gradio 6 (en 6.0 desaparecio el evento submit)
pytest>=8.0
nbformat>=5.9        # validacion del notebook y ejecucion con nbclient (ver docs/VERIFICACION.md)
```

assets/cat_to_name.json

Los 102 nombres de especie, para no depender de internet. (104 líneas)


```
{
 "0": "pink primrose",
 "1": "hard-leaved pocket orchid",
 "2": "canterbury bells",
 "3": "sweet pea",
 "4": "english marigold",
 "5": "tiger lily",
 "6": "moon orchid",
 "7": "bird of paradise",
 "8": "monkshood",
 "9": "globe thistle",
 "10": "snapdragon",
 "11": "colt's foot",
 "12": "king protea",
 "13": "spear thistle",
 "14": "yellow iris",
 "15": "globe-flower",
 "16": "purple coneflower",
 "17": "peruvian lily",
 "18": "balloon flower",
 "19": "giant white arum lily",
 "20": "fire lily",
 "21": "pincushion flower",
 "22": "fritillary",
 "23": "red ginger",
 "24": "grape hyacinth",
 "25": "corn poppy",
 "26": "prince of wales feathers",
 "27": "stemless gentian",
 "28": "artichoke",
 "29": "sweet william",
 "30": "carnation",
 "31": "garden phlox",
 "32": "love in the mist",
 "33": "mexican aster",
 "34": "alpine sea holly",
 "35": "ruby-lipped cattleya",
 "36": "cape flower",
 "37": "great masterwort",
 "38": "siam tulip",
 "39": "lenten rose",
 "40": "barbeton daisy",
 "41": "daffodil",
 "42": "sword lily",
 "43": "poinsettia",
 "44": "bolero deep blue",
 "45": "wallflower",
 "46": "marigold",
 "47": "buttercup",
 "48": "oxeye daisy",
 "49": "common dandelion",
 "50": "petunia",
 "51": "wild pansy",
 "52": "primula",
 "53": "sunflower",
 "54": "pelargonium",
 "55": "bishop of llandaff",
 "56": "gaura",
 "57": "geranium",
 "58": "orange dahlia",
 "59": "pink-yellow dahlia?",
 "60": "cautleya spicata",
 "61": "japanese anemone",
 "62": "black-eyed susan",
```

```
 "63": "silverbush",
 "64": "californian poppy",
 "65": "osteospermum",
 "66": "spring crocus",
 "67": "bearded iris",
 "68": "windflower",
 "69": "tree poppy",
 "70": "gazania",
 "71": "azalea",
 "72": "water lily",
 "73": "rose",
 "74": "thorn apple",
 "75": "morning glory",
 "76": "passion flower",
 "77": "lotus",
 "78": "toad lily",
 "79": "anthurium",
 "80": "frangipani",
 "81": "clematis",
 "82": "hibiscus",
 "83": "columbine",
 "84": "desert-rose",
 "85": "tree mallow",
 "86": "magnolia",
 "87": "cyclamen",
 "88": "watercress",
 "89": "canna lily",
 "90": "hippeastrum",
 "91": "bee balm",
 "92": "ball moss",
 "93": "foxglove",
 "94": "bougainvillea",
 "95": "camellia",
 "96": "mallow",
 "97": "mexican petunia",
 "98": "bromelia",
 "99": "blanket flower",
 "100": "trumpet creeper",
 "101": "blackberry lily"
}
```

Anexo B. El cuaderno de Colab, celda por celda

Un cuaderno de Colab es una lista de celdas: unas contienen texto explicativo y otras el código que se ejecuta. Aquí se reproduce el código de todas las celdas de código, en el mismo orden en que se ejecutan.

Celda de codigo 1

```
import importlib
import subprocess
import sys


def importable(nombre):
    # True si el modulo se puede importar en este kernel.
    try:
        importlib.import_module(nombre)
        return True
    except Exception:
        return False


# --- 1.1 torch / torchvision: SOLO se comprueban, jamas se instalan ---------
TORCH_OK = importable("torch") and importable("torchvision")
if TORCH_OK:
    print("torch y torchvision ya estan disponibles: no se tocan (regla de Colab).")
else:
    print("AVISO: falta torch y/o torchvision en este entorno.")
    print("  - En Colab no deberia pasar: revisa 'Entorno de ejecucion > T4 GPU'.")
    print("  - Fuera de Colab instalalos tu a proposito: pip install torch torchvision")
    print("  (aqui NO se hace: en Colab romperia la GPU.)")

# --- 1.2 paquetes que Colab no garantiza: pip SOLO si faltan ---------------
REQUERIDOS = {
    "gradio": "gradio",           # demo interactivo (paso 9)
    "sklearn": "scikit-learn",    # metricas y matriz de confusion (paso 7)
    "matplotlib": "matplotlib",   # figuras de curvas y confusion
    "scipy": "scipy",             # Flowers102 lee los .mat con scipy.io
}
faltantes = [pip_name for modulo, pip_name in REQUERIDOS.items() if not importable(modulo)]

if faltantes:
    print("Instalando por pip (solo lo que falta):", faltantes)
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", *faltantes])
else:
    print("Todo lo necesario ya estaba instalado: no se llama a pip.")

# --- 1.3 paquetes de apoyo (informativo) -----------------------------------
print("\nDependencias de apoyo:")
for modulo in ("numpy", "pandas", "PIL", "tqdm"):
    print("  %-8s: %s" % (modulo, "OK" if importable(modulo) else "FALTA"))

# --- 1.4 resumen de versiones ----------------------------------------------
print("\n=== Resumen del entorno ===")
try:
    import torch
    import torchvision

    print("torch          :", torch.__version__)
    print("torchvision    :", torchvision.__version__)
    hay_cuda = torch.cuda.is_available()
    print("CUDA disponible:", hay_cuda)
    if hay_cuda:
        props = torch.cuda.get_device_properties(0)
        print("CUDA runtime   :", torch.version.cuda)
        print("GPU            :", torch.cuda.get_device_name(0))
        print("memoria        : %.1f GB" % (props.total_memory / 1024 ** 3))
    else:
```

```
        print("AVISO: no hay GPU. Entorno de ejecucion > Cambiar tipo de")
        print("       entorno de ejecucion > T4 GPU > Guardar. El notebook")
        print("       funciona en CPU, pero la corrida completa tarda muchisimo mas.")
except ImportError:
    print("torch no importable -> el entrenamiento fallara. Arreglalo antes de seguir.")
print("============================")
```

Celda de codigo 2

```
import shutil
import sys
from pathlib import Path

# ------------------------------------------------------------------ ajustes ----
USAR_DRIVE = True      # ponlo en False si no quieres persistir nada en Drive
DRIVE_DIR = None       # se rellena si Drive queda montado
EN_COLAB = False       # True si estamos dentro de Google Colab

try:
    import google.colab  # noqa: F401  (este modulo solo existe en Colab)

    EN_COLAB = True
except ImportError:
    EN_COLAB = False

# ------------------------------------------------------------------ montaje ---
if EN_COLAB and USAR_DRIVE:
    try:
        from google.colab import drive

        drive.mount("/content/drive")
        DRIVE_DIR = Path("/content/drive/MyDrive/IA_flores")
        DRIVE_DIR.mkdir(parents=True, exist_ok=True)
        print("Drive montado. Carpeta de respaldo:", DRIVE_DIR)
    except Exception as exc:
        DRIVE_DIR = None
        print("No se pudo montar Drive:", exc)
        print("El notebook continua igual; descarga el ZIP del paso 10 antes de cerrar.")
elif EN_COLAB:
    print("USAR_DRIVE=False: los resultados solo viviran durante esta sesion.")
else:
    print("Fuera de Colab (o sin google.colab): se omite el montaje de Drive.")
    print("Nota: los avisos de 'Acceso denegado' al escribir en MyDrive fuera de Colab")
    print("      son normales; el notebook funciona igual en modo local.")

# ---------------------------------------- restaurar dataset desde Drive --
# Solo se copia si data/ no existe todavia: si ya esta, no se toca.
RAIZ_ACTUAL = Path.cwd()
DATA_LOCAL = RAIZ_ACTUAL / "data"

if DRIVE_DIR is None:
    print("\nCache del dataset: sin Drive, se omite la restauracion.")
elif DATA_LOCAL.exists():
    print("\nCache del dataset: ya existe", DATA_LOCAL, "-> no se restaura nada.")
else:
    DATA_DRIVE = DRIVE_DIR / "data"
    try:
        if DATA_DRIVE.is_dir() and any(DATA_DRIVE.iterdir()):
            print("\nRestaurando data/ desde", DATA_DRIVE, "...")
            shutil.copytree(DATA_DRIVE, DATA_LOCAL)
            print("Dataset restaurado: se evita la descarga de ~330 MB.")
        else:
            print("\nCache del dataset: no hay copia en Drive todavia (se descargara).")
    except Exception as exc:
        print("\nNo se pudo restaurar data/ desde Drive:", exc)
        print("Se descargara el dataset otra vez; el notebook continua.")
```

Celda de codigo 3

```
import os
import shutil
import zipfile
from pathlib import Path

# Via de obtencion del codigo: "auto" | "zip" | "clone" | "drive"
MODO_CODIGO = "auto"

# En Colab el disco de trabajo es /content; fuera de Colab, el directorio actual.
TRABAJO = "/content" if Path("/content").is_dir() else "."

# Opcion "clone": descomenta y pon tu repositorio.
# REPO_GIT = "https://github.com/TU_USUARIO/IA-flores.git"
REPO_GIT = ""
RAIZ_CLON = TRABAJO + "/IA_flores"

# Opcion "zip": nombre del ZIP que sube el usuario.
NOMBRE_ZIP = "IA_flores_colab.zip"
RUTA_ZIP = TRABAJO + "/" + NOMBRE_ZIP

# Opcion "drive": carpeta del proyecto dentro de MyDrive.
ORIGEN_DRIVE = TRABAJO + "/drive/MyDrive/IA_flores"


def raiz_con_config(base):
    # Devuelve la carpeta que contiene src/config.py, o None si no existe.
    base = Path(base)
    if not base.exists():
        return None
    if (base / "src" / "config.py").is_file():
        return base.resolve()
    for sub in sorted(base.glob("*/src/config.py")):
        return sub.parent.parent.resolve()
    for sub in sorted(base.rglob("src/config.py")):
        return sub.parent.parent.resolve()
    return None


def detectar_proyecto():
    # Busca el proyecto en el cwd y en sus carpetas padre (caso Jupyter local).
    actual = Path.cwd().resolve()
    for candidato in [actual, *actual.parents]:
        raiz = raiz_con_config(candidato)
        if raiz is not None:
            return raiz
    return None


raiz = detectar_proyecto()

if MODO_CODIGO == "auto":
    if raiz is not None:
        print("MODO_CODIGO='auto': proyecto detectado en", raiz)
        MODO_CODIGO = "local"
    else:
        print("MODO_CODIGO='auto': no hay proyecto local -> se pide el ZIP.")
        MODO_CODIGO = "zip"

if MODO_CODIGO == "local":
    # (a) El proyecto ya esta aqui (o en una carpeta padre): no hay nada que copiar.
    print("Se usa el proyecto local en", raiz)

elif MODO_CODIGO == "clone":
    # (b) Clonar el repositorio. Si es privado, usa un token en la URL.
```

```
    if not REPO_GIT:
        raise ValueError("MODO_CODIGO='clone' pero REPO_GIT esta vacio: pon la URL del repositorio.")
    if Path(RAIZ_CLON).exists():
        print("Ya existe", RAIZ_CLON, "-> no se vuelve a clonar.")
    else:
        os.system('git clone --depth 1 "%s" "%s"' % (REPO_GIT, RAIZ_CLON))
    raiz = raiz_con_config(RAIZ_CLON)

elif MODO_CODIGO == "zip":
    # (c) Subir el ZIP del proyecto desde el PC.
    if not Path(RUTA_ZIP).exists():
        try:
            from google.colab import files   # solo existe en Colab

            print("Selecciona el ZIP del proyecto (" + NOMBRE_ZIP + ")...")
            subidos = files.upload()
            for nombre in subidos:
                if Path(nombre).resolve() != Path(RUTA_ZIP).resolve():
                    shutil.move(nombre, RUTA_ZIP)
        except ImportError:
            print("Fuera de Colab: copia el ZIP a mano en", RUTA_ZIP)

    if not Path(RUTA_ZIP).exists():
        raise FileNotFoundError(
            "No encuentro %s. Sube el ZIP (python empaquetar_colab.py) o usa MODO_CODIGO='clone'."
            % RUTA_ZIP
        )

    extraido = Path(TRABAJO) / "_extraido"
    if extraido.exists():
        shutil.rmtree(extraido)
    with zipfile.ZipFile(RUTA_ZIP) as zf:
        zf.extractall(extraido)
    raiz = raiz_con_config(extraido)
    if raiz is None:
        raise FileNotFoundError(
            "El ZIP no contiene src/config.py. Comprime la carpeta del proyecto, no los archivos
sueltos."
        )
    print("ZIP extraido en", raiz)

elif MODO_CODIGO == "drive":
    # (d) Copiar la carpeta del proyecto que ya esta en Drive.
    if not Path(ORIGEN_DRIVE).exists():
        raise FileNotFoundError("No encuentro %s. Sube la carpeta del proyecto a Drive." % ORIGEN_DRIVE)
    destino = Path(TRABAJO) / "IA_flores"
    shutil.copytree(ORIGEN_DRIVE, destino, dirs_exist_ok=True)
    raiz = raiz_con_config(destino)
    print("Proyecto copiado desde Drive a", destino)

else:
    raise ValueError("MODO_CODIGO debe ser 'auto', 'local', 'zip', 'clone' o 'drive'.")

if raiz is None:
    raise FileNotFoundError(
        "No encuentro src/config.py. Revisa la via elegida en MODO_CODIGO (ahora: %r)." % MODO_CODIGO
    )

# La raiz del proyecto pasa a ser el directorio de trabajo: los imports y las rutas
# relativas de Config (data/, resultados/) cuelgan de aqui.
os.chdir(raiz)

assert Path("src/config.py").is_file(), "Falta src/config.py en la raiz del proyecto."
print("Raiz del proyecto:", Path.cwd())
print("src/config.py    : OK")
print("Contenido        :", sorted(x.name for x in Path(".").iterdir()))
```

Celda de codigo 4

```
import json
import sys
import time
from pathlib import Path

RAIZ_PROYECTO = str(Path.cwd())
if RAIZ_PROYECTO not in sys.path:
    sys.path.insert(0, RAIZ_PROYECTO)   # permite "from src.xxx import ..."

import matplotlib.pyplot as plt                                          # noqa: E402

from src.config import Config, FLOWERS102_CLASSES, IMAGE_SIZE            # noqa: E402
from src.utils import describe_device, load_json, resolve_device, set_seed  # noqa: E402

print("Imports del proyecto OK.")
print("Raiz en sys.path :", RAIZ_PROYECTO)
print("Contrato         : img_size = %d px | num_classes = %d" % (IMAGE_SIZE, FLOWERS102_CLASSES))
```

Celda de codigo 5

```
# Cambia a False para la corrida completa (25 epocas, batch 32, ~15-25 min en T4).
MODO_DEBUG = True

if MODO_DEBUG:
    cfg = Config(
        debug=True,                  # subconjunto + ajustes automaticos de Config
        debug_per_class=4,           # imagenes por clase en train/val/test
        epochs=2,
        batch_size=32,
        out_dir="resultados/debug",  # separado de los resultados definitivos
        device="auto",               # CUDA si esta disponible, si no CPU
    )
else:
    cfg = Config(
        debug=False,
        epochs=25,
        batch_size=32,
        backbone="efficientnet_b0",
        dropout=0.3,
        pretrained=True,
        lr=3e-4,                     # learning rate de la cabeza
        backbone_lr=3e-5,            # LR bajo del backbone al descongelar
        warmup_epochs=2,
        freeze_epochs=3,
        early_stop_patience=6,
        amp=True,                    # precision mixta (se ignora en CPU)
        out_dir="resultados",
        device="auto",
    )

cfg.ensure_dirs()                   # crea out_dir, figuras/, predicciones/ y data/
set_seed(cfg.seed)
device = resolve_device(cfg.device)

print("Modo         :", "DEBUG (subconjunto)" if MODO_DEBUG else "COMPLETO (dataset entero)")
print("Dispositivo  :", describe_device(device))
print("Epocas       : %d | batch: %d | img: %d" % (cfg.epochs, cfg.batch_size, cfg.img_size))
print("Backbone     : %s (pretrained=%s, dropout=%s)" % (cfg.backbone, cfg.pretrained, cfg.dropout))
print("Salida       :", Path(cfg.out_dir).resolve())
print("Checkpoint   :", cfg.checkpoint_path)
print("Metricas     :", cfg.metrics_path)
print("num_workers  :", cfg.num_workers, "(0 en debug; en Colab/Linux 2 es correcto)")
print("data_root    :", Path(cfg.data_root).resolve(), "(aqui se cachea Flowers-102)")
```

Celda de codigo 6

```
import shutil
import time
import warnings
from pathlib import Path

from src.train import train


def respaldar_en_drive(origen="resultados", destino=None):
    # Copia la carpeta de resultados a Drive (si Drive esta montado).
    if DRIVE_DIR is None:
        print("Sin Drive montado: se omite el respaldo (modo local).")
        return None
    origen = Path(origen)
    if not origen.exists():
        print("Nada que respaldar todavia:", origen)
        return None
    destino = Path(destino) if destino is not None else DRIVE_DIR / "resultados"
    try:
        destino.mkdir(parents=True, exist_ok=True)
        shutil.copytree(origen, destino, dirs_exist_ok=True)
        print("Respaldo en Drive:", destino)
        return destino
    except Exception as exc:
        print("No se pudo respaldar en Drive:", exc)
        return None


# La descarga de pesos ImageNet se bloquea en entornos sin salida a esa URL: el
# contrato de src.model degrada solo a pretrained=False. Aqui se traduce el aviso
# crudo por uno mas legible (el entrenamiento sigue siendo valido).
with warnings.catch_warnings(record=True) as capturados:
    warnings.simplefilter("always")
    t0 = time.time()
    historial = train(cfg)             # DEBUG o COMPLETO segun MODO_DEBUG (paso 5)
    minutos = (time.time() - t0) / 60.0

for aviso in capturados:
    if "preentrenados" in str(aviso.message):
        print("\nNOTA: no se pudieron descargar los pesos ImageNet del backbone")
        print("      (red bloqueada). El contrato de src/model degrada solo a")
        print("      pretrained=False: se entrena desde cero y todo lo demas funciona.")
        print("      En Colab la descarga suele funcionar; si no, es lo mismo.")

print("\nEntrenamiento terminado en %.1f min" % minutos)
print("Mejor val_acc: %s (epoca %s)" % (historial.get("best_val_acc"), historial.get("best_epoch")))
print("Checkpoint   :", cfg.checkpoint_path, "| existe:", Path(cfg.checkpoint_path).is_file())

# Respaldo inmediato: si Colab se cae ahora, el checkpoint ya esta en Drive.
respaldar_en_drive("resultados")

# --- curvas del historial ----------------------------------------------
hist = load_json(cfg.history_path)
epocas = hist["epochs"]
fig, ejes = plt.subplots(1, 3, figsize=(16, 4))
ejes[0].plot(epocas, hist["train_loss"], marker="o", label="train")
ejes[0].plot(epocas, hist["val_loss"], marker="o", label="val")
ejes[0].set_xlabel("epoca"); ejes[0].set_ylabel("loss"); ejes[0].legend()
ejes[0].set_title("Perdida"); ejes[0].grid(alpha=0.3)
ejes[1].plot(epocas, hist["train_acc"], marker="o", label="train")
ejes[1].plot(epocas, hist["val_acc"], marker="o", label="val")
ejes[1].plot(epocas, hist["val_top5"], marker="s", alpha=0.6, label="val top-5")
ejes[1].set_xlabel("epoca"); ejes[1].set_ylabel("accuracy"); ejes[1].legend()
ejes[1].set_title("Accuracy (top-1 y top-5)"); ejes[1].grid(alpha=0.3)
ejes[2].plot(epocas, hist["lr"], marker="o", color="tab:green")
ejes[2].set_xlabel("epoca"); ejes[2].set_ylabel("learning rate")
```

```
ejes[2].set_title("LR de la cabeza (warmup + coseno)"); ejes[2].grid(alpha=0.3)
plt.tight_layout(); plt.show()

print("Historial:", cfg.history_path)
print("Figura de curvas:", cfg.figures_dir() / "curvas.png")
```

Celda de codigo 7

```
from src.evaluate import evaluate

metricas = evaluate(cfg.checkpoint_path, cfg, split="test")

print("\nClaves de metrics.json:", sorted(metricas.keys()))
print("top1 = %s | top5 = %s | macro_f1 = %s" % (
    metricas.get("top1"), metricas.get("top5"), metricas.get("macro_f1")))
```

Celda de codigo 8

```
# --- 7.b Lectura de metrics.json, matriz de confusion y peores clases ------
import warnings

metricas = load_json(cfg.metrics_path)   # lo acaba de escribir evaluate()

# Resumen sin las estructuras grandes (per_class / per_class_f1 / top_confusions).
resumen = {k: v for k, v in metricas.items() if k not in ("per_class", "per_class_f1",
"top_confusions")}
print(json.dumps(resumen, indent=2, ensure_ascii=False))
print("\nTop-1: %.2f%% | Top-5: %.2f%% | macro-F1: %.4f | imagenes: %s | %.1f s" % (
    100.0 * float(metricas["top1"]), 100.0 * float(metricas["top5"]),
    float(metricas["macro_f1"]), metricas["num_images"], float(metricas["segundos"])))

print("\nConfusiones mas frecuentes (real, pred, n):")
for fila in (metricas.get("top_confusions") or [])[:10]:
    real, pred, n = fila[0], fila[1], fila[2]
    print("  %-32s -> %-32s n=%s" % (real, pred, n))

# La matriz de confusion 102x102 la guarda evaluate() como PNG.
ruta_cm = Path(metricas.get("ruta_confusion") or (cfg.figures_dir() / "confusion_matrix.png"))
if ruta_cm.is_file():
    plt.figure(figsize=(9, 9))
    plt.imshow(plt.imread(ruta_cm))
    plt.axis("off")
    plt.title("Matriz de confusion (%dx%d) - split %s" % (
        cfg.num_classes, cfg.num_classes, metricas.get("split", "test")))
    with warnings.catch_warnings():
        # Si un modulo del proyecto forzo el backend "Agg" (lo hace src.evaluate
        # para no abrir ventanas), plt.show() avisa de que no puede dibujar en
        # pantalla. La figura se genera igual y se incrusta en el notebook.
        warnings.filterwarnings("ignore", message=".*non-interactive.*")
        plt.show()
    print("Figura:", ruta_cm)
else:
    print("No encuentro la matriz de confusion en", ruta_cm)

# Peores clases por F1 (especies que mas se confunden).
per_class_f1 = metricas.get("per_class_f1")
if isinstance(per_class_f1, dict) and per_class_f1:
    try:
        peores = sorted(per_class_f1.items(), key=lambda kv: float(kv[1]))[:5]
        print("\nClases con peor F1 (especies que mas se confunden):")
        for nombre, f1 in peores:
            print("  %-40s F1=%.3f" % (nombre, float(f1)))
    except (TypeError, ValueError):
        print("\nper_class_f1 no tiene el formato nombre -> f1; se omite el ranking.")
else:
```

```
    print("\nmetrics.json no trae per_class_f1 como diccionario; se omite el ranking.")
```

Celda de codigo 9

```
# --- 8.a Resumen pegable en INFORME.md (seccion 8) --------------------------
from pathlib import Path

metricas = load_json(cfg.metrics_path)
hist = load_json(cfg.history_path)


def _pct(valor):
    try:
        return "%.2f %%" % (100.0 * float(valor))
    except (TypeError, ValueError):
        return "s/d"


def _segundos(valor):
    try:
        total = float(valor)
    except (TypeError, ValueError):
        return "s/d"
    return "%.1f s (%.1f min)" % (total, total / 60.0)


def _entero(valor):
    try:
        return f"{int(valor):,}".replace(",", " ")
    except (TypeError, ValueError):
        return "s/d"


lineas = []
lineas.append("<!-- Copiado del notebook de Colab (paso 8) -->")
lineas.append("")
lineas.append("**Corrida:** %s | dispositivo: %s | backbone: %s" % (
    "DEBUG (subconjunto)" if MODO_DEBUG else "COMPLETA (1020/1020/6149)",
    metricas.get("device", "s/d"),
    metricas.get("backbone", cfg.backbone)))
lineas.append("")
lineas.append("### 8.1 Metricas principales (split oficial de test)")
lineas.append("")
lineas.append("| Metrica | Valor | Fuente del dato |")
lineas.append("|---|---|---|")
origen = Path(cfg.out_dir).as_posix()
lineas.append("| Exactitud top-1 | %s | `%s/metrics.json` -> `top1` |" % (_pct(metricas.get("top1")),
origen))
lineas.append("| Exactitud top-5 | %s | `%s/metrics.json` -> `top5` |" % (_pct(metricas.get("top5")),
origen))
lineas.append("| Macro-F1 | %.4f | `%s/metrics.json` -> `macro_f1` |" % (float(metricas.get("macro_f1",
0.0)), origen))
lineas.append("| Imagenes evaluadas | %s | `%s/metrics.json` -> `num_images` |" %
(_entero(metricas.get("num_images")), origen))
lineas.append("| Tiempo de evaluacion | %s | `%s/metrics.json` -> `segundos` |" %
(_segundos(metricas.get("segundos")), origen))
lineas.append("| Tiempo de entrenamiento | %s | `%s/history.json` -> `segundos` |" %
(_segundos(hist.get("segundos")), origen))
lineas.append("| Mejor epoca | %s | `%s/history.json` -> `best_epoch` |" %
(_entero(hist.get("best_epoch")), origen))
lineas.append("| Epocas ejecutadas | %s | `%s/history.json` -> `epochs_run` |" %
(_entero(hist.get("epochs_run")), origen))
lineas.append("| Mejor exactitud de validacion | %s | `%s/history.json` -> `best_val_acc` |" %
(_pct(hist.get("best_val_acc")), origen))
lineas.append("| Dispositivo utilizado | %s | `%s/history.json` -> `device` |" % (str(hist.get("device",
"s/d")), origen))
```

```
lineas.append("| Numero de parametros | %s | `%s/history.json` -> `num_params` |" %
(_entero(hist.get("num_params")), origen))
lineas.append("")
lineas.append("### 9.2 Confusiones mas frecuentes (real -> predicha)")
lineas.append("")
lineas.append("> Bloque para la seccion 9.2 del informe (analisis de errores).")
lineas.append("")
lineas.append("| # | Especie real | Especie predicha | Imagenes |")
lineas.append("|---|---|---|---:|")
confusiones = list(metricas.get("top_confusions") or [])[:10]
if confusiones:
    for posicion, fila in enumerate(confusiones, start=1):
        lineas.append("| %d | %s | %s | %s |" % (posicion, fila[0], fila[1], fila[2]))
else:
    lineas.append("| - | (sin confusiones registradas) | - | 0 |")
lineas.append("")
lineas.append("**Figuras:** `%s` (curvas), `%s` (matriz de confusion), `%s` (top confusiones)." % (
    cfg.figures_dir() / "curvas.png",
    cfg.figures_dir() / "confusion_matrix.png",
    cfg.figures_dir() / "top_confusions.png"))

if MODO_DEBUG:
    lineas.append("")
    lineas.append("> AVISO: estas cifras son del MODO DEBUG (subconjunto). Para el informe hay")
    lineas.append("> que repetir la corrida con `MODO_DEBUG = False` y volver a copiar el bloque.")

bloque = "\n".join(lineas)
print("-----------------------------8<-----------------------------")
print(bloque)
print("----------------------------->8-----------------------------")

# Se guarda tambien en disco para no perderlo.
ruta_resumen = Path(cfg.out_dir) / "resumen_informe.md"
ruta_resumen.write_text(bloque + "\n", encoding="utf-8")
print("\nResumen guardado en:", ruta_resumen)
```

Celda de codigo 10

```
from src.predict import predict_image

# Imagen de ejemplo: la primera disponible en data/ (o assets/ si no hay dataset).
candidatas = sorted(Path("data").rglob("*.jpg")) if Path("data").is_dir() else []
if not candidatas:
    candidatas = sorted(Path("assets").rglob("*.jpg"))

# min_conf=0.0 -> se muestra SIEMPRE el top-k. Con el umbral por defecto (0.25) un
# modelo poco entrenado (por ejemplo en MODO_DEBUG) no llega ni al 3 % de confianza,
# porque el softmax queda muy plano, y esta celda imprimiria solo "no reconocida".
MIN_CONF = 0.0

if candidatas and Path(cfg.checkpoint_path).is_file():
    imagen_ejemplo = candidatas[0]
    print("Imagen de ejemplo:", imagen_ejemplo)
    print("Checkpoint       :", cfg.checkpoint_path)
    resultados = predict_image(
        imagen_ejemplo, cfg.checkpoint_path, topk=cfg.topk, min_conf=MIN_CONF
    )
    for posicion, (nombre, probabilidad) in enumerate(resultados, start=1):
        print("%2d. %-42s %6.2f%%" % (posicion, nombre, 100.0 * probabilidad))
    maxima = max((p for _, p in resultados), default=0.0)
    if maxima < 0.25:
        print()
        print("NOTA: la confianza maxima es %.2f %%, por debajo del 25 %%." % (100.0 * maxima))
        print("      Es normal si entrenaste en MODO_DEBUG o sin pesos preentrenados:")
        print("      el softmax queda plano. Con la corrida completa (MODO_DEBUG=False)")
        print("      la confianza sube y la especie propuesta es mas fiable.")
else:
```

```
    print("Sin imagen de ejemplo o sin checkpoint: se omite la prediccion suelta.")
```

Celda de codigo 11

```
from pathlib import Path

from src.app import build_app

CHECKPOINT = str(cfg.checkpoint_path)   # resultados/best.pt (o resultados/debug/best.pt)
print("Checkpoint de la demo:", CHECKPOINT, "| existe:", Path(CHECKPOINT).is_file())

# Umbral por debajo del cual la demo anade un AVISO de "confianza baja".
# La demo SIEMPRE muestra el top-k con las especies; este umbral ya no borra la
# prediccion (antes respondia "no reconocida" incluso acertando la especie).
MIN_CONF = 0.25

demo = build_app(checkpoint_path=CHECKPOINT, topk=cfg.topk, min_conf=MIN_CONF)
print("Interfaz construida:", type(demo).__name__, "| tiene launch:", hasattr(demo, "launch"))

# share=False -> solo el iframe dentro de Colab (privado y mas rapido).
# share=True  -> enlace publico temporal (https://xxxx.gradio.live, ~72 h),
#                para abrirlo desde el movil o ensenar la demo a alguien.
USAR_SHARE = False

# prevent_thread_lock=True lanza el servidor en segundo plano y devuelve el control:
# asi la demo queda viva (iframe/enlace) pero la ejecucion no se queda bloqueada.
BLOQUEAR_CELDA = False

resultado = demo.launch(share=USAR_SHARE, debug=False, prevent_thread_lock=not BLOQUEAR_CELDA)

url_local = None
url_publica = None
if isinstance(resultado, tuple) and len(resultado) >= 3:
    url_local, url_publica = resultado[1], resultado[2]
elif isinstance(resultado, str):
    url_local = resultado

print("Demo lanzada.")
if url_local:
    print("  URL local    :", url_local)
if USAR_SHARE and url_publica:
    print("  URL publica  :", url_publica)
elif USAR_SHARE:
    print("  AVISO: se pidio share=True pero Gradio no devolvio enlace publico.")
print("  Para detenerla: ejecuta la celda de abajo (demo.close()).")
print("  Si falta el checkpoint, la interfaz muestra el aviso de entrenar primero.")
```

Celda de codigo 12

```
# --- Celda auxiliar OPCIONAL: parar la demo de Gradio ----------------------
# Ejecutala solo si quieres liberar el puerto (por ejemplo antes de relanzarla).
try:
    demo.close()
    print("Demo detenida.")
except NameError:
    print("No hay ninguna demo levantada en esta sesion.")
except Exception as exc:
    print("No se pudo cerrar la demo:", exc)
```

Celda de codigo 13

```
import shutil
import zipfile
from pathlib import Path
```

```
RAIZ_RESULTADOS = Path("resultados")            # incluye resultados/debug si existe
NOMBRE_ZIP = (Path("/content") if Path("/content").is_dir() else Path(".")) / "resultados_flores.zip"

if not RAIZ_RESULTADOS.exists():
    raise FileNotFoundError("No hay resultados/: ejecuta antes el paso 6 (entrenamiento).")

with zipfile.ZipFile(NOMBRE_ZIP, "w", zipfile.ZIP_DEFLATED) as zf:
    for ruta in sorted(RAIZ_RESULTADOS.rglob("*")):
        if ruta.is_file():
            zf.write(ruta, ruta.as_posix())

print("ZIP creado: %s (%.1f MB)" % (NOMBRE_ZIP, NOMBRE_ZIP.stat().st_size / 1024 ** 2))
with zipfile.ZipFile(NOMBRE_ZIP) as zf:
    for nombre in zf.namelist()[:20]:
        print("   ", nombre)

# Descarga al PC (solo funciona en Colab) y copia a Drive para no perderlo.
try:
    from google.colab import files

    files.download(str(NOMBRE_ZIP))
except ImportError:
    print("Fuera de Colab: el ZIP esta en", NOMBRE_ZIP)
except Exception as exc:
    print("No se pudo lanzar la descarga:", exc)

if DRIVE_DIR is not None:
    try:
        shutil.copy2(NOMBRE_ZIP, DRIVE_DIR / NOMBRE_ZIP.name)
        print("Copia del ZIP en Drive:", DRIVE_DIR / NOMBRE_ZIP.name)
    except Exception as exc:
        print("No se pudo copiar el ZIP a Drive:", exc)

# --- cache del dataset en Drive (evita re-descargar 330 MB la proxima vez) --
if DRIVE_DIR is None:
    print("\nCache del dataset: sin Drive, se omite (el dataset se queda en la sesion).")
elif not Path("data").is_dir():
    print("\nCache del dataset: no hay data/ que copiar.")
else:
    try:
        destino_data = DRIVE_DIR / "data"
        print("\nCopiando data/ a Drive (330 MB, puede tardar un par de minutos)...")
        shutil.copytree("data", destino_data, dirs_exist_ok=True)
        print("Dataset cacheado en:", destino_data)
        print("La proxima sesion lo restaurara en el paso 2 y no volvera a descargarlo.")
    except Exception as exc:
        print("No se pudo cachear data/ en Drive:", exc)
        print("El notebook termina igual; la proxima sesion volvera a descargar el dataset.")

# --- inventario final de artefactos ----------------------------------------
print("\n=== Artefactos de la corrida ===")
for etiqueta, ruta in [
    ("checkpoint", cfg.checkpoint_path),
    ("historial", cfg.history_path),
    ("config usada", cfg.config_path),
    ("metricas", cfg.metrics_path),
    ("resumen informe", Path(cfg.out_dir) / "resumen_informe.md"),
    ("curvas", cfg.figures_dir() / "curvas.png"),
    ("matriz de confusion", cfg.figures_dir() / "confusion_matrix.png"),
    ("top confusiones", cfg.figures_dir() / "top_confusions.png"),
    ("predicciones", cfg.predictions_dir() / "test_predicciones.csv"),
    ("ZIP de resultados", NOMBRE_ZIP),
]:
    print("  %-18s %s  %s" % (etiqueta, "OK " if Path(ruta).exists() else "FALTA", ruta))
```

Anexo C. Estructura del proyecto y cómo regenerarlo

Así está organizado el proyecto. Las carpetas data y resultados no viajan en el paquete final: se crean solas al ejecutarlo (el dataset se descarga la primera vez).

```
IA flores/
  README.md                 guía rápida para quien llega nuevo
  CONTRATOS.md              interfaces acordadas antes de programar
  PLAN_TRABAJO.md           plan inicial y riesgos previstos
  INFORME.md                informe académico (con huecos para tus resultados)
  requirements.txt          librerías necesarias
  pytest.ini                configuración de las pruebas
  empaquetar_colab.py       crea el ZIP que se sube a Colab
  IA_flores_colab.zip       el paquete que se sube a Colab
  assets/
    cat_to_name.json        los 102 nombres de especie (sin depender de internet)
  docs/
    VERIFICACION.md         auditoría independiente del proyecto
    construir_documentacion.py  genera este documento
    contenido/              las secciones de este documento en Markdown
  notebooks/
    flores_colab.ipynb      el cuaderno que se ejecuta en Colab
  src/
    config.py               todos los ajustes
    utils.py                utilidades comunes
    data.py                 dataset, transformaciones y lotes
    model.py                la red neuronal y los checkpoints
    train.py                entrenamiento
    evaluate.py             métricas y matriz de confusión
    predict.py              predicción de una foto
    app.py                  demo de Gradio
  tests/
    conftest.py             arreglo de los temporales
    test_data.py            pruebas de datos
    test_model.py           pruebas del modelo
    test_train.py           pruebas del entrenamiento
    test_eval_predict.py    pruebas de evaluación y predicción
    test_smoke.py           prueba de humo de todo el pipeline
  data/                     (se crea al ejecutar) el dataset, 330 MB
  resultados/               (se crea al ejecutar) checkpoints, métricas y figuras
```

Cómo volver a generar el paquete para Colab

Si se modifica cualquier archivo del proyecto, este comando vuelve a crear el archivo comprimido que se sube a Colab:

```
python empaquetar_colab.py

# y para volver a generar esta documentación:
python docs/construir_documentacion.py
```